

MEASURING AND MITIGATING SOLUTION MODE COLLAPSE IN RLVR

Anonymous authors

Paper under double-blind review

ABSTRACT

A language model (LM) can usually answer the same question in more than one way, but reinforcement learning with verifiable rewards (RLVR) is indifferent to which correct answer a model produces. A solution will earn the same reward whether it is the thousandth copy of a familiar answer or one the model has never produced before. Yet, there is potential value in having the model retain multiple correct solutions as it is trained. For instance, multiple modes may give users a choice and provide problem-solving strategies that improve overall model performance. Here, we measure solution diversity through RLVR training using MODEBENCH, a set of reasoning tasks, where the verifier returns both correctness and mode discovered. We find that RLVR post-training concentrates probability onto fewer correct modes even as accuracy holds or improves, and moreover, that frontier models are already highly concentrated. We then introduce Re:Max, which uses the verifier to index a memory containing one verified example per discovered mode and rehearses those modes uniformly. A solution found once is, therefore, practiced as often as one found repeatedly. Across three model scales, two RL objectives, and harder task constructions, rehearsing verified successes raises how often a policy succeeds, and rehearsing them uniformly is what preserves how many ways it can succeed.

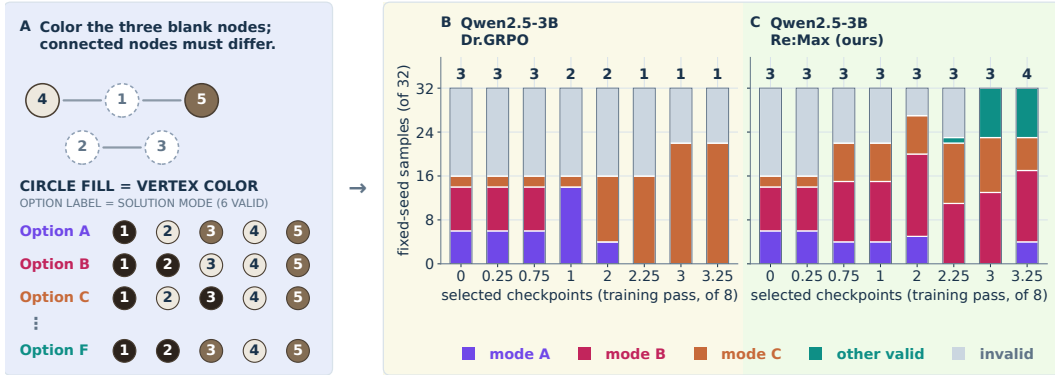


Figure 1: **Motivation.** Consider the Graph coloring problem shown in A (left), with six valid solutions. We evaluate Qwen2.5-3B across training, tracking the number and diversity of correct solutions. At each training pass, we show 32 sampled solutions, pooling eight samples from four matched runs. Training with Dr. GRPO (B) improves accuracy, but the model progressively collapses onto a single solution. In contrast, Re:Max (C) improves accuracy and expands the diversity of correct solutions.

1 INTRODUCTION

We usually evaluate reinforcement learning with verifiable rewards (RLVR) post-training by how often a model succeeds. But test-time search and reranking depend on having alternatives to choose from (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Wang



ModeBench dataset and models



Code repository

Links anonymized for review.

et al., 2023). This raises a question: as a model gets better at finding a correct answer during RLVR, *what happens to other correct alternatives?*

RLVR trains against an executable check (Guo et al., 2025), but its binary reward does not distinguish a solution the model has produced a thousand times from one it has never produced before. Nothing in the objective asks it to maintain diversity. Fig. 1 shows what this can look like on a graph-coloring problem with six valid answers: under Dr.GRPO (Liu et al., 2025), the policy becomes more accurate while its correct outputs concentrate onto a single mode.

Mode collapse, or narrowing of the output space, has been observed in alignment (Kirk et al., 2024; Cui et al., 2025), creative generation (Doshi & Hauser, 2024), brainstorming and ideation (Anderson et al., 2024), public argument (Kim et al., 2026), and LM-assisted writing (Abdulhai et al., 2026). Quantifying this narrowing requires deciding when two outputs are actually the same answer. Token-level diversity (Li et al., 2016; Zhu et al., 2018) counts two spellings of one solution as variety, while a judge that groups answers by meaning (Farquhar et al., 2024; Zheng et al., 2023) puts a second model between the data and the claim.

Here, we investigate the prevalence and impact of mode collapse in problem-solving tasks under RLVR post-training. Our contributions are threefold:

1. **MODEBENCH**: We contribute a benchmark for empirically studying mode collapse in reasoning tasks. **MODEBENCH** provides both output correctness and maintains a unique identity for each available modal solution to a given problem. The benchmark spans five executable domains, each with five difficulty levels (Sec. 2).
2. We conduct comprehensive experiments on **MODEBENCH** across 4 open-weight model families and 7 frontier models. Across open-weight models and at 3 model scales, the number of solution modes collapsed in every domain under both Dr.GRPO and GRPO. Further, in inference, all frontier models we measured reached fewer than 1.5 effective modes (Sections 5.1 and 2.3).
3. We propose *mode replay*: store one verified exemplar per mode and rehearse modes equally. Adding replay to MaxRL (Tajwar et al., 2026) gives Re:Max, which improves accuracy and correct-mode diversity across scales and task levels. A matched weighting ablation isolates the contribution of equal rehearsal to additional verified modes (Sections 3 and 5.2–5.3).

Overall, our findings shed light on solution mode collapse and suggest that incorporating mode replay as part of RLVR post-training can lead to both increased diversity of solutions and improved model performance.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

Producing a benchmark to study solution mode collapse is challenging for two reasons. First, because we must be able to control that a specific question has several valid discrete solutions. Second, because we must be able to identify, from reasoning traces, which solution was reached. For example, in Fig. 2D, we show algebra problems in our benchmark. Note that we must both generate questions for which multiple solutions exist and, given a correct answer, be able to determine which solution mode is present.

Ensuring these two properties hold is domain-specific. Fig. 2 illustrates a subset of solution modes across the considered domains; e.g., in **Countdown** and **MathIR**, modes mean the model reached

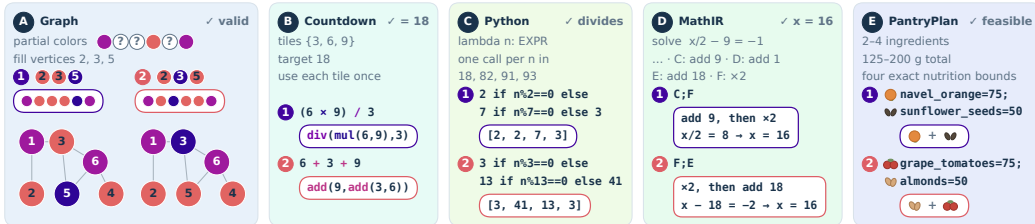


Figure 2: **MODEBENCH** comprises questions with many acceptable solutions across domains. (1) and (2) identify different solution modes across the five domains, which include Graph coloring, a normalized computation tree, executable Python code, algebraic problems and constraint optimization.

the same answer by a different route, whereas in Graph and PantryPlan, modes mean a different valid answer altogether. App. B and App. C give the per-domain rules and the prompts.

For each domain, we provide a verifier that evaluates whether a solution is correct and which solution mode a solution belongs to. This was built by creating in advance a comprehensive set of potential solutions to the problem, which at training time allows us to quickly sort the reasoning traces associated with a solution into a pre-computed solution mode. Precisely, for a prompt x with specification s_x , the validator maps an output y either to failure $\perp$ or to a key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs are the same mode if and only if their keys agree. The full set of valid keys $\mathcal{C}_x$ is never revealed, neither its members nor its size. A policy, and the memory that stores its successes, only ever sees keys for answers the policy actually produced.

2.1 SUCCESS AND VERIFIED-MODE DIVERSITY

To measure performance on MODEBENCH, we must consider two distinct properties: how often the output is accurate, and, given that it is right, which of the available right answers has been given. A benchmark reporting only accuracy cannot capture the failure we study.

That obvious way to measure modal diversity is to count distinct correct answers in K samples. Let P be the chance a sample is correct and q_c be how that correct probability is divided among the valid answers. Then

$$\underbrace{\Pr[D_K > 0]}_{\text{pass@K}} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct@K}} = \sum_c [1 - (1 - Pq_c)^K], \quad (1)$$

where D_K counts the distinct correct answers. However, both metrics increase with P , and thus a policy that is right more often may have higher pass@K and distinct@K while producing less diverse solutions.

We therefore propose an alternative metric, pairwise correct-mode diversity:

$$\text{PCMD} = 1 - \sum_c q_c^2 = \Pr[Z_1 \neq Z_2 \mid \text{both responses verified}], \quad (2)$$

measuring the joint probability of different solution modes given two correct responses. A PCMD of zero thus implies every correct answer belongs to the same solution mode, and an even spread over m yields a PCMD of $1 - 1/m$, e.g., if answers spread evenly across two possible solution modes, then the PCMD is $1/2$. We note that when infinite solution modes are possible, the maximum PCMD converges to 1, and that when the model outputs no correct solutions, we cannot calculate the metric. We only calculate PCMD when models yield at least 30 correct solutions for a specific problem, to avoid excessively noisy measurements.

Fig. 3 shows how PCMD captures diversity of responses independently of accuracy. GPT-5.6 Sol has near-perfect pass@8 for all five tasks considered, yet much weaker models in the

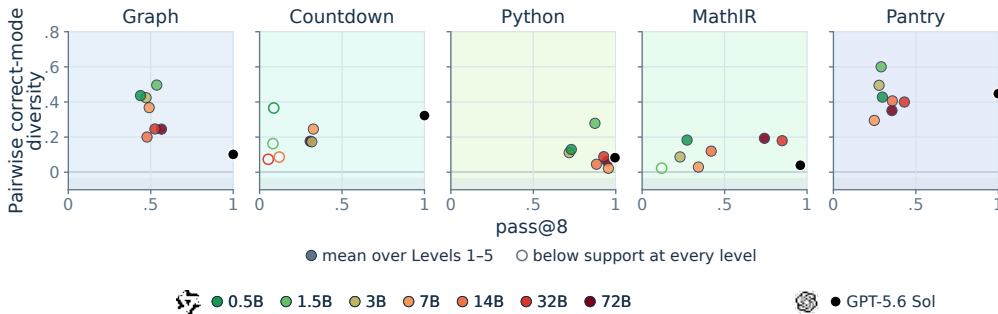


Figure 3: **Higher performance is not associated with increased solution mode diversity.** We show the relationship between a performance-related metric (pass@8; x -axis) and our proposed solution mode diversity metric (PCMD; y -axis) for GPT-5.6 Sol (in black), and Qwen2.5-Instruct models of various sizes (where parameter count is indicated by color). Note that accuracy and model size are not correlated with solution mode diversity.

Qwen2.5-Instruct family have higher PCMD. For the context of this work, we define a *solution mode collapse* as a decline in PCMD during RLVR.

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

We construct MODEBENCH so that tasks can be made more or less challenging, which helps prevent saturation. For each domain, we include 5 independent difficulty levels, e.g. for our Graph domain we allow more hidden vertices, and for Countdown we give the model more operands. Each difficulty level holds constant (1) our definition of mode, (2) our approximate distribution of modes, (3) our verifier constant, and (4) the sample size of 384 train and 128 test problems. To ensure (2) holds, and moreover, that the problems sample are in fact more challenging, we fit each level with consideration to $\text{pass}@1$ and $\text{pass}@8$ of a reference model, e.g. Level 4 was tuned such that a Qwen2.5-Instruct-7B model has the same $\text{pass}@1$ and $\text{pass}@8$ scores as a Qwen2.5-Instruct-0.5B on Level 1.

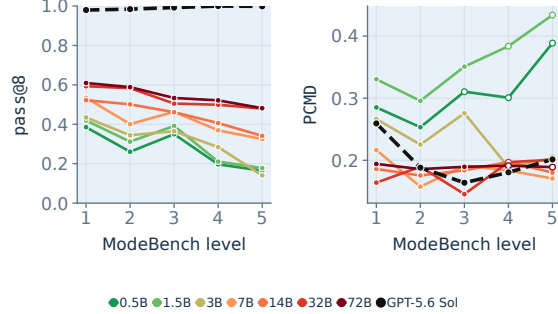


Figure 4: **Harder problems of the same domain exhibit fewer solution modes.** When we ask the Qwen2.5-Instruct family and GPT-5.6 Sol increasingly difficult questions from harder levels of MODEBENCH, $\text{pass}@8$ declined and so too does PCMD. For our benchmark, as problems get harder, solution mode collapse worsens.

LMs with a higher parameter count generally have higher $\text{pass}@8$ and can solve more problems. Yet, in Fig. 4, for our harder problems, we find that solution mode diversity declines, even as our underlying mode count per level remains stable. This is most noticeable when frontier models are evaluated. Even though correctness is saturated, we are able to use PCMD to isolate solution mode collapse (App. N), and this phenomena happen across MODEBENCH. Frontier models answer .991 of questions from level 1-3 correctly, but their mean PCMD is .232, which is below the .570 of a 1.5B model that answers .273 of its own prompts.

2.3 CONCENTRATION IN FRONTIER MODELS

Repeated sampling can increase solution coverage (Brown et al., 2024; Yue et al., 2025), but our discovery curves flatten far below the available modes (Fig. 5). On one Level-3 Graph problem, GPT-5.6 Sol answers correctly in all 1,024 draws yet produces only one of six valid colorings. For Python, 512 draws recover one to four return vectors from hundreds available:

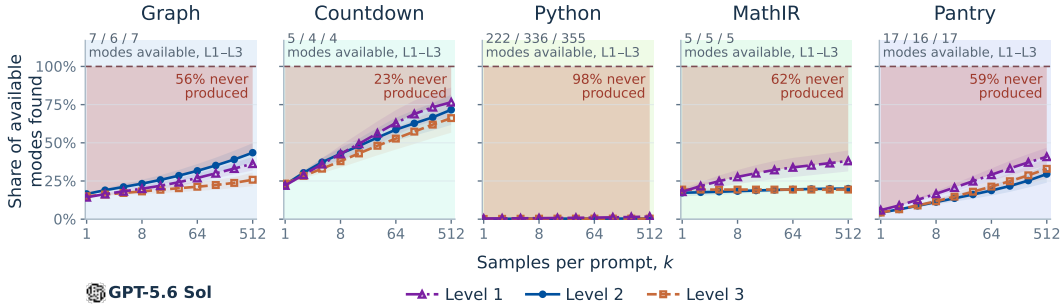


Figure 5: **Asking the model more times may not recover all available modes.** We ask each frontier model the same question eight times in our early experiments. However, a potential critique could be that we simply need to ask the model more times. So, we ask GPT-5.6 Sol, with medium reasoning on; 32 problems per domain and level, 512 responses each. Curves are distinct verified modes in random k -subsets as a share of the modes those problems admit, and the shaded band is what no level ever produced. Shading is the pointwise 95% problem bootstrap. Printed counts are the enumerated support those problems admit, in Level 1/2/3 order, and every panel shares one scale (App. K.3).

under 2% of the support, and under 1% at Levels 2 and 3. All seven hosted deployments narrow from Level 1 to Level 3 despite near-perfect correctness (Fig. 16).

Larger sampling budgets also leave trained local policies concentrated: quadrupling to $K = 32$ changes diversity by thousandths, and MathIR at 64 draws remains as concentrated as at eight (Apps. J and K). Disabling explicit reasoning lowers overall correctness and mode counts; conditional diversity changes vary by deployment (Table 56, App. O.6). Mixing independent models recovers more modes than either alone (App. L.2); raising temperature also adds modes, at an accuracy cost (Fig. 38).

3 RE:MAX: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

3.1 THEORETICAL INTUITION FOR MODE COLLAPSE

Binary rewards treat all correct modes alike, while fresh samples follow the policy’s current preferences. A mode with an early probability advantage is therefore sampled and reinforced more often (Fig. 6). Under independent logit updates, this feedback can produce winner-take-all concentration. Changing the binary-reward estimator alone does not change the trajectory of this concentration, as the following result makes precise.

For one isolated prompt, let a finite categorical policy have $m \geq 2$ correct modes and $n \geq 1$ incorrect categories, with $p = \text{softmax}(z)$, correctness $P = \sum_c p_c$, and conditional mode probabilities $q_c = p_c/P$. Each category has an independent logit. Draw responses $Z_1, \dots, Z_G$ independently with $G \geq 2$, with binary rewards R_i and $R = \sum_i R_i$. Assume finite initial logits, common response-length normalization, and on-policy Euclidean expected-gradient flow without KL, entropy, momentum, or preconditioning (Assumption P.1).

Theorem 3.1 (Conditional inertness of outcome-binary objectives). *Under Assumption P.1, let $a(\cdot, \cdot)$ be any detached advantage measurable with respect to the group’s binary reward vector, applied as $\hat{g}_a = G^{-1} \sum_i a(R_i, R) \nabla_z \log p_{Z_i}$ at the on-policy point. Then*

$$\mathbb{E}[\hat{g}_a] = c_a(P) \nabla_z P, \quad c_a(P) = \frac{1}{G} \left(\frac{\mathbb{E}[R a(1, R)]}{P} - \frac{\mathbb{E}[(G - R) a(0, R)]}{1 - P} \right). \quad (3)$$

If $c_a > 0$ on $(0, 1)$, then under the time change $d\tau = c_a(P)P(1 - P) dt$ the conditional distribution q obeys Equation (18) exactly, for every such a . Dr.GRPO, reward-standardized GRPO, a leave-one-out baseline, the centered binary MaxRL estimator, and any reweighting shaped by a pass@ k target are points in this class. The choice among them rescales optimization time along the correctness axis and leaves both the trajectory in q and the identity of the surviving mode exactly where they were.

The conditional dynamics are $dq_c/d\tau = q_c(q_c - \sum_d q_d^2)$: above-average modes grow at the expense of others. With a unique initial maximum and unbounded τ , one mode eventually dominates (Theorem P.4). The proof of Theorem 3.1 is in App. P.3. These are statements about the specified logit geometry, not guarantees for stochastic neural training. Replay changes the available targets by retaining successes that fresh sampling may stop producing.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Replay revisits a discovered success even when fresh sampling no longer produces it, drawing on established uses of memory in RL (Lin, 1992; Mnih et al., 2015; Rolnick et al., 2019). Under a fixed-bank analysis, a constant replay dose keeps every retained mode above a positive probability floor throughout training (Theorem P.21).

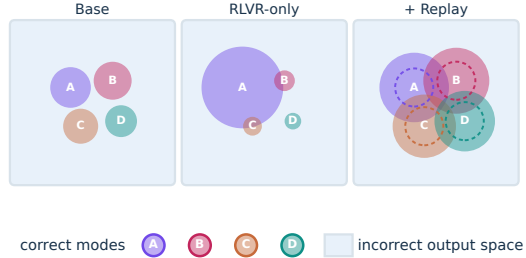


Figure 6: **RLVR early "winner" (A) becomes the pre-dominant way to solve.** Under the base model, we see that before RLVR, many modes exist. However, RLVR-only training concentrates it on the mode found first. Our uniform replay approach keeps every banked mode and leaves room to find more.

For each training prompt x , we maintain an initially empty, bounded bank $\mathcal{B}_x$. The first time a verified mode appears, we store one exemplar e_c that realizes it (subsequent rediscoveries do not modify the bank). A deterministic schedule revisits nonempty banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (4)$$

Uniform weighting breaks the link between discovery frequency and rehearsal frequency. Once a mode enters the bank, it is rehearsed as often as every other retained mode, so frequently sampled modes receive no additional replay.

Adding this loss to MaxRL (Tajwar et al., 2026) gives Re:Max; adding it to Dr.GRPO (Liu et al., 2025) gives Re:Dr. Banked responses contribute only through the separate replay loss, leaving the on-policy reward group unchanged. Banks contain only training-prompt responses, so held-out evaluation measures the learned policy (App. D). Fig. 7 illustrates the two learning paths. Unlike methods that modify fresh rewards or advantages to favor rare outcomes (Song et al., 2025; Li et al., 2026a), mode identity determines which successes memory retains and rehearses.

4 EXPERIMENTAL DESIGN

We compare Dr.GRPO (Liu et al., 2025) and MaxRL (Tajwar et al., 2026), each with and without replay, on all five MODEBENCH domains. Level 1 uses Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B with five seeds, eight training epochs, and $G = 16$ fresh rollouts per prompt. Replay contributes a likelihood gradient on at most $M = 16$ stored exemplars within the same optimizer step (Table 6). Optimizer updates and fresh-rollout counts are matched; total FLOPs and wall-clock time are not. The zero-derivative control retains replay computation but does not redirect that work to extra task-gradient updates, fresh rollouts, or longer training. These comparisons test the replay mechanism, not superiority at equal total compute (App. Q.1). We repeat the comparison at Levels 2 and 3 with Qwen2.5-0.5B. At Level 1 and 0.5B, we also test frequency-weighted replay, UCPO (Lochab et al., 2026), RLEP (Zhang et al., 2025), GAPO (Anschel et al., 2025), SetPO (Li et al., 2026a), and fixed Semantic-MaxEnt.

Our separate inference-only studies evaluate GPT-5.6 Sol, GPT-5.4, Grok 4.3, Kimi K3, and Claude Opus 4.8 on Levels 1–3, with eight responses per prompt and a fixed formatting normalizer (App. O.6). Open-weight evaluations span the available scales of SmoLLM2, Qwen2.5, Falcon3, and OLMo-2 across Levels 1–5 (App. B.6). Deployment identifiers and protocols are recorded with these results; no training or replay is applied in these studies.

5 RESULTS

We test three claims: RLVR concentrates correct responses onto fewer modes; verified replay improves accuracy and diversity; and balancing replay across modes contributes additional diversity. We then examine harder task levels and compare replay with anchoring to the initial policy.

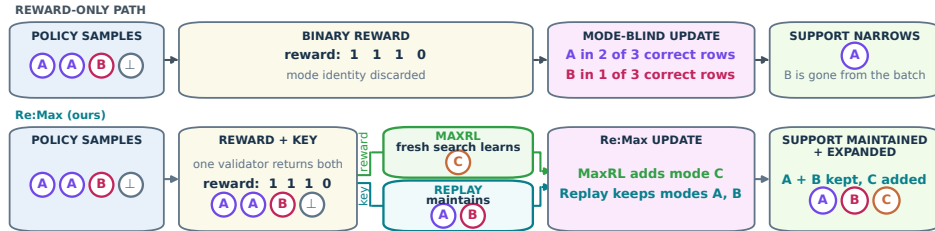


Figure 7: **Replay preserves successes that on-policy learning may stop seeing.** MaxRL learns from the policy’s current samples, while replay stores one verified exemplar per discovered mode and revisits those exemplars with equal weight. Together, these two learning paths allow Re:Max to preserve previously discovered solution modes while continuing to explore for new ones.

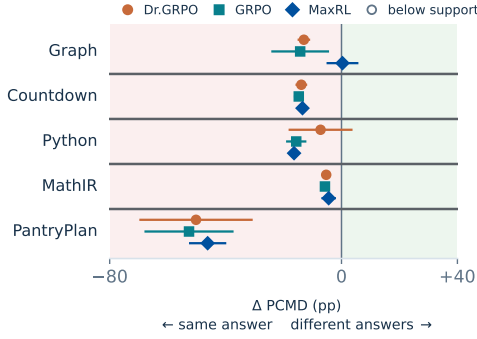


Figure 8: **Training reduces solution mode diversity.** Change in PCMD at Qwen2.5-3B, Level 1, across three objectives and five domains. Marks are five-seed means with nominal 95% intervals. Diversity falls in 14 of 15 comparisons. Other scales are in Fig. 26.

5.1 RLVR CONCENTRATES SOLUTION MODES

RLVR replaces diverse correct attempts with the same repeated answers. We compare each policy’s solution-mode distribution before and after training using PCMD. We evaluate whether training under Dr.GRPO, GRPO, and MaxRL results in fewer modes across five domains and three model scales. Fig. 8 shows Qwen2.5-3B; Fig. 26 gives the other scales. The starkest case is `Python`, where Qwen2.5-3B goes from solving 73 of 128 held-out prompts in multiple ways to as few as 22 after Dr.GRPO post-training. At Qwen2.5-3B, PCMD falls in 14 of the 15 objective–domain comparisons.

As outputs concentrate, rollouts under RLVR lose training signal. A group containing only successes or only failures gives zero fresh task advantage under Dr.GRPO and MaxRL. Across all 3,072 Qwen2.5-0.5B `Python` updates, the five-seed mean mixed-group fraction is 1.5% under Dr.GRPO versus 18.5% under Re:Dr. Saved control completions also become byte-identical within each prompt by steps 288–480 and remain so through training. Replay supplies a separate likelihood gradient even when the fresh task gradient vanishes; the complete audit is in App. H.3.

5.2 REUSE RAISES ACCURACY; EQUAL WEIGHTING PRESERVES DIVERSITY

The combined method improves accuracy and reasoning diversity across objectives and scales. We add the same verified-mode replay mechanism to Dr.GRPO and MaxRL and compare each against its matched no-replay control across five MODEBENCH domains and three model scales. Replay improves the averages on both axes at each scale (Fig. 9). At Qwen2.5-0.5B, Re:Dr improves `pass@8` by .337 and PCMD by .294 over Dr.GRPO; Re:Max improves them by .358 and .320 over the same control (Table 1).

Balancing retained modes contributes additional diversity. We change only the replay target weights: uniform over stored modes versus proportional to their verified fresh-rollout counts. Bank construction, scheduling, and update code stay fixed, although the resulting policies can discover different bank contents. At Qwen2.5-0.5B, Level 1, we compare five matched seeds Table 1: **Compared objectives under Qwen2.5-0.5B, MODEBENCH Level 1.** Differences from terminal Dr.GRPO; Avg weights five domains equally within seed (five seeds). The separate untrained-checkpoint row is the initial checkpoint minus terminal Dr.GRPO. Bold marks the best trained method; dashes indicate insufficient correct-response pairs. Colors scale within each column.

	$\Delta \text{pass@8}$						ΔPCMD					
	Count	Count	Count	Count	Pantry	Pantry	Count	Count	Count	Count	Pantry	Pantry
	Graph	down	Python	MathIR	Plan	Avg	Graph	down	Python	MathIR	Plan	Avg
Re:Dr (<i>ours</i>)	+ .646	+ .191	+ .356	+ .285	+ .207	+ .337	+ .559	+ .480	+ .082	+ .017	+ .334	+ .294
Re:Max (<i>ours</i>)	+ .621	+ .185	+ .509	+ .277	+ .199	+ .358	+ .525	+ .504	+ .249	+ .005	+ .317	+ .320
GAPO	+ .175	+ .011	+ .009	− .054	+ .279	+ .084	+ .131	+ .340	.000	.000	+ .653	+ .225
SetPO	+ .064	− .050	+ .497	+ .051	+ .154	+ .143	+ .039	+ .007	+ .013	+ .003	+ .266	+ .066
MaxRL (no replay)	+ .213	+ .101	.000	− .066	+ .009	+ .051	+ .086	+ .015	.000	.000	.000	+ .020
RLEP-Dr	+ .098	+ .103	.000	+ .126	.000	+ .066	+ .035	+ .012	.000	− .001	.000	+ .009
UCPO	− .003	+ .133	.000	+ .031	+ .053	+ .043	− .003²	+ .019	.000²	− .001	.000²	+ .009
Semantic-MaxEnt	+ .002	+ .141	.000	− .121	+ .201	+ .045	+ .004	+ .020	.000	+ .001	—	+ .006
GRPO	+ .037	+ .159	.000	− .052	+ .023	+ .034	+ .008	+ .001	.000	+ .001	.000	+ .002
Untrained checkpoint	+ .132	− .383	− .172	− .292	+ .380	− .067	+ .108	—	—	+ .027	+ .591	+ .242

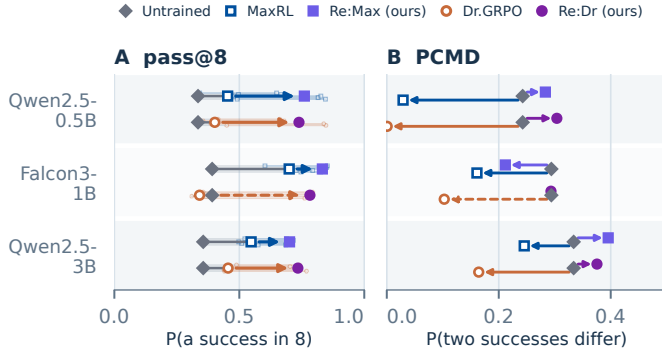


Figure 9: **Replay improves accuracy and diversity.** Diamonds mark initial models; open/filled marks are without/with replay. (A) averages five domains within seed; (B) uses the domains where all arms clear the support bar. Faint paths show seeds. Solid arrows use five seeds; thinner pairs print their counts.

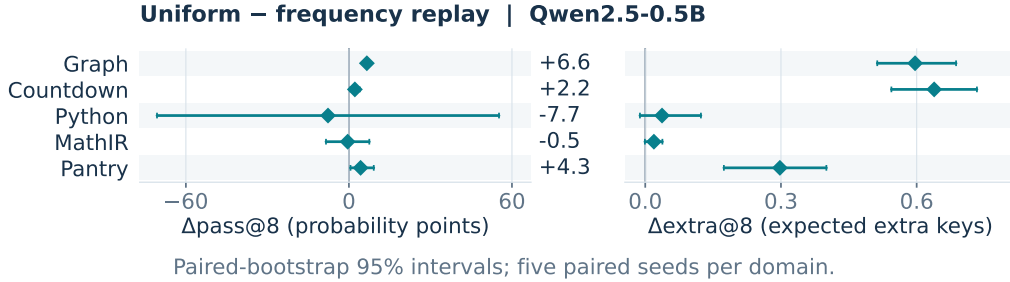


Figure 10: **Uniform weighting preserves more verified alternatives.** Uniform minus frequency-weighted replay at Qwen2.5-0.5B, Level 1, with five matched seeds. Extra modes are $B_8 = \text{distinct@8} - \text{pass@8}$. Bars show nominal 95% paired-seed bootstrap intervals. The mean gain is .318 extra modes, with an inconclusive correctness effect. Extra modes remain coupled to correctness; they do not measure success-conditional PCMD.

across all five domains. Because these runs lack resampled PCMD estimates, we measure extra verified modes beyond the first, $B_8 = \text{distinct@8} - \text{pass@8}$.

Uniform weighting adds .318 extra modes per prompt, with a nominal 95% paired-seed bootstrap interval [.272, .358], averaging domains equally within seed (Fig. 10). Graph, Countdown, and PantryPlan drive the gain; Python and MathIR are inconclusive. The pass@8 difference is .010 with interval [-.119, .139], establishing neither a correctness gain nor equivalence. The ablation supports balancing as a source of additional verified diversity, while its correctness effect remains unresolved. Reusing verified successes connects to rejection-sampling fine-tuning (Dong et al., 2023); the full ablation is in App. F.6.

Other diversity mechanisms give smaller aggregate gains. Table 1 compares UCPO, RLEP, GAPO, SetPO, and fixed Semantic-MaxEnt. Re:Max has the largest mean gain in both pass@8 (.358) and PCMD (.320) over Dr.GRPO. GAPO is the strongest alternative on mean PCMD (.225) and leads on PantryPlan (.653), showing that the aggregate advantage does not imply dominance in every domain.

5.3 REPLAY REMAINS EFFECTIVE BEYOND THE BASE TRAINING SETTING

Replay’s gains persist on harder task constructions. We repeat the matched comparisons within each of the first three MODEBENCH levels at Qwen2.5-0.5B. Replay improves average pass@8 and PCMD under both objectives at every level (Fig. 11).

Reference KL and replay preserve different sources of diversity. A KL penalty anchors the policy to its initial distribution; replay rehearses modes discovered during training. Beyond $\beta = .01$ in our sweep, increasing the KL coefficient preserves more diversity at lower correctness (Fig. 12). On the four-domain aggregate, matching replay’s diversity with KL costs correctness. The analysis in App. P.12 distinguishes the reference’s initial diversity from the modes the bank discovers.

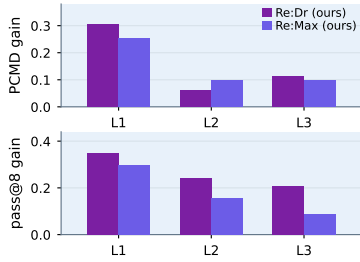


Figure 11: **Replay across task levels.** Re:Dr and Re:Max gains over their matched fresh objectives at Qwen2.5-0.5B, Levels 1–3. App. F.2 gives domain-level results.

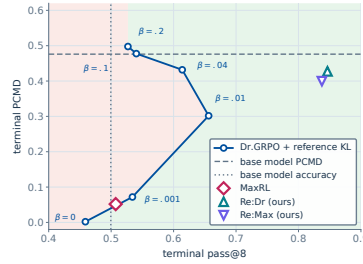


Figure 12: **Reference KL trades correctness for diversity.** Four-domain means (excluding Python) at Qwen2.5-0.5B across KL coefficients β . Green marks replay’s correctness advantage at matched diversity (App. P.12).

6 RELATED WORK

Alignment narrows what a model produces. RLHF and reasoning RL reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). The literature establishes the impact of solution mode collapse. Our work tries to quantify the impact and establish both a benchmark for evaluation and a potential path forward to resolve solution mode collapse.

What is a different answer? Diversity can only be measured once sameness is defined. Semantic clustering infers it from language (Farquhar et al., 2024), which is general but depends on a judge. MODEBENCH takes identity from the same executable check that assigns reward, so two outputs differ exactly when they execute differently; narrower, cheaper, and shared by training and evaluation. Alternatives matter because sampling-based selection consumes proposals (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024).

Expanding diversity. Outcome bonuses, set prediction, distribution matching and uniform-correct objectives all reshape the fresh update (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026), and MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); all act on what the policy samples now, the pressure our analysis identifies. Memory sidesteps it, but its weighting decides what survives: replay mitigates forgetting (Lin, 1992; Rolnick et al., 2019) and RLEP reuses successful reasoning trajectories (Zhang et al., 2025), yet both revisit successes in proportion to how they were collected. Balancing modes independently of discovery frequency separates our memory from those and connects it to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates reliable answering from verified-mode diversity. Re:Max retains discovered successes after fresh sampling stops producing them, improving accuracy and diversity across our controlled comparisons. The weighting ablation supports equal rehearsal as a source of additional verified modes; its correctness effect remains unresolved.

Limits. Replay retains only modes that sampling discovers. Our theory assumes independent categorical logits, and our benchmark uses executable mode identities. Distinct modes need not have equal practical value, so broader applications require both a defensible notion of equivalence and evaluation of the alternatives’ usefulness. Future work can test whether replay can maintain a chosen distribution over solution modes in open-ended tasks.

AI USE STATEMENT

Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The authors reviewed and verified the resulting content and take responsibility for the final manuscript.

ETHICS STATEMENT

The benchmark and experiment-specific data are synthetic; this study collects no human-subject or personal data. More verified modes are not inherently better: a distinct execution may be risky or costly even when it passes the validator, and PCMD does not measure downstream value. Systems that evaluate or optimize diversity should report correctness, invalidity, and application-specific costs alongside mode counts.

REPRODUCIBILITY STATEMENT

Every reported number is computed from a versioned result file. App. S records protocols, hashes, validations, and regeneration commands, and links the public dataset and model archive; App. B records split identities. The AI Use Statement above discloses generative-AI assistance.

REFERENCES

- Marwa Abdulhai, Isadora White, Yanming Wan, Ibrahim Qureshi, Joel Leibo, Max Kleiman-Weiner, and Natasha Jaques. How LLMs distort our written language, 2026. URL <https://arxiv.org/abs/2603.18161>.
- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Emmanuelle Anceaume, Yann Busnel, Ernst Schulte-Geers, and Bruno Sericola. Optimization results for a generalized coupon collector problem, 2015. URL <https://arxiv.org/abs/1504.03878v1>.
- Barrett R. Anderson, Jash Hemant Shah, and Max Kreminski. Homogenization effects of large language models on human creative ideation. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 413–425. Association for Computing Machinery, 2024. doi: 10.1145/3635636.3656204.
- Oron Anschel, Alon Shoshan, Adam Botach, Shunit Haviv Hakimi, Asaf Gendler, Emanuel Ben Baruch, Nadav Bhonker, Igor Kviatkovsky, Manoj Aggarwal, and Gerard Medioni. Group-aware reinforcement learning for output diversity in large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 32394–32415, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.1649.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025. URL <https://arxiv.org/abs/2505.22617v1>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward rAnked FineTuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2304.06767>.
- Anil R. Doshi and Oliver P. Hauser. Generative ai enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024. doi: 10.1126/sciadv.adn5290.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.

- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2160–2169. PMLR, 2019. URL <https://proceedings.mlr.press/v97/geist19a.html>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Marc Harper. Information geometry and evolutionary game theory. *arXiv preprint arXiv:0911.1383*, 2009. URL <https://arxiv.org/abs/0911.1383v1>.
- Xixiang He, Qiyao Sun, Ao Cheng, Xingming Li, Xuanyu Ji, Hailun Lu, Runke Huang, and Qingyong Hu. Advantage collapse in group relative policy optimization: Diagnosis and mitigation, 2026. URL <https://arxiv.org/abs/2605.21125v2>.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. Time-uniform Chernoff bounds via nonnegative supermartingales. *Probability Surveys*, 17:257–317, 2020. doi: 10.1214/18-PS321. URL <https://doi.org/10.1214/18-PS321>.
- Yekyung Kim, Yapei Chang, Chau Minh Pham, and Mohit Iyyer. Argument collapse: LLMs flatten long-form public debate, 2026. URL <https://arxiv.org/abs/2606.01736>.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PX03FAVHJT>.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062v1>.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 110–119, 2016. doi: 10.18653/v1/N16-1014. URL <https://aclanthology.org/N16-1014/>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430v4>.
- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.

- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding RL-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6820–6829. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. doi: 10.1038/nature14236.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017. URL <https://arxiv.org/abs/1701.06548>.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.
- Tim van Erven and Peter Harremoës. Rényi divergence and Kullback–Leibler divergence. *IEEE Transactions on Information Theory*, 2014. doi: 10.1109/TIT.2014.2320500. URL <https://arxiv.org/abs/1206.2459v2>.
- Nino Vieillard, Tadashi Kozuno, Bruno Scherrer, Olivier Pietquin, Rémi Munos, and Matthieu Geist. Leverage the average: An analysis of KL regularization in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/8e2c381d4dd04f1c55093f22c59c3a08-Abstract.html>.
- Xiao Wang, Shiqian Ma, Donald Goldfarb, and Wei Liu. Stochastic quasi-newton methods for nonconvex stochastic optimization. *SIAM Journal on Optimization*, 27(2):927–956, 2017. doi: 10.1137/15M1053141. URL <https://doi.org/10.1137/15M1053141>.

- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou. RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. Texygen: A benchmarking platform for text generation models. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 1097–1100, 2018. doi: 10.1145/3209978.3210080. URL <https://arxiv.org/abs/1802.01886>.

APPENDIX ORGANIZATION

The supplement is organized in eight parts, mapped here to their pages.

Part I. Scope and measurement	
A Metric and Objective Details	18
A.1 Measuring success and diversity	18
A.2 Endpoints and the reported diversity axis	19
A.3 Why balance within the replay bank	20
Part II. Benchmark and method	
B Benchmark and Evaluation Protocol	21
B.2 Dataset splits and validation	22
B.3 Python execution contract	22
B.4 Matched level constructions	23
B.5 Frozen base models across all five levels	23
B.1 What a mode is, by domain, and whether the effect depends on it	21
B.6 Other model families at matched scale	24
B.7 Training and evaluation settings	24
C Exact Domain Prompts	26
C.1 Graph coloring	27
C.2 Countdown	28
C.3 Python factors	28
C.4 MathIR action menu	29
C.5 PantryPlan	29
D Replay Algorithm	29
D.1 Fresh task advantages	29
D.2 Optimizer update	30
D.3 Admission, capacity, and deterministic scheduling	30
D.4 Zero-derivative replay control	30
D.5 Verifier requirements and applicability	31
D.6 Exemplar quality and length	31
E Fixed Semantic MaxEnt Comparator	31
Part III. Training results	
F Empirical Results and Comparisons	33
F.1 Terminal comparisons across scales	33
F.2 Terminal comparison on the harder level	35
F.3 Accuracy and verified modes throughout training	37
F.4 Supporting comparators	37
F.5 Diversity-preserving comparators	40
F.6 Uniform versus fresh-frequency replay	41
G Fixed Semantic-Entropy Factorials	41
H Collapse and Replay Mechanism Checks	42
H.1 Conditional concentration of correct outputs	42
H.2 Diversity before and after RLVR-only training	47
H.3 Fresh-gradient degeneracy and observed output collapse	48
H.4 Control behavior at fixed learning rates	50
H.5 Fixed-bank exemplar score trajectories	51
Part IV. What diversity represents	
I Matched Prompt-Hint Ablation	52
I.1 Removing strategy hints from unchanged problems	52
J Decoding Temperature, Nucleus, and Budget	54
K Sampling-Budget Ablation	56
K.1 Sampling design and estimands	56
K.2 Hosted discovery: guidance, formatting, and late modes	57
K.3 All five domains and all three levels at 512 draws	59
K.4 Observed sampling-budget effects	61
L Inference: Adaptation, Model Mixtures, and Coarser Keys	64
L.1 Pantry portfolios under changed requirements	64
L.2 Complementarity across hosted models	66
L.3 Sensitivity to task-relevant equivalences	68
M Portfolio Survival under Withdrawn Options	68
M.1 Two withdrawals at once	70
M.2 Decoding settings as a substitute	71
M.3 Recovery after the change	71
Part V. Fixed deployments	
N Inference-Only Concentration in a Hosted Model	73
N.1 Hosted concentration on the success-conditional axis	75
O Comparison across Hosted Deployments	76
O.1 Python prompt sensitivity	78
O.2 Requested-temperature sensitivity	78
O.3 GPT-5.6 Sol: temperature, pass@8 and verified modes	79
O.4 Native provider outcomes and undefined concentration	82
O.5 The revised Python wording on a second deployment	88
O.6 Matched reasoning-control comparison	89
Part VI. Idealized analysis	
P Mode Collapse and Verified Replay	91
P.1 Categorical abstraction and mean updates	91
P.2 Winner-take-all collapse in the categorical mean flow	93
P.3 Conditional trajectories of outcome-binary mean flows	94
P.4 Neural score identities and the limits of conditional inertness	95
P.5 Diversity at finite steps and sampling budgets	97

810	P.6	A replay signal when fresh groups supply none	100
811	P.7	Fixed-bank replay and qualitative retention	100
812	P.8	When replay reverses concentration, and how fast	103
813	P.9	Complete responses and finite evaluation visibility	105
814	P.10	When a replay gradient improves a neural exemplar	107
815	P.11	Reference KL retains the reference, not the discovery	108
816	P.12	Two ceilings, and which one a domain sits under	112
817	P.13	Why a memory rather than an anchor, once the reference is collapsed	114
818	P.14	Scope relative to entropy, KL, admission, and neural optimization	115
819	Q	What Verified Replay Costs	117
820	Q.1	What the control matches, and what it does not	117
821	Q.2	Token-based cost accounting and its limits	117
822	Q.3	The missing comparison at equal total compute	118
823	Part VII. Extended comparison		
824	R	Diversity-Preserving RLVR: Extended Comparison	119
825	Part VIII. Records		
826	S	Source Integrity and Reproducibility	121
827	S.1	Admissible sources and missing checkpoints	121
828	S.2	Conflicted initial checkpoints, admitted by provenance	121
829	S.3	Conflicted scheduled checkpoints, admitted by provenance	122
830	S.4	Data, models, and scientific records	122
831	S.5	Exact continuation and validation	122
832			
833			
834			
835			
836			
837			
838			
839			
840			
841			
842			
843			
844			
845			
846			
847			
848			
849			
850			
851			
852			
853			
854			
855			
856			
857			
858			
859			
860			
861			
862			
863			

NOTATION

Table 2 defines the symbols used in the benchmark, replay algorithm, and idealized analysis.

Table 2: **Notation.** Symbols reused across the benchmark, the algorithm, and the idealized analysis.

Symbol	Meaning
<i>Benchmark and metrics</i>	
$V(y, s_x)$	Validator: a canonical execution key, or $\perp$ for failure
$\mathcal{C}_x$	Set of valid keys for prompt x (never revealed to the learner)
m	Number of correct modes a prompt admits, $m = \mathcal{C}_x $
P	Probability that a sampled response is verified correct
q_c	Conditional probability of key c given correctness
Z_i	Canonical key of the i th draw, or $\perp$
$C(q)$	Conditional collision $\sum_c q_c^2$; the concentration measure
PCMD	Success-conditional diversity $1 - C(q)$; the reported diversity axis
$N_{\text{eff}}^{(2)}$	Effective number of verified modes, $1/(1 - \text{PCMD})$
K	Draws per evaluation group ($K = 8$ for every headline number)
n_c, n_+	Draws verified on key c , and verified in all, $n_+ = \sum_c n_c$
D_K, F_K	Distinct verified keys, and correct fraction, in K draws
$\bar{D}_K$	Expected distinct verified keys, $\mathbb{E}[D_K] = \text{distinct}@K$
B	Extra verified modes, $\text{distinct}@K - \text{pass}@K$
<i>Algorithm</i>	
$\mathcal{B}_x$	Replay bank for prompt x ; $M = 16$ its capacity
k	Number of keys a bank holds, $k = \mathcal{B}_x \leq M$; separately, the budget index in $\text{pass}@k$
$P_{\mathcal{B}}$	Total probability mass of banked keys
e_c	Stored exemplar realizing key c ; $ e_c $ its token length
ℓ_c	Mean response-token log probability of the exemplar e_c
G	Fresh rollout group size ($G = 16$ in training)
R_i, R	Binary reward of response i , and group success count $\sum_i R_i$
A_i	Fresh task advantage of response i
σ	Serialized round-robin replay cursor
η, κ	Scale and clip constants of the Semantic-MaxEnt comparator
<i>Idealized analysis</i>	
$\mathcal{A}$	Category set, correct modes $\mathcal{C}$ and incorrect $\mathcal{N}$
$c_G(P), \Psi$	Mean-flow correctness coefficient and its potential
ρ	Effective replay dose in the mean flow; λ_{replay} its implemented weight
τ	Effective optimization time in the mean flow
C_T	Energy constant in the retention bound
μ, β	Reference policy and its KL coefficient in (A4')
D	Reverse divergence $\text{KL}(p \parallel \mu)$ to that reference

Part I. Scope and measurement

A METRIC AND OBJECTIVE DETAILS

Correctness and verified-mode diversity are distinct properties of a policy. For a prompt x with reference specification s_x , the validator maps a response to either failure or a canonical key:

$$V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}.$$

Here $\mathcal{C}_x$ is the set of valid solution keys and $\perp$ denotes failure. The learner observes a key only for a response it generates; neither the full set $\mathcal{C}_x$ nor its size is revealed.

For $Y \sim \pi_\theta(\cdot | x)$, the policy induces both a total probability of verified success and a distribution over how that probability is allocated:

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c | x) = \Pr[V(Y, s_x) = c | V(Y, s_x) \neq \perp].$$

The main text writes these two quantities as P and q_c , dropping the policy and the prompt where they are clear. The conditional distribution is defined only when $P_\theta^+(x) > 0$; when $P_\theta^+(x) = 0$, all three sampling metrics below are zero, while PCMD is undefined rather than zero. The first quantity measures correctness. The second describes the distribution of verified behaviors conditional on being correct. We call concentration of p_θ^+ onto fewer keys *mode collapse*, particularly when P_θ^+ remains stable or increases. Estimating PCMD requires at least two verified responses per prompt. The full-support threshold for absolute estimates is 30 defining prompts; paired training comparisons require at least 20 per arm (App. A.2). Moving correct probability onto keys that already hold more of it can only lower PCMD = $1 - \sum_c q_c^2$ (Lemma A.1), so the concentration defined here always lowers PCMD (2) that the main text reports. The converse does not follow: PCMD summarizes an allocation in one number, so a fall in it need not arise from any such ordered movement of mass. Neither statement claims that a key reached probability zero, which finite samples cannot establish.

A.1 MEASURING SUCCESS AND DIVERSITY

Write n_c for the number of a prompt’s draws verified on key c , and $n_+ = \sum_c n_c$ for its verified draws in all. Where $n_+ \geq 2$, the estimator of (2) is the fraction of verified pairs that differ:

$$\widehat{\text{PCMD}} = 1 - \frac{\sum_c \binom{n_c}{2}}{\binom{n_+}{2}} = 1 - \frac{\sum_c n_c(n_c - 1)}{n_+(n_+ - 1)}, \quad (5)$$

which is unbiased for (2) at every $n_+ \geq 2$: conditioning on the number of verified draws leaves that many independent labels drawn from q , so every ordered pair agrees with probability $\sum_c q_c^2$. The estimator uses the verified count n_+ , not the draw budget: under the pooling rule below a local prompt’s four $K = 8$ groups contribute 32 draws, a hosted prompt contributes its eight, and n_+ counts the verified ones. The identity is promptwise, and does not survive joint selection on other conditions or dependent prompts. Two equivalent readings are sometimes useful: *rarefied-distinct@2* = $1 + \text{PCMD}$ is the expected number of distinct modes in exactly two verified draws, which fixes sampling depth rather than success count; and $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ is an effective number of verified modes. Both are monotone transforms of PCMD and order policies identically.

Each PCMD estimate averages this prompt-level estimator. Where a protocol draws more than one group per prompt, those draws are pooled before scoring — q is a property of the policy and prompt rather than of the draw budget, so pooling estimates the same quantity from more pairs and defines it far more often. The local evaluations draw four groups of eight and pool to 32; the hosted deployments answer each prompt eight times, with nothing to pool. Either way a reported value is the unweighted mean over the prompts where PCMD is defined, carried with its across-prompt standard error. Each prompt receives equal weight. Weighting a prompt by its number of verified pairs — equivalently, pooling verified pairs across prompts before taking the ratio — lets the prompts a policy solves most often dominate the estimate, which reintroduces the accuracy dependence PCMD exists to remove.

For K independent samples $Y_{1:K}$, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

MODEBENCH reports three sampling averages with different units. $\text{distinct@}K = \mathbb{E}[D_K]$ is an expected count in $[0, K]$ of unique verified keys; $\text{pass@}K = \Pr[D_K > 0]$ and $\text{mean@}K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$.

Lemma A.1 (Success, sampled diversity, and PCMD). *Fix a prompt, a finite or countable valid-key set, and $K \geq 1$ independent draws from the same fixed sampling law. Let $\mu_c = \Pr[V(Y, s_x) = c]$ and $P = \sum_c \mu_c$. Then*

$$\text{pass@}K = 1 - (1 - P)^K, \quad \bar{D}_K := \text{distinct@}K = \sum_c [1 - (1 - \mu_c)^K]. \quad (6)$$

For a finite valid-key set of size $m \geq 1$ and $P > 0$, write $q_c = \mu_c/P$. At fixed P , concentration lowers $\bar{D}_K$: if q majorizes q' (its decreasingly sorted partial sums are at least those of q'), then $\bar{D}_K(P, q) \leq \bar{D}_K(P, q')$. In particular,

$$1 - (1 - P)^K \leq \bar{D}_K(P, q) \leq m[1 - (1 - P/m)^K]. \quad (7)$$

For $K \geq 2$, the lower bound is attained only by concentration on one key, and the upper bound only by uniform q . For $K = 1$, $\bar{D}_1 = P$ for every q . At fixed q , increasing P cannot decrease $\bar{D}_K$. By contrast $\text{PCMD}(q) = 1 - \sum_c q_c^2$ does not involve P : it is unchanged when correctness rises or falls at a fixed allocation. Concentration lowers it: if q majorizes q' then $\text{PCMD}(q) \leq \text{PCMD}(q')$, strictly unless q' is a permutation of q . It runs from 0 at a point mass to $1 - 1/m$ at the uniform allocation, so its ceiling is set by the number of valid keys rather than by correctness.

Proof. All K draws fail with probability $(1 - P)^K$. For each key c , its appearance indicator has expectation $1 - (1 - \mu_c)^K$; summing indicators gives the second identity, also for countably many keys by nonnegativity. For fixed $P > 0$ and $K \geq 2$, the function $f(v) = 1 - (1 - Pv)^K$ is strictly concave on $[0, 1]$. Transferring mass from a larger coordinate to a smaller one without reversing their order increases $\sum_c f(q_c)$; successive such transfers give the majorization assertion. The point mass and uniform vector are its extreme cases, giving the bounds and equality conditions. Finally, $\partial \bar{D}_K / \partial P = K \sum_c q_c (1 - Pq_c)^{K-1} \geq 0$, which proves monotonicity in correctness. For the final claim, $\sum_c q_c^2$ is symmetric and convex, so the same transfer strictly decreases it whenever the two coordinates differ; PCMD is one minus that sum and no term in it involves P . The point mass and the uniform vector again give its extremes, by the same succession of transfers that gave the bounds on $\bar{D}_K$. $\square$

These are promptwise identities, before averaging over prompts. In particular, applying the nonlinear formula to a mean correctness generally changes the result. They explain the shared measurement in the training and hosted studies: success depends on total correct mass; sampled diversity also depends on its allocation.

Raw $\text{distinct@}K$ measures sampled coverage, not the total number of positive-probability modes. For an uninformed reference, suppose a finite space has N actions and N_c actions map to valid key c . Uniform independent action sampling then gives $\sum_c [1 - (1 - N_c/N)^K]$ expected verified keys. The simpler expression $m[1 - (1 - 1/N)^K]$ applies when the m valid actions have distinct keys, as in our reported finite-action references. Open-ended program and derivation spaces have no canonical nominal “uniform” generator. Table 20 applies these references to the trained policies.

A.2 ENDPOINTS AND THE REPORTED DIVERSITY AXIS

The terminal metrics are $\text{pass@}K$, the probability of at least one correct output, and raw $\text{distinct@}K$, the number of verified keys found. The diversity axis is PCMD, which depends only on the conditional allocation q (Lemma A.1). Raw $\text{distinct@}K$ also depends on correctness: it grows with P at fixed q (6). Across the 375 untrained measurements, its correlation with $\text{pass@}8$ is .936; PCMD correlates -.332 on the 225 measurements with sufficient support. These correlations use different populations because PCMD is undefined at low success rates.

The distinction is visible in Fig. 15. From 7B up, Python is answered correctly on at least .988 of held-out prompts at Levels 1–3, while its PCMD never exceeds .044: near-perfect accuracy coexists with repeated use of the same solution mode.

Support thresholds. PCMD is undefined for a prompt with fewer than two verified responses. The fraction of prompts defining it correlates .988 with `pass@8`, so policies can differ in both diversity and the population on which diversity is measurable. Each estimate includes its number of defining prompts. Filled markers indicate at least 30 such prompts; hollow markers indicate 5–29 and include standard errors. Estimates based on fewer than five prompts are omitted. Of the 375 frozen-model measurements, 225 meet the 30-prompt threshold. The gaps concentrate where success is rarest: 62 of the 150 are Countdown. An absent estimate indicates insufficient verified responses, not zero diversity. For paired training comparisons, each arm must define PCMD on at least 20 evaluation prompts; comparisons failing either threshold are omitted. The eligible prompt subsets can differ between arms.

PCMD is one minus the conditional-collision U-statistic of App. H.1, computed from the canonical keys of verified responses. The contrast `distinct@K` – `pass@K` = $\mathbb{E}[D_K - \mathbf{1}[D_K > 0]]$ instead measures verified modes beyond the first. It requires no knowledge of the full valid set $\mathcal{C}_x$, but remains coupled to the number of successful draws. At fixed $K = 8$, dividing `distinct@8` by eight is a linear rescaling.

A.3 WHY BALANCE WITHIN THE REPLAY BANK

Uniform key sampling decouples replay frequency from discovery frequency. Consider an equal-weight categorical surrogate without length normalization (Fig. 6); App. D gives the implemented response-level objective. A stored response’s probability need not equal its key’s total probability. For $k = |\mathcal{B}_x|$ banked keys, write $p(c)$ for their probabilities and P_B for their total mass. Then

$$-\frac{1}{k} \sum_{c \in \mathcal{B}_x} \log p(c) = -\log P_B + \log k + \text{KL}(U_k \parallel p(c \mid c \in \mathcal{B}_x)). \quad (8)$$

The objective rewards greater banked verified mass and a more balanced conditional distribution, aligning with `pass@K` and `distinct@K`. The combined update need not improve both at every step: banked-key coverage contributes $\sum_{c \in \mathcal{B}_x} [1 - (1 - p(c))^K]$ to expected `distinct@K` and, at fixed P_B and k , concavity makes it largest under uniform allocation, while $1 - (1 - P_B)^K$, the probability of any banked key, depends only on total mass (Fig. 13). Uniformity also maximizes the smallest conditional banked-key mass, making equal retention explicit.

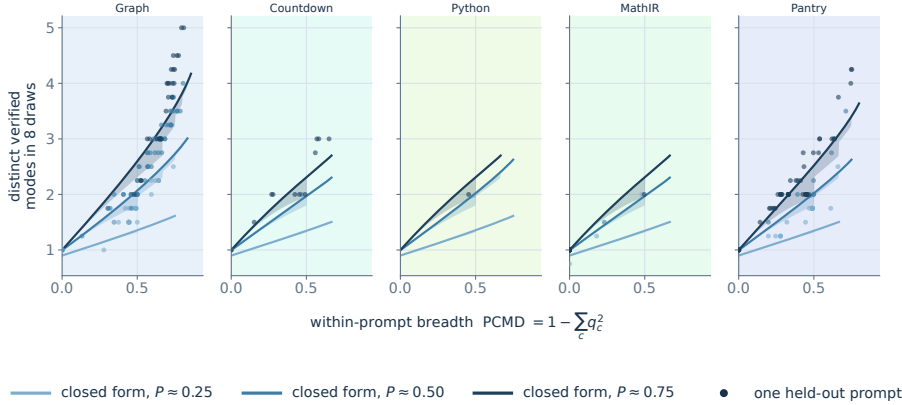


Figure 13: Balanced mode probabilities maximize expected coverage at fixed accuracy and mode count. Points show held-out Level-1 prompts from terminal Re:Dr and Re:Max checkpoints of Qwen2.5-0.5B (seed 43), grouped by their verified response fraction P . Bands show $\sum_c [1 - (1 - Pq_c)^8]$ for $P \in \{.25, .50, .75\}$ as allocations vary from a point mass to uniform over domain-specific mode-count ranges. They are analytic ranges, not confidence intervals. Both coordinates of each point use the same sampled responses; evaluation prompts have no replay bank.

These properties describe the categorical objective on training-prompt banks. Generalization is measured on held-out prompts, whose responses never enter the banks. The surrogate alone does not establish that the implemented response-level objective increases held-out PCMD.

Part II. Benchmark and method

B BENCHMARK AND EVALUATION PROTOCOL

MODEBENCH takes correctness and identity from one executable validator, the map $V(\cdot, s_x)$ of App. A: invalid outputs receive reward zero and no key, valid outputs reward one and a canonical execution key. Neither the valid-key catalogue nor its size is available to training, replay, scheduling, or stopping. Table 3 specifies the five domains and which surface forms merge. The datasets are available in the public release. The training comparisons reported here use Levels 1 to 3.

Table 3: **MODEBENCH at Level 1**. Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt with the per-prompt range beside it, exact over the released Level-1 multi-answer split. *Reached* is what one frontier deployment actually produces on the same construction — distinct verified modes per prompt at 512 draws, over the 32 problems per domain of Fig. 5, an outcome-independent subset of the same evaluation split — so the two columns separate the modes a domain admits from the modes anything has been observed to give on the same prompts. MathIR is the domain where that gap is structural rather than a sampling limit, because its modes are derivations of one answer and most of the certified ones are longer routes. Each higher level is a separate construction with its own support distribution (App. B.4). The exact prompt each policy receives is given in App. C, including the chat template.

Domain	Generated answer / check	Executed key	Aliases that merge	Valid modes	Reached
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27	2.53
				4–12	
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52	3.47
				2–8	
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44	3.75
				16–1,512	
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00	1.91
				5	
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19	6.91
				8–40	

B.1 WHAT A MODE IS, BY DOMAIN, AND WHETHER THE EFFECT DEPENDS ON IT

MODEBENCH keys a mode three ways, and the difference is worth stating plainly rather than leaving to the per-domain rules. In Countdown and MathIR two accepted outputs are different when they reach the same answer by a different route. In Python they are different when the program behaves differently on the test inputs. In Graph and PantryPlan they are different when the answer itself differs. The response budgets differ accordingly (Table 6): a PantryPlan response is a short plan and a Countdown response is a derivation, and neither is a long chain of deliberation. So a reader may fairly ask whether what collapses is reasoning or only an answer set, since three of the five domains key outcomes.

The question is decidable rather than terminological, because the two keyings are measured side by side on the same training runs. Grouping the frozen-to-terminal change in conditional collision by how the domain keys a mode, over RLVR-only arms, the route-keyed domains move +0.202 across 98 seed-blocks, the behaviour-keyed domain +0.201 over 15, and the answer-keyed domains +0.404 over 106. Concentration appears under every keying, so it is not an artifact of the outcome-keyed domains. It is larger where modes are answers, and we do not claim otherwise.

The domain that settles the narrow question is Countdown, where a mode is unambiguously a different derivation of the same number and nothing about the answer distinguishes two of them. It concentrates by +0.383 over 46 seed-blocks, more than Graph at +0.230 and close to the answer-keyed mean. Whatever the right name for the aggregate, the effect is present at full strength in the domain that most cleanly instantiates a reasoning route, which an answer-set reading would not predict. MathIR is the weak case at +0.042, for the reason its own entry gives: its certified support is keyed to derivations of one answer and most of those routes are longer ones no policy produces, so much of its support is unreachable rather than abandoned.

Where the models do deliberate. The hosted comparison holds an 8,192-token output budget fixed and switches explicit reasoning off at the provider (App. O.6). Every deployment finds fewer verified modes per prompt without it, GPT-5.6 Sol falling from 1.62 to 0.99 and GPT-5.4 from 1.85 to 1.18. The quantity this paper counts therefore responds to whether the model reasons, at a budget where a reasoning trace has room to exist. That is the sense in which these are reasoning modes: they are what the deliberation arrives at, and there are fewer of them when it is removed. Their identity remains the verifier’s equivalence class, which is a route in two domains and an outcome in three.

B.2 DATASET SPLITS AND VALIDATION

Each domain has disjoint training and evaluation splits (Table 6). PantryPlan and Level 2 also have development splits, excluded from the evaluation results (App. B.4). Every Python problem has at least two functions with different valid keys, verified by execution. MathIR validates each solution route with the same interpreter used for model outputs.

Table 4: **Dataset sizes and valid-mode counts.** Python and MathIR use disjoint training and evaluation splits. Mode-count ranges refer to the training split; Table 3 gives evaluation-split counts.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt (train)	16–3,600	5

PantryPlan. PantryPlan has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8–45 over the training split and 8–40 over the 128 evaluation prompts, which average 18.19 feasible supports. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

B.3 PYTHON EXECUTION CONTRACT

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean `and/or`, unary `plus/minus/not`, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and other identifiers are rejected. The specification format admits J distinct integers in $[4, 1000]$, with J between two and eight. Every released Level-1 row uses 4 of them, drawn from $[6, 96]$, and carries at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm; the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield $\perp$ and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For each of the J frozen inputs n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor:  $d_1, \dots, d_J$` ; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

B.4 MATCHED LEVEL CONSTRUCTIONS

Levels 2–5 increase the search space while retaining the verifier, canonicalizer, and definition of a solution mode. Calibration compares larger models with a fixed Level-1 reference; the criteria and the Level-4 MathIR exception are given below.

Level 2. Level 2 changes problem structure while preserving the verifier and canonicalizer. Its 384/128/128 training/development/evaluation rows have disjoint identities across the Level-1 and Level-2 splits. Training histograms match the native Level-1 training sets. Development and evaluation histograms match designated Level-1 development and confirmation reserves in Graph, Countdown, Python, and MathIR; Pantry uses native references, with the counts of its 64-row Level-1 development histogram scaled by two to give the 128-row target that its Level-2 development split is built to match.

These construction reserves are not the native Level-1 terminal test sets. The displayed terminal evaluations have different valid-key histograms between levels in Graph, Countdown, and Python. Cross-level changes therefore combine task structure and test population; they do not isolate difficulty at fixed test-set support. Replay comparisons remain paired within each level on the same prompts, seeds, fresh objective, and training budget.

Table 5: **What Level 2 changes.** The verifier, the canonicalizer and the notion of a mode are the same at both levels; only the structure to be searched grows.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

For frozen Qwen2.5-0.5B on the development split, each Level-2 domain has `pass@8` in $[\cdot10, \cdot90]$ and strictly below its Level-1 construction reference under identical decoding.

Scale-calibrated Level 3. Level 3 uses frozen Qwen2.5-3B against the same fixed Qwen2.5-0.5B Level-1 reference. Construction parameters are fitted on development problems and evaluated on held-out prompts. The calibration criteria are absolute `pass@1` and `pass@8` differences of at most $\cdot04$ and $\cdot08$ in every domain, using four independent groups of eight samples per prompt. `distinct@8` is not a fitting objective. The interface uses native chat, temperature 1, top- p 1, and 192 generated tokens: boxed-direct answers for Graph and the hybrid solver with legal-syntax guidance for the other domains. This matches fixed measured targets empirically; it is not a two-sided equivalence test.

The Python Level-3 interface used in the hint ablation removes the strategy hints (App. I), so its difficulty is not covered by that calibration.

Levels 4 and 5. Levels 4 and 5 use 7B and 14B models, respectively, with the same Qwen2.5-0.5B Level-1 reference and `pass@1`/`pass@8` tolerances of $\cdot04$ and $\cdot08$. Level 5 meets these criteria in all five domains. Level 4 meets them in Graph, Countdown, Python, and PantryPlan, but not MathIR.

For MathIR, the target `pass@8`/`pass@1` ratio is 5.50, while the largest ratio among the 7B constructions in this family is 4.16. A ratio of mixture averages cannot exceed the largest component ratio, so no mixture of these constructions reaches the target. The 7B MathIR models have heterogeneity gaps of $\cdot169$ – $\cdot398$, compared with $\cdot060$ for the Level-1 reference. All Level-4 domains have 384/128/128 training/development/evaluation prompts, disjoint from every other level. Level-4 MathIR is included in Figs. 3 and 15 but is not difficulty-matched to the reference.

B.5 FROZEN BASE MODELS ACROSS ALL FIVE LEVELS

Fig. 3 reports the frozen Qwen2.5-Instruct scales across all five levels and five domains; Fig. 17 adds the other three families, for 375 model–domain–level measurements in all. Every model uses the same 128 held-out prompts within a domain and level, with four independent groups of eight draws per prompt, so each plotted point rests on 4,096 responses. Each group contributes an

indicator of any verified output to pass@8 and the number of distinct verified canonical keys to distinct@8 ; we average groups within prompts and then average prompts. PCMD instead pools a prompt’s four groups before scoring it, as defined in App. A.1. For pass@8 and distinct@8 the four groups stay separate $K = 8$ observations throughout; only PCMD pools them.

These measurements use native chat, $T = 1$, $\text{top-}p = 1$, and a 192-token output limit, with boxed-direct Graph answers and a hybrid solver interface with legal-syntax guidance elsewhere. This interface differs from the one used to evaluate training runs. Under the training template and its larger evaluation sample, the untrained Level-1 Python model defines PCMD on 73 of 128 prompts, with mean .168 (Section 5.1); the grid uses the same weights with different prompts and sampling settings. Comparisons therefore use the untrained reference from the corresponding evaluation interface.

Success need not decrease monotonically with level: pass@8 ends below its Level-1 value in 33 of 35 model–domain series. Scale and level also affect diversity differently. The median range of PCMD is .251 across scales at a fixed level and .104 across levels at a fixed scale.

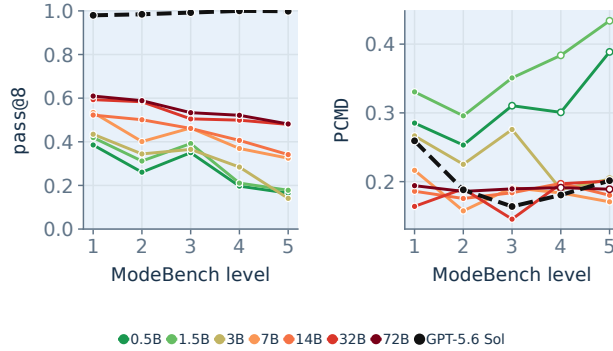


Figure 14: **Accuracy and solution diversity respond differently to harder levels.** Left: pass@8 ; right: PCMD, averaged equally over domains for each Qwen2.5-Instruct scale (coloured lines) and GPT-5.6 Sol (black dashed line). Qwen evaluations combine 128 held-out prompts with 384 training-split prompts where available; these checkpoints receive no benchmark training. Filled diversity marks have at least 30 prompts with two correct responses in every domain. Hollow marks average the defined estimates with weaker support in at least one domain.

B.6 OTHER MODEL FAMILIES AT MATCHED SCALE

Fig. 17 compares SmolLM2, Qwen2.5, Falcon3, and OLMo-2 on the same 128 held-out prompts per domain and level, with four groups of eight draws, the same verifier, and the same support thresholds. Family comparisons use the 4 shared levels; only Qwen2.5 covers Level 5. Colour indicates parameter count.

At matched parameter count, the families differ in diversity. Near 7B, Qwen2.5, Falcon3, and OLMo-2 separate on PCMD despite similar pass@8 . This ordering persists in the 10–15B range, where Falcon3 and OLMo-2 are close. Within parameter-count bands, the median range of PCMD across family means is .166, compared with .198 across scales within a family, over the 4 shared levels. The .251 range above instead spans all seven Qwen scales at each level. These comparisons characterize the checkpoints; they do not identify which pretraining or post-training choices cause the differences.

B.7 TRAINING AND EVALUATION SETTINGS

Responses are independent within each evaluation draw. For each training seed, pass@8 and raw distinct@8 average over all 128 prompts and four draws; mean@8 measures the fraction of correct responses. Evaluation supplies no training data, bank entries, checkpoint selection, or stopping rule. Complete paired five-seed blocks use nominal pointwise 95% Student- t intervals. Incomplete primary endpoint blocks give exact seed counts and descriptive means without intervals. Support-filtered diversity comparisons specify their eligible seed sets and interval rules separately. Intervals condition on the fixed prompts and evaluation requests. Terminal comparisons use only terminal checkpoints; missing endpoints are not replaced by earlier checkpoints. Each comparison specifies its seed intersection and averaging rule; the replay-weighting comparison uses a paired-seed bootstrap.



Figure 15: **High accuracy can coexist with low solution diversity.** Each row shows one frozen checkpoint across five domains, with $\text{pass}@8$ on the horizontal axis and PCMD on the vertical axis. Colour indicates parameter count and marker shape indicates level. Each cell uses four groups of eight responses to 128 held-out prompts at $T = 1$; PCMD pools the groups within each prompt. Filled marks have at least 30 prompts with two correct responses; hollow marks have 5-29, with one-standard-error bars. Ticks below the axis indicate fewer than five such prompts.

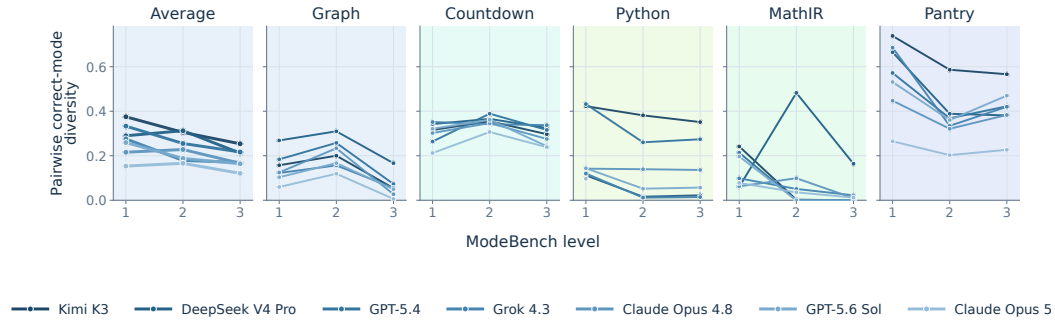


Figure 16: **Every hosted deployment has lower mean solution diversity at Level 3 than at Level 1.** Lines show PCMD for 7 deployments, using eight responses to each of 128 held-out prompts per cell. Panels show individual domains and their equally weighted average over a domain set fixed across levels for each deployment. Darker lines indicate higher overall diversity; hollow marks fall below the 30-prompt support threshold and are excluded from connecting lines. Higher levels increase task difficulty and add strategy guidance in MathIR, Python, and PantryPlan (App. I).

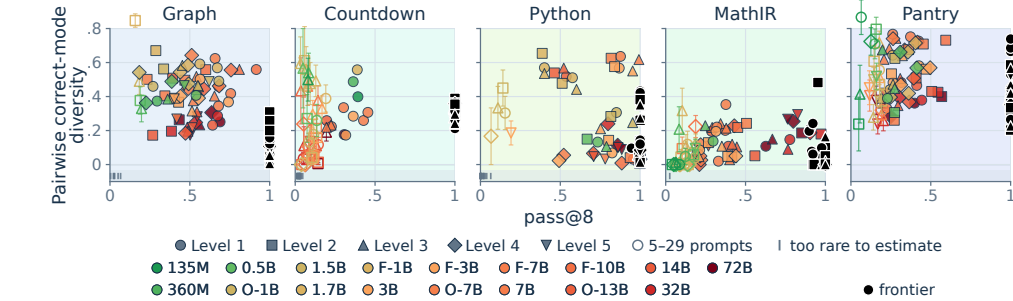


Figure 17: **Parameter count does not determine solution diversity across model families.** Frozen SmoLLM2, Qwen2.5, Falcon3, and OLMo-2 checkpoints are plotted by `pass@8` and PCMD across five domains and the available levels. Colour indicates parameter count on a logarithmic scale; marker shape indicates level. Black marks show the 7 hosted deployments. Frozen checkpoints use four groups of eight responses to 128 held-out prompts per cell; hosted deployments use eight responses per prompt. Filled marks, hollow marks with standard-error bars, and ticks follow the support thresholds in Fig. 15.

C EXACT DOMAIN PROMPTS

Each domain uses one system message and one user message, with no few-shot examples. Training and evaluation use the same template within each method. The examples below show Qwen’s chat format. Falcon3 uses the same system and user text with its own `<|system|>`/`<|user|>` markers. Each checkpoint in the frozen-model comparison uses its tokenizer’s native chat template (App. B.5). The examples are the first prompts of the Level-1 evaluation splits; Levels 2–5 use separate problems with the same interface (App. B.4). A two-space continuation indent indicates display wrapping: joining it to the preceding line with a single space recovers the prompt text. In the PantryPlan example, a break without that indent is a newline in the prompt.

Table 6: **Training settings for paired comparisons.** Settings are shared within each model–domain–seed comparison, including the zero-derivative replay control. Both arms use the same optimizer-update and fresh-rollout budgets and retain the replay scoring/backward path. Realized bank workloads, total FLOPs, and wall-clock time are not matched. The control does not reallocate replay work to additional useful baseline training (App. Q.1).

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$ except in the reference-KL cohort of App. P.12, which moves that coefficient alone; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Optimizer	Fused Adam in <code>bf16</code> with decoupled weight decay, $\beta_1 = .9$, $\beta_2 = .95$, weight decay 0, gradient-norm clip 1.0, gradient checkpointing on. Offload swaps the fused kernel for its CPU twin and changes no hyperparameter.
Policy update	One PPO epoch per batch with parameters synchronized every update, so each rollout is used once and on-policy. The surrogate keeps the standard symmetric clip at $\varepsilon = .2$; at one epoch the importance ratio is 1 when the clip is evaluated, so it binds on no update.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .

Table 7: **Base-model revisions.** Both arms of each paired training comparison use the same initialization from the model revisions below.

Base model	Revision
Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfd
Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

C.1 GRAPH COLORING

384 training and 128 evaluation prompts.

```
<|im_start|>system
Return only the final answer inside \boxed{. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3),
(3,5), (3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no
edge has the same color on both endpoints. A partial coloring is 2??1?2, where ?
means uncolored. Keep every shown digit fixed. There are exactly 3 question marks.
Return exactly one final answer and no explanation: exactly 3 digits, each one 1,
2, or 3, for the missing positions from left to right, inside \boxed{.<|im_end|>
<|im_start|>assistant
```

C.2 COUNTDOWN

384 training and 128 evaluation prompts.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use
each given number exactly once. You may use +, -, *, /, and parentheses. Return
exactly one final answer and no explanation: the expression inside
\boxed{ }.<|im_end|>
<|im_start|>assistant
```

C.3 PYTHON FACTORS

384 training and 128 evaluation prompts.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external
Python tool will call it once for each n in [18, 82, 91, 93]. For every call,
return an integer d satisfying 1 < d < n and n % d == 0. Different valid return
vectors are different solution modes. You may use integer literals, n, +, -, *,
/, %, comparisons, Boolean operators, and conditional expressions; calls,
imports, attributes, containers, and other names are forbidden. Return exactly the
one-line lambda inside \boxed{ } and no explanation.<|im_end|>
<|im_start|>assistant
```

C.4 MATHIR ACTION MENU

384 training and 128 evaluation prompts.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed
by exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c.
Action menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E
means sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations,
repeated states, and paths that do not finish with x isolated are rejected. Return
only the IDs separated by semicolons inside the answer box; for example formatting
only: \boxed{A;C}. Do not return x's numeric value.<|im_end|>
<|im_start|>assistant
```

C.5 PANTRYPLAN

384 training and 128 evaluation prompts.

```
<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the
corresponding Pantry row in printed order. A trusted environment will choose exact
quantities on precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:
- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal,
  protein=21.5g, fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal,
  protein=0.74g, fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal,
  protein=0.941g, fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant
```

D REPLAY ALGORITHM

Verified replay combines a fresh task objective with uniform rehearsal over stored solution modes. For each prompt x , $\mathcal{B}_x$ contains at most M verified policy exemplars, one per canonical key. A round-robin cursor σ selects one prompt bank per update. Table 2 defines the notation; App. A.3 relates the replay objective to the diversity metric.

D.1 FRESH TASK ADVANTAGES

For a group of G fresh responses with verified binary rewards R_i and $R = \sum_i R_i$, Dr.GRPO uses $A_i = R_i - R/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} GR_i/R - 1, & R > 0, \\ 0, & R = 0. \end{cases}$$

The centered MaxRL estimator (Tajwar et al., 2026) gives failures advantage -1 in mixed groups and weights each success more strongly when successes are scarce. All-correct and all-failure groups have zero task advantage. Lemma P.3 gives its harmonic pass@ k mean objective. These advantages apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

D.2 OPTIMIZER UPDATE

Algorithm 1 Verified replay adds uniform rehearsal to fresh task learning. One optimizer update with $o = \text{Dr.GRPO}$ gives Re:Dr; $o = \text{MaxRL}$ gives Re:Max.

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \sigma)$

Ensure: Updated $(\theta', \mathcal{B}', \sigma')$

- 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward R_i
- 4: Set $c_i \leftarrow \tilde{c}_i$ iff $R_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 5: **end for**
- 6: Compute fresh task advantages A_i^o from $\{R_i\}_{i=1}^G$; for $o = \text{MaxRL}$ use the formula in App. D.1
- 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
- 8: $(z, \sigma') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \sigma)$ by a fixed round-robin cursor over prompts; the replayed prompt z need not be the prompt x that supplied the fresh group
- 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
- 10: For each exemplar $e_c \in \mathcal{B}_z$, compute $\ell_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} \ell_c$
- 12: **else**
- 13: $L_{\text{replay}} \leftarrow 0$
- 14: **end if**
- 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$

D.3 ADMISSION, CAPACITY, AND DETERMINISTIC SCHEDULING

Each bank corresponds to one exact prompt-token sequence. A response is eligible when it has positive task reward and a valid canonical key; responses truncated at the generation cap are subject to the same rule. All responses in a group are validated before any new key is added. For each new key, the lexicographically smallest normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is therefore invariant to rollout-row order. A bank stores one exemplar per key, up to $M = 16$, with no eviction or replacement. Once capacity is reached, later keys receive no replay support.

At each update, the scheduler sorts nonempty banks by prompt hash and selects one by a round-robin cursor; singleton banks are eligible. Selection is independent of reward, evaluation, desired support, and entropy. The selected bank’s original prompt and response tokens form a right-padded teacher-forcing batch, with prompt and padding positions masked. Each key contributes its exemplar’s mean response-token log probability. These scores are averaged uniformly across the selected bank and multiplied by $\lambda_{\text{replay}}(G - 1)/G^2$, as in Algorithm 1. The factor $(G - 1)/G^2$ equals the weight of a lone success in a centered Dr.GRPO group: its advantage is $(G - 1)/G$, and averaging over the G responses supplies the additional $1/G$ (Lemma P.2). Neither exhaustive support nor evaluation outputs enter the replay loss.

D.4 ZERO-DERIVATIVE REPLAY CONTROL

Replay and its control share validation, bank admission, scheduling, and replay scoring. The control sets the replay score-gradient vector to exact zeros while retaining the replay forward and backward passes. Fresh-task and replay gradients accumulate before the same optimizer step; the replay derivative is the only difference in the learning objective. Subsequent policies can produce different samples, bank contents, selected banks, and exemplar lengths. Optimizer-update and fresh-rollout budgets are matched, but realized FLOPs and wall-clock time are not. The control also does not

allocate replay computation to additional task-gradient updates, fresh rollouts, or longer training. It isolates the replay learning signal without establishing superiority at equal total compute (App. Q.1).

D.5 VERIFIER REQUIREMENTS AND APPLICABILITY

Replay requires a deterministic equivalence relation on verified outputs. The canonical key must be computable during training, inexpensive enough to evaluate for every response, and consistent with the task’s notion of a solution. In MODEBENCH, the executable verifier supplies both correctness and canonical identity, so admission requires no learned judge or embedding threshold.

Final-answer grading on tasks such as MATH, GSM8K, and AIME does not identify distinct derivations. For a prompt with one correct answer, an answer-based key maps every successful response to the same mode. Its bank then holds one exemplar, and replay reinforces that exemplar without balancing alternative derivations. Distinguishing derivations requires a richer identity. MathIR uses executed state trajectories and includes certified four-state routes. None appears among 27,450 verified draws. Among the 115 solved Level-1 prompts, these draws cover 67% of the certified two-state modes and 52% of the three-state modes. These modes rarely coexist within a prompt. MathIR’s terminal PCMD averages .006, equivalent to 1.01 effective verified modes per prompt. These results limit the evidence for replay on derivation-based identities.

MathIR is included in every five-domain mean. In all 6 cells with complete five-domain results, its smaller gain reduces the aggregate by .010–.074 PCMD relative to the corresponding four-domain mean. At Qwen2.5-0.5B, the Re:Dr gain is +.294 over all five domains and +.364 over the four domains with alternative-answer modes. App. N.1 also reports the four-domain comparison.

For open-ended derivations without an executable identity, a learned judge could supply a novelty criterion. Such a criterion requires separate validation: judge decisions may be inconsistent, may disagree with task correctness, and may reward distinctions that replay can exploit without improving solution diversity. A fixed model, prompt, and decoding rule alone do not establish a valid equivalence relation. Agreement with executable keys can be evaluated where both are available, while diversity measured by those keys remains independent of the judge’s training signal. Judge-based replay is not evaluated here.

D.6 EXEMPLAR QUALITY AND LENGTH

Canonical verification establishes output validity, not the quality of every intermediate reasoning step. A verified exemplar may be verbose or contain an unsound explanation. Uniform replay assigns weight by solution key and does not rank exemplars by reasoning quality.

Each exemplar contributes its mean response-token log probability, $\ell_c = |e_c|^{-1} \log \pi_\theta(e_c \mid z)$, and each key occupies one bank entry. Thus a longer response receives neither additional entries nor loss weight proportional to its token count. In the categorical surrogate with one fixed-length exemplar per key, the corresponding key weights are proportional to L_b^{-1} ; the full-coverage limit favors shorter exemplars and is uniform only at equal lengths (App. P.7). This surrogate does not imply uniform key probabilities in a language model. Mean-token normalization also does not guarantee concise or high-quality explanations.

A quality criterion can restrict bank admission in addition to correctness and canonical validity. Examples include a token budget, a process-reward threshold, or a trace-quality classifier; a quality score could also guide exemplar selection within a key or eviction at capacity. These changes alter the replay target and are not part of the uniform-replay comparison. Quality and diversity can conflict when alternative modes require longer solutions. MathIR alternatives include trajectories through additional states, and Python’s rarer return vectors require longer lambdas than its modal vector. A brevity filter can therefore remove valid alternatives; diversity is measured after applying the filter.

E FIXED SEMANTIC MAXENT COMPARATOR

Fixed Semantic MaxEnt augments the fresh-rollout advantage using the estimated frequency of each verified solution mode. It retains counts of successful canonical keys but supplies no likelihood gradient on stored exemplars. For row i in a group of G responses to prompt x , let $n_x(a)$ count

previous successful responses with key a , and let $m_{-i}(a)$ count successful peers with that key. Define $N_x = \sum_a n_x(a)$, $S_{-i} = \sum_a m_{-i}(a) \leq G - 1$, and let $\mathcal{K}_i$ be the union of historical and peer-observed keys. Unit pseudocounts and one reserved unseen bucket give, for $a \in \mathcal{K}_i$,

$$\hat{p}_i(a) = \frac{n_x(a) + m_{-i}(a) + 1}{N_x + S_{-i} + |\mathcal{K}_i| + 1}, \quad \hat{p}_i(\text{UNSEEN}) = \frac{1}{N_x + S_{-i} + |\mathcal{K}_i| + 1}. \quad (9)$$

A sampled key outside $\mathcal{K}_i$ receives the unseen-bucket probability. Unsuccessful and invalid peers contribute no count.

Prompt-specific success counts. Each exact prompt-token sequence has an independent count table. Counts include only responses with positive task reward and a valid canonical key; invalid and unsuccessful responses affect neither the predictor nor its history. Counts accumulate throughout training, with no window, decay, eviction, or reset. Each eligible response is scored against the preceding groups' counts and its own successful leave-one-out peers. The current group's counts are added after all its responses have been scored, so a response never contributes to its own predictor. The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}_i(a_i), \kappa\} - \mathbb{E}_{\hat{p}_i}[\min\{-\log \hat{p}_i(a), \kappa\}]}{\kappa}, \quad \eta = .10, \kappa = 5. \quad (10)$$

The score does not depend on the token index, so row i carries the same $A_{i,t}^{\text{sem}}$ at every t . Invalid and unsuccessful rows receive zero, and the centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome when that outcome appears in a fresh rollout, but supplies no gradient for an absent stored exemplar. If neither history nor successful peers provide a key, the unseen bucket has probability one and the bonus is zero. App. F.4 reports the empirical comparison.

For the true conditional distribution $q_\theta = p_\theta(\cdot \mid \mathcal{C})$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (11)$$

The empirical predictor does not yield an unbiased estimator of this entropy gradient. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{Z \sim p} [\mathbf{1}\{Z \in \mathcal{C}\} (s_Z - b) \nabla_\theta \log p_Z] = P \mathbb{E}_{c \sim q} [s_c \nabla_\theta \log q_c] + (\mathbb{E}_q[s] - b) \nabla_\theta P.$$

The predictor baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla_\theta H(q)$. For a single correct key, conditional entropy is zero, yet the unseen bucket can give that key negative semantic advantage once it has been observed. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

Part III. Training results

F EMPIRICAL RESULTS AND COMPARISONS

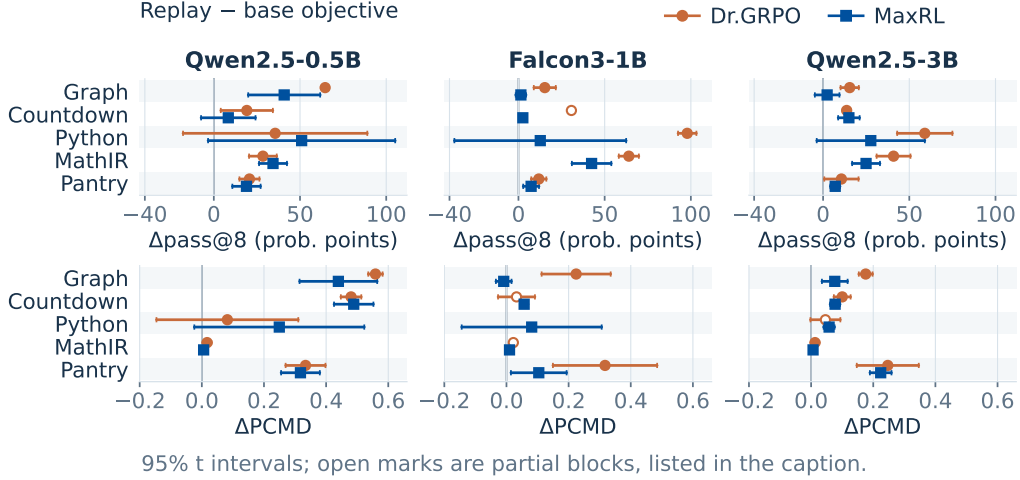


Figure 18: **Replay improves correctness across all three model scales.** Terminal paired effects of Re:Dr minus Dr.GRPO (circles) and Re:Max minus MaxRL (squares), in percentage points of pass@8 and changes in PCMD. Open marks have fewer than five seeds. Correctness uses four seeds only for Falcon Countdown Dr.GRPO. For Re:Dr diversity, $n = 4$ for Falcon Countdown, $n = 1$ for Falcon MathIR, and $n = 2$ for Qwen2.5-3B Python; Falcon Python has no eligible pair. Other comparisons use five seeds. Diversity requires at least 20 defining prompts per arm. Bars are nominal paired 95% Student- t intervals for five-seed correctness comparisons and diversity comparisons with at least two seeds.

F.1 TERMINAL COMPARISONS ACROSS SCALES

Within each model–domain–seed comparison, methods share initialization, training and evaluation prompts, the eight-pass horizon, evaluation requests, verifier, and replay schedule. Controls use the same bank and scoring path with a zero replay derivative. Policies can develop different banks and workloads, so the comparison does not match total training compute (App. Q.1).

Dr.GRPO retention contrasts. Tables 8 and 9 show absolute arm means and paired replay effects, respectively. Dr.GRPO replay increases mean pass@8 and distinct@8 in all 14 complete five-seed blocks; distinct@8 increases in all 74 terminal seed pairs. Because distinct@8 depends on correctness as well as diversity, PCMD provides a separate conditional measure.

At Qwen2.5-0.5B, Countdown pass@8 rises from .480 to .672, while PCMD rises by +.480 (95% interval $[+.447, +.513]$). Python correctness rises from .172 to .528, with a diversity effect of +.082 ($[-.146, +.310]$); MathIR rises from .512 to .796, with a diversity effect of +.017 ($[+.003, +.031]$). These are nominal, unadjusted Student- t intervals over five paired training seeds. Countdown’s terminal PCMD values are .007 for Dr.GRPO and .487 for Re:Dr. Its keys distinguish computation routes, although these held-out endpoints do not identify which previously discovered routes survived training.

Fig. 19 gives the absolute results by domain. All MaxRL/Re:Max correctness comparisons use five paired seeds, including seeds 70–74 in every Qwen2.5-3B domain. Dr.GRPO/Re:Dr also uses five except Falcon Countdown, which uses four. Diversity has separate support requirements and seed populations. A factorial interaction requires the common four-arm seed intersection.

The diversity effects in Fig. 18 and Table 9 pair training seeds for which each arm defines PCMD on at least 20 evaluation prompts. Absolute diversity means in Table 8 and Fig. 19 use a 30-prompt threshold separately for each arm. Thus correctness, absolute diversity, and paired diversity effects can have different seed populations. Eligible prompt subsets can also differ between arms. Diversity-effect intervals use Student- t with $n - 1$ degrees of freedom when $n \geq 2$; single-seed

Table 8: **Terminal correctness and solution diversity with and without replay.** Dr.GRPO and Re:Dr after eight training passes. Correctness uses five paired seeds except Falcon Countdown ($n = 4$). Each diversity column averages its own eligible seeds, requiring at least 30 prompts with two correct responses per seed; superscripts give counts below five. Dashes indicate no eligible estimate. These arm-specific means can use different seeds and need not subtract to the paired effects in Table 9, which uses a 20-prompt threshold per arm. Shading increases from red to green on a shared scale for both arms of each metric.

Model	Domain	Dr.GRPO		Re:Dr	
		pass@8	PCMD	pass@8	PCMD
Qwen2.5-0.5B	Graph coloring	.323	.001	.969	.560
	Countdown	.480	.007	.672	.487
	Python factors	.172	—	.528	.000 ²
	MathIR	.512	.001	.796	.018
	PantryPlan	.522	.000	.729	.334
Falcon3-1B	Graph coloring	.656	.204	.809	.428
	Countdown ⁴	.325	.102	.632	.134
	Python factors	.000	—	.979	.060
	MathIR	.095	—	.736	.022
	PantryPlan	.633	.000	.751	.317
Qwen2.5-3B	Graph coloring	.682	.224	.836	.399
	Countdown	.609	.012	.746	.112
	Python factors	.069	—	.659	.098
	MathIR	.278	.001	.686	.014
	PantryPlan	.638	.105	.744	.351

Table 9: **Replay improves average correctness and solution diversity at every tested scale.** Terminal Re:Dr minus Dr.GRPO, by domain and model scale. Superscripts give paired seed counts below five; dashes indicate no eligible comparison. Diversity effects require at least 20 defining prompts per arm. Each Avg weights domains equally within seed on a fixed domain set: all five for correctness, and those supported by every seed for diversity. The latter excludes Python at both larger scales and additionally Countdown and MathIR at Falcon3-1B. Eligible prompt subsets can differ between arms.

	Δ pass@8					Δ PCMD				
	Graph	Countdown	Python	MathIR	PantryPlan	Avg	Graph	Countdown	Python	MathIR
Qwen2.5-0.5B	+.646	+.191	+.356	+.285	+.207	+.337	+.480	+.082	+.017	+.334
Falcon3-1B	+.153	+.307 ⁴	+.979	+.640	+.118	+.442 ⁴	+.224	+.032 ⁴	—	+.022 ¹
Qwen2.5-3B	+.154	+.137	+.590	+.409	+.106	+.279	+.176	+.100	+.046 ²	+.013

effects have no interval. Correctness and raw-count intervals require five paired seeds. All intervals are nominal 95% estimates without multiple-comparison adjustment.

At Qwen2.5-0.5B, Re:Max raises PCMD by +.439, +.489, and +.317 on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph has a negative mean diversity effect of $-.009$ ($[-.034, .016]$) and an inconclusive correctness effect of $+.014$ ($[-.016, .044]$). Positive cross-domain averages therefore do not imply a gain in every domain. At Qwen2.5-3B, the five-seed diversity effects are +.076, +.077, +.058, +.006, and +.224 for Graph, Countdown, Python, MathIR, and PantryPlan. Every interval excludes zero, although the MathIR gain is small. Correctness intervals exclude zero for Countdown, MathIR, and PantryPlan; Graph and Python are inconclusive. Re:Max has higher mean PCMD than MaxRL in 14 of 15 measurable comparisons, and Re:Dr exceeds Dr.GRPO in 14 of 14. These counts describe the direction of the means, not statistical significance.

Cross-domain factorial summaries. Averaging five domains equally within each paired seed, Re:Max minus MaxRL improves pass@8 by +.307 at Qwen2.5-0.5B (95% interval $[+.208, .406]$), +.133 at Falcon3-1B ($[+.043, .223]$), and +.154 at Qwen2.5-3B ($[+.086, .222]$), each using five seeds. Re:Dr minus Dr.GRPO improves correctness by +.337, +.442, and +.279, respectively. The Falcon Dr.GRPO average uses four common seeds and has no interval; the other two use five.

At Qwen2.5-3B, MaxRL/Re:Max arm means are .547/.701 for correctness and .744/1.107 for distinct@8. The paired raw-count effect is +.363 ($[+.285, .441]$). The four-arm factorial has 14 complete five-seed blocks and four seeds for Falcon Countdown. Cross-domain PCMD averages in Table 9 use a fixed set of domains supported by every seed; per-domain effects appear in Fig. 18.

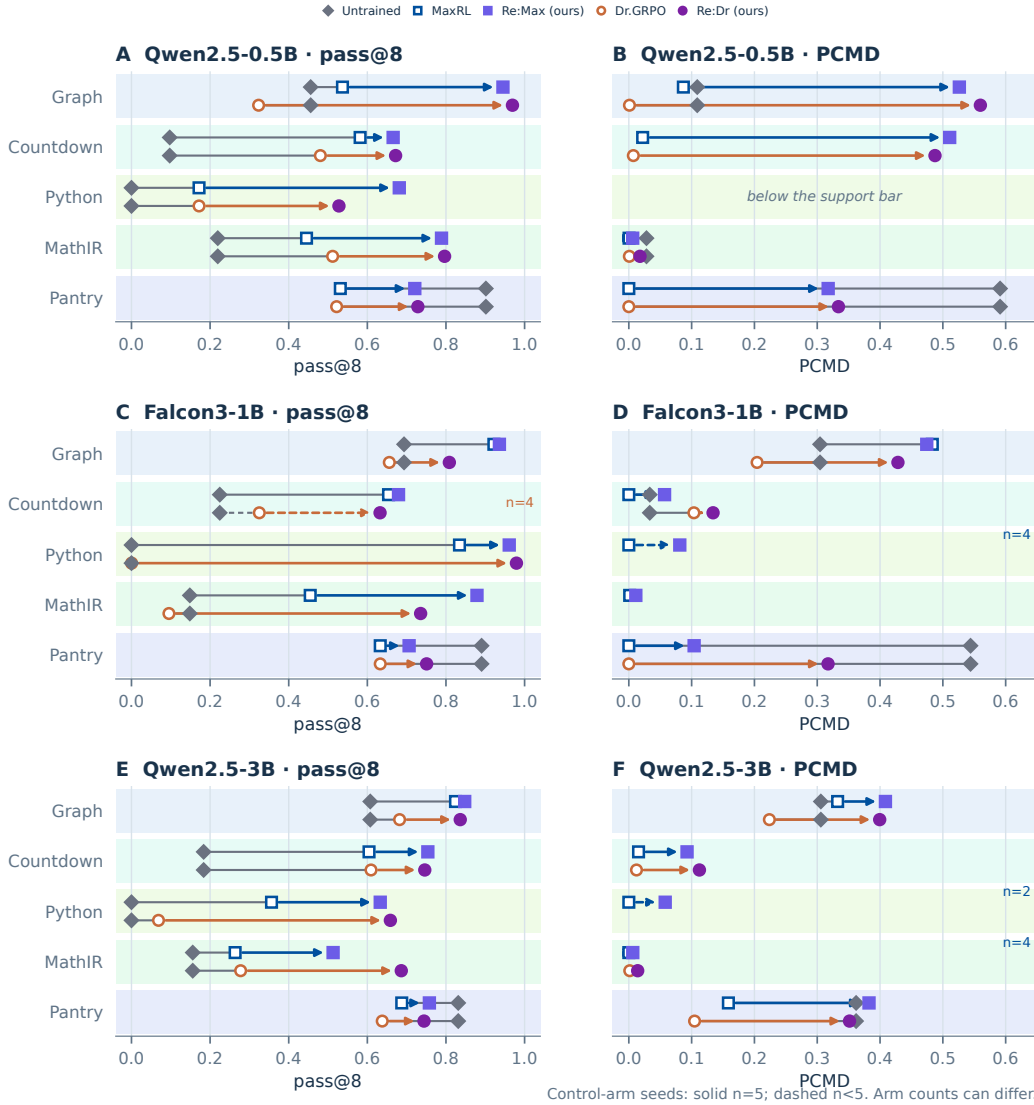
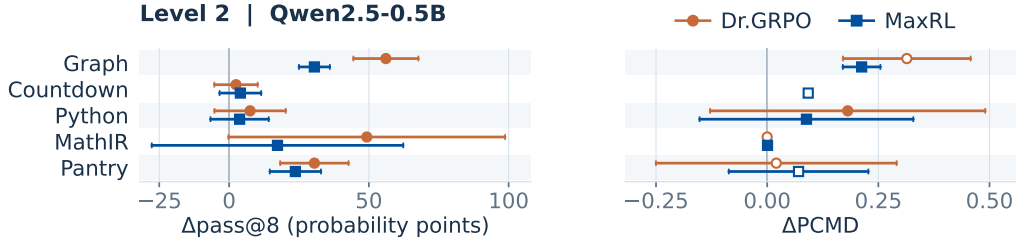


Figure 19: **Replay gains vary by domain and model scale.** Terminal pass@8 and PCMD for the three model scales. Open/filled marks denote the fresh objective/its replay variant; circles denote Dr.GRPO, squares MaxRL, and gray diamonds the untrained checkpoint. Correctness uses five paired seeds except Falcon Countdown Dr.GRPO ($n = 4$). Diversity averages each arm’s eligible seeds separately, requiring at least 30 defining prompts per seed. Line style follows the control’s seed count: solid for five, dashed for fewer. Replay counts can differ; Falcon Countdown uses five Dr.GRPO and four Re:Dr diversity seeds. Missing tracks or diamonds indicate insufficient support. No uncertainty intervals are shown. Table 9 gives paired diversity effects at a 20-prompt threshold.

F.2 TERMINAL COMPARISON ON THE HARDER LEVEL

Fig. 11 compares replay gains at Levels 1–3 using fixed domain sets. Correctness averages Graph, Countdown, Python, and PantryPlan equally within each of five common training seeds. Diversity averages Graph, MathIR, and PantryPlan equally, after averaging each domain’s eligible seeds separately for each arm. These diversity means can use different seeds across arms and levels. Their differences are descriptive and have no intervals. At Level 2, only PantryPlan has sufficient untrained diversity support, so an initial three-domain diversity mean is undefined.

Within Level 2, all five domains have five terminal seeds for each of the four methods (100 endpoints). The paired diversity contrasts in Fig. 20 use their own eligible seed sets. Graph improves under both objectives: Re:Max adds $+.212$ ($[+.170, +.255]$, five seeds) and Re:Dr adds $+.314$ ($[+.170, +.458]$, three seeds) to PCMD. Python’s mean diversity effects are positive ($+.088$ and



Replay – base; 95% t intervals. Open marks are partial blocks; the panels do not share a cohort.

Figure 20: **Replay improves Level-2 Graph accuracy and solution diversity.** Paired terminal effects of Re:Dr minus Dr.GRPO (circles) and Re:Max minus MaxRL (squares) for Qwen2.5-0.5B, in percentage points of `pass@8` and changes in `PCMD`. Correctness uses five paired seeds per domain. Diversity requires at least 20 defining prompts per arm. Re:Dr uses three seeds for Graph and PantryPlan, four for MathIR, and none for Countdown; Re:Max uses one for Countdown and four for PantryPlan. Other comparisons use five. Open marks indicate fewer than five pairs; blanks indicate no eligible pair. Bars are nominal paired 95% Student-*t* intervals where at least two seeds contribute.

+181), with intervals spanning zero; MathIR’s are near zero. PantryPlan correctness improves by +.305 under Re:Dr and +.237 under Re:Max, both using five seeds.

Diversity has less support than correctness. Re:Dr uses three paired seeds for Graph and PantryPlan, four for MathIR, none for Countdown, and five for Python. Re:Max uses one for Countdown, four for PantryPlan, and five for the other domains. An absent comparison indicates insufficient support, not a zero effect. Each Level-2 MathIR prompt admits one shortest derivation and four longer alternatives; concentration on the shortest route gives near-zero diversity without exhausting the domain’s valid modes. Levels use distinct held-out problems and can differ in support distributions and prompt guidance (App. B.4), so cross-level differences do not isolate the causal effect of difficulty.

For each common seed and endpoint Y , the replay interaction is

$$I = (Y^{\text{Re:Max}} - Y^{\text{MaxRL}}) - (Y^{\text{Re:Dr}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises `pass@8` relative to Dr.GRPO by +.233 ([+.140, +.325]), while the replay interaction on `pass@8` is −.256 [−.329, −.183]. This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 10: **A stronger fresh objective need not increase replay’s additional benefit.** Five common seeds per domain for all four methods. The interaction is $(\text{Re} : \text{Max} - \text{MaxRL}) - (\text{Re} : \text{Dr} - \text{Dr.GRPO})$. Here $P_8 = \text{pass@8}$, $D_8 = \text{distinct@8}$, and $B_8 = D_8 - P_8$. Entries give paired means and nominal 95% Student-*t* intervals. Both count metrics remain coupled to correctness. Four-arm `PCMD` comparisons have five eligible seeds in Python, four in MathIR, three in Graph, two in PantryPlan, and none in Countdown, so they do not share this table’s five-seed population.

Domain	n	Contrast	ΔP_8	ΔD_8	ΔB_8
Graph	5	MaxRL – Dr.GRPO	+.233 [+.140, +.325]	+.322 [+.162, +.482]	+.089 [+.019, +.159]
Graph	5	Re:Max – Re:Dr	−.023 [−.062, +.016]	−.029 [−.198, +.140]	−.006 [−.142, +.130]
Graph	5	Replay × MaxRL interaction	−.256 [−.329, −.183]	−.351 [−.446, −.255]	−.095 [−.186, −.003]
Countdown	5	MaxRL – Dr.GRPO	−.014 [−.073, +.045]	−.014 [−.073, +.045]	+.000 [+.000, +.000]
Countdown	5	Re:Max – Re:Dr	+.002 [−.008, +.012]	−.002 [−.047, +.044]	−.004 [−.042, +.035]
Countdown	5	Replay × MaxRL interaction	+.016 [−.047, +.078]	+.012 [−.068, +.092]	−.004 [−.042, +.035]
Python	5	MaxRL – Dr.GRPO	+.000 [+.000, +.000]	+.000 [+.000, +.000]	+.000 [+.000, +.000]
Python	5	Re:Max – Re:Dr	−.037 [−.232, +.157]	−.188 [−.988, +.613]	−.150 [−1.014, +.713]
Python	5	Replay × MaxRL interaction	−.037 [−.232, +.157]	−.188 [−.988, +.613]	−.150 [−1.014, +.713]
MathIR	5	MaxRL – Dr.GRPO	+.283 [−.193, +.759]	+.283 [−.193, +.759]	+.000 [+.000, +.000]
MathIR	5	Re:Max – Re:Dr	−.036 [−.158, +.085]	−.035 [−.158, +.088]	+.002 [−.003, +.006]
MathIR	5	Replay × MaxRL interaction	−.320 [−.864, +.225]	−.318 [−.860, +.224]	+.002 [−.003, +.006]
PantryPlan	5	MaxRL – Dr.GRPO	+.019 [−.067, +.105]	+.019 [−.081, +.119]	+.000 [−.021, +.021]
PantryPlan	5	Re:Max – Re:Dr	−.049 [−.113, +.014]	−.061 [−.249, +.128]	−.012 [−.145, +.122]
PantryPlan	5	Replay × MaxRL interaction	−.068 [−.168, +.032]	−.080 [−.279, +.119]	−.012 [−.134, +.110]

Level-2 Graph endpoint detail. For frozen Qwen2.5-0.5B on the development split, Graph `pass@8` falls from .438 at Level 1 to .148 at Level 2. After eight training passes at Level 2,

Dr.GRPO/MaxRL reach .202/.434, compared with .763/.739 for Re:Dr/Re:Max. The paired correctness gains are +.561 and +.305 (95% Student- t intervals $[+.445, +.677]$ and $[+.250, +.360]$, five seeds each). The lowest replay accuracy (.711) exceeds the highest non-replay accuracy (.477). Replay adds 1.019/.668 raw distinct modes, yielding `distinct@8` of 1.243/1.214.

For the three seeds eligible for the paired Dr.GRPO diversity comparison, PCMD rises from .039 to .353; for the five MaxRL pairs, it rises from .112 to .325. These conditional-diversity gains accompany the correctness gains, but do not establish that broader support causes better correctness.

F.3 ACCURACY AND VERIFIED MODES THROUGHOUT TRAINING

Figs. 21–24 show accuracy and diversity during training. Accuracy and raw distinct-mode counts use terminal paired-seed cohorts, averaging the available members at each checkpoint; ringed points identify partial cohorts. Evaluations use four groups of eight responses to each of 128 held-out prompts. The PCMD curves use method-specific cohorts and evaluation grids, with at least five defining prompts per contributing seed. Their seed populations can therefore differ from the accuracy curves and change over training. Bands show seed ranges, not confidence intervals. Lines connect observed means without smoothing and break where neither a value nor an eligible seed is available.

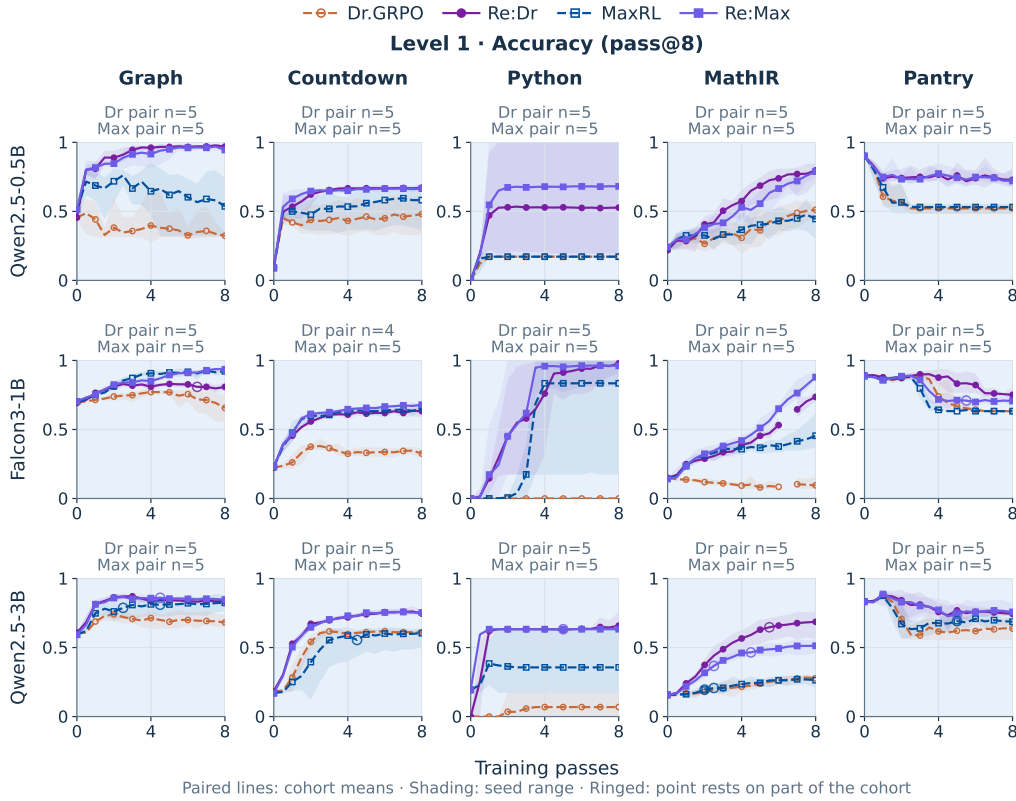


Figure 21: **Replay improves terminal accuracy across scales.** `pass@8` over eight training passes for the four primary methods. Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show domains. Each objective pair uses five terminal training seeds except Falcon Countdown Dr.GRPO/Re:Dr ($n = 4$). Lines average available seeds within these fixed cohorts; bands span their minimum and maximum. Ringed points use fewer than the full cohort. Both paired curves omit a checkpoint when either arm has no observation; the only such gap is Falcon MathIR Dr.GRPO/Re:Dr at pass 6.5.

F.4 SUPPORTING COMPARATORS

GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse RLEP-Dr reuses verified successes without uniform weighting by canonical key. GRPO has 75

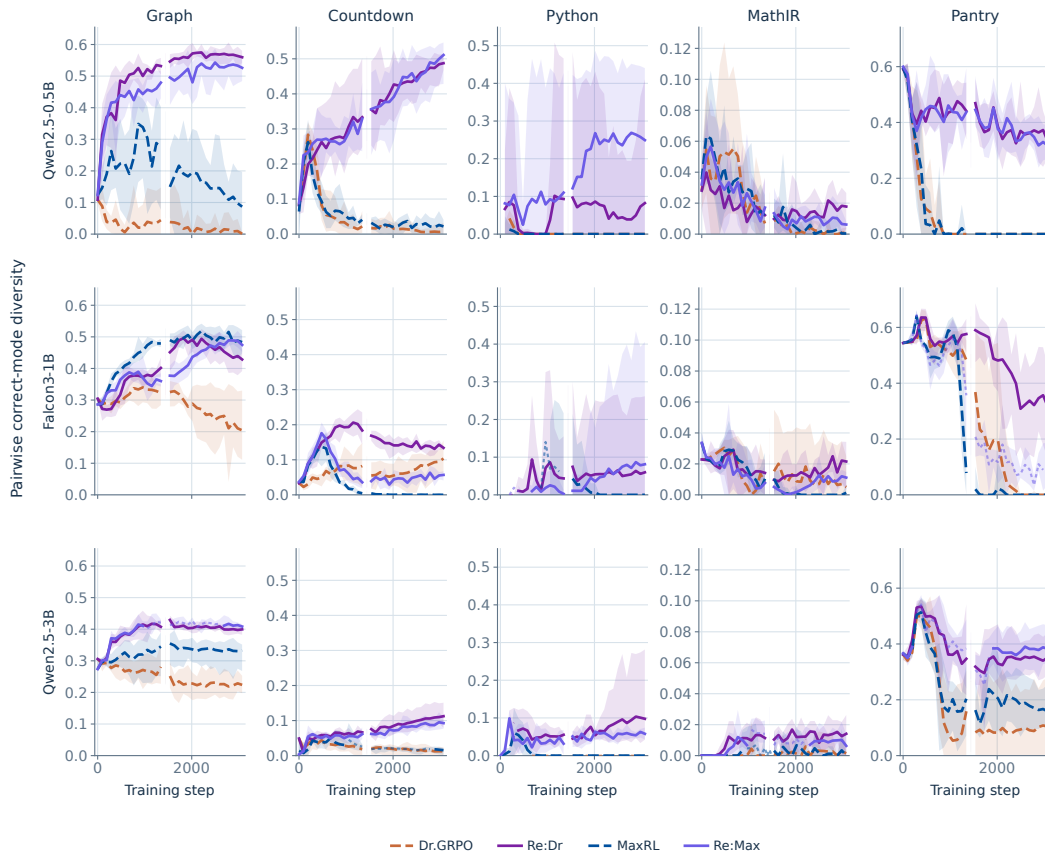


Figure 22: **Replay limits the loss of PantryPlan solution diversity at every scale.** PCMD throughout training, with model rows and method colours as in Fig. 21. Both fresh objectives reach zero terminal PantryPlan diversity at 0.5B and 1B, whereas both replay variants retain multiple solution modes. Lines average one to five eligible seeds per checkpoint; bands show seed ranges and are absent for a single seed. Each method has its own cohort and evaluation grid. A seed contributes when at least five prompts yield two correct responses. Fine dotted segments use incomplete cohorts, whose changing membership can affect the mean. Gaps mark checkpoints with no eligible seed. Vertical scales are shared within each domain.

terminal runs across the three model scales. UCPO and RLEP-Dr use the two smaller scales, with 50 and 47 terminal runs, respectively; RLEP has two seeds for Falcon Python and five elsewhere. Fixed Semantic MaxEnt provides a fresh-rollout frequency bonus (App. E). The four-arm semantic factorial covers ten model–domain cells at the two smaller scales, with five common seeds except Falcon Countdown ($n = 4$). The 3B comparison uses one seed and tests adding the bonus to replay (App. G).

The GRPO, UCPO, and RLEP diversity contrasts in this subsection average per-prompt differences over prompts where both arms have at least two correct responses, requiring at least 30 such common prompts per seed. This differs from Table 1, which uses separate 20-prompt thresholds per arm. On Qwen2.5-0.5B Graph, one UCPO seed and four RLEP seeds meet the common-prompt criterion; no Python comparison at any scale meets it.

At Qwen2.5-0.5B, UCPO raises Countdown $\text{pass}@8$ by $+0.133$ ($[+0.005, +0.262]$), with an inconclusive PCMD effect of $+0.002$ ($[-0.018, +0.023]$). Its MathIR effects are also inconclusive. RLEP-Dr raises MathIR correctness by $+0.126$ ($[+0.030, +0.222]$), with zero measured diversity change; its Countdown and Falcon PantryPlan effects are inconclusive. Across GRPO, UCPO, and RLEP, seven of 34 complete five-seed correctness intervals lie above zero. None of 19 complete diversity intervals does; one lies below zero, for Falcon Countdown GRPO: -0.105 ($[-0.147, -0.063]$). At Qwen2.5-3B, all five GRPO correctness intervals and the three available diversity intervals include zero. Intervals are nominal paired 95% Student- t intervals over training seeds.

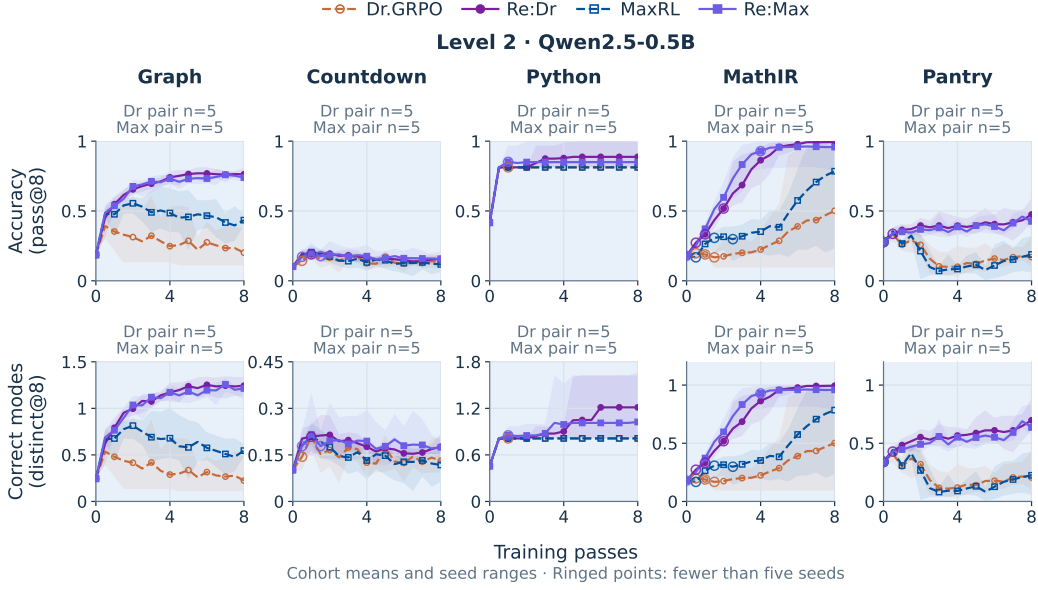


Figure 23: **Replay improves final accuracy and verified-mode counts at Level 2.** Qwen2.5-0.5B trajectories for Dr.GRPO, Re:Dr, MaxRL, and Re:Max across five domains. Rows show `pass@8` and `distinct@8`; the latter depends on both correctness and solution diversity. Each objective pair has five terminal seeds. Lines average available seeds within these cohorts; bands show seed ranges, and ringed points use fewer than five seeds. Both metrics use the same checkpoints and contributing seeds. Fig. 24 shows success-conditional diversity.

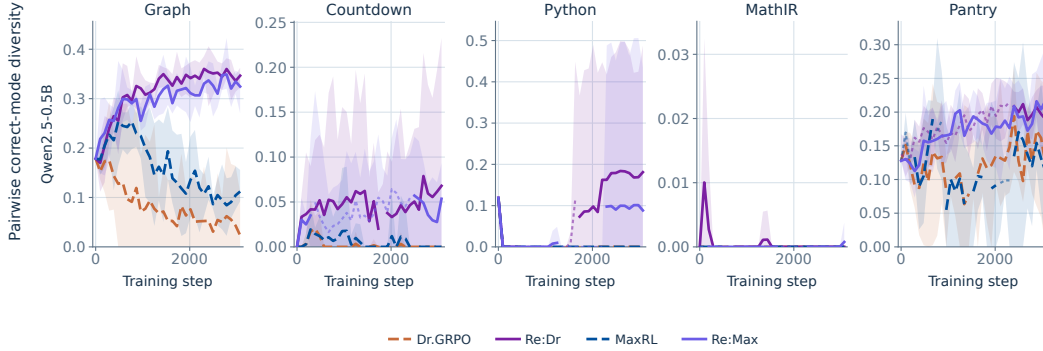


Figure 24: **Replay preserves greater solution diversity on Level-2 Graph.** PCMD during Qwen2.5-0.5B training for the four primary methods. At the terminal checkpoint, Re:Dr and Re:Max reach .346 and .325, compared with .023 for Dr.GRPO and .112 for MaxRL. MathIR remains near zero under all four methods. Lines average eligible seeds on each method’s evaluation grid; bands show seed ranges. Each seed requires at least five prompts with two correct responses, so cohorts can differ from those in Fig. 23. Fine dotted segments use incomplete cohorts; gaps have no eligible seed.

Uniform versus frequency-weighted replay uses nine five-seed comparisons: all five Qwen2.5-0.5B domains and Graph/PantryPlan at both larger scales. Changing only the within-bank weights adds +.318 extra verified modes per prompt in the Qwen2.5-0.5B aggregate ([+.272, +.358], paired-seed bootstrap), with an inconclusive correctness effect of +.010 ([−.119, +.139]). This supports an additional benefit from balancing beyond frequency-weighted reuse; extra-mode counts remain coupled to correctness (App. F.6).

RLEP’s offline pool uses temperature .7 and top- $p = .95$, whereas evaluation uses temperature 1 and top- $p = 1$. Offline collection also differs from the online bank construction used by verified replay.

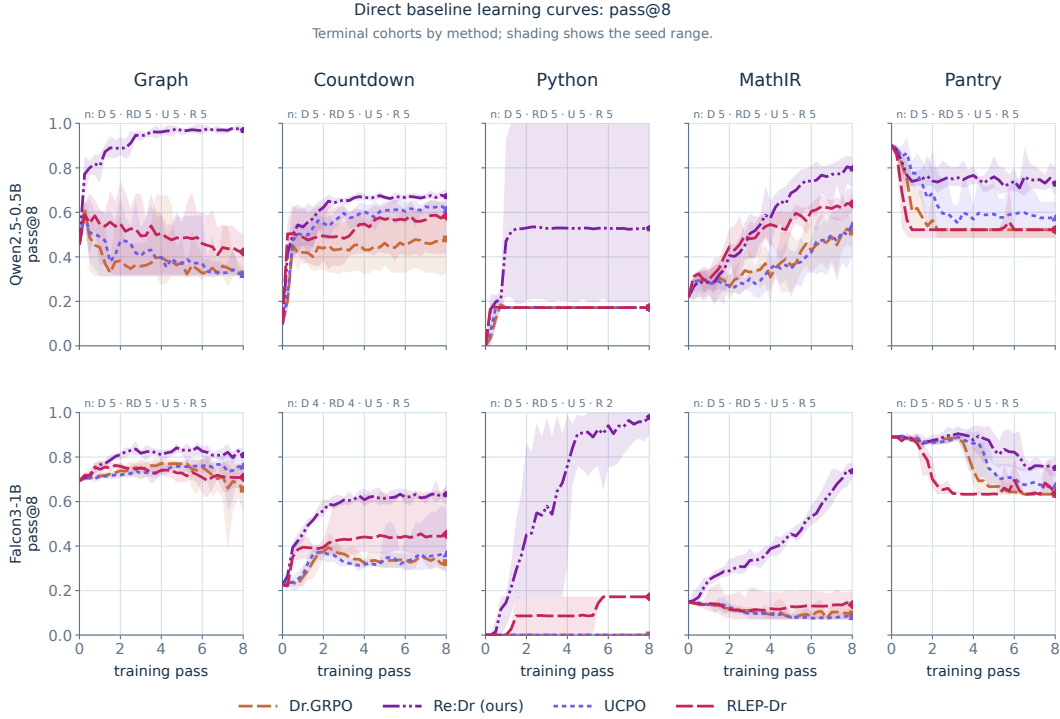


Figure 25: **Re:Dr has the highest terminal mean accuracy in every displayed comparison.** pass@8 during Level-1 training for Dr.GRPO, Re:Dr, UCPO, and sparse RLEP-Dr. Rows show Qwen2.5-0.5B and Falcon3-1B; columns show the five domains. Lines average available members of each method’s terminal cohort; bands span the seed range, and missing checkpoints leave gaps. Panel labels give cohort sizes: five except Falcon Countdown Dr.GRPO/Re:Dr ($n = 4$) and Falcon Python RLEP-Dr ($n = 2$). Dr.GRPO/Re:Dr use paired cohorts; comparator cohorts are method-specific.

F.5 DIVERSITY-PRESERVING COMPARATORS

GAPO (Anschel et al., 2025) and SetPO (Li et al., 2026a) target output diversity within the fresh rollout group. Both are evaluated on Level-1 Qwen2.5-0.5B across five domains and five seeds. GAPO assigns verifier-positive rollouts the reward $[1 - (f_i - 1/L) + 1]/2$, with zero for failures, where f_i is the key’s frequency among the group’s verified rollouts and L is the prompt’s certified number of valid modes. SetPO adds each rollout’s leave-one-out marginal contribution to kernelized set diversity to its advantage, with coefficient $\lambda = 1$. It uses frozen all-MiniLM-L6-v2 sentence embeddings with mean pooling and cosine similarity clamped to $[0, 1]$. The embedding forward pass adds computation.

GAPO increases PCMD on Countdown (+.340, [+ .112, +.567]) and PantryPlan (+.653, [+ .610, +.696]), while its PythonFactors effect is near zero with an interval spanning zero. PythonFactors has more valid modes than the sixteen-rollout group can represent on 95% of training prompts, with up to 3,600 modes. This limits within-group coverage but does not prevent a policy from assigning mass across the full support. MathIR effects are small across methods; the largest is Re:Dr’s +.017 ([+.003, +.031]). SetPO increases PCMD on PantryPlan (+.266, [+ .177, +.356]); its other four domain intervals include zero. Five-domain mean effects are +.225 for GAPO, +.066 for SetPO, and +.320 for Re:Max, although GAPO’s PantryPlan effect

is larger than Re:Max’s. GAPO reshapes rewards using fresh group frequencies, whereas replay balances rehearsal across a bank of verified modes accumulated over training.

GAPO and SetPO share a separate Dr.GRPO control under their hardware and disjoint-draw evaluation configuration. Its terminal PCMD differs from the primary control by a mean of $+ .003$ over twenty paired cells, with a range of $- .014$ to $+ .034$. These observed differences are small relative to the five-domain effects above, but do not establish equivalence between the controls.

F.6 UNIFORM VERSUS FRESH-FREQUENCY REPLAY

The weighting ablation uses the same bank construction and update rule, changing only the within-bank target: uniform weights over retained keys versus weights proportional to verifier-positive fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below favor uniform weighting. The two primary outcomes are $B_8 = \text{distinct@8} - \text{pass@8}$, extra verified modes per prompt, and $P_8 = \text{pass@8}$. The five-domain Qwen2.5-0.5B aggregate averages domain effects within each matched seed, giving equal domain weight.

Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired training seeds and use linearly interpolated 2.5th and 97.5th percentiles. Each resampled seed carries its complete domain vector for the five-domain aggregate. These nominal intervals quantify seed variability within the fixed benchmark; prompts and domains are not resampled, and intervals are not adjusted across comparisons.

The Qwen2.5-0.5B five-domain effect is $+ .318$ extra modes per prompt ($[+ .272, + .358]$), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The P_8 effect is $+ .010$ ($[- .119, + .139]$), establishing neither a gain nor equivalence. Python’s P_8 effects range from $- .781$ to $+ .828$ across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes ($+ .233$, $[+ .182, + .320]$). Its correctness measures differ: $\Delta P_8 = + .045$ ($[- .0004, + .0910]$) is inconclusive, while mean per-response correctness falls by $- .0152$ ($[- .0214, - .0090]$), with an interval entirely below zero. Falcon Pantry is inconclusive on both outcomes: $\Delta B_8 = - .146$ ($[- .564, + .414]$) and $\Delta P_8 = - .021$ ($[- .063, + .026]$). Qwen2.5-3B Graph has five pairs and an extra-mode effect of $+ .079$ ($[+ .011, + .135]$); its P_8 interval includes zero. Qwen2.5-3B Pantry also has five pairs, with $\Delta B_8 = + .207$ ($[+ .038, + .377]$) and inconclusive ΔP_8 ($+ .011$, $[- .020, + .043]$).

Rehearsing self-generated verified successes is related to rejection-sampling fine-tuning (Dong et al., 2023); here the likelihood term is length-normalized and combined with the on-policy objective. This ablation isolates the effect of uniform versus fresh-frequency weighting within that replay procedure. It supports an additional sampled-diversity benefit in the stated domains. The inconclusive P_8 effects do not establish that weighting leaves accuracy unchanged or that replay’s accuracy and diversity effects have separate causes.

G FIXED SEMANTIC-ENTROPY FACTORIALS

A four-arm factorial compares Dr.GRPO, Re:Dr, and each objective with a fixed semantic-entropy bonus across five domains at Qwen2.5-0.5B and Falcon3-1B. All three contrasts within a model-domain cell use the same training seeds: five in nine cells and four (55–58) for Falcon Countdown. Table 11 reports terminal paired means after eight training passes, with nominal pointwise 95% Student- t intervals using $n - 1$ degrees of freedom. These intervals describe training-seed variability on the fixed evaluation prompts and are unadjusted across comparisons.

At Qwen2.5-3B, seed 70 compares Re:Dr with and without the bonus across all five domains. These single-seed effects have no interval; without a semantic-only arm, a factorial interaction is undefined. The results characterize the evaluated fixed coefficients and do not identify the best entropy strength. Extra-mode counts remain coupled to correctness and do not measure success-conditional solution diversity.

Table 11: **Fixed semantic entropy helps Qwen2.5-0.5B PantryPlan with and without replay.** Without replay is semantic-only minus Dr.GRPO; on replay is semantic-plus-replay minus Re:Dr; interaction subtracts the former effect from the latter. Extra modes equal `distinct@8` minus `pass@8`. Entries give paired means and nominal 95% Student-*t* intervals; the single-seed 3B effects have no intervals.

Model	Domain	<i>n</i>	Contrast	Δ pass@8	Δ distinct@8	Δ extra modes
Qwen2.5-0.5B	Graph	5	Without replay	0.002 [-0.028, 0.031]	0.004 [-0.034, 0.041]	0.002 [-0.010, 0.014]
	Graph	5	On replay	-0.002 [-0.027, 0.023]	-0.022 [-0.179, 0.135]	-0.020 [-0.161, 0.120]
	Graph	5	Interaction	-0.004 [-0.035, 0.028]	-0.026 [-0.193, 0.141]	-0.022 [-0.161, 0.116]
	Countdown	5	Without replay	0.141 [-0.047, 0.330]	0.184 [0.011, 0.358]	0.043 [0.001, 0.085]
	Countdown	5	On replay	-0.003 [-0.033, 0.028]	0.115 [0.022, 0.209]	0.118 [0.004, 0.232]
	Countdown	5	Interaction	-0.144 [-0.326, 0.037]	-0.069 [-0.233, 0.095]	0.075 [-0.051, 0.201]
	Python	5	Without replay	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
	Python	5	On replay	-0.002 [-0.717, 0.714]	0.016 [-0.747, 0.780]	0.018 [-0.164, 0.200]
	Python	5	Interaction	-0.002 [-0.717, 0.714]	0.016 [-0.747, 0.780]	0.018 [-0.164, 0.200]
	MathIR	5	Without replay	-0.121 [-0.349, 0.106]	-0.121 [-0.347, 0.105]	0.001 [-0.006, 0.008]
	MathIR	5	On replay	0.031 [-0.035, 0.097]	0.035 [-0.044, 0.114]	0.004 [-0.047, 0.055]
	MathIR	5	Interaction	0.153 [-0.018, 0.324]	0.156 [-0.050, 0.362]	0.003 [-0.052, 0.058]
	Pantry	5	Without replay	0.201 [0.119, 0.283]	1.120 [0.888, 1.352]	0.919 [0.713, 1.124]
	Pantry	5	On replay	0.141 [0.044, 0.238]	1.095 [0.368, 1.823]	0.955 [0.321, 1.589]
	Pantry	5	Interaction	-0.061 [-0.161, 0.040]	-0.025 [-0.826, 0.777]	0.036 [-0.682, 0.754]
Falcon3-1B	Graph	5	Without replay	0.036 [-0.192, 0.264]	0.205 [-0.483, 0.893]	0.169 [-0.292, 0.630]
	Graph	5	On replay	0.001 [-0.040, 0.043]	-0.006 [-0.113, 0.101]	-0.007 [-0.091, 0.077]
	Graph	5	Interaction	-0.035 [-0.237, 0.168]	-0.211 [-0.886, 0.464]	-0.176 [-0.649, 0.297]
	Countdown	4	Without replay	0.004 [-0.092, 0.100]	-0.042 [-0.258, 0.173]	-0.047 [-0.168, 0.074]
	Countdown	4	On replay	0.036 [-0.028, 0.101]	0.047 [-0.082, 0.177]	0.011 [-0.064, 0.087]
	Countdown	4	Interaction	0.032 [-0.098, 0.161]	0.090 [-0.188, 0.367]	0.058 [-0.099, 0.216]
	Python	5	Without replay	0.002 [-0.003, 0.008]	0.002 [-0.003, 0.008]	0.000 [0.000, 0.000]
	Python	5	On replay	-0.133 [-0.308, 0.043]	-0.091 [-0.503, 0.321]	0.042 [-0.296, 0.380]
	Python	5	Interaction	-0.135 [-0.310, 0.040]	-0.093 [-0.507, 0.321]	0.042 [-0.296, 0.380]
	MathIR	5	Without replay	-0.002 [-0.063, 0.060]	-0.002 [-0.063, 0.060]	0.000 [0.000, 0.000]
	MathIR	5	On replay	0.086 [0.004, 0.168]	0.108 [0.003, 0.214]	0.022 [-0.009, 0.054]
	MathIR	5	Interaction	0.087 [0.029, 0.146]	0.110 [0.043, 0.176]	0.022 [-0.009, 0.054]
	Pantry	5	Without replay	0.011 [-0.019, 0.041]	0.091 [-0.161, 0.342]	0.080 [-0.142, 0.301]
	Pantry	5	On replay	0.025 [-0.044, 0.094]	0.382 [-0.702, 1.465]	0.356 [-0.670, 1.382]
	Pantry	5	Interaction	0.014 [-0.052, 0.081]	0.291 [-0.646, 1.228]	0.277 [-0.615, 1.168]
Qwen2.5-3B	Graph	1	On replay	-0.016	-0.010	0.006
	Countdown	1	On replay	0.039	0.146	0.107
	Python	1	On replay	-0.590	-1.215	-0.625
	MathIR	1	On replay	0.033	0.010	-0.023
	Pantry	1	On replay	0.035	0.393	0.357

H COLLAPSE AND REPLAY MECHANISM CHECKS

H.1 CONDITIONAL CONCENTRATION OF CORRECT OUTPUTS

Conditional concentration measures how strongly correct outputs favor the same solution keys, separately from the probability of correctness. For a fixed prompt with correct mass $P > 0$, write $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$ and define

$$C(q) = \sum_c q_c^2, \quad \bar{D}_2 = 2P - P^2 C(q). \quad (12)$$

Here C is the probability that two independent correct outputs share a key, and $\bar{D}_2$ is the expected number of distinct verified keys in two outputs. The identity follows from $\bar{D}_2 = \sum_c [1 - (1 - Pq_c)^2]$. At $P = 0$, C is undefined and $\bar{D}_2 = 0$. Expected distinct keys, $\bar{D}_K$, differ from extra verified modes, $B = \text{distinct}@K - \text{pass}@K$. Sampled diversity can change through either correctness or conditional concentration. Lower C alone implies neither majorization nor larger total support, and collisions do not determine whether any particular unobserved key has zero probability.

Connection to the conditional theory. Under the assumptions of Theorem P.4, the time change in its proof gives $dq_c/d\tau = q_c(q_c - C)$. Consequently,

$$\frac{dC}{d\tau} = 2 \left(\sum_c q_c^3 - C^2 \right) = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (13)$$

The variance vanishes exactly when q is uniform on its positive support. Concentration therefore increases strictly along this ideal categorical flow whenever the positive coordinates are unequal. This result does not establish monotonicity for neural optimization or a monotonic decrease under finite-bank replay.

Estimator and comparison population. For a prompt with $R \geq 2$ correct samples, the collision estimator is $\hat{C} = 1 - \widehat{\text{PCMD}}$, using the pair statistic in App. A.1. It is unbiased for promptwise C under independent sampling from a fixed policy. Estimates are undefined at $R < 2$; joint selection of eligible prompts across conditions imposes additional assumptions, as detailed in App. A.1.

For paired training seed s , let E_s contain the prompts with defined estimates in both conditions. When E_s is nonempty, the seed contrast is $|E_s|^{-1} \sum_{x \in E_s} (\hat{C}_{B,x} - \hat{C}_{A,x})$. Defined seeds receive equal weight. Before/after comparisons use each method’s own initial evaluation and step 3,072; replay comparisons use paired terminal evaluations. A missing initial evaluation leaves the corresponding before/after contrast unavailable. Nominal Student- t intervals require all five seeds; partial groups are descriptive. These intervals measure variation among the observed seed means, without simultaneous coverage or uncertainty over unseen prompts.

The analysis includes 475 training runs and 902 checkpoint evaluations. Before conditional eligibility, Level 1 has 74 Dr.GRPO and 75 MaxRL terminal replay pairs, and Level 2 has 25 pairs per objective. Complete terminal evaluations do not ensure enough correct samples to estimate concentration, or supply missing initial evaluations.

The hosted analysis instead pools correct pairs across prompts. Under independent sampling at a common budget, its ratio of expected colliding to correct pair counts is $\sum_x P_x^2 C(q_x) / \sum_x P_x^2$; the finite-sample ratio is not generally unbiased for this target. Training comparisons weight eligible prompts equally. Changes in correctness can alter both eligibility and pooled pair weights, so these aggregates estimate different quantities.

Overlapping sampling streams. Four eight-output groups per prompt do not supply 32 independently seeded responses. Parent seeds increase by one between groups, and the vLLM V0 sampler sets each output’s seed to the parent seed plus its zero-based index. The four ranges therefore contain eleven distinct nominal stream identifiers. For each prompt and checkpoint, the estimator uses the earliest draw/output position for each identifier, selected independently of correctness and key identity. Outputs sharing an identifier sometimes differ, so equal seed integers do not imply identical outputs. The seed mapping assumes that the vLLM V0 rule applies across runs; the exact runtime version is not established for every run.

Two sensitivity comparisons use disjoint stream identifiers across conditions. Split 0 assigns the lower five identifiers to condition A and the upper six to B; split 1 reverses the assignment. The identifiers are disjoint within every prompt in both splits across all 627 paired seed-level comparisons. Each split has its own eligible prompts and number of defined seeds. The two splits agree with the primary contrast’s direction in eleven of the twelve Graph/PantryPlan Dr.GRPO/GRPO before/after comparisons and all six Re:Dr-minus-Dr.GRPO comparisons.

Disjoint identifiers do not establish independent draws. Identifiers also repeat across prompts, so neither the splits nor the jointly eligible prompt means imply independent prompt observations. Independent sampling from a fixed policy remains an assumption of the estimator identity. Comparisons under shared streams are descriptive where joint-selection assumptions do not hold.

The original mean@8, pass@8, and distinct@8 average the four intact groups. Stream overlap affects dependence, but not the linearity of these marginal averages. Additional sensitivity comparisons use individual intact groups, the first group alone, prompt intersections fixed across seeds, and four-condition change differences. Pooling all 32 positions repeats stream identifiers and serves only as an overlap diagnostic. These comparisons share outputs and are not independent replications; missing conditions remain unavailable.

Concentration before and after training. At Level 1, all twelve Graph/PantryPlan Dr.GRPO/GRPO comparisons across the three scales show increased concentration, with nominal intervals above zero. Per-sample correctness also increases on their jointly eligible prompts. Graph’s increases range from .098 to .274 and PantryPlan’s from .447 to .886. Both disjoint splits agree in direction in eleven of these twelve comparisons; Falcon Graph with GRPO is the exception. Thus the observed concentration increase has a sampling sensitivity and does not establish exact mode disappearance or concentration in every RLVR run.

Replay contrasts. At Level 1, Re:Dr has lower concentration than Dr.GRPO in all six Graph and PantryPlan comparisons. Changes range from $-.756$ to $-.273$ in Graph and $-.433$ to $-.300$ in PantryPlan; all six nominal intervals lie below zero, and both disjoint splits agree in direction. These comparisons also have lower per-sample correctness under replay on their jointly eligible prompts. The collision contrast therefore describes a change in the conditional correct-key distribution on that population; it neither fixes correctness nor implies improvement in every accuracy measure. Full-test pass@8 and sampled diversity measure separate outcomes.

The other domains have narrower identifiable populations. Level-1 Dr.GRPO/GRPO Python initial evaluations yield no jointly eligible prompts. Some Countdown before/after comparisons contain only one or two eligible prompts per seed. Several MathIR conditions already have observed collision one, leaving little detectable change. A diversity increase alone therefore does not imply reduced conditional concentration.

For Qwen2.5-3B MaxRL at Level 1, replay lowers concentration in Countdown, Python, and PantryPlan, with nominal intervals below zero; MathIR’s contrast is zero. Graph’s primary contrast is negative, but its two disjoint splits disagree in direction, as they do for Falcon Graph with MaxRL. At Level 2, both Graph replay contrasts are negative and both splits agree. PantryPlan remains sparse despite complete terminal evaluations: Re:Dr has three defined seeds with 2–16 eligible prompts and mean $+.495$; Re:Max has four defined seeds with 1–19 eligible prompts and mean $+.439$. Both splits also give positive means, with fewer defined seeds. Neither contrast has a five-seed interval. Full-test diversity gains can therefore coexist with greater conditional concentration on these small jointly correct subsets.

Table conventions. Positive $\Delta\hat{C}$ indicates greater concentration. Eligible prompts have $R \geq 2$ correct representatives in both conditions, with eligibility determined separately for each contrast and split. The range $|E_s|$ counts eligible prompts out of 128 across available seed pairs, including zero; n counts seeds with a defined estimate, out of five. Within each seed, eligible prompts receive equal weight; defined seeds then receive equal weight. The nominal 95% intervals are unadjusted, untruncated Student- t intervals and appear only at $n = 5$. Dashes denote unavailable or unidentified contrasts. Replay-table rows give Re:Dr minus Dr.GRPO or Re:Max minus MaxRL at step 3,072; they do not measure the control’s own initial-to-final change.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.242	[+0.110, +0.373]	17–40	+0.257 (5)	+0.284 (5)
Graph	GRPO	5	+0.221	[+0.097, +0.344]	15–28	+0.268 (5)	+0.287 (5)
Graph	MaxRL	5	+0.145	[−0.033, +0.324]	18–40	+0.115 (5)	+0.174 (5)
Graph	Re:Dr	5	−0.507	[−0.558, −0.456]	45–49	−0.519 (5)	−0.465 (5)
Graph	Re:Max	5	−0.399	[−0.458, −0.339]	40–51	−0.436 (5)	−0.348 (5)
Countdown	Dr.GRPO	5	+1.000	[+1.000, +1.000]	1–2	− (0)	+1.000 (5)
Countdown	GRPO	5	+0.964	[+0.863, +1.065]	1–2	− (0)	+0.920 (5)
Countdown	MaxRL	4	+0.977	–	0–2	− (0)	+1.000 (4)
Countdown	Re:Dr	5	+0.467	[+0.117, +0.816]	1–3	− (0)	+0.713 (4)
Countdown	Re:Max	5	+0.420	[+0.359, +0.480]	1–2	− (0)	+0.493 (5)
Python	Dr.GRPO	0	–	–	0	− (0)	− (0)
Python	GRPO	0	–	–	0	− (0)	− (0)
Python	MaxRL	0	–	–	0	− (0)	− (0)
Python	Re:Dr	0	–	–	0	− (0)	− (0)
Python	Re:Max	0	–	–	0	− (0)	− (0)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	5–8	+0.000 (5)	+0.000 (1)
MathIR	GRPO	5	+0.040	[−0.071, +0.151]	4–10	+0.000 (5)	+0.250 (2)
MathIR	MaxRL	5	+0.022	[−0.039, +0.084]	5–9	+0.000 (5)	+0.067 (5)
MathIR	Re:Dr	5	+0.059	[−0.046, +0.163]	8–11	+0.000 (5)	+0.367 (2)
MathIR	Re:Max	5	+0.063	[−0.009, +0.134]	9–11	+0.000 (5)	+0.250 (5)
Pantry	Dr.GRPO	5	+0.833	[+0.819, +0.847]	54–64	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.849	[+0.845, +0.853]	64–65	+1.000 (5)	+1.000 (5)
Pantry	MaxRL	5	+0.859	[+0.836, +0.881]	54–65	+1.000 (5)	+1.000 (5)
Pantry	Re:Dr	5	+0.413	[+0.345, +0.481]	78–89	+0.543 (5)	+0.538 (5)
Pantry	Re:Max	5	+0.446	[+0.324, +0.567]	80–88	+0.593 (5)	+0.550 (5)

Table 12: **Initial-to-final conditional concentration: Qwen2.5-0.5B, Level 1.** Each row compares the same method at initialization and step 3,072; positive $\Delta\hat{C}$ means greater concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.233	[+0.124, +0.343]	41–54	+0.059 (5)	+0.428 (5)
Graph	GRPO	5	+0.098	[+0.026, +0.170]	48–56	-0.186 (5)	+0.273 (5)
Graph	MaxRL	3	-0.113	–	48–52	-0.083 (3)	+0.039 (3)
Graph	Re:Dr	5	-0.007	[-0.065, +0.050]	37–45	+0.048 (5)	+0.045 (5)
Graph	Re:Max	2	-0.062	–	51–52	-0.300 (2)	-0.015 (2)
Countdown	Dr.GRPO	5	+0.022	[-0.038, +0.082]	6–8	+0.000 (5)	– (0)
Countdown	GRPO	5	+0.180	[+0.128, +0.231]	6–11	+0.126 (5)	– (0)
Countdown	MaxRL	2	+0.182	–	11	+0.143 (2)	– (0)
Countdown	Re:Dr	4	+0.035	–	10–11	+0.158 (4)	– (0)
Countdown	Re:Max	1	+0.135	–	11	+0.143 (1)	– (0)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	0	–	–	0	– (0)	– (0)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	0	–	–	–	– (0)	– (0)
MathIR	Dr.GRPO	5	+0.105	[+0.088, +0.122]	6	– (0)	+0.098 (5)
MathIR	GRPO	5	+0.054	[-0.099, +0.206]	3–7	– (0)	+0.233 (5)
MathIR	MaxRL	1	+0.278	–	6	+1.000 (1)	+0.133 (1)
MathIR	Re:Dr	5	+0.086	[+0.026, +0.146]	6–7	– (0)	+0.111 (5)
MathIR	Re:Max	2	+0.167	–	5–6	+1.000 (2)	+0.000 (2)
Pantry	Dr.GRPO	5	+0.886	[+0.886, +0.886]	76	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.863	[+0.798, +0.928]	76	+0.945 (5)	+1.000 (5)
Pantry	MaxRL	1	+0.886	–	76	+1.000 (1)	+1.000 (1)
Pantry	Re:Dr	4	+0.437	–	75–83	+0.451 (4)	+0.618 (4)
Pantry	Re:Max	2	+0.746	–	76–78	+0.740 (2)	+0.978 (2)

Table 13: **Initial-to-final conditional concentration: Falcon3-1B, Level 1.** Each row compares the same method at initialization and step 3,072; positive $\Delta\hat{C}$ means greater concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.274	[+0.232, +0.316]	44–47	+0.030 (5)	+0.339 (5)
Graph	GRPO	5	+0.274	[+0.207, +0.341]	46–49	+0.046 (5)	+0.364 (5)
Graph	MaxRL	5	+0.072	[-0.002, +0.145]	45–55	-0.208 (5)	+0.215 (5)
Graph	Re:Dr	5	-0.038	[-0.060, -0.016]	48–53	-0.227 (5)	+0.121 (5)
Graph	Re:Max	5	-0.145	[-0.198, -0.093]	46–52	-0.220 (5)	+0.165 (5)
Countdown	Dr.GRPO	5	+0.160	[+0.109, +0.211]	11–12	+0.200 (5)	-0.016 (5)
Countdown	GRPO	5	+0.182	[+0.182, +0.182]	11	+0.200 (5)	+0.000 (5)
Countdown	MaxRL	5	+0.056	[-0.046, +0.159]	9–11	+0.050 (5)	+0.000 (5)
Countdown	Re:Dr	5	+0.020	[-0.060, +0.099]	11–15	+0.169 (5)	-0.071 (5)
Countdown	Re:Max	5	-0.100	[-0.145, -0.055]	10–12	+0.000 (5)	-0.050 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	+0.000	[+0.000, +0.000]	2–9	+0.000 (2)	+0.000 (5)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	5	-0.023	[-0.050, +0.003]	9	+0.000 (5)	-0.040 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	11–14	+0.000 (5)	+0.000 (5)
MathIR	GRPO	5	+0.000	[+0.000, +0.000]	9–11	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	7–14	+0.000 (5)	+0.000 (5)
MathIR	Re:Dr	4	-0.012	–	14	+0.000 (4)	+0.000 (4)
MathIR	Re:Max	5	+0.000	[+0.000, +0.000]	12–15	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	+0.447	[+0.266, +0.627]	49–67	+0.478 (5)	+0.245 (5)
Pantry	GRPO	5	+0.469	[+0.411, +0.526]	49–62	+0.514 (5)	+0.271 (5)
Pantry	MaxRL	5	+0.431	[+0.297, +0.565]	59–69	+0.394 (5)	+0.246 (5)
Pantry	Re:Dr	5	+0.184	[+0.081, +0.288]	64–71	+0.087 (5)	+0.037 (5)
Pantry	Re:Max	5	+0.142	[+0.018, +0.266]	68–75	+0.046 (5)	-0.015 (5)

Table 14: **Initial-to-final conditional concentration: Qwen2.5-3B, Level 1.** Each row compares the same method at initialization and step 3,072; positive $\Delta\hat{C}$ means greater concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.625	[+0.295, +0.955]	2–10	+0.633 (2)	+0.860 (5)
Graph	MaxRL	5	+0.452	[+0.387, +0.518]	6–10	+0.767 (4)	+0.484 (5)
Graph	Re:Dr	5	-0.204	[-0.282, -0.126]	9–11	+0.033 (5)	-0.372 (5)
Graph	Re:Max	5	-0.128	[-0.262, +0.006]	9–11	+0.333 (5)	-0.267 (5)
Countdown	Dr.GRPO	1	+0.000	–	0–1	– (0)	– (0)
Countdown	MaxRL	1	+0.000	–	0–1	– (0)	– (0)
Countdown	Re:Dr	5	+0.000	[+0.000, +0.000]	1	– (0)	– (0)
Countdown	Re:Max	3	+0.000	–	0–1	– (0)	– (0)
Python	Dr.GRPO	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	MaxRL	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	Re:Dr	5	+0.094	[-0.259, +0.448]	32	-0.224 (5)	+0.146 (5)
Python	Re:Max	5	+0.199	[-0.088, +0.485]	32	-0.070 (5)	+0.258 (5)
MathIR	Dr.GRPO	3	+0.000	–	0–5	– (0)	+0.000 (3)
MathIR	MaxRL	4	+0.000	–	0–6	– (0)	+0.000 (4)
MathIR	Re:Dr	5	+0.000	[+0.000, +0.000]	6	– (0)	+0.000 (5)
MathIR	Re:Max	4	+0.000	–	6	– (0)	+0.000 (4)
Pantry	Dr.GRPO	3	-0.578	–	0–3	-0.500 (1)	-1.000 (1)
Pantry	MaxRL	3	-0.238	–	1–5	-1.000 (1)	-1.000 (1)
Pantry	Re:Dr	2	-0.049	–	15–16	-0.075 (2)	-0.050 (2)
Pantry	Re:Max	3	+0.020	–	18–20	+0.092 (3)	-0.174 (3)

Table 15: **Initial-to-final conditional concentration: Qwen2.5-0.5B, Level 2.** Each row compares the same method at initialization and step 3,072; positive $\Delta\hat{C}$ means greater concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.756	[-0.831, -0.680]	37–40	-0.853 (5)	-0.807 (5)
Graph	MaxRL	5	-0.587	[-0.759, -0.414]	41–73	-0.625 (5)	-0.605 (5)
Countdown	Dr.GRPO	5	-0.529	[-0.605, -0.452]	42–78	-0.529 (5)	-0.535 (5)
Countdown	MaxRL	5	-0.547	[-0.635, -0.459]	45–84	-0.578 (5)	-0.530 (5)
Python	Dr.GRPO	5	-0.090	[-0.341, +0.160]	22	-0.115 (5)	-0.062 (5)
Python	MaxRL	5	-0.266	[-0.577, +0.044]	22	-0.298 (5)	-0.272 (5)
MathIR	Dr.GRPO	5	-0.008	[-0.017, +0.001]	44–68	-0.006 (5)	-0.008 (5)
MathIR	MaxRL	5	-0.005	[-0.013, +0.003]	32–56	-0.007 (5)	-0.002 (5)
Pantry	Dr.GRPO	5	-0.433	[-0.518, -0.347]	54–66	-0.442 (5)	-0.494 (5)
Pantry	MaxRL	5	-0.417	[-0.547, -0.288]	53–71	-0.363 (5)	-0.508 (5)

Table 16: **Replay and conditional concentration: Qwen2.5-0.5B, Level 1.** Each row gives Re:Dr minus Dr.GRPO or Re:Max minus MaxRL at step 3,072; negative $\Delta\hat{C}$ means lower concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.309	[-0.523, -0.094]	31–64	-0.129 (5)	-0.452 (5)
Graph	MaxRL	5	-0.021	[-0.040, -0.002]	95–100	+0.100 (5)	-0.013 (5)
Countdown	Dr.GRPO	4	-0.168	–	33–42	-0.139 (4)	-0.233 (4)
Countdown	MaxRL	5	-0.063	[-0.079, -0.047]	78–81	-0.062 (5)	-0.063 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	-0.093	[-0.350, +0.165]	22–128	-0.039 (5)	-0.090 (5)
MathIR	Dr.GRPO	5	+0.004	[-0.007, +0.015]	9–14	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.010, +0.009]	40–72	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.404	[-0.600, -0.209]	74–81	-0.435 (5)	-0.352 (5)
Pantry	MaxRL	5	-0.129	[-0.223, -0.035]	81	-0.203 (5)	-0.043 (5)

Table 17: **Replay and conditional concentration: Falcon3-1B, Level 1.** Each row gives Re:Dr minus Dr.GRPO or Re:Max minus MaxRL at step 3,072; negative $\Delta\hat{C}$ means lower concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.273	[-0.299, -0.247]	54–71	-0.363 (5)	-0.151 (5)
Graph	MaxRL	5	-0.162	[-0.261, -0.062]	68–83	-0.169 (5)	+0.059 (5)
Countdown	Dr.GRPO	5	-0.137	[-0.183, -0.090]	70–75	-0.153 (5)	-0.139 (5)
Countdown	MaxRL	5	-0.105	[-0.126, -0.084]	56–76	-0.113 (5)	-0.095 (5)
Python	Dr.GRPO	2	-0.190	–	0–22	-0.221 (2)	-0.141 (2)
Python	MaxRL	5	-0.153	[-0.265, -0.041]	22–81	-0.203 (5)	-0.083 (5)
MathIR	Dr.GRPO	5	-0.010	[-0.022, +0.003]	23–34	-0.017 (5)	+0.003 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	21–27	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.300	[-0.420, -0.179]	71–83	-0.337 (5)	-0.305 (5)
Pantry	MaxRL	5	-0.292	[-0.319, -0.265]	81–89	-0.355 (5)	-0.237 (5)

Table 18: **Replay and conditional concentration: Qwen2.5-3B, Level 1.** Each row gives Re:Dr minus Dr.GRPO or Re:Max minus MaxRL at step 3,072; negative $\Delta\hat{C}$ means lower concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

Domain	Control	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.577	[-0.736, -0.418]	9–25	-0.599 (5)	-0.544 (5)
Graph	MaxRL	5	-0.405	[-0.472, -0.339]	28–42	-0.262 (5)	-0.416 (5)
Countdown	Dr.GRPO	4	-0.230	–	0–9	-0.281 (4)	-0.292 (4)
Countdown	MaxRL	4	-0.073	–	0–10	+0.000 (3)	-0.118 (4)
Python	Dr.GRPO	5	-0.209	[-0.563, +0.146]	104	-0.219 (5)	-0.234 (5)
Python	MaxRL	5	-0.106	[-0.397, +0.184]	104	-0.076 (5)	-0.121 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	12–122	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.004, +0.002]	29–125	+0.000 (5)	-0.001 (5)
Pantry	Dr.GRPO	3	+0.495	–	0–16	+0.656 (1)	+0.554 (2)
Pantry	MaxRL	4	+0.439	–	0–19	+0.267 (3)	+0.208 (2)

Table 19: **Replay and conditional concentration: Qwen2.5-0.5B, Level 2.** Each row gives Re:Dr minus Dr.GRPO or Re:Max minus MaxRL at step 3,072; negative $\Delta\hat{C}$ means lower concentration among correct outputs. n counts defined seeds and $|E_s|$ eligible prompts. Splits 0 and 1 use opposite assignments of disjoint sampling streams, with separate eligibility. Nominal 95% Student- t intervals require five defined seeds.

H.2 DIVERSITY BEFORE AND AFTER RLVR-ONLY TRAINING

Unmodified Dr.GRPO and GRPO are evaluated at passes 0 and 8 on all three models and five domains: 150 runs, each evaluated with four $K = 8$ groups per checkpoint. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. The `pass@8` mean falls in 10 of the 30 domain–method–scale comparisons, while macro `distinct@8` ends above its initial value at Qwen2.5-3B. The loss of multiple sampled correct outcomes can therefore coexist with increased probability of sampling at least one correct outcome. These finite observations do not establish zero probability for unobserved keys.

Macro PCMD uses domains measurable at both endpoints: Graph, MathIR, and PantryPlan at 0.5B; Graph, Countdown, and PantryPlan at 1B; and Graph and PantryPlan at 3B. It declines by 98.5% and 99.7% at Qwen2.5-0.5B under GRPO and Dr.GRPO, respectively, and by 50.8%–65.1% at the larger scales. Terminal macro PCMD rounds to .001 for Dr.GRPO at Qwen2.5-0.5B. Thus the measured decline also appears in diversity conditional on success. Eligibility depends on the number of correct samples, so these descriptive macro-averages do not hold the prompt population or correctness fixed; App. H.1 reports contrasts on jointly eligible prompts. Falcon3-1B Countdown is an exception to the Dr.GRPO decline: its PCMD rises from .033.

Each method uses its own initial evaluation. For Qwen2.5-3B MathIR and Pantry, Dr.GRPO control seeds 71–74 use different initial sampling requests from plain GRPO: initial `pass@8` averages are .154 versus .162 and .832 versus .828, respectively. Those cells therefore have separate initial references.

Graph and PantryPlan contain substantial initial diversity at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-diversity percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. Re:Dr does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four seeds. Conditional diversity comparisons depend on which prompts produce enough correct responses. App. H.1 gives paired

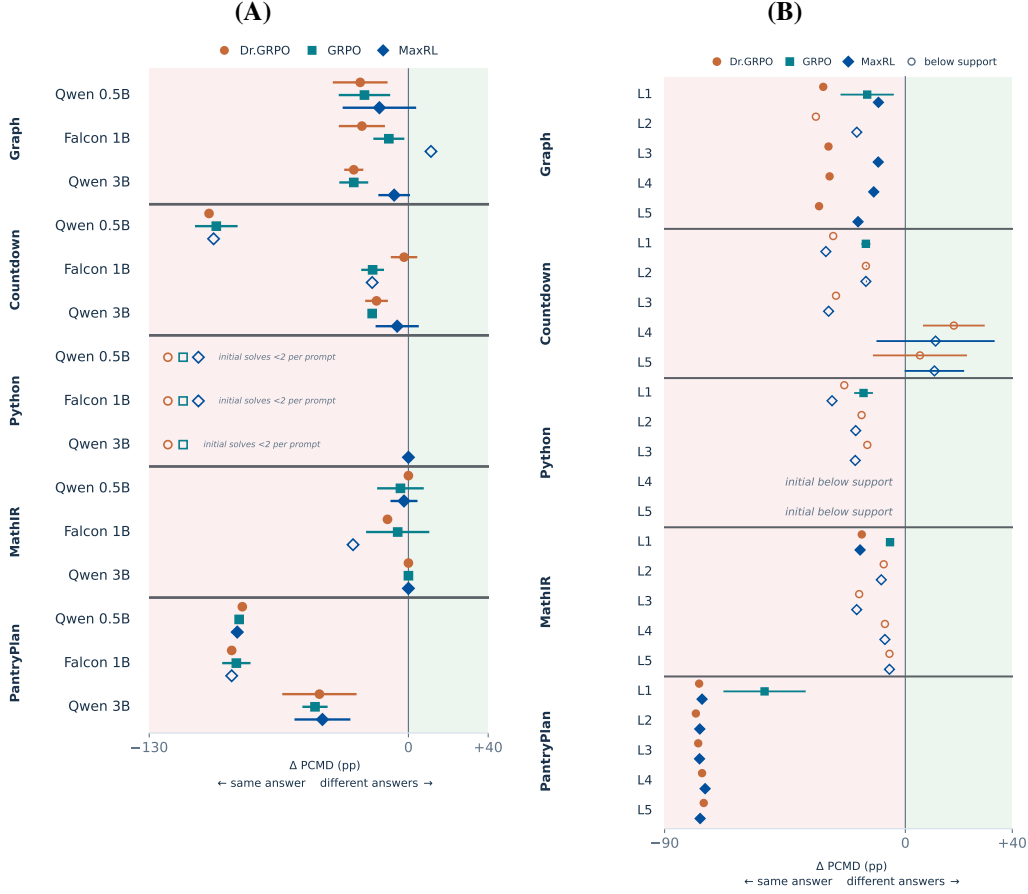


Figure 26: Graph and PantryPlan lose conditional solution diversity across scales and task levels. Changes in PCMD, in percentage points. **(A)** Initial-to-final changes at Level 1 across three model scales and three objectives. Each seed averages over prompts with at least two correct responses at both checkpoints. Filled marks and bars show five-seed means and nominal paired 95% Student-*t* intervals; open marks show partial-seed means without intervals. Annotated hollow symbols identify objectives with no jointly eligible prompts. Dr.GRPO and GRPO reduce diversity in Graph and PantryPlan at every scale. **(B)** Level-1-trained policies evaluated on Levels 1–5, with Levels 2–5 measuring transfer. Available evaluations are averaged within each scale, then available scales receive equal weight. Repeated evaluations under different context limits remain separate entries, without representing additional training seeds. Each scale uses one initial checkpoint evaluation. Bars appear only when one scale contributes and give nominal 95% Student-*t* intervals over its training seeds. Open marks indicate that at least one contributing scale has fewer than 30 eligible initial prompts or fewer than 30 eligible trained prompt observations in total. Missing initial references have fewer than two eligible prompts. GRPO appears only at Level 1. Graph and PantryPlan means decrease at every level; Countdown increases at Levels 4–5.

contrasts on jointly eligible prompts, with their seed counts and uncertainty. These held-out measurements do not track the survival of individual stored modes.

Uniform-action references are meaningful for the enumerable Graph and Pantry representations, and App. A.1 derives them. No uniform-action reference is specified for the other domains. Pantry’s bit-uniform reference exceeds every final model’s sampled `distinct@8`; the trained policies do not approach that reference.

H.3 FRESH-GRADIENT DEGENERACY AND OBSERVED OUTPUT COLLAPSE

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer

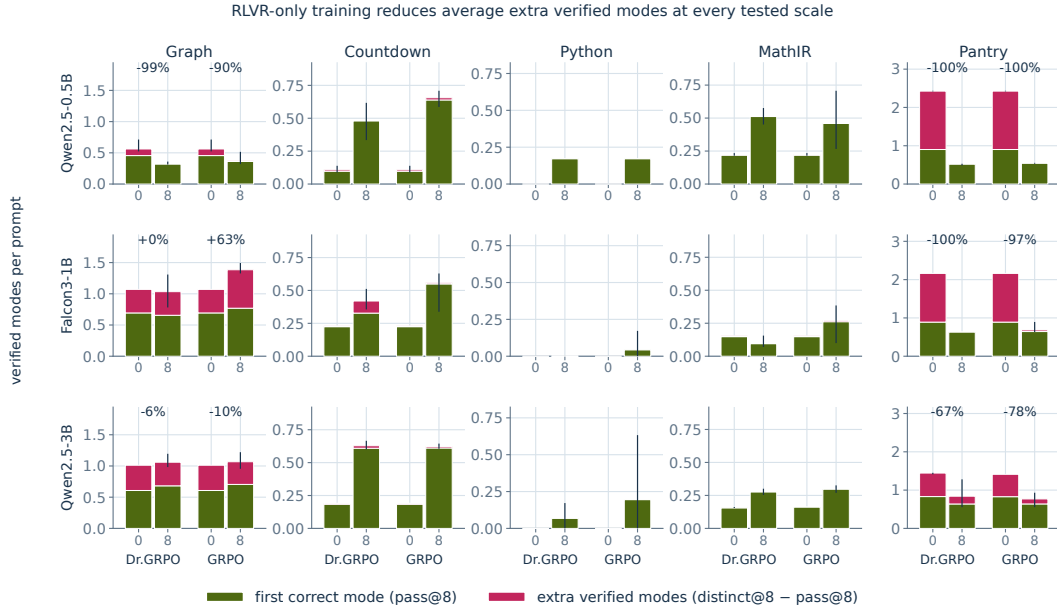


Figure 27: **RLVR-only training reduces average extra verified modes at every tested scale.** Bars decompose `distinct@8` into `pass@8` and extra verified modes before training and after eight passes. Panels show Dr.GRPO and GRPO across three models and five domains: 150 runs, with five seeds per cell and four eight-response groups per checkpoint. Whiskers show the seed range; each method uses its own initial evaluation. Badges give the percentage change in extra modes only where the initial mean exceeds .05; the other three domains start at .011 or less.

Table 20: **Uniform actions and untrained checkpoints provide absolute diversity references.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Initial is the matched pass-0 model; the final two columns are pass-8 means. Values are raw `distinct@8`; dashes indicate domains without a specified uniform-action reference.

Model	Domain	Uniform	Initial	Dr.GRPO	Re:Dr
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Domain	Dr.GRPO	Re:Dr
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

All five Qwen2.5-0.5B Python control seeds exhibit observed output collapse. At pass 0, each eight-response group contains an average of 7.99 unique raw completions per prompt. By steps 288–480, all 32 recorded temperature-1 responses per prompt are byte-identical within each seed and match its greedy completion, on all 128 prompts. This persists at every subsequent evaluation through step 3,072. The 32 responses span four groups with overlapping sampling streams

Table 21: **Per-response accuracy increases in every control cell, while `pass@8` can fall.** Accuracy uses each run’s initial and last available evaluations. Control and replay means use five runs per cell separately; Falcon Countdown includes an additional replay seed relative to the paired endpoint comparison. `mean@8` is per-response correctness. Mixed gives the mean fraction of recorded rollout groups containing both correct and incorrect responses. An unmixed group has zero fresh task advantage, although optimizer momentum can still change the policy. Training schedules match update counts and fresh-rollout budgets; total compute is discussed in App. Q.2.

Model	Domain	control <code>pass@8</code>			control <code>mean@8</code>		mixed groups	Re:Dr <code>pass@8</code>
		initial	terminal	Δ	initial	terminal		
Qwen2.5-0.5B	Graph	.456	.323	−.132	.178	.323	.144	.969
	Countdown	.097	.480	+.383	.014	.465	.140	.672
	Python	.000	.172	+.172	.000	.172	.015	.528
	MathIR	.220	.512	+.292	.037	.464	.364	.796
	Pantry	.902	.522	−.380	.344	.522	.123	.729
Falcon3-1B	Graph	.693	.656	−.037	.178	.300	.777	.809
	Countdown	.225	.328	+.103	.037	.268	.400	.636
	Python	.000	.000	+.000	.000	.000	.009	.979
	MathIR	.148	.095	−.053	.023	.068	.158	.736
	Pantry	.891	.633	−.258	.318	.633	.523	.751
Qwen2.5-3B	Graph	.607	.682	+.075	.215	.360	.706	.836
	Countdown	.184	.609	+.426	.049	.569	.281	.746
	Python	.000	.069	+.069	.000	.069	.011	.659
	MathIR	.156	.278	+.122	.061	.168	.152	.686
	Pantry	.831	.638	−.193	.267	.558	.482	.744

(App. H.1); they are not 32 independent draws. Within eight-response groups, the terminal mean is 1.00 unique completion per prompt for the control and 1.97 for replay. The repeated completion can be incorrect. These finite observations do not rule out low-probability alternatives.

This sparse binary-reward setting allows fresh task gradients to vanish. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider decoding could alter that behavior; their effects are not identified by this factorial.

H.4 CONTROL BEHAVIOR AT FIXED LEARNING RATES

The controls use a constant learning rate of 2×10^{-7} at Qwen2.5-0.5B and Falcon3-1B and a peak rate of 10^{-7} with cosine decay at Qwen2.5-3B (Table 6). No learning-rate sweep is available. Table 21 describes how accuracy and fresh reward information change at these operating points. The comparisons establish effects under the tested schedules, without identifying the best schedule for either arm.

Accuracy changes during training. Of 15 control cells, 8 finish above their initial `pass@8` and 6 below it, with a mean change of +0.039. The largest increase is +0.426 (Countdown at Qwen2.5-3B); the largest decrease is −0.380 (Pantry at Qwen2.5-0.5B). Per-response correctness rises in all 15 cells, including 7 where `pass@8` does not increase. In Pantry at Falcon3-1B, per-response correctness rises from 0.318 to 0.633, while `pass@8` falls from 0.891 to 0.633. This combination is consistent with prior observations of increased single-response accuracy alongside reduced sampling coverage (Yue et al., 2025).

The gap between `pass@8` and `mean@8` is at least .02 in 12 initial cells and below .02 in 7 terminal cells. This gap measures variation in correctness within response groups; it does not by itself identify repeated strings or conditional diversity among correct keys. The raw-completion observations in App. H.3 establish repetition directly for the Qwen2.5-0.5B Python controls.

Sparse fresh reward information. Initial `pass@8` is zero on Python at all three scales. In Python at Falcon3-1B, only a fraction 0.009 of updates contain mixed rewards, leaving zero fresh task advantage in 99.1% of updates. A learning-rate change cannot make an identically zero current task gradient nonzero, but it can change the effect of informative updates, optimizer momentum, and subsequent sampling. The Falcon3-1B Python control’s terminal `pass@8` rounds to .000, while Re:Dr ends at

.979. This is an accuracy contrast in a sparse-success setting; conditional mode diversity requires at least two correct responses per prompt and is undefined when that sampling condition is unmet.

Scope of the comparison. Replay and control share the learning rate, schedule, initialization, prompt stream, optimizer, update count, and fresh-rollout budget. Their differences therefore measure the effect of replay at the tested operating point. These controls do learn, as measured by their increased per-response correctness, but that does not establish adequate tuning or exclude a stronger baseline. A different learning rate could change accuracy, diversity, and the relative benefit of replay. Neither the sign nor the magnitude of those changes is determined without a learning-rate comparison.

H.5 FIXED-BANK EXEMPLAR SCORE TRAJECTORIES

The fixed-bank study follows Qwen2.5-0.5B Re:Dr on Graph for seeds 43–47. Bank membership and fresh counts are fixed before step 384’s fresh rollout group. No further modes are added, and uniform replay continues through step 3,072. Every exemplar in the fixed bank is tracked by prompt and canonical mode. At each replay visit, its mean token log probability s and sequence log probability ℓ are evaluated. Changes compare each exemplar’s first and final visits after the bank is fixed; visit steps can differ across exemplars.

The 3,406 exemplars span 1,575 seed–prompt pairs and have finite scores at all scheduled visits: 8–9 visits per exemplar and 29,041 observations. The pooled median change is $+.038$ nat/token (95% interval $[+.025, +.048]$), while the 10th percentile is $-.256$ and 2.61% of exemplars lose more than $.5$ nat/token ($[1.54, 3.77]\%$). The worst intermediate change is -1.328 nat/token. Sequence log probability has median change $+.154$ nat/sequence ($[+.104, +.198]$), with 21.99% of exemplars losing more than $.5$ nat/sequence. Lengths range from 4 to 100 tokens, so the token and sequence thresholds measure different changes.

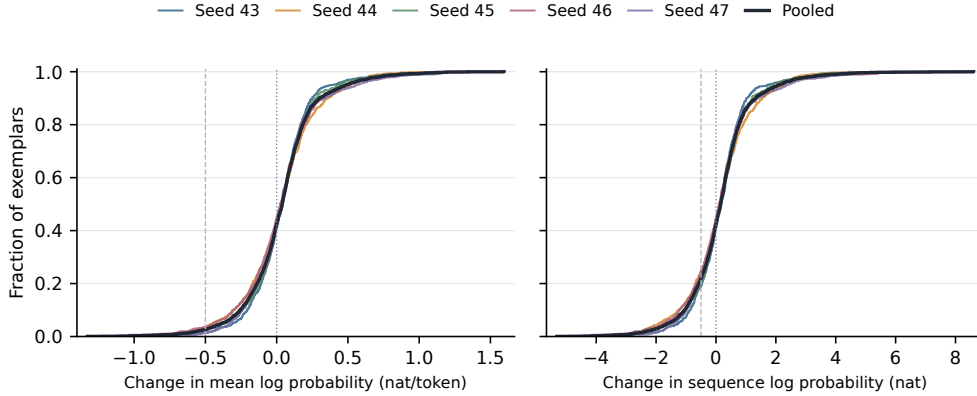


Figure 28: **Median exemplar scores rise, while some exemplars decline.** Empirical cumulative distributions of first-to-final score changes for 3,406 fixed-bank exemplars from five Qwen2.5-0.5B Re:Dr Graph seeds. Panels show mean token and sequence log probability. Colored curves represent individual seeds; the pooled curve weights exemplars equally. Dotted and dashed vertical lines mark zero and $-.5$, respectively. Positive changes indicate higher teacher-forced likelihood. These scores do not measure the probability of generating each canonical solution mode.

Pooled summaries weight exemplars equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap replicates (random seed 121): prompts are resampled within each original seed, followed by five seeds with replacement. Repeated seeds reuse their within-replicate prompt sample. Percentile intervals use linearly interpolated empirical quantiles.

Scores are observed at replay visits, rather than after every update. The declining tail shows that some fixed exemplars lose likelihood despite continued replay. Without a matched fixed-bank control lacking replay, these trajectories do not identify a causal replay effect or validate the probability floor in Theorem P.21.

Part IV. What diversity represents

I MATCHED PROMPT-HINT ABLATION

I.1 REMOVING STRATEGY HINTS FROM UNCHANGED PROBLEMS

We compare original and neutral prompts on Python factors, MathIR, and Pantry at Levels 2 and 3. Each of these six cells contains the 32 problems with the smallest SHA-256 hashes of (20260911, level, domain, row index) from the 128-problem evaluation split. Both arms generate eight draws per identical problem. The original arm uses the standard prompt. The neutral arm removes the Python nested-conditional example and suggestions to test small divisors or dispatch on listed values, the MathIR ordered algebraic-isolation suggestion, and the Pantry preference for high-energy/protein, low-sodium ingredients such as seeds or oats. MathIR changes “those operations” to “the operations” to preserve the menu-ID instruction. The user problem, answer specification, executable verifier, and remaining format requirements are unchanged.

The paired effects are neutral minus original on empirical $P_8 = \text{pass@8}$ and $D_8 = \text{distinct@8}$, with $B_8 = D_8 - P_8$ as additional verified breadth. Incorrect, empty, and truncated outputs remain in their eight-draw groups; only verifier-accepted outputs contribute a verified key. Strict verification is the primary grading rule. A formatting-normalization sensitivity analysis preserves every strict success and canonical key and applies the same rules to both arms.

Intervals use 20,000 whole-problem bootstrap resamples, paired across arms and stratified by domain and level (seed 20260911). Domain–level cells receive equal weight in averages across cells. The intervals are pointwise 95% percentile intervals without multiplicity adjustment. With 32 problems per cell, an interval crossing zero does not establish equivalence. A zero-width interval reflects zero variation in the observed bootstrap resamples, including all-tied P_8 outcomes, and does not establish population equivalence. Correct-pair collision and its certified-support uniform reference use the same correct-pair weights. Eight draws per prompt do not identify total unseen support.

The local panel comprises the initial Qwen2.5-0.5B-Instruct checkpoint and Level-2-trained Dr-GRPO and Re:Dr checkpoints. Python and MathIR use matched training seeds 43–47; Pantry uses seeds 43 and 46. Level 3 tests transfer from Level 2 training. All checkpoints evaluate the same selected problems under the local syntax constraints and a 192-token generation budget. Five-seed intervals resample whole training seeds and paired problems; Pantry intervals condition on its two checkpoints, and initial-model intervals condition on one instruction-tuned checkpoint. These effects concern the specified models, problems, and interface.

PCMD is reported for five of the 18 cells in Table 22. Each requires at least thirty checkpoint–problem groups with two verified draws under each wording. The same 32 problems are evaluated at each checkpoint, so these group counts include repeated problems across training seeds. Within a cell, PCMD weights eligible groups equally; checkpoints with more eligible problems therefore receive more weight. The population depends on success under both wordings and need not represent all evaluated problems. Three reportable cells are MathIR: all observed verified pairs within each eligible group share a canonical key, giving PCMD = .000 under both wordings. This observation does not limit the number of valid keys. In the two Re:Dr cells on Python factors, hint removal changes PCMD from .231 to .173 at Level 2 and from .256 to .323 at Level 3, on 66 and 57 eligible checkpoint–problem groups, respectively. The two point estimates have opposite signs; without intervals, they do not establish a shared direction or the reliability of either concentration effect. At Level 3, seed 46 produces 1 verified draw among 256 neutral outputs, compared with 256 of 256 original outputs. No problem at that checkpoint has two verified draws in both arms, so it contributes no groups to PCMD. Thus four checkpoints contribute to PCMD, while P_8 and D_8 average all five. A dash denotes an unavailable conditional estimate, not a zero effect.

Formatting normalization adds no verified responses for any local checkpoint in this panel; all tabulated estimates and intervals therefore agree under the two grading rules.

Two-seed Pantry effects. The Pantry intervals condition on two checkpoints and do not quantify training-seed variability. DrGRPO L2 (seed 43: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); DrGRPO L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.06$;

Table 22: **Removing Python guidance reduces correctness and observed breadth.** Local Qwen2.5-0.5B checkpoints under strict verification, with original $\rightarrow$ neutral wording. $P_8 = \text{pass@8}$, $D_8 = \text{distinct@8}$, and $B_8 = D_8 - P_8$ use eight draws on each of 32 problems per domain and level. Effects are neutral minus original. P_8 , D_8 , and B_8 average the displayed number s of checkpoints. Brackets are pointwise 95% bootstrap intervals: paired problems and training seeds are resampled for five-seed groups; Pantry intervals condition on its two checkpoints, and initial-model intervals on one checkpoint. PCMD is the mean probability that two verified draws on a problem differ, pooled over checkpoint–problem groups with at least two verified draws in each arm. Its paired population depends on correctness, and repeated problems at different checkpoints count separately. Only 5 of 18 cells meet the minimum of 30 such groups; dashes denote the others. PCMD has no reported interval.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8	PCMD: O $\rightarrow$ N	Δ PCMD
Initial (s = 1)	Python L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531 [-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03	-	-
Initial (s = 1)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00	-	-
Initial (s = 1)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00	-	-
Initial (s = 1)	Python L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688 [-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09	-	-
Initial (s = 1)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000 [-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00	-	-
Initial (s = 1)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031 [+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00	-	-
DrGRPO (s = 5)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025 [-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00	.000 $\rightarrow$.000	+0.000
DrGRPO (s = 5)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037 [-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00	-	-
DrGRPO (s = 2)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016 [-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02	-	-
DrGRPO (s = 2)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000 [-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02	-	-
DrGRPO (s = 5)	Python L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812 [-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00	-	-
DrGRPO (s = 5)	Python L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000 [-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00	-	-
Re:Dr (s = 5)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019 [-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00	.000 $\rightarrow$.000	+0.000
Re:Dr (s = 5)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037 [-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00	.000 $\rightarrow$.000	+0.000
Re:Dr (s = 2)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016 [-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00	-	-
Re:Dr (s = 2)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047 [+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06	-	-
Re:Dr (s = 5)	Python L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331 [-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24	.231 $\rightarrow$.173	-.058
Re:Dr (s = 5)	Python L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506 [-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26	.256 $\rightarrow$.323	+0.067

seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$; Re:Dr L2 (seed 43: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.06$; seed 46: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$); Re:Dr L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.12$; seed 46: $\Delta P_8 = +0.062$, $\Delta D_8 = +0.09$).

Scope of the wording intervention. The Python intervention removes an executable nested-conditional example along with divisor-testing and dispatch guidance, and it shortens the prompt. Its effect concerns this combined change; it does not isolate a preference cue from example or length effects. Lower distinct@8 together with lower pass@8 can reflect reduced correctness rather than increased concentration among correct outputs. The PCMD columns in Table 22 measure conditional diversity on jointly eligible checkpoint–problem groups. Eligibility itself depends on correctness, so these comparisons do not hold the full evaluation population or its accuracy fixed.

Observed prompt dependence. Python shows a large loss of correctness after guidance removal. Across the five DrGRPO seeds, mean P_8 falls from .8125 to zero at Level 2 and from 1.0 to zero at Level 3; D_8 falls identically and $B_8 = 0$ in both arms. With no correct neutral DrGRPO outputs, its conditional-correct concentration is undefined. Re:Dr retains neutral $P_8 = .55625$ and $.49375$ and $D_8 = .675$ and $.65625$ at Levels 2 and 3, respectively, while losing additional diversity: $\Delta B_8 = -.2375$ and $-.25625$. The initial checkpoint also loses Python correctness (Table 22). These observations identify substantial dependence on the combined Python wording intervention; a raw mode-count decline cannot be interpreted solely as a redistribution among correct answers.

Python responses after hint removal. Across 5,120 DrGRPO Python responses from five seeds, two levels, and both wordings, all 2,560 original-wording outputs yield the same extracted lambda, whose body matches the conditional example in the prompt; 2,320 of these responses pass verification:

```
lambda n: 2 if n % 2 == 0 else 3 if n % 3 == 0 else 5
```

Of the 2,560 neutral-wording responses, 1,816 (70.94%) are invalid Python expressions, 59 violate other restricted-language rules, 376 return non-integers, 276 fail the proper-divisor condition, and 33 (1.29%) have no extracted candidate after reaching the token limit. Syntax errors are the largest failure category. The categories follow the first rejection under the extractor, parser, and executor; they do not identify an internal training mechanism.

Concentration after hint removal. Every trained MathIR checkpoint has $B_8 = 0$ under both wordings at both levels. Within each problem’s eight-draw group, every observed pair of correct

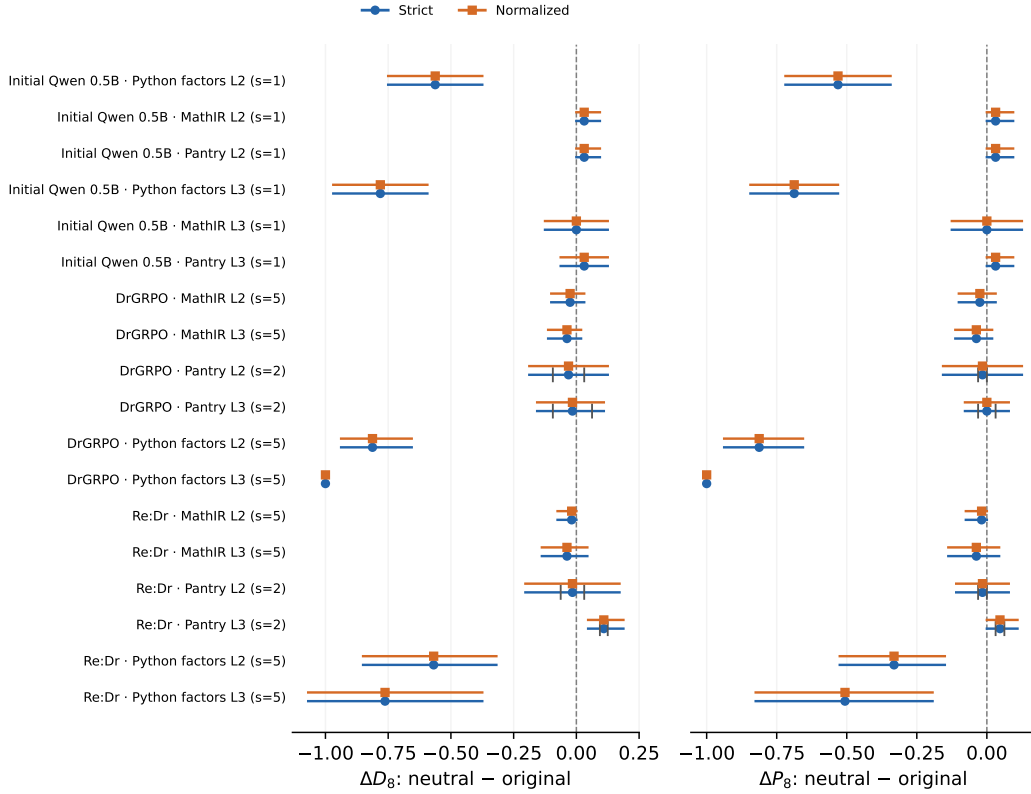


Figure 29: **Python guidance removal produces the largest wording effects in this panel.** Neutral-minus-original D_8 (left) and P_8 (right) for the initial Qwen2.5-0.5B checkpoint and Level-2-trained DrGRPO and Re:Dr on Python factors, MathIR, and Pantry at Levels 2 and 3, with 32 paired problems and eight draws per arm. Blue circles use strict verification; orange squares use formatting normalization. Horizontal bars are pointwise 95% bootstrap intervals; the vertical dashed line marks zero. Five-seed groups resample training seeds and paired problems; Pantry and initial-model intervals condition on two checkpoints and one checkpoint, respectively. Gray marks show individual Pantry seed effects.

outputs shares a canonical key, in every trained checkpoint-level cell: correct-pair collision is 1.0, versus the uniform reference of .2 over five certified modes. Table 22 reports P_{CMD} .000 under both wordings in every eligible MathIR cell. Removing this hint therefore leaves observed correct outputs concentrated. At Level 2, Re:Dr retains mean $P_8 = .975$ without the order hint, compared with .99375 originally. Across its five checkpoints, the neutral arm has 4,179 colliding pairs out of 4,179 correct pairs, from 154 eligible checkpoint-problem groups; the original arm has 4,382 out of 4,382 pairs from 157 such groups. Both arms evaluate the same 32 problems at each checkpoint. This preserves the observed concentration despite removing the specified order hint; it does not measure all unseen modes or remove the other interface constraints. Pantry effects are mixed. Re:Dr at Level 3 gains .109375 in D_8 , comprising .046875 in P_8 and .0625 in B_8 , but this comparison has only two training seeds. The 64-draw hosted experiment evaluates both wordings on sixteen problems per cell in App. K; its population and draw budget remain distinct from this local eight-draw experiment.

J DECODING TEMPERATURE, NUCLEUS, AND BUDGET

Temperature at fixed checkpoints. Qwen2.5-0.5B Dr.GRPO and Re:Dr checkpoints after eight training passes are evaluated at $T \in \{.5, .7, 1.0, 1.3, 1.6, 2.0\}$, with top- $p = 1$ and four eight-response groups per prompt. Five domains, two methods, and five seeds give 300 checkpoint-setting evaluations (Fig. 30A). The prompt template, evaluation split, response limit, and sampling seeds are fixed across temperatures. Correct keys are pooled across each prompt’s response groups, whose sampling streams overlap as in App. H.1. These estimates describe the sampled

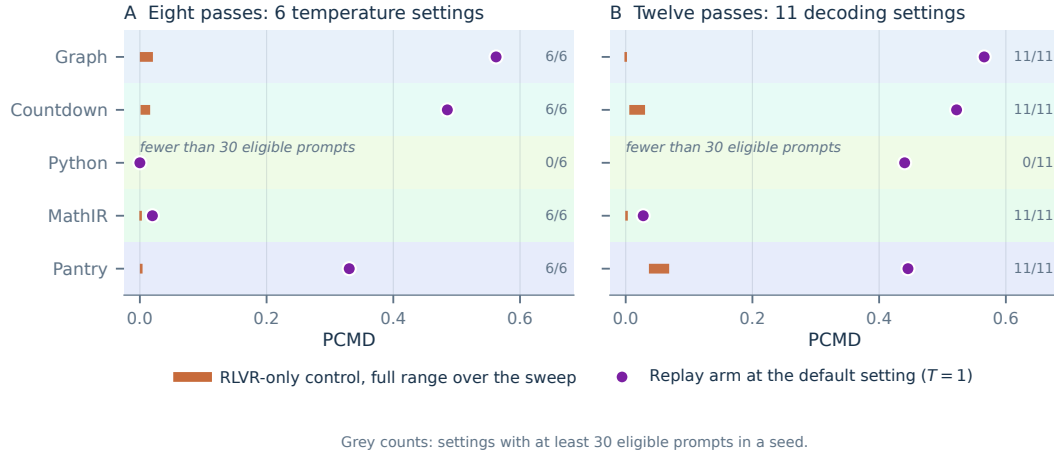


Figure 30: Control diversity remains low across the tested decoding settings. Qwen2.5-0.5B terminal PCMD across five domains. (A) Eight-pass Dr.GRPO/Re:Dr checkpoints at six temperatures ($T = .5-2.0$): 300 checkpoint-setting evaluations. (B) A separate twelve-pass cohort at eleven temperature, group-size, and nucleus settings: 550 evaluations, using a different replay objective. Bars span control means across settings; they are not confidence intervals. Markers show the corresponding replay variant at $T = 1$, top- $p = 1$, and $K = 8$. Means use seeds with at least 30 prompts having at least two correct responses, so control and replay populations need not match. Control bars use five seeds outside Python; Python controls have none, while its replay markers use two seeds in A and three in B. Counts give eligible settings. Ranges narrower than .004 are displayed at width .004 for visibility.

pools. A seed contributes to a PCMD mean when at least 30 prompts have at least two correct responses. Eligible prompts and seeds can differ across settings and methods.

Across the six temperatures, the largest control PCMD is .021 in Graph, .016 in Countdown, .002 in MathIR, and .004 in PantryPlan. Re:Dr at $T = 1$ has corresponding means .562, .485, .020, and .330. These four domains retain five eligible control seeds at every temperature. Python controls have no seed reaching the 30-prompt threshold, so their conditional diversity is unidentified under this reporting rule. The sweep therefore shows low measured control diversity in the four eligible domains, without establishing that unobserved modes have zero probability or that every decoding choice would behave the same way.

Hardware sensitivity. All evaluations in the temperature sweep use NVIDIA RTX A5000 GPUs. A paired comparison of A100 and A5000 evaluations at $T = 1$ gives mean absolute PCMD difference .0032 across twelve comparisons, with maximum .0131. These differences describe the tested hardware comparisons; they do not bound hardware sensitivity at other checkpoints or settings.

Larger groups and nucleus truncation. A separate Qwen2.5-0.5B cohort trained for twelve passes is evaluated at eleven settings (Fig. 30B): the six temperatures above; two groups of $K = 32$ at $T \in \{1.0, 1.3, 1.6\}$; and four groups of $K = 8$ with top- $p = .95$ at $T \in \{1.0, 1.6\}$. The $K = 32$ settings quadruple the group size and double total responses per prompt from 32 to 64. Five domains, two methods, and five seeds give 550 checkpoint-setting evaluations. Its replay variant combines separate mass and within-bank balancing terms with novelty and semantic-entropy shaping. These settings and the longer training schedule differ from the eight-pass Re:Dr comparison.

Control PCMD is zero throughout the grid in Graph. Its range is .006 to .031 in Countdown, .000 to .003 in MathIR, and .037 to .069 in PantryPlan. All eleven settings retain five eligible control seeds in these domains; Python controls remain below the reporting threshold. The replay markers provide a descriptive reference for this separate variant, without identifying the effect of the Re:Dr objective.

Scope of the decoding comparison. Both grids evaluate fixed terminal policies at one model scale. They do not measure the effects of changing rollout temperature, optimizer settings, or training dura-

tion. Correctness and conditional eligibility can change with decoding settings; a missing PCMD estimate indicates insufficient eligible prompts, rather than disappearance of all correct solution modes.

K SAMPLING-BUDGET ABLATION

K.1 SAMPLING DESIGN AND ESTIMANDS

Large-budget `pass@k` evaluation can reveal differences hidden at small sampling budgets (Yue et al., 2025). This evaluation uses sixteen problems per domain-level cell, selected by the sixteen lowest ranks in the outcome-independent hash ordering of the prompt-hint ablation. The six cells cover Python, MathIR, and Pantry at Levels 2 and 3. Each problem has original and neutral wordings, with 64 sampled responses per wording and checkpoint or deployment. The eight-draw estimates below are subsets of these 64-draw pools; responses from the separate eight-draw evaluation are excluded. The initial Qwen2.5-0.5B-Instruct checkpoint has no training-seed replication. DrGRPO and Re:Dr use five matched seeds in Python and MathIR and two in Pantry. Trained checkpoints use Level-2 training, so Level-3 evaluation measures transfer.

For a 64-draw pool, let c be the number of correct outputs and n_j the count of verified canonical key j . At $k \in \{1, 2, 4, 8, 16, 32, 64\}$, the curves average all size- k subsets without replacement:

$$P_k = 1 - \frac{\binom{64-c}{k}}{\binom{64}{k}}, \quad D_k = \sum_j \left[1 - \frac{\binom{64-n_j}{k}}{\binom{64}{k}} \right], \quad B_k = D_k - P_k.$$

An infeasible numerator combination is zero. Incorrect, empty, refused, and truncated responses remain in the pool; only verifier-accepted responses contribute keys. These correlated curve points describe finite-sample discovery and do not estimate complete unseen support. The gains $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$ distinguish first-success gains from additional solution modes.

Strict verification is primary. A formatting-normalization sensitivity preserves every strict success and canonical key. Intervals use 20,000 whole-problem bootstrap replicates (seed 20260911), paired across wordings and stratified by domain and level. Five-seed intervals also resample matched training seeds with shared problem indices; Pantry intervals condition on its two checkpoints. The intervals describe sixteen problems per cell and do not adjust for multiple comparisons. Zero-width intervals indicate no observed resampled variation and do not establish equivalence.

To compare breadth at a fixed number of correct draws, rarefy m outputs from the correct responses alone:

$$R_m = \sum_j \left[1 - \frac{\binom{c-n_j}{m}}{\binom{c}{m}} \right].$$

Paired comparisons include only prompts with $c \geq m$ under both wordings. This conditions on observed success; it does not equate accuracy or recover diversity on problems excluded by the threshold. Eligibility varies with m , method, and grading. Trained checkpoint means weight seeds equally and are undefined if any seed lacks eligible prompts. Conditional intervals use bootstrap replicates in which the corresponding mean is defined.

The full-64 summaries use two prompt weightings. Pooled collision C divides the total number of equal-key correct pairs by the total number of correct pairs within each checkpoint or deployment; a prompt therefore receives weight proportional to $\binom{c}{2}$. Promptwise PCMD averages $1 - \sum_j \binom{n_j}{2} / \binom{c}{2}$ equally over prompts with $c \geq 2$. Each wording’s absolute estimate uses its own eligible prompts; the neutral-minus-original contrast Δ_{PCMD} uses only jointly eligible prompts. Trained checkpoint estimates then receive equal weight across seeds.

Support references use certified lower bounds $L \leq M$: two valid modes in Python, five in MathIR, and the problem-specific certificate count in Pantry. Thus $1/L$ is an upper reference for collision under a uniform distribution over full support, and $L[1 - (1 - 1/L)^m]$ is a lower reference for uniform expected breadth. These are uniform-distribution references, not bounds on model breadth; observed distinct counts may exceed L .

In the tables, O/N denotes original/neutral wording, and ΔG_D is neutral minus original. U denotes the uniform reference under the corresponding weighting. E_O/E_N count separately eligible checkpoint–problem groups with $c \geq 2$; Q_O/Q_N count correct pairs. These counts sum across checkpoints for description, whereas the estimates weight checkpoints equally. Separate eligibility counts do not give the size of the jointly eligible population used for a contrast.

Table 23: **Discovery gains from eight to 64 draws.** Strict verification; O/N denotes original/neutral wording. Each row is a fixed hosted deployment evaluated on sixteen problems per cell. Intervals resample paired problems, with no replication across model training seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.014/0.000	2.017/3.051	2.003/3.051	+1.033 [−0.044, +2.371]
GPT-5.6 Sol	Python factors L2	0.000/0.405	0.000/0.866	0.000/0.462	+0.866 [+0.545, +1.231]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [−0.583, +1.128]
GPT-5.4	Python factors L3	0.025/0.000	2.100/2.340	2.076/2.340	+0.240 [−0.379, +0.930]
GPT-5.6 Sol	Python factors L3	0.000/0.586	0.000/0.759	0.000/0.174	+0.759 [+0.581, +0.952]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [−0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.086/0.101	0.917/1.789	0.831/1.688	+0.872 [+0.245, +1.575]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.032/0.129	1.625/1.842	1.592/1.713	+0.217 [−0.489, +0.921]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

Table 24: **Discovery gains from eight to 64 draws.** Formatting normalization; O/N denotes original/neutral wording. Each row is a fixed hosted deployment evaluated on sixteen problems per cell. Intervals resample paired problems, with no replication across model training seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.000/0.000	2.156/3.216	2.156/3.216	+1.060 [−0.071, +2.386]
GPT-5.6 Sol	Python factors L2	0.008/0.010	0.113/0.794	0.105/0.784	+0.681 [+0.351, +1.066]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [−0.583, +1.128]
GPT-5.4	Python factors L3	0.000/0.000	2.431/2.604	2.431/2.604	+0.173 [−0.657, +1.075]
GPT-5.6 Sol	Python factors L3	0.011/0.007	0.120/0.490	0.109/0.483	+0.370 [+0.112, +0.674]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [−0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.004/0.004	0.954/1.834	0.950/1.830	+0.880 [+0.237, +1.547]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.001/0.020	1.586/2.462	1.585/2.443	+0.877 [+0.272, +1.436]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

K.2 HOSTED DISCOVERY: GUIDANCE, FORMATTING, AND LATE MODES

The hosted evaluation contains 36,864 responses from GPT-5.4, GPT-5.6 Sol, and Grok 4.3: sixteen problems in each of six Python/MathIR/Pantry level cells, two wordings, and 64 draws per condition. All three deployments use medium reasoning effort and an 8,192-token output cap. These settings differ from the reasoning-disabled main-table evaluation. The Python cells use the same mathematical problems under original and neutral wording; this hosted evaluation includes no task-specific training.

Neutral wording broadens GPT’s correct MathIR solutions. Both GPT deployments yield at most one verified mode per Level-2 problem in 64 original-wording draws. Among problems with at least eight correct responses under both wordings, neutral wording increases mean breadth at

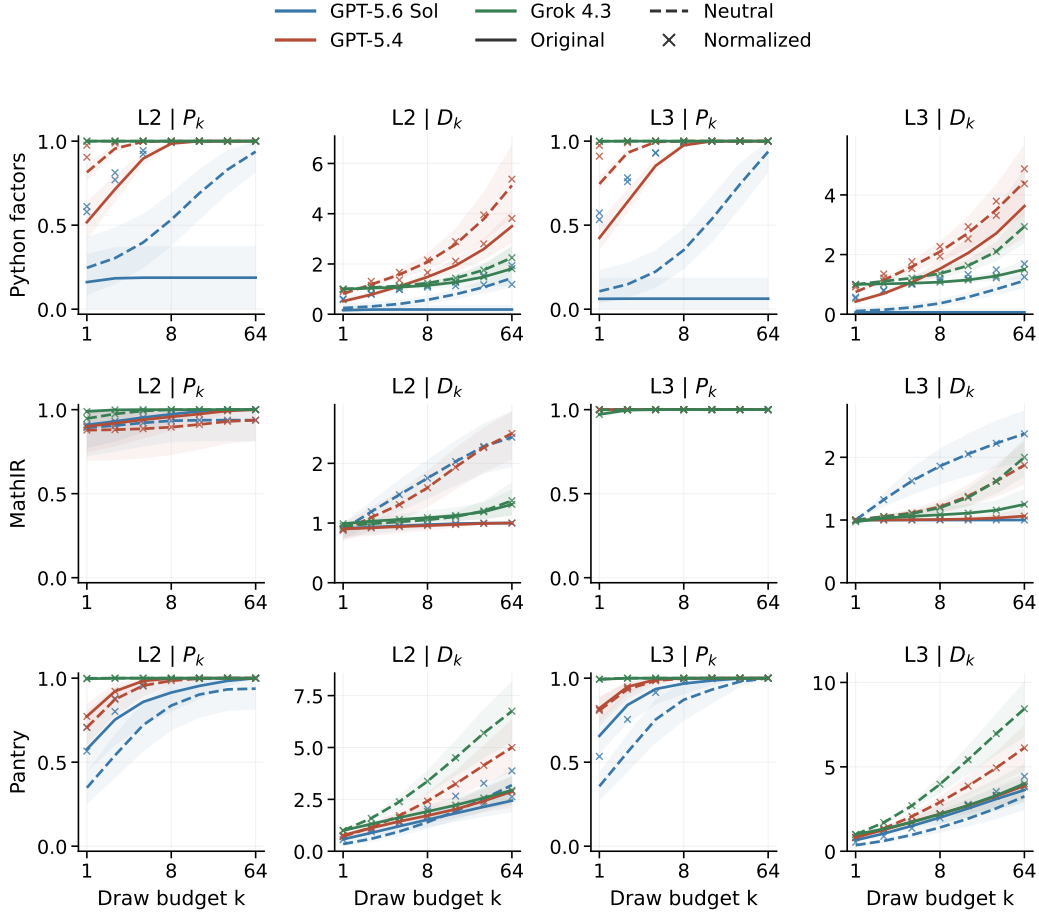


Figure 31: **Additional draws reveal solution modes beyond early success.** Hosted deployments; rows show Python, MathIR, and Pantry, with paired P_k and D_k columns for Levels 2 and 3. Curves show pass probability P_k and distinct verified modes D_k against draw budget k , obtained by rarefying sixteen 64-draw problem pools per cell. Colours identify deployments; solid/dashed lines denote original/neutral wording. Lines use strict verification, crosses formatting normalization, and shading pointwise 95% bootstrap intervals. Intervals resample paired problems within each cell and condition on the deployment.

eight correct draws by .794 ([.623, .990]) for GPT-5.4 and .872 ([.606, 1.123]) for GPT-5.6 Sol. The Level-3 effects are .204 ([.106, .320]) and .860 ([.563, 1.141]). These comparisons include fourteen and fifteen jointly eligible problems at Level 2, respectively, and sixteen for each GPT deployment at Level 3. Grok’s corresponding intervals include zero at both levels. The intervals resample the sixteen paired problems in each cell and are pointwise, with no replication across hosted model training seeds. The breadth contrasts describe jointly eligible problems and do not equate the wordings’ accuracy.

Pantry continues discovering modes after first success. Under neutral wording, GPT-5.4 gains 2.583 and 3.219 verified modes between eight and 64 draws at Levels 2 and 3; Grok gains 3.377 and 4.466. Grok’s empirical pass probability is already 1.0 at eight draws in both cells, so these gains consist entirely of additional modes. At Level 3 its distinct count rises from 3.972 to 8.438. On jointly eligible problems at eight correct draws, neutral minus original breadth is 1.781 ([1.226, 2.338]) for Grok and .934 ([.670, 1.184]) for GPT-5.4; GPT-5.6 Sol’s Level-3 contrast is inconclusive. These counts measure finite-sample discovery rather than exhaustive solution coverage.

Formatting limits GPT-5.6 Sol’s strict Python scores. GPT-5.6 Sol’s original-wording strict pass probability is approximately .1875 at Level 2 and .0625 at Level 3 from eight through 64 draws. Formatting normalization gives $P_{64} = 1.0$ under both wordings at both levels, with all four P_8 values above .989. The low strict scores therefore include failures to express valid so-

Table 25: **Pair weighting changes the summary of correct-solution diversity.** Strict 64-draw estimates for hosted deployments. Collision C weights prompts by their number of correct pairs; PCMD weights eligible prompts equally. Absolute estimates for each wording use their own eligible prompts, whereas Δ_{PCMD} uses only jointly eligible prompts. Thus Δ_{PCMD} need not equal the difference between the displayed levels, and E_O/E_N are separate eligibility counts. Each deployment is fixed; intervals resample paired problems.

Model	Cell	C_O	C_N	PCMD_O	PCMD_N	U_O^C	U_O^{PCMD}	E_O/E_N	Q_O/Q_N	Δ_{PCMD} [95%]
GPT-5.4	Python factors L2	0.790	0.665	0.257	0.341	0.500	0.500	16/16	10374/21720	+0.084 [-0.006, +0.186]
GPT-5.6 Sol	Python factors L2	1.000	0.974	0.000	0.184	0.500	0.500	3/13	4467/6074	+0.021 [+0.000, +0.032]
Grok 4.3	Python factors L2	0.963	0.938	0.037	0.062	0.500	0.500	16/16	32256/32256	+0.025 [+0.002, +0.048]
GPT-5.4	Python factors L3	0.732	0.620	0.334	0.384	0.500	0.500	16/16	6747/18043	+0.050 [-0.026, +0.128]
GPT-5.6 Sol	Python factors L3	1.000	0.966	0.000	0.130	0.500	0.500	1/10	1953/2097	+0.031 [+0.031, +0.031]
Grok 4.3	Python factors L3	0.981	0.893	0.019	0.107	0.500	0.500	16/16	32193/32256	+0.087 [+0.041, +0.154]
GPT-5.4	MathIR L2	1.000	0.756	0.000	0.228	0.200	0.800	16/15	28440/28227	+0.228 [+0.161, +0.302]
GPT-5.6 Sol	MathIR L2	1.000	0.678	0.000	0.302	0.200	0.800	16/15	28645/28377	+0.302 [+0.200, +0.402]
Grok 4.3	MathIR L2	0.968	0.985	0.033	0.014	0.200	0.800	16/16	31600/29677	-0.020 [-0.076, +0.017]
GPT-5.4	MathIR L3	0.998	0.944	0.002	0.056	0.200	0.800	16/16	32256/32256	+0.054 [+0.027, +0.087]
GPT-5.6 Sol	MathIR L3	1.000	0.676	0.000	0.324	0.200	0.800	16/16	32256/32256	+0.324 [+0.208, +0.432]
Grok 4.3	MathIR L3	0.966	0.949	0.033	0.051	0.200	0.800	16/16	30520/32193	+0.019 [-0.046, +0.064]
GPT-5.4	Pantry L2	0.680	0.499	0.252	0.483	0.069	0.934	16/16	20097/17436	+0.231 [+0.069, +0.395]
GPT-5.6 Sol	Pantry L2	0.674	0.624	0.275	0.474	0.076	0.934	16/15	12941/5001	+0.180 [+0.058, +0.327]
Grok 4.3	Pantry L2	0.684	0.412	0.315	0.589	0.066	0.934	16/16	32005/32193	+0.274 [+0.143, +0.411]
GPT-5.4	Pantry L3	0.546	0.392	0.431	0.610	0.065	0.937	16/16	22332/21975	+0.179 [+0.103, +0.257]
GPT-5.6 Sol	Pantry L3	0.566	0.652	0.422	0.441	0.061	0.937	16/16	15229/4946	+0.019 [-0.136, +0.172]
Grok 4.3	Pantry L3	0.673	0.294	0.330	0.703	0.063	0.937	16/16	31830/31881	+0.373 [+0.224, +0.523]

Table 26: **Discovery gains from eight to 64 draws.** Strict verification; O/N denotes original/neutral wording. Qwen2.5-0.5B-Instruct checkpoint means use five training seeds in Python and MathIR and two in Pantry; $s = 1$ denotes the single initial checkpoint. Five-seed intervals resample seeds and paired prompts; Pantry intervals condition on its two checkpoints. Formatting normalization gives the same values in every local cell.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	-0.138 [-0.217, -0.061]
Re:Dr ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	-0.497 [-0.766, -0.270]
Re:Dr ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	-0.004 [-0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	-0.109 [-0.383, +0.164]
Re:Dr ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [-0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	-0.008 [-0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [-0.123, +0.095]
Re:Dr ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	-0.008 [-0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [-0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [-0.201, +0.392]
Re:Dr ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [-0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [-0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [-0.109, +0.399]
Re:Dr ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	-0.107 [-0.321, +0.098]

lutions in the required format. The normalized neutral-minus-original increase in late mode discovery is .681 ([.351, 1.066]) at Level 2 and .370 ([.112, .674]) at Level 3. Normalization changes response formatting before the same verifier and canonical-key extraction.

K.3 ALL FIVE DOMAINS AND ALL THREE LEVELS AT 512 DRAWS

We evaluate GPT-5.6 Sol on thirty-two problems from each of five domains at Levels 1, 2, and 3, using the original prompt wording. Problems are selected by an outcome-independent SHA256 ranking (seed 20260911). Each problem receives 512 responses: 480 problems and 245,760 responses in total. Requests use medium reasoning, an 8,192-token output limit, and unspecified temperature and top- p , with model snapshot gpt-5.6-sol-2026-07-09. All problems and response slots are included; invalid, refused, and truncated answers remain in the sampling denominator. Transport retries do not count as additional responses.

Figure 5 uses formatting-normalized grading; Table 28 also gives strict-verifier results on the same response pools. In draw-index order, the first 64 responses solve 474 of 480 problems after normalization and 418 under strict grading. At 512 responses, these counts are 477 and 436, respectively.

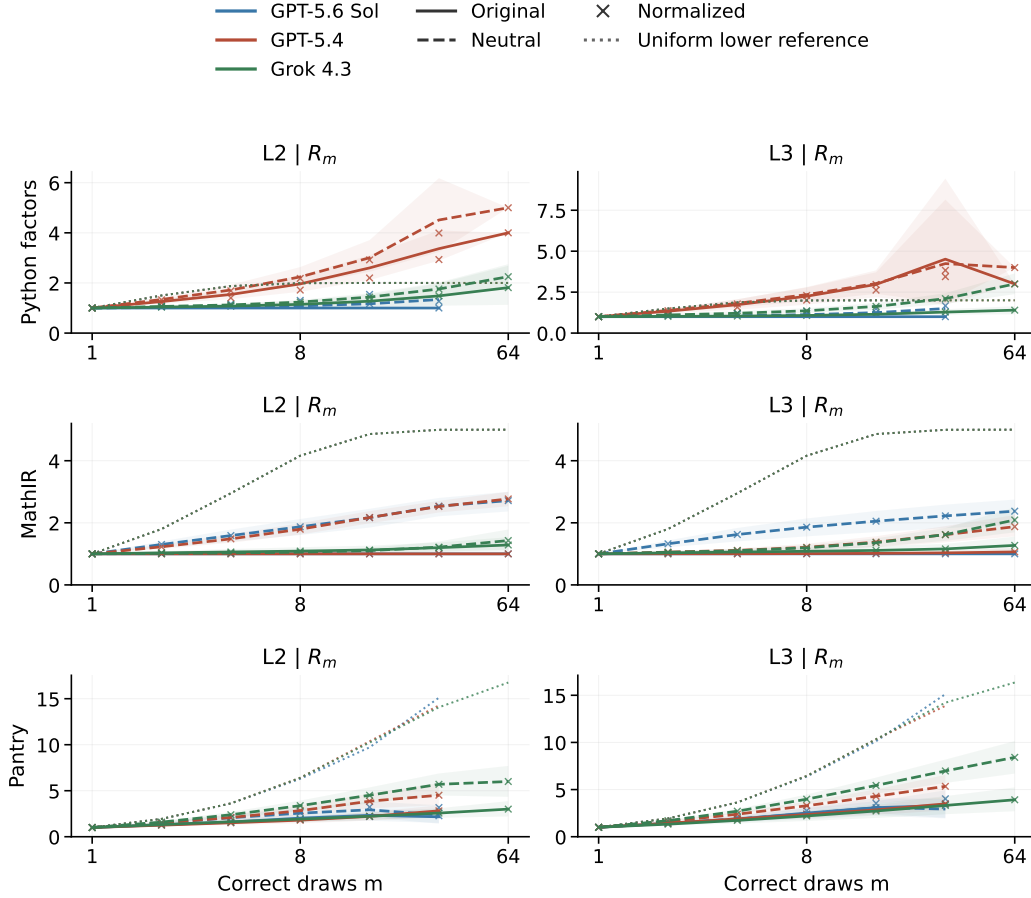


Figure 32: **Neutral wording broadens GPT solutions in MathIR at fixed correct-draw budgets.** Hosted deployments; rows show Python, MathIR, and Pantry, and columns Levels 2 and 3. Curves show R_m , the expected distinct verified modes in m correct draws, on prompts with at least m correct outputs under both wordings. Eligibility varies with m , method, and grading; gaps mark undefined means. Dotted lines give uniform lower references on prompts eligible under strict grading in both wordings, with the same prompt and checkpoint weighting as the strict curves. Colours identify deployments; solid/dashed lines denote original/neutral wording. Lines use strict verification, crosses formatting normalization, and shading pointwise 95% bootstrap intervals. Intervals resample paired problems within each cell and condition on the deployment.

For each pool of size $N = 512$, with c correct responses and verified-mode multiplicities n_j , the expected success indicator and number of distinct verified modes in a uniformly sampled k -subset are

$$P_N(k) = 1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}, \quad D_N(k) = \sum_j \left[1 - \frac{\binom{N-n_j}{k}}{\binom{N}{k}} \right],$$

respectively. We average all thirty-two problems equally within each domain and level, including problems with no correct response. Every curve uses the complete 512-response pools at budgets $k \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$. The rarefied estimate at $k = 64$ need not equal the result from the first 64 responses in draw-index order. The final-doubling contrast is $D_{512}(512) - D_{512}(256)$. Pointwise 95% intervals use 20,000 whole-problem bootstrap replicates within domain and level (seed 20260911); the same resampled problems form both sides of each contrast. Intervals do not provide simultaneous coverage across budgets or cells.

Graph support exhausts all fixed-color-compatible assignments. Countdown references enumerate binary expression trees; the validator also accepts unary negation and retains it in canonical keys, so these counts are lower bounds. The Python, MathIR, and PantryPlan references are also certified lower bounds. Discovered modes can exceed a lower bound without exhausting the available modes.

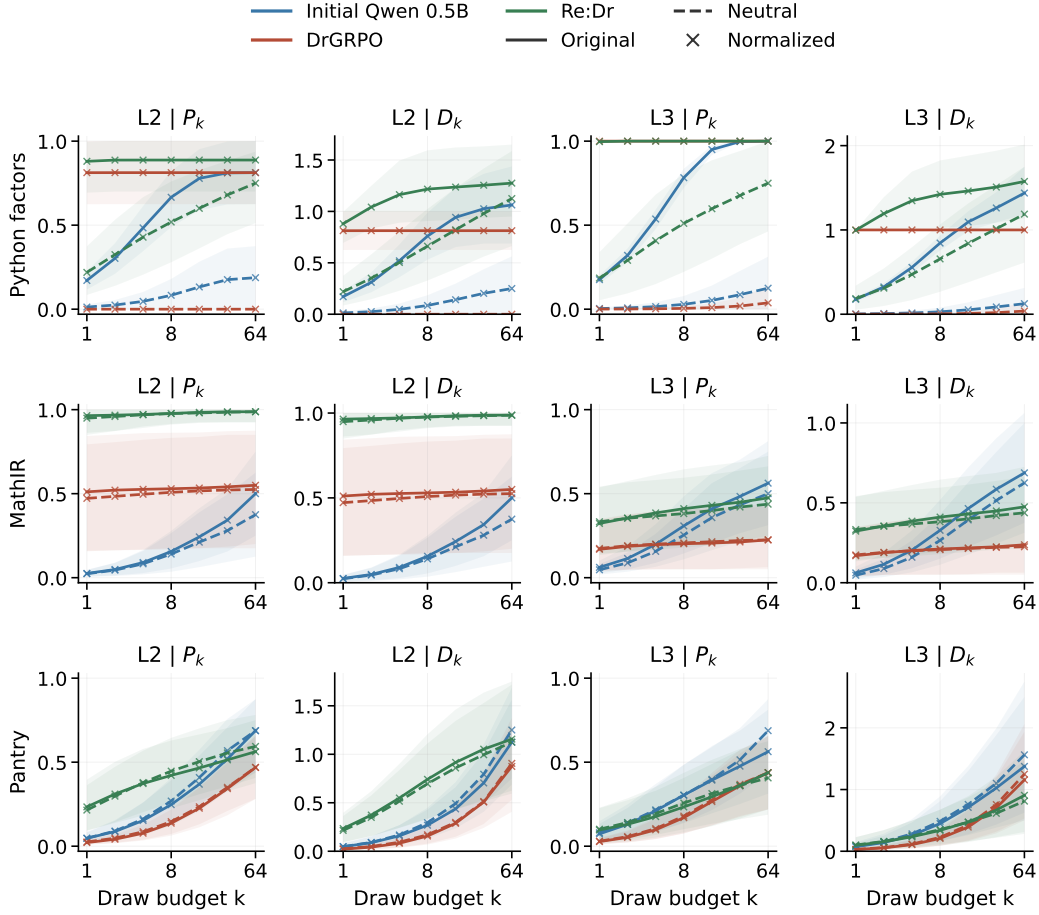


Figure 33: **Additional draws reveal few new MathIR modes after training.** Qwen2.5-0.5B-Instruct checkpoints; rows show Python, MathIR, and Pantry, with paired P_k and D_k columns for Levels 2 and 3. Curves show pass probability P_k and distinct verified modes D_k against draw budget k , obtained by rarefying sixteen 64-draw problem pools per cell. Colours identify methods; solid/dashed lines denote original/neutral wording. Lines use strict verification, crosses formatting normalization, and shading pointwise 95% bootstrap intervals. Means weight checkpoints equally: five training seeds in Python and MathIR, two in Pantry, and one initial checkpoint. Five-seed intervals resample seeds and paired problems; Pantry and initial intervals condition on the observed checkpoints.

Figure 5 divides the mean distinct-mode count by the mean enumerated mode count for the same thirty-two problems. This is a ratio of means, rather than the mean fraction recovered per problem; the denominator is held fixed when scaling interval endpoints. The independently certified references in Table 28 can be smaller than these enumerated counts. In Python, the table certifies at least two modes per problem, while the mean enumerated counts used in the figure are 222, 335.75, and 354.75 at Levels 1, 2, and 3.

These curves describe random subsets of the observed response pools. Interpreting them as predictions for fresh sampling at each budget also requires stable sampling behavior across requests; a common model snapshot and decoding settings do not ensure this. Small or zero gains at the largest budgets do not rule out rare unseen modes.

K.4 OBSERVED SAMPLING-BUDGET EFFECTS

The 64-draw comparison in App. K uses 36,864 hosted and 110,592 local responses, 147,456 in total. Each of the six hosted model-wording cohorts contributes 6,144 responses. The local panel contains 25 checkpoints: each trained checkpoint is evaluated in its own domain at Levels 2 and 3, and the initial checkpoint covers all six cells. Every response remains in its sampling denominator. Formatting

Table 27: **Pair weighting changes the summary of correct-solution diversity.** Strict 64-draw estimates for Qwen2.5-0.5B-Instruct checkpoints. Collision C weights prompts by their number of correct pairs; PCMD weights eligible prompts equally. Absolute estimates for each wording use their own eligible prompts, whereas Δ PCMD uses only jointly eligible prompts. Thus Δ PCMD need not equal the difference between the displayed levels, and E_O/E_N are separate eligibility counts. Checkpoint estimates receive equal weight; a seed with no eligible prompt makes the mean undefined.

Model	Cell	C_O	C_N	PCMD _O	PCMD _N	U_O^C	U_N^{PCMD}	E_O/E_N	Q_O/Q_N	Δ PCMD [95%]
DrGRPO	Python factors L2	1.000	--	0.000	--	0.500	0.500	65/0	130977/0	--
Initial Qwen 0.5B	Python factors L2	0.791	0.875	0.097	0.167	0.500	0.500	13/3	1270/24	-0.252 [-0.395, +0.019]
Re:Dr	Python factors L2	0.809	0.800	0.191	0.156	0.500	0.500	71/53	140722/18289	-0.037 [-0.139, +0.037]
DrGRPO	Python factors L3	1.000	--	0.000	--	0.500	0.500	80/0	161280/0	--
Initial Qwen 0.5B	Python factors L3	0.875	1.000	0.101	0.000	0.500	0.500	16/1	1010/3	-0.513 [-0.513, -0.513]
Re:Dr	Python factors L3	0.805	0.797	0.195	0.227	0.500	0.500	80/52	160275/12893	+0.030 [-0.023, +0.117]
DrGRPO	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	43/42	80766/74119	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	3/3	64/64	+0.000 [+0.000, +0.000]
Re:Dr	MathIR L2	1.000	1.000	0.000	0.000	0.200	0.800	79/79	155016/151485	+0.000 [+0.000, +0.000]
DrGRPO	MathIR L3	0.980	1.000	0.038	0.000	0.200	0.800	16/18	24457/25393	-0.038 [-0.202, +0.000]
Initial Qwen 0.5B	MathIR L3	0.883	0.912	0.128	0.162	0.200	0.800	7/6	282/181	+0.013 [-0.014, +0.052]
Re:Dr	MathIR L3	1.000	1.000	0.000	0.000	0.200	0.800	34/34	46891/50301	+0.000 [+0.000, +0.000]
DrGRPO	Pantry L2	0.374	0.276	0.733	0.780	0.047	0.953	9/10	101/193	+0.220 [-0.030, +0.470]
Initial Qwen 0.5B	Pantry L2	0.849	0.787	0.587	0.623	0.052	0.945	7/10	212/178	-0.078 [-0.300, +0.064]
Re:Dr	Pantry L2	0.600	0.613	0.343	0.218	0.051	0.942	16/18	10144/8510	-0.093 [-0.220, +0.007]
DrGRPO	Pantry L3	0.203	0.228	0.808	0.680	0.047	0.950	10/12	134/165	-0.156 [-0.359, +0.081]
Initial Qwen 0.5B	Pantry L3	0.375	0.313	0.307	0.341	0.049	0.945	7/6	531/705	-0.017 [-0.109, +0.091]
Re:Dr	Pantry L3	0.636	0.666	0.337	0.198	0.051	0.941	10/11	4006/3946	-0.112 [-0.319, +0.008]

Domain	Level	Known support (range)	D/P at 512		ΔD , 256 to 512 [95% CI]	
			Normalized	Strict	Normalized	Strict
Graph	1	6.97 (4–12)	2.531/100.0	2.531/100.0	0.222 [0.142, 0.305]	0.222 [0.142, 0.305]
Graph	2	6.03 (4–12)	2.625/100.0	2.625/100.0	0.268 [0.170, 0.373]	0.268 [0.170, 0.373]
Graph	3	6.66 (4–12)	1.719/100.0	1.719/100.0	0.137 [0.059, 0.228]	0.137 [0.059, 0.228]
Countdown	1	≥ 4.53 (2–6)	3.469/100.0	3.375/100.0	0.148 [0.070, 0.243]	0.164 [0.083, 0.259]
Countdown	2	≥ 4.40 (3–7)	3.156/100.0	3.094/100.0	0.218 [0.123, 0.320]	0.235 [0.136, 0.344]
Countdown	3	≥ 4.25 (2–8)	2.812/100.0	2.562/100.0	0.181 [0.090, 0.289]	0.191 [0.097, 0.297]
Python	1	≥ 2.00 (2)	3.750/100.0	2.875/100.0	0.592 [0.438, 0.750]	0.492 [0.334, 0.656]
Python	2	≥ 2.00 (2)	1.438/100.0	0.562/43.8	0.073 [0.020, 0.135]	0.148 [0.078, 0.227]
Python	3	≥ 2.00 (2)	1.344/100.0	0.281/28.1	0.013 [0.000, 0.032]	0.086 [0.031, 0.156]
MathIR	1	≥ 5.00 (5)	1.906/93.8	1.906/93.8	0.071 [0.012, 0.153]	0.071 [0.012, 0.153]
MathIR	2	≥ 5.00 (5)	1.000/100.0	1.000/100.0	0.002 [0.000, 0.006]	0.002 [0.000, 0.006]
MathIR	3	≥ 5.00 (5)	0.969/96.9	0.969/96.9	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
PantryPlan	1	≥ 16.90 (8–39)	6.906/100.0	6.906/100.0	0.622 [0.458, 0.799]	0.622 [0.458, 0.799]
PantryPlan	2	≥ 16.50 (8–39)	4.875/100.0	4.625/100.0	0.702 [0.476, 0.951]	0.702 [0.472, 0.962]
PantryPlan	3	≥ 16.87 (8–40)	5.562/100.0	5.250/100.0	0.719 [0.527, 0.914]	0.678 [0.489, 0.875]

Table 28: **Late mode discovery is largest in PantryPlan and Level-1 Python.** GPT-5.6 Sol with original wording and 512 responses for each of thirty-two problems per domain and level. D/P gives mean distinct verified modes and the percentage of problems solved at least once. Support gives each cell’s mean and range: exact for Graph and a certified lower bound elsewhere. Strict and normalized grading use identical pools. The paired contrast $\Delta D = D_{512}(512) - D_{512}(256)$ measures the gain between random subsets of the complete pool, not chronological new modes. Brackets give pointwise 95% whole-problem bootstrap intervals.

normalization adds no verified local responses, so the two grading rules give identical local estimates. These sixteen-problem cells are separate from the thirty-two-problem, 512-draw study in App. K.3.

Prompt weighting changes the diversity summary. Tables 25 and 27 compare two summaries of the full 64-response pools. Within each checkpoint or deployment, collision C pools correct pairs across prompts, weighting each prompt in proportion to $\binom{C}{2}$. Promptwise PCMD instead gives each prompt with at least two verified responses equal weight. Trained checkpoint scores then receive equal weight across seeds. Across 70 defined model–cell–wording entries, the median absolute difference between promptwise PCMD and $1 - C$ is .003, but nineteen entries differ by at least .05. For the initial checkpoint under original wording, Level-2 Pantry has $1 - C = .151$ and promptwise PCMD of .587; Level 3 has .625 and .307, respectively. Both entries use seven of sixteen problems. The direction of the difference depends on which prompts supply the most correct pairs.

Neither weighting holds overall accuracy or the eligible population fixed. For example, initial-checkpoint Level-3 Python has sixteen eligible problems under original wording and one under neutral wording. Absolute columns use each wording’s own eligible prompts, while

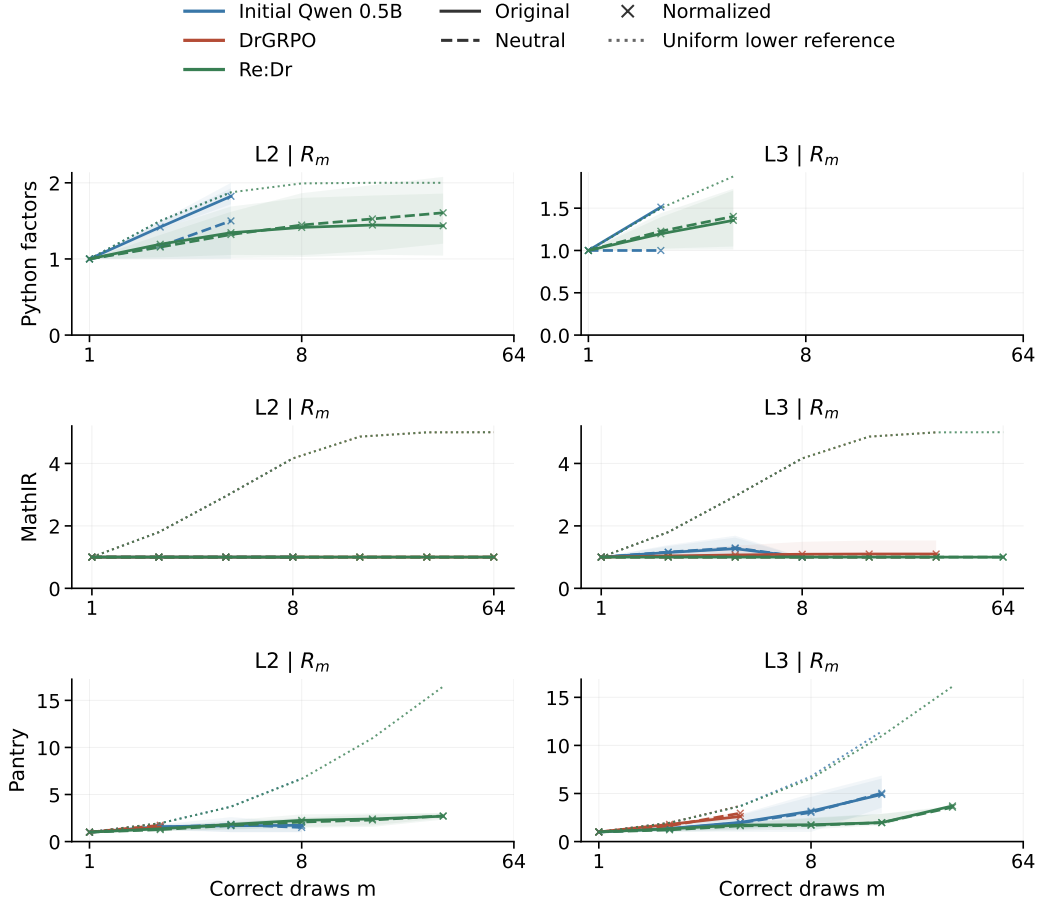


Figure 34: **MathIR correct solutions remain concentrated after training.** Qwen2.5-0.5B-Instruct checkpoints; rows show Python, MathIR, and Pantry, and columns Levels 2 and 3. Curves show R_m , the expected distinct verified modes in m correct draws, on prompts with at least m correct outputs under both wordings. Eligibility varies with m , method, and grading; gaps mark undefined means. Dotted lines give uniform lower references on prompts eligible under strict grading in both wordings, with the same prompt and checkpoint weighting as the strict curves. Colours identify methods; solid/dashed lines denote original/neutral wording. Lines use strict verification, crosses formatting normalization, and shading pointwise 95% bootstrap intervals. Means weight checkpoints equally: five training seeds in Python and MathIR, two in Pantry, and one initial checkpoint. Five-seed intervals resample seeds and paired problems; Pantry and initial intervals condition on the observed checkpoints.

paired differences use jointly eligible prompts. Equal prompt weights describe conditional diversity within these selected populations (App. A.1).

MathIR: concentration largely persists through 64 draws. At Level 2, both trained methods have $B_k = 0$ at every measured budget under both wordings. Re:Dr has original $P_8 = .9774$ and $P_{64} = .9875$; neutral wording gives $.9767$ and $.9875$. Its D_k equals P_k throughout, so the small late gain consists entirely of first successes. At exactly eight correct draws, both wordings have $R_8 = 1$ on the same 76 eligible checkpoint–problem groups, versus the certified-support uniform lower reference $5[1 - (4/5)^8] = 4.1611$. These groups reuse the same 16 problems across five checkpoints. At Level 3, all neutral trained groups and all original Re:Dr groups also contain at most one observed correct key. There is one exception in original DrGRPO: a single problem at seed 46 contains two keys with counts 28 and 9, giving mean $B_{64} = .0125$ and $B_{64} - B_8 = .0035$. Its equal-seed correct-pair collision is $.9803$; the other trained MathIR cells have collision 1.0, against the uniform upper reference $.2$. Thus the larger budget reveals a rare additional mode without supporting an exact single-mode claim for every trained condition.

Python: prompt dependence and late discovery coexist. Original DrGRPO has unchanged $P_k = D_k = .8125$ at Level 2 and 1.0 at Level 3 from eight through 64 draws. Neutral DrGRPO has zero correct responses out of 5,120 at Level 2 and three out of 5,120 at Level 3. Each of those three successes occurs alone in its checkpoint–problem group, so Level 3 mean $P_{64} = D_{64} = .0375$ while correct-pair collision remains undefined. Re:Dr behaves differently. Under original wording its pass probability is already unchanged from eight to 64, but D_k gains .0582 at Level 2 and .1529 at Level 3. Under neutral wording, P_k rises from .5186 to .7500 and from .5109 to .7500, respectively; D_k rises from .6618 to 1.1250 and from .6561 to 1.1875. These final mode counts remain below the original 1.2750 and 1.5750. The corresponding additional-mode gains are .2318 [.0574, .4565] and .2923 [.0659, .5445], with the seed-and-prompt intervals. Larger budgets therefore recover both first successes and additional verified alternatives in the neutral replay condition. These unconditional gains do not identify a change in the conditional distribution of correct solutions by themselves.

Pantry: eight draws understate observed discovery. Across the trained Pantry cells and both wordings, $D_{64} - D_8$ ranges from .4151 to 1.0247, with additional-mode gains ranging from .2739 to .7664. The initial checkpoint also gains .8565–1.0778 modes. In the two observed training seeds, Level 3 point estimates change method ordering: original DrGRPO/Re:Dr have $D_8 = .2075/.3381$ but $D_{64} = 1.1563/.9063$; neutral wording gives .2253/.3515 at eight and 1.2500/.8125 at 64. This is a descriptive ranking of these checkpoints, not evidence of a population-level method advantage. Pantry has only two training seeds, and fixed-correct-draw comparisons at larger m often have few or no jointly eligible problems. These comparisons condition on observed success, and an equal-seed mean is undefined when either seed has no eligible problem.

Several trained conditions remain concentrated at 64 draws, while others continue discovering solution modes. Even a flat finite-pool curve leaves open the possibility of rare unseen modes. Here the $k = 8$ points are rarefaction estimates from sixteen 64-response pools per cell; App. I uses a separate eight-draw evaluation on thirty-two problems per cell. The trained checkpoints use Level-2 training, so their Level-3 results measure transfer. Hosted results concern fixed deployments.

L INFERENCE: ADAPTATION, MODEL MIXTURES, AND COARSER KEYS

We evaluate adaptation to changed requirements, complementarity between models, and sensitivity to the definition of a solution mode. Model weights remain fixed during inference. Pointwise 95% percentile intervals use 20,000 paired problem-bootstrap replicates (seed 20260911). Each resampled problem carries all its perturbations, model conditions, and training seeds together; hosted comparisons resample within each domain–level cell. These intervals condition on the observed response pools and fixed training seeds. They do not estimate variability across new response pools or training seeds, and the 21 model pairs are dependent comparisons without multiplicity adjustment.

L.1 PANTRY PORTFOLIOS UNDER CHANGED REQUIREMENTS

We evaluate the initial Qwen2.5-0.5B checkpoint and Level-2 Dr.GRPO and Re:Dr checkpoints at fixed seeds 43 and 46 on 32 held-out problems. All 192 single-ingredient outages come from task specifications independently of generated answers; exhaustive search of the original quantity grid finds 181 feasible revised tasks. Verified feasibility witnesses are withheld from the model. The five feasible additional dietary restrictions are reported separately because only five problem clusters qualify.

Each portfolio contains eight plans. We test these plans unchanged against the revised requirements, then allow at most eight sequential recovery calls if none survives, stopping at the first valid plan. Unresolved portfolios consume all eight calls. The common recovery interface shows the revised task, initial portfolio, previous recovery outputs and deterministic verifier feedback. Cumulative success includes portfolios that needed no recovery. We average perturbations within each source problem, then weight problems equally.

The three sampling strategies use the same checkpoints, verifier, and 192-token response cap. Ordinary sampling uses temperature 1.0. Tuned temperature is chosen from $\{0.7, 1.0, 1.3\}$ by outage survival on 16 separate development problems (384 responses per checkpoint). Diversity prompting makes eight sequential calls, requesting a new support while retaining the

complete output history, including failures. These strategies match initial response counts; their context lengths and total token costs differ.

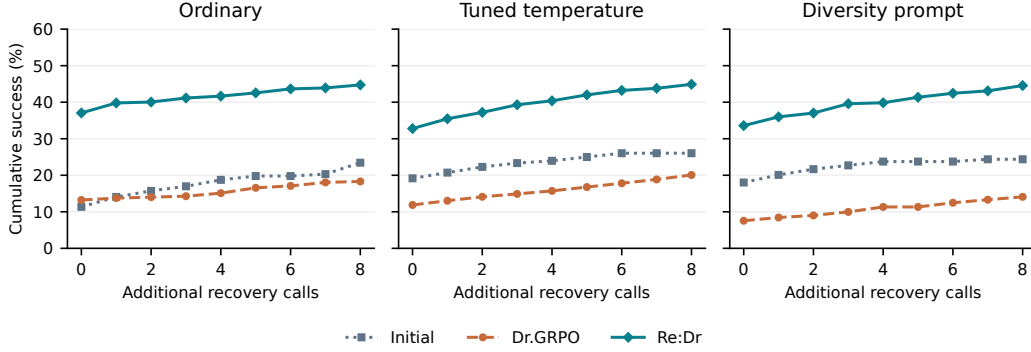


Figure 35: **Re:Dr improves portfolio survival and success within eight recovery calls.** Panels compare ordinary sampling, tuned temperature, and diversity prompting on the same 181 feasible ingredient outages from 32 Pantry problems. The horizontal axis is the additional recovery-call budget; the vertical axis includes success without recovery at zero calls. Curves average outages within problems, then problems equally. Solid diamonds show Re:Dr, dashed circles Dr.GRPO, and dotted squares the initial checkpoint; trained curves average fixed seeds 43 and 46. Pointwise 95% intervals for paired differences appear in Table 30.

Table 29: **Re:Dr portfolios adapt better to ingredient outages across all three sampling strategies.** Eight initial plans face 181 feasible outages from 32 problems or five additional dietary restrictions from five problems. Survival is success without recovery; By 8 includes success within eight recovery calls (both percentages). Calls count unresolved cases as eight. Token totals charge the initial portfolio once per revised task. All entries average perturbations within problems, then problems equally; dietary estimates use only five problems.

Change	Checkpoint	Strategy	Survival	By 8	Calls	Input tokens	Output tokens
Outages	Initial	Ordinary	11.35	23.44	6.63	11301.75	376.49
	Initial	Tuned temperature	19.17	26.04	6.13	10864.88	370.17
	Initial	Diversity prompt	18.02	24.38	6.22	11794.81	352.87
	Dr.GRPO 43	Ordinary	9.79	14.01	7.09	12439.22	823.70
	Dr.GRPO 43	Tuned temperature	13.44	16.77	6.79	12017.57	775.44
	Dr.GRPO 43	Diversity prompt	5.21	8.39	7.44	14103.23	848.09
	Dr.GRPO 46	Ordinary	16.67	22.60	6.47	11265.39	637.59
	Dr.GRPO 46	Tuned temperature	10.31	23.39	6.75	11531.40	644.94
	Dr.GRPO 46	Diversity prompt	9.90	19.79	6.90	13124.10	731.00
	Re:Dr 43	Ordinary	39.38	47.97	4.47	9078.92	297.98
	Re:Dr 43	Tuned temperature	28.23	40.94	5.19	9718.01	310.50
	Re:Dr 43	Diversity prompt	36.93	47.19	4.68	10465.22	344.02
	Re:Dr 46	Ordinary	34.74	41.51	4.94	9547.16	308.52
	Re:Dr 46	Tuned temperature	37.34	48.85	4.53	9148.77	300.05
	Re:Dr 46	Diversity prompt	30.26	41.93	5.06	10584.02	313.50
Dietary	Initial	Ordinary	0.00	20.00	6.80	11392.80	371.20
	Initial	Tuned temperature	20.00	40.00	6.40	11175.20	387.20
	Initial	Diversity prompt	0.00	0.00	8.00	13630.20	405.00
	Dr.GRPO 43	Ordinary	0.00	0.00	8.00	13310.20	849.20
	Dr.GRPO 43	Tuned temperature	0.00	0.00	8.00	13394.60	856.00
	Dr.GRPO 43	Diversity prompt	0.00	0.00	8.00	15045.40	939.60
	Dr.GRPO 46	Ordinary	0.00	20.00	7.40	12063.00	650.40
	Dr.GRPO 46	Tuned temperature	0.00	0.00	8.00	12544.20	615.40
	Dr.GRPO 46	Diversity prompt	0.00	0.00	8.00	14132.40	725.60
	Re:Dr 43	Ordinary	20.00	60.00	5.20	9732.60	326.00
	Re:Dr 43	Tuned temperature	40.00	40.00	4.80	9402.80	307.60
	Re:Dr 43	Diversity prompt	40.00	40.00	4.80	10693.60	355.60
	Re:Dr 46	Ordinary	40.00	80.00	3.20	7853.20	271.20
	Re:Dr 46	Tuned temperature	40.00	80.00	3.60	8127.80	265.20
	Re:Dr 46	Diversity prompt	20.00	40.00	5.00	10738.20	341.00

With ordinary sampling, Re:Dr improves survival over Dr.GRPO by 23.83 percentage points [12.16, 35.89], reduces capped recovery burden by 2.08 calls [1.14, 3.03], and improves success within eight extra calls by 26.43 points [14.69, 38.18]. Mean input and output token totals fall by 2,539.27 [1, 618.01, 3, 468.86] and 427.40 [386.83, 466.30], respectively. However, original correctness also increases by 1.234 correct responses out of eight [.578, 1.984]. Restricting each

Table 30: **Re:Dr raises outage survival and reduces recovery calls.** Entries are Re:Dr minus Dr.GRPO, averaged over fixed seeds 43 and 46 on 32 paired problems. Survival and success by eight calls are percentage-point differences; calls include unresolved cases. Brackets are pointwise 95% paired problem-bootstrap intervals.

Strategy	Δ survival	Δ by 8 calls	Δ calls
Ordinary	23.83 [12.16, 35.89]	26.43 [14.69, 38.18]	-2.08 [-3.03, -1.14]
Tuned temperature	20.91 [9.19, 32.89]	24.82 [14.04, 35.52]	-1.91 [-2.78, -1.05]
Diversity prompt	26.04 [16.93, 35.65]	30.47 [21.51, 39.27]	-2.30 [-3.04, -1.57]

seed comparison to equal observed correct counts leaves a 1.05-point survival gain [0.00, 3.53]: within each problem we average its eligible seed comparisons, then average the 19 problems with at least one eligible comparison. This descriptive subset changes both problem and seed composition; equal observed counts do not establish equal underlying accuracy. The adaptation gains therefore do not isolate diversity as their causal mechanism.

Per-perturbation token totals represent deploying one portfolio and recovering from one change in requirements. Input counts include reused prefixes and do not measure the compute saved by prefix caching. Table 31 counts each response once across calibration, the two controlled portfolio strategies, and recovery for all three strategies. It excludes ordinary initial-portfolio generation, whose timing is unavailable. Training costs are outside both inference cost summaries.

Table 31: **Inference costs include temperature selection and recovery.** Each checkpoint selects T using 384 responses on 16 development problems. Inference totals include this calibration, initial portfolios for tuned-temperature and diversity prompting, and recovery under all three strategies on 32 evaluation problems. Ordinary initial-portfolio generation is excluded. Each response is counted once; input counts include reused prefixes. Seconds measure generation time, exclude training, and are not hardware-normalized compute.

Checkpoint	Scope	T	Responses	Input tokens	Output tokens	Seconds
Dr.GRPO 43	Calibration	1.3	384	229392	19871	136.3
Dr.GRPO 43	Inference total	1.3	4874	4870884	272484	3302.0
Dr.GRPO 46	Calibration	1.3	384	229392	15544	126.5
Dr.GRPO 46	Inference total	1.3	4634	4365524	220458	2792.9
Re:Dr 43	Calibration	0.7	384	229392	8552	111.1
Re:Dr 43	Inference total	0.7	3559	3160284	92227	1527.7
Re:Dr 46	Calibration	1.0	384	229392	8585	112.5
Re:Dr 46	Inference total	1.0	3592	3169337	88300	1510.2
Initial	Calibration	1.3	384	229392	9521	123.5
Initial	Inference total	1.3	4416	4035069	116619	1957.4

L.2 COMPLEMENTARITY ACROSS HOSTED MODELS

Seven hosted models each produce eight responses to the same 1,920 prompts (five domains, three levels, 128 prompts per cell), totaling 107,520 responses. Grading is strict, with the original Opus 5 Python interface. These conditions differ from the normalized hosted comparison with the revised Opus 5 interface. For each of the 21 model pairs, a mixed portfolio draws four responses without replacement from each model’s eight-response pool. We compute its expected distinct-mode count exactly over all $\binom{8}{4}^2 = 4,900$ subset pairs, then compare with each constituent’s complete eight-response portfolio and their average. These are expectations within the finite response pools, conditional on the outputs generated.

Every pair increases expected distinct solution modes over the average constituent by .115–.266. Ten pairs improve over both constituents in point estimates; nine have positive pointwise intervals against both. On prompts with at least four correct responses from each model, we also compare a mixture of two correct draws per model with four correct draws from either constituent. All pairwise gains over the average constituent remain positive (.073–.177 modes). This fixes the number of correct draws used to measure diversity; it conditions on a different eligible subset for each pair and does not match accuracy or the number of attempts needed to obtain them.

Cross-model collision is .624–.740 versus .704–.819 for mean within-model collision, averaging prompts with ≥ 2 correct draws per model equally per pair. Absence of observed overlap does not establish disjoint supports. Average-constituent gains persist with coarser keys and formatting-normalized grading. Equal response counts leave provider tokens, compute, costs, and reasoning interfaces unmatched; these finite-pool comparisons do not identify causal training mechanisms.

Table 32: **Every model pair increases expected distinct modes over its average constituent.** A 4+4 mixture samples without replacement from the two eight-response pools; A and B name the constituents in order. Panel A compares with their average: fine and coarse keys use all 1,920 prompts, while the four-correct-draw comparison uses n prompts with at least four correct responses per model (2+2 mixed versus four per constituent). Panel B compares fine-key distinct modes with each constituent and pass@8 with their average. Entries are distinct-mode differences except pass@8, in percentage points. Brackets are pointwise 95% paired problem-bootstrap intervals, stratified by domain and level; comparisons share models and prompts.

A									
Pair		Fine		Coarse		Four correct draws		n	
DeepSeek V4 Pro + Kimi K3	0.148	[0.136, 0.160]	0.142	[0.131, 0.154]	0.107	[0.098, 0.115]		1858	
DeepSeek V4 Pro + Opus 4.8	0.198	[0.185, 0.211]	0.183	[0.171, 0.197]	0.153	[0.142, 0.164]		1847	
DeepSeek V4 Pro + Opus 5	0.262	[0.249, 0.275]	0.252	[0.240, 0.265]	0.177	[0.165, 0.190]		1510	
DeepSeek V4 Pro + GPT-5.4	0.133	[0.122, 0.143]	0.127	[0.116, 0.138]	0.098	[0.090, 0.107]		1695	
DeepSeek V4 Pro + GPT-5.6 Sol	0.235	[0.223, 0.247]	0.224	[0.212, 0.237]	0.154	[0.142, 0.166]		1396	
DeepSeek V4 Pro + Grok 4.3	0.123	[0.112, 0.134]	0.117	[0.106, 0.127]	0.091	[0.083, 0.099]		1859	
Kimi K3 + Opus 4.8	0.187	[0.174, 0.200]	0.172	[0.159, 0.185]	0.138	[0.127, 0.148]		1885	
Kimi K3 + Opus 5	0.266	[0.253, 0.279]	0.255	[0.242, 0.268]	0.162	[0.150, 0.175]		1543	
Kimi K3 + GPT-5.4	0.127	[0.115, 0.139]	0.112	[0.101, 0.124]	0.083	[0.075, 0.092]		1720	
Kimi K3 + GPT-5.6 Sol	0.202	[0.190, 0.214]	0.195	[0.183, 0.207]	0.093	[0.083, 0.104]		1417	
Kimi K3 + Grok 4.3	0.115	[0.104, 0.126]	0.110	[0.099, 0.121]	0.073	[0.066, 0.080]		1900	
Opus 4.8 + Opus 5	0.187	[0.176, 0.199]	0.182	[0.171, 0.193]	0.098	[0.087, 0.109]		1534	
Opus 4.8 + GPT-5.4	0.201	[0.186, 0.215]	0.185	[0.172, 0.199]	0.156	[0.144, 0.168]		1713	
Opus 4.8 + GPT-5.6 Sol	0.247	[0.234, 0.261]	0.237	[0.223, 0.250]	0.151	[0.138, 0.164]		1409	
Opus 4.8 + Grok 4.3	0.180	[0.167, 0.194]	0.171	[0.158, 0.184]	0.141	[0.130, 0.153]		1894	
Opus 5 + GPT-5.4	0.228	[0.215, 0.241]	0.217	[0.204, 0.230]	0.158	[0.145, 0.171]		1493	
Opus 5 + GPT-5.6 Sol	0.169	[0.157, 0.182]	0.161	[0.149, 0.174]	0.143	[0.130, 0.156]		1370	
Opus 5 + Grok 4.3	0.231	[0.219, 0.244]	0.225	[0.213, 0.237]	0.145	[0.133, 0.157]		1547	
GPT-5.4 + GPT-5.6 Sol	0.156	[0.145, 0.167]	0.147	[0.136, 0.158]	0.094	[0.084, 0.104]		1400	
GPT-5.4 + Grok 4.3	0.130	[0.119, 0.142]	0.121	[0.111, 0.132]	0.095	[0.086, 0.104]		1722	
GPT-5.6 Sol + Grok 4.3	0.211	[0.199, 0.224]	0.204	[0.192, 0.216]	0.118	[0.107, 0.130]		1418	
B									
Pair	ΔD vs A		ΔD vs B		Δ pass@8 vs mean				
DeepSeek V4 Pro + Kimi K3	0.279	[0.254, 0.304]	0.017	[-0.008, 0.043]	0.22	[0.08, 0.39]			
DeepSeek V4 Pro + Opus 4.8	0.089	[0.064, 0.114]	0.307	[0.282, 0.332]	0.31	[0.15, 0.49]			
DeepSeek V4 Pro + Opus 5	-0.061	[-0.082, -0.039]	0.585	[0.560, 0.610]	7.48	[7.19, 7.78]			
DeepSeek V4 Pro + GPT-5.4	0.097	[0.074, 0.121]	0.168	[0.145, 0.192]	0.64	[0.41, 0.90]			
DeepSeek V4 Pro + GPT-5.6 Sol	-0.003	[-0.025, 0.019]	0.473	[0.447, 0.500]	7.16	[6.71, 7.62]			
DeepSeek V4 Pro + Grok 4.3	0.059	[0.037, 0.082]	0.186	[0.163, 0.210]	0.34	[0.17, 0.52]			
Kimi K3 + Opus 4.8	-0.053	[-0.080, -0.027]	0.427	[0.401, 0.452]	0.19	[0.07, 0.32]			
Kimi K3 + Opus 5	-0.188	[-0.212, -0.165]	0.719	[0.692, 0.747]	7.34	[7.08, 7.61]			
Kimi K3 + GPT-5.4	-0.040	[-0.064, -0.015]	0.293	[0.267, 0.320]	0.73	[0.48, 1.00]			
Kimi K3 + GPT-5.6 Sol	-0.167	[-0.191, -0.144]	0.571	[0.543, 0.600]	7.38	[6.92, 7.85]			
Kimi K3 + Grok 4.3	-0.080	[-0.104, -0.056]	0.309	[0.284, 0.335]	0.12	[0.03, 0.23]			
Opus 4.8 + Opus 5	-0.027	[-0.045, -0.009]	0.401	[0.379, 0.423]	7.42	[7.14, 7.71]			
Opus 4.8 + GPT-5.4	0.274	[0.248, 0.300]	0.127	[0.101, 0.153]	0.93	[0.65, 1.24]			
Opus 4.8 + GPT-5.6 Sol	0.118	[0.096, 0.141]	0.377	[0.350, 0.403]	7.51	[7.03, 7.99]			
Opus 4.8 + Grok 4.3	0.226	[0.202, 0.250]	0.135	[0.110, 0.160]	0.11	[0.02, 0.23]			
Opus 5 + GPT-5.4	0.515	[0.489, 0.541]	-0.060	[-0.082, -0.037]	6.30	[5.90, 6.71]			
Opus 5 + GPT-5.6 Sol	0.254	[0.232, 0.277]	0.084	[0.063, 0.106]	2.11	[1.66, 2.57]			
Opus 5 + Grok 4.3	0.490	[0.467, 0.514]	-0.028	[-0.048, -0.007]	7.41	[7.13, 7.69]			
GPT-5.4 + GPT-5.6 Sol	-0.046	[-0.067, -0.025]	0.359	[0.335, 0.383]	5.61	[5.15, 6.08]			
GPT-5.4 + Grok 4.3	0.102	[0.078, 0.127]	0.159	[0.135, 0.182]	0.93	[0.65, 1.23]			
GPT-5.6 Sol + Grok 4.3	0.386	[0.360, 0.412]	0.037	[0.015, 0.058]	7.55	[7.08, 8.04]			

L.3 SENSITIVITY TO TASK-RELEVANT EQUIVALENCES

Graph groups global color renamings that fix every anchored color; vertex identities remain fixed. Python groups opposite members d and n/d of the same unordered proper factor pair, retaining input-vector order. Countdown flattens associative additions and multiplications and sorts their children, preserving operands, multiplicity, and subtraction/division boundaries. MathIR keeps its simplified state trajectory. Pantry keeps ingredient supports, whose differences can matter under outages. The same domain-specific map applies to each model’s correct responses. Correctness is unchanged; deterministic merging cannot increase the observed distinct-mode count or reduce correct-pair collision.

Uniform mass over fine keys generally induces unequal mass across coarse groups. For Graph and Python, exhaustive enumeration determines both the number of coarse groups and their sizes, allowing separate collision references for uniform fine-key mass and uniform coarse-group mass. Countdown’s full verifier support is unknown, so neither uniform reference has a numerical value.

Table 33: **Coarser keys reduce observed mode counts and increase collision.** Strict grading uses eight responses per hosted model on 1,920 prompts. $D@8$ averages all prompts; collision averages prompts with at least two correct responses, with eligibility varying by model. Brackets are pointwise 95% paired problem-bootstrap intervals, stratified by domain and level.

Model	Fine $D@8$		Coarse $D@8$		Fine collision		Coarse collision	
DeepSeek V4 Pro	1.826	[1.792, 1.859]	1.762	[1.729, 1.795]	0.720	[0.710, 0.731]	0.739	[0.729, 0.750]
Kimi K3	2.087	[2.047, 2.129]	1.952	[1.913, 1.992]	0.688	[0.678, 0.699]	0.723	[0.712, 0.733]
Opus 4.8	1.608	[1.575, 1.641]	1.556	[1.525, 1.588]	0.795	[0.785, 0.805]	0.811	[0.801, 0.821]
Opus 5	1.180	[1.155, 1.206]	1.152	[1.128, 1.177]	0.856	[0.845, 0.866]	0.867	[0.857, 0.877]
GPT-5.4	1.755	[1.718, 1.792]	1.679	[1.644, 1.715]	0.729	[0.717, 0.741]	0.750	[0.739, 0.762]
GPT-5.6 Sol	1.349	[1.317, 1.382]	1.312	[1.281, 1.345]	0.778	[0.765, 0.790]	0.794	[0.782, 0.806]
Grok 4.3	1.698	[1.665, 1.732]	1.645	[1.613, 1.679]	0.791	[0.782, 0.800]	0.807	[0.798, 0.816]

Table 34: **Python diversity gains depend on the factor-pair equivalence and prompt wording.** Entries are Re:Dr minus Dr:GRPO in distinct modes from eight responses, under strict grading. Level-2 trained Qwen2.5-0.5B endpoints are evaluated on 32 paired problems at each level and averaged over the displayed fixed seeds. Brackets are pointwise 95% paired problem-bootstrap intervals, conditional on those checkpoints. MathIR and Pantry use identical fine and coarse keys.

Domain	Level	Wording	Seeds	Fine $\Delta D@8$		Coarse $\Delta D@8$	
Python	2	original	5	0.431	[0.406, 0.456]	0.075	[0.025, 0.138]
MathIR	2	original	5	0.463	[0.387, 0.531]	0.463	[0.387, 0.531]
Pantry	2	original	2	0.469	[0.203, 0.750]	0.469	[0.203, 0.750]
Python	3	original	5	0.419	[0.400, 0.444]	0.000	[0.000, 0.000]
MathIR	3	original	5	0.119	[0.019, 0.225]	0.119	[0.019, 0.225]
Pantry	3	original	2	0.125	[−0.109, 0.375]	0.125	[−0.109, 0.375]
Python	2	neutral	5	0.675	[0.519, 0.838]	0.556	[0.438, 0.675]
MathIR	2	neutral	5	0.469	[0.375, 0.562]	0.469	[0.375, 0.562]
Pantry	2	neutral	2	0.484	[0.234, 0.766]	0.484	[0.234, 0.766]
Python	3	neutral	5	0.656	[0.537, 0.775]	0.500	[0.419, 0.575]
MathIR	3	neutral	5	0.119	[0.025, 0.213]	0.119	[0.025, 0.213]
Pantry	3	neutral	2	0.250	[−0.031, 0.562]	0.250	[−0.031, 0.562]

The original-wording Python Level-3 gain falls from .419 to zero; Level 2 falls from .431 to .075. Neutral-wording coarse gains remain .556 at Level 2 and .500 at Level 3. Thus the original Python effect can reflect opposite members of the same factor pair, and the conclusion depends on wording and the meaning of output identity. Neither divisor vectors nor factor-pair vectors identify different algorithms.

M PORTFOLIO SURVIVAL UNDER WITHDRAWN OPTIONS

A portfolio can remain useful when a solved task loses an option. We evaluate this survival across all five domains in 474 terminal checkpoint evaluations, each with 128 prompts and four groups of eight responses per prompt. The groups have overlapping sampling streams (App. H.1). Replay’s largest gains in expected distinct modes at four verified draws occur in Graph, Countdown, Python and Pantry. Across 38 matched domain–scale–level–objective comparisons, PCMD gains correlate with survival gains at four verified draws (Pearson $r = .57$).

One withdrawn option. Each task loses one option: a colour at one vertex (Graph), an arithmetic operation (Countdown), a returned divisor value (Python), a menu action (MathIR), or an ingredient (Pantry; see App. L.1). Options are enumerated from the task specification. The solution-mode

enumeration matches the certified counts for all 1,280 prompt rows. A surviving certified mode establishes feasibility; absence from this enumeration does not establish infeasibility when the certificate is a lower bound on the verifier’s full support. Of 9,864 options, 9,118 retain at least one certified mode. Among these, 8,308 are binding: they also remove at least one certified mode. We evaluate both the binding set and the full set with a feasibility certificate.

What counts as surviving. Each portfolio contains the verified draws among eight responses. Survival means that at least one retained solution mode remains valid. Graph, Countdown and Python determine this directly from the outcome key, including verified keys outside the enumeration. Countdown contributes 2 such keys, observed 50 times among the cohort’s 846,398 verified draws: the verifier accepts unary negation, which the expression enumeration omits. For MathIR, re-enumerating the reduced menu tests whether the same executed state path remains realizable, possibly through a different action sequence. Pantry checks the certified witness plan for each ingredient-support key. All observed MathIR and Pantry keys belong to their respective enumerations.

Survival conditional on correct draws. Raw survival depends on both correctness and the distribution of valid modes. At budgets $m = 1, 2, 4$, we compute the exact expected survival of a uniformly selected m -element subset of a portfolio’s verified draws, as in App. L.2. Replay-control contrasts use only prompt-group pairs with at least m verified draws in both arms; the eligible population can change with m . This fixes the number of correct draws, rather than policy accuracy. Differences can still reflect individual modes’ resistance to a withdrawal; an increasing gain with m alone does not isolate diversity. Expected distinct modes at the same budget provide a separate measure of portfolio breadth.

Table 35: **Binding withdrawals remove certified modes while retaining a feasibility certificate.** Each row covers 128 prompts. Support is the mean certified mode count per prompt; Options counts single withdrawals. Binding counts options that retain at least one certified mode and remove at least one. Kept is the mean retained fraction of certified modes across binding options. Pairs counts binding pairs of withdrawals, and Kept₂ gives the corresponding retained fraction.

Domain	Level	Prompts	Support	Options	Binding	Kept	Pairs	Kept ₂
Graph	1	128	6.3	1152	818	0.57	1957	0.34
Graph	2	128	6.1	1536	1068	0.57	3457	0.35
Countdown	1	128	4.5	512	147	0.48	21	0.44
Countdown	2	128	4.4	512	185	0.49	40	0.37
Python	1	128	229.4	1433	1433	0.68	7716	0.50
Python	2	128	251.7	1647	1647	0.71	10381	0.55
MathIR	1	128	5.0	768	448	0.40	384	0.23
MathIR	2	128	5.0	768	512	0.45	512	0.25
Pantry	1	128	18.2	768	651	0.44	1231	0.18
Pantry	2	128	18.2	768	653	0.45	1228	0.19

Table 36: **Replay improves survival in Graph, Python and Pantry at four verified draws.** Each replay arm is compared with its fresh objective. Survival differences are percentage points; ΔD_4 is the difference in expected distinct modes at four verified draws. Raw uses all eight responses; Budget columns use the indicated number of verified draws. Fixed-correct-draw results pool jointly eligible prompt-group pairs across matched seeds, scales and levels; raw results use all paired groups. Binding options are used except in the all column, which includes every option with a feasibility certificate. Brackets are pointwise 95% intervals from 20,000 paired bootstrap replicates (seed 20260917). Each resampled cluster is a prompt within one level and scale, with all its groups and training seeds kept together; copies at different scales are separate clusters. Intervals condition on the observed training seeds.

Domain	Arm	Raw	Budget 1	Budget 2	Budget 4	Budget 4 (all)	ΔD_4
Graph	Re:Dr	34.78	0.00 [0.00, 0.00]	10.51 [8.76, 12.31]	19.65 [15.61, 23.82]	15.66	1.290
Graph	Re:Max	18.98	0.00 [-0.00, 0.00]	6.62 [5.23, 7.98]	10.99 [8.25, 13.81]	8.31	0.646
Countdown	Re:Dr	5.32	3.51 [1.88, 5.30]	6.52 [4.27, 8.94]	9.58 [6.81, 12.63]	2.53	0.626
Countdown	Re:Max	1.95	-2.40 [-4.62, -0.21]	-1.25 [-3.70, 1.14]	0.10 [-2.62, 2.78]	-0.05	0.490
Python	Re:Dr	38.99	-0.22 [-1.25, 0.91]	4.09 [3.17, 5.08]	7.67 [6.82, 8.58]	7.67	0.317
Python	Re:Max	22.84	1.53 [0.87, 2.24]	3.85 [3.04, 4.72]	5.68 [4.80, 6.64]	5.68	0.198
MathIR	Re:Dr	33.61	0.01 [0.00, 0.05]	0.12 [0.03, 0.23]	0.14 [0.00, 0.30]	0.08	0.004
MathIR	Re:Max	21.64	0.00 [-0.04, 0.05]	0.03 [-0.03, 0.10]	0.09 [-0.01, 0.21]	0.06	0.003
Pantry	Re:Dr	16.55	-1.11 [-1.65, -0.54]	4.36 [3.82, 4.87]	9.78 [8.90, 10.65]	9.20	0.810
Pantry	Re:Max	13.04	-0.64 [-1.04, -0.21]	3.12 [2.72, 3.53]	6.61 [5.83, 7.39]	6.20	0.547

Raw survival improves in 39 of 40 domain-scale-level-objective comparisons. At four verified draws, the point estimate remains positive in 28 of 39 eligible comparisons. These con-

ditional comparisons use a smaller population than raw survival. Re:Dr’s expected distinct-mode gain is largest in Graph, Countdown, Python and Pantry, with pooled survival gains reaching 19.6 percentage points. In MathIR, the corresponding gains are only .004 distinct modes and .14 survival points; the latter interval rounds to [.00, .30]. These small positive estimates agree descriptively with the small PCMD gain in Table 9.

Graph’s one-draw contrast is zero under this withdrawal distribution. Each valid colouring loses one colour choice per variable vertex, so every single mode survives the same fraction of binding withdrawals. Including nonbinding options reduces the contrast because those options retain every certified mode. For example, 63% of Countdown Level-1 options with a feasibility certificate are nonbinding. The all-options column and the binding column therefore describe different withdrawal populations.

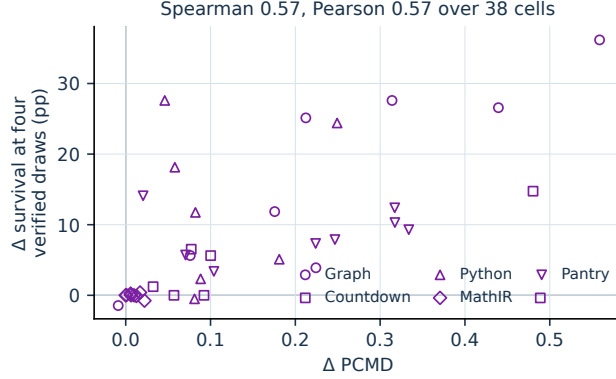


Figure 36: **Diversity gains correlate with survival gains after a withdrawal.** Each mark is one replay–fresh-objective contrast in a domain, scale and level. The horizontal axis is the terminal PCMD difference; the vertical axis is the percentage-point survival difference at four verified draws, averaged equally over eligible seed-specific contrasts. Survival uses jointly eligible prompt–group pairs within each seed. Marker shape identifies the domain; zero lines mark no change. The 38 comparisons have Pearson and Spearman correlations of .57; points show estimates without intervals. The association is descriptive, and the two metrics need not use the same eligible population.

M.1 TWO WITHDRAWALS AT ONCE

We evaluate pairs drawn from the binding single withdrawals. Of 29,213 pairs, 26,927 retain at least one certified mode and are included in the survival comparison. For Graph, Countdown, Python and Pantry, a certified mode survives the pair exactly when it survives both individual withdrawals. MathIR can differ: removing two actions can eliminate every realization of a state path even when either action alone can be removed. Re-enumeration gives equality with the single-withdrawal intersection for all 1,344 MathIR pairs in this evaluation.

Table 37: **Two withdrawals retain survival gains in Graph, Python and Pantry.** Entries are replay–fresh-objective differences in percentage points at two or four verified draws. One and Two use binding single options and binding pairs with a feasibility certificate, respectively; their eligible prompt populations can differ. Estimates pool jointly eligible groups. Brackets are pointwise 95% paired prompt-bootstrap intervals with the clustering in Table 36.

Domain	Arm	One @2		One @4	Two @2	Two @4
Graph	Re:Dr	10.51	19.65	[15.61, 23.82]	10.02	23.65 [19.14, 28.25]
Graph	Re:Max	6.62	10.99	[8.25, 13.81]	5.93	12.51 [9.37, 15.69]
Countdown	Re:Dr	6.52	9.58	[6.81, 12.63]	3.29	6.11 [0.87, 12.22]
Countdown	Re:Max	-1.25	0.10	[−2.62, 2.78]	-5.53	−4.87 [−11.80, 0.76]
Python	Re:Dr	4.09	7.67	[6.82, 8.58]	5.37	10.10 [8.77, 11.55]
Python	Re:Max	3.85	5.68	[4.80, 6.64]	5.64	8.16 [6.79, 9.64]
MathIR	Re:Dr	0.12	0.14	[0.00, 0.30]	0.18	0.20 [0.01, 0.45]
MathIR	Re:Max	0.03	0.09	[−0.01, 0.21]	0.04	0.12 [−0.01, 0.29]
Pantry	Re:Dr	4.36	9.78	[8.90, 10.65]	2.85	7.23 [6.51, 7.95]
Pantry	Re:Max	3.12	6.61	[5.83, 7.39]	2.03	4.94 [4.35, 5.57]

Two withdrawals retain less certified support on average (Table 35). Re:Dr’s pooled survival gain at four verified draws rises in Graph, Python and MathIR, and falls in Countdown and Pantry. Countdown Level 1 starts with only 4.5 certified modes per prompt; Pantry Level 1 retains only 18%

of its certified modes under a binding pair. These support differences accompany the effect-size differences but do not identify their cause. Across domain–scale–level–objective comparisons, 25 of 39 point estimates are positive at four verified draws, compared with 28 for single withdrawals.

M.2 DECODING SETTINGS AS A SUBSTITUTE

We evaluate binding Level-1 withdrawals on 128 prompts per domain using the separate twelve-pass Qwen2.5-0.5B cohort in App. J. Its replay variant combines separate mass and within-bank balancing terms with novelty and semantic-entropy shaping; it differs from Re:Dr in both objective and training duration. Five seeds per arm are evaluated at 11 settings: six temperatures from .5 to 2.0, three settings with larger groups, and two with top- $p = .95$. Larger groups increase K from 8 to 32, but use two groups rather than four: total responses per prompt double from 32 to 64. The reference is $T = 1$, top- $p = 1$, and four groups of eight. For each domain, the control setting with the highest four-verified-draw survival is selected on these evaluation prompts; replay uses the reference setting. Table 38: **Replay survival exceeds the selected control setting in all five domains.** Survival at four verified draws is expressed as percentages. Control, Control grid and Replay are equal-seed means on each setting’s own eligible groups; the grid column gives the control range and Best its maximizing temperature. Δ instead pools jointly eligible groups for reference replay minus selected control, so it need not equal the difference of the displayed means. Brackets are pointwise 95% intervals from 20,000 paired prompt-bootstrap replicates (seed 20260917), keeping all groups and five seeds of a prompt together. The selected setting stays fixed in the bootstrap; intervals condition on that selection and the observed seeds.

Domain	Control	Control grid	Best	Replay	Δ vs best
Graph	58.61	58.59–58.85	T^2	95.91	36.97 [35.53, 38.42]
Countdown	15.49	12.57–16.16	$T^{1.6}$	40.09	28.87 [18.98, 38.87]
Python	70.54	70.36–70.54	$T^{0.5}$	77.56	8.33 [7.15, 9.69]
MathIR	72.23	72.18–72.55	T^2	73.60	1.37 [0.61, 2.30]
Pantry	50.65	50.02–51.79	T^2	64.09	12.73 [10.22, 15.31]

Control survival spans at most 3.6 percentage points across the grid. The paired replay gains range from 1.4–37.0 points, with all five pointwise intervals above zero. Every selected control setting uses eight-response groups and top- $p = 1$. Thus the tested decoding changes do not close the survival gap in this cohort. The finite samples do not establish that alternative modes have zero probability, and selecting a setting on evaluation outcomes does not measure its performance on new prompts.

M.3 RECOVERY AFTER THE CHANGE

Recovery is evaluated at the Level-1 Qwen2.5-0.5B terminal Dr.GRPO and Re:Dr checkpoints, using seeds 43 and 44: 20 checkpoint evaluations across five domains. Each domain has 32 test and 16 temperature-calibration prompts in a disjoint split. Prompts without a binding withdrawal that retains a certified mode are excluded, leaving 28 test and 15 calibration prompts in Countdown and all prompts elsewhere. Up to four binding options per test prompt give 533 distinct withdrawals, shared by both arms and seeds.

Each initial portfolio contains eight responses. Ordinary sampling uses its terminal evaluation group at $T = 1$. The temperature strategy generates a group at the temperature in $\{.7, 1.0, 1.3\}$ maximizing mean survival on the calibration prompts. The diversity strategy generates eight responses sequentially, including the preceding responses in each prompt. All strategies receive the same recovery procedure: if no initial mode survives, generate one response per call at $T = 1$, up to eight calls, with the revised task, initial portfolio and previous failed attempts in context. Stop at the first verified response whose mode survives the withdrawal.

Survival uses the mode predicates defined above. In MathIR, a retained or newly generated state path can count as surviving even if realizing it under the reduced menu requires a different action sequence. Pantry responses are six-bit ingredient-support masks projected onto verified quantities by the environment (App. C.5); survival is evaluated on the resulting support key. These outcomes measure retained mode feasibility, not necessarily reuse of the original response text unchanged.

With ordinary or temperature-tuned initial portfolios, replay resolves more withdrawals and uses fewer recovery responses in every domain. All ten resolution contrasts and all ten response-count contrasts have pointwise intervals excluding zero. Ordinary sampling saves 0.55–5.37 responses per withdrawal.

Table 39: **Replay improves ordinary and temperature-tuned recovery in every domain.** Saved is the percentage of withdrawals with a surviving initial mode; By 8 includes those resolved within eight additional responses. Calls is the mean additional response count, with zero for initial survival and eight for unresolved cases. Each row pools the same withdrawals over two matched seeds. Fresh denotes Dr.GRPO and Replay denotes Re:Dr; the three strategies differ in initial portfolio generation and share the recovery procedure.

Domain	Strategy	Fresh saved	Fresh by 8	Fresh calls	Replay saved	Replay by 8	Replay calls
Graph	Ordinary	15.62	22.66	6.38	83.20	91.02	1.01
Graph	Tuned temperature	15.62	20.31	6.55	77.34	89.06	1.36
Graph	Diversity prompt	29.69	30.47	5.58	62.50	74.61	2.44
Countdown	Ordinary	2.70	2.70	7.78	9.46	10.81	7.23
Countdown	Tuned temperature	2.70	2.70	7.78	9.46	13.51	7.05
Countdown	Diversity prompt	2.70	2.70	7.78	5.41	9.46	7.41
Python	Ordinary	14.06	14.06	6.88	69.92	77.73	2.21
Python	Tuned temperature	14.06	14.06	6.88	71.88	75.78	2.12
Python	Diversity prompt	14.06	14.06	6.88	62.50	67.97	2.79
MathIR	Ordinary	38.39	44.20	4.60	58.93	65.18	2.92
MathIR	Tuned temperature	37.05	45.09	4.61	63.84	67.86	2.67
MathIR	Diversity prompt	41.07	45.09	4.58	47.32	53.57	3.85
Pantry	Ordinary	31.25	31.64	5.48	47.66	52.34	4.04
Pantry	Tuned temperature	31.25	44.53	4.81	53.12	62.11	3.38
Pantry	Diversity prompt	56.64	58.20	3.39	33.98	45.70	4.71

Table 40: **Recovery benefits depend on the initial portfolio strategy.** Entries are paired Re:Dr–Dr.GRPO differences on the same withdrawals under each strategy. Saved and By 8 are percentage points; Calls is additional responses. Brackets are pointwise 95% intervals from 20,000 paired prompt-bootstrap replicates (seed 20260917). Each prompt’s withdrawals and both training seeds move together; intervals condition on those two seeds.

Domain	Strategy	Δ saved	Δ by 8	Δ calls
Graph	Ordinary	67.58 [57.81, 76.56]	68.36 [59.38, 76.95]	−5.37 [−6.05, −4.65]
Graph	Tuned temperature	61.72 [51.95, 71.09]	68.75 [61.33, 75.78]	−5.20 [−5.84, −4.52]
Graph	Diversity prompt	32.81 [21.09, 44.14]	44.14 [34.77, 53.12]	−3.14 [−3.95, −2.30]
Countdown	Ordinary	6.76 [1.35, 13.89]	8.11 [1.43, 16.18]	−0.55 [−1.12, −0.11]
Countdown	Tuned temperature	6.76 [1.35, 13.89]	10.81 [3.33, 19.32]	−0.73 [−1.35, −0.21]
Countdown	Diversity prompt	2.70 [0.00, 6.58]	6.76 [1.32, 13.64]	−0.38 [−0.76, −0.06]
Python	Ordinary	55.86 [44.92, 65.23]	63.67 [51.56, 74.22]	−4.67 [−5.45, −3.76]
Python	Tuned temperature	57.81 [46.88, 67.19]	61.72 [50.39, 71.48]	−4.76 [−5.54, −3.87]
Python	Diversity prompt	48.44 [37.11, 59.38]	53.91 [41.80, 65.62]	−4.09 [−4.99, −3.14]
MathIR	Ordinary	20.54 [11.16, 30.09]	20.98 [12.71, 29.82]	−1.68 [−2.37, −1.02]
MathIR	Tuned temperature	26.79 [18.26, 35.65]	22.77 [13.89, 32.27]	−1.94 [−2.66, −1.28]
MathIR	Diversity prompt	6.25 [−4.09, 16.67]	8.48 [−0.46, 18.26]	−0.73 [−1.50, 0.03]
Pantry	Ordinary	16.41 [8.20, 25.39]	20.70 [11.72, 30.47]	−1.44 [−2.18, −0.76]
Pantry	Tuned temperature	21.88 [13.28, 31.25]	17.58 [10.94, 24.61]	−1.43 [−2.00, −0.91]
Pantry	Diversity prompt	−22.66 [−31.25, −13.67]	−12.50 [−21.88, −2.73]	1.32 [0.55, 2.05]

Summed over all three strategies and both seeds, replay uses 9,872 recovery responses and the control uses 18,335. These totals cover recovery only; they exclude initial portfolios and temperature calibration. Response counts do not match token lengths, context lengths or training compute.

The diversity prompt increases the control’s initial survival in Graph and Pantry, relative to ordinary sampling, with intervals above zero. For replay, initial-survival point estimates decrease in every domain. The intervention changes both the request for alternatives and the accumulated context, so their effects are not separated. With both arms using the diversity prompt, Pantry favours the control by 22.66 points of initial survival and 12.50 points of resolution by eight calls. Replay’s initial-survival difference is positive in the other four domains, ranging from 2.7 to 48.4 points, although the Countdown and MathIR intervals include zero. Pantry’s six-bit response format permits enumerating support masks, but this comparison does not establish that enumerability explains the reversal.

Interpretation and limits. In Graph, Countdown, Python and Pantry, Re:Dr’s broader four-draw portfolios accompany larger survival gains than in MathIR. The positive association with PCMD is descriptive. Fixing the number of verified draws conditions on jointly eligible groups; it does not fix policy accuracy, and eligibility can change with the draw budget. Feasibility is established by a surviving certified mode, which may belong to a lower bound on the verifier’s full support. The interventions cover single and paired exclusions, rather than arbitrary task revisions. Recovery uses domain-specific mode predicates and yields a strategy-dependent reversal in Pantry. Finally, the intervals quantify prompt-sampling uncertainty within the observed training cohorts; they do not estimate variability over new training seeds.

Part V. Fixed deployments

N INFERENCE-ONLY CONCENTRATION IN A HOSTED MODEL

GPT-5.6 Sol’s verified responses concentrate on a small subset of solution modes under the MODEBENCH interface. This inference-only evaluation measures the output distribution of one deployment, independently of the paper’s matched training interventions.

Evaluation setup. The deployment is `gpt-5.6-sol`, with model snapshot `gpt-5.6-sol-2026-07-09`. Each of five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. Temperature is 1.0 and top- p is .98, the service defaults. The hosted natural-language interface does not enforce the local training evaluation’s token-level grammar constraints.

Level 1 uses the native evaluation splits; Levels 2 and 3 use the matched constructions described in App. B.4. Level 3 extends the test population beyond the two levels of the training intervention. All cells contain multi-answer problems. The Level-2/Level-3 support histograms match those of a separate Level-1 reference population, which differs from the evaluated Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Each prompt has eight returned responses with distinct response identifiers. The 107 failed API or transport attempts were retried and are excluded from the 15,360 responses; they are not scored as incorrect answers. Stateless requests and distinct identifiers do not establish independent provider randomness. Hidden reasoning content is not exposed.

Table 41: **Correct responses frequently share a solution mode at all three levels.** GPT-5.6 Sol with medium reasoning, under strict grading. Each level averages five domains equally, with 128 prompts and eight responses per prompt in each domain. M and P are mean@8 and pass@8 (%); D is mean distinct@8. C is correct-pair collision (%), weighted by correct pairs within each domain. Brackets are pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8; pass@8 is the fraction of prompts with at least one verified response. Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise percentile intervals for the empirical prompt population; generations are not resampled separately.

Formatting sensitivity. Strict grading applies the executable validators directly. The normalized comparison applies a fixed set of typography rules to the same responses, then uses those validators without repairing calculations or program logic. The rules convert a leading LaTeX lambda to Python

Table 42: **High Graph correctness coexists with concentrated solution modes.** GPT-5.6 Sol with medium reasoning and strict executable grading. Each cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is mean distinct@8, including unsolved prompts. C is correct-pair collision (%), weighted by correct pairs, with pointwise 95% whole-prompt bootstrap intervals. U is the uniform-correct-key collision reference (%); a dash denotes unknown total support.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

λ , unescape percent signs and underscores, remove LaTeX spacing commands, simplify sized parentheses and translate balanced LaTeX fractions into parenthesized division. Numeric/parenthesis adjacency becomes explicit multiplication only when the containing group has no division sign. Normalization leaves Graph, MathIR and Level-1 PantryPlan unchanged. Strict successes retain their original keys. Normalization verifies 2,580 additional outputs; the added correct responses change both the eligible correct pairs and their weights, so collision can increase or decrease. This comparison measures sensitivity to these formatting conventions on the evaluated responses.

Table 43: **Formatting normalization improves correctness while substantial collision remains.** The same 128 prompts and 1,024 responses per cell as Table 42, with typography normalization before executable verification. M, P, D, C and U use the same definitions; C weights correct pairs and includes pointwise 95% whole-prompt bootstrap intervals. Additional verified responses change the eligible pairs and their weights.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration across levels. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized comparison gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not monotonic within every domain. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level

label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 correct pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). The domain tables show the corresponding correctness, diversity and collision estimates for every domain and level.

Interface and interpretation. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Observed keys provide a lower bound on accessible support: eight draws cannot establish that an unseen mode has zero probability. The results describe concentration under the specified prompts and deployment configuration; they identify neither a training cause nor a causal difficulty or model-size effect.

N.1 HOSTED CONCENTRATION ON THE SUCCESS-CONDITIONAL AXIS

Table 44 reports promptwise PCMD under strict grading and the original task wording. Within each cell it averages equally over prompts with at least two verified responses; a cell is reportable when at least thirty prompts qualify. The collision tables above instead weight prompts by their number of correct pairs. For GPT-5.6 Sol under strict grading, $1 - \hat{C}$ and promptwise PCMD summarize the same responses with different weights (App. A.1). Fig. 39 shows the corresponding strict, original-wording cell estimates in its diversity row. Its accuracy row uses formatting normalization and revised Opus 5 Python wording, so the two rows have different grading and, for that deployment and domain, different prompt conditions.

For GPT-5.6 Sol, strict MathIR PCMD is .000 at Levels 2 and 3: none of the 110 and 125 eligible prompts, respectively, contains two different verified keys. Level-3 Graph has PCMD .049, although its prompts admit between four and twelve valid colorings. PantryPlan has higher diversity, with PCMD .531, .386 and .455 across the three levels. Python at Levels 2 and 3 has only 14 and 15 eligible prompts out of 128, below the thirty-prompt reporting threshold. These missing entries reflect strict formatting failures, including LaTeX-escaped Python; this deployment has no provider refusals in those cells. Opus 5’s corresponding missing entries have a different cause: 1,021 and 1,024 of its 1,024 responses are refusals.

The Level-4 probe. The GPT-5.6 Sol Level-4 points in Fig. 3 average to .181 across 5 domains, using the pair-pooled $1 - \hat{C}$ statistic and formatting normalization. This evaluation uses the direct OpenAI service, whereas Levels 1–3 use Azure. The deployments may differ, so their contrast does not isolate a level effect. Python is particularly sensitive to grading: strict grading verifies two of 1,024 responses, compared with 675 after normalization. Normalization changes the correct-response population on which diversity is measured. This difference measures sensitivity to output formatting; it does not establish correctness for the remaining responses.

Concentration across deployments. The seven strict-grading macros range from .142 for Claude Opus 5 to .320 for DeepSeek V4 Pro. They use the thirteen domain–level cells that are reportable for every deployment, excluding Python at Levels 2 and 3. The transformed summaries $N_{\text{eff}}^{(2)} = 1/(1 - \text{macro PCMD})$ range from 1.17 to 1.47. These are transforms of the mean diversity, not averages of per-cell effective mode counts or bounds on unseen modes. At the largest observed macro, the corresponding mean probability of repeating a correct mode is about .68 under these prompt and cell weights.

MathIR defines modes by canonical derivation paths; the other domains use alternative output structures. Removing the three MathIR cells raises every deployment’s macro. Across the ten

remaining common cells, the largest transformed summary is 1.58, for Kimi K3. Python contributes only Level 1 to this comparison, so this is an equal-cell mean, not an equal-domain mean over four domains. The low diversity summaries therefore persist without MathIR, although their magnitudes depend on the mode definitions and averaging population.

Averaging each deployment over its own reportable cells instead changes the ranking: DeepSeek V4 Pro has .280 across its fifteen cells and falls below Kimi K3. Domain rankings also differ: GPT-5.4 and Kimi K3 have higher Python diversity, while DeepSeek V4 Pro has higher Graph diversity. Every deployment’s per-level mean is lower at Level 3 than at Level 1, although the intermediate level need not be monotone. Fig. 16 shows the normalized cohort one deployment per row.

Table 44: Seven hosted deployments have low diversity among correct solutions. Strict grading, original wording, 128 prompts per domain–level cell and eight stateless requests per prompt. Within-cell PCMD weights eligible prompts equally. Macro PCMD averages the same thirteen cells for every deployment, omitting Python L2/L3; $N_{\text{eff}}^{(2)}$ transforms that macro. L1–L3 columns average each deployment’s own reportable cells: five domains per level, except four at L2/L3 for GPT-5.6 Sol and Opus 5. Thus the macro is not the mean of those three columns. A cell requires thirty prompts with at least two verified responses. Entries are point estimates.

Deployment	Macro PCMD	$N_{\text{eff}}^{(2)}$	Macro PCMD by level		
			L1	L2	L3
DeepSeek V4 Pro	.320	1.47	.289	.319	.232
Kimi K3	.302	1.43	.374	.307	.253
GPT-5.4	.264	1.36	.338	.247	.224
Grok 4.3	.239	1.31	.275	.179	.171
GPT-5.6 Sol	.226	1.29	.256	.229	.183
Claude Opus 4.8	.215	1.27	.217	.229	.168
Claude Opus 5	.142	1.17	.140	.164	.121

Level comparisons also change prompt guidance. The Level-2 and Level-3 system messages add strategy guidance to MathIR, Python and PantryPlan: an algebraic isolation order, a nested small-divisor chain and a preference over ingredient families. Level 1 lacks this guidance. In the normalized original-wording cohort, pair-pooled $1 - \hat{C}$ falls from Level 1 to Level 3 by .115 on average across the 20 measured deployment–domain pairs in these domains, compared with .048 across the 14 pairs in Graph and Countdown. Averaging the two unguided domains within each model, 7 of 7 deployments decline. The guided and unguided domains also differ in their tasks and solution spaces, so this comparison does not estimate the effect of guidance.

The prompt ablations test that effect within specified conditions. Removing MathIR guidance from the local 0.5B checkpoints leaves observed PCMD at .000 under both wordings (App. I). The separate 64-draw hosted study finds higher Level-2 MathIR diversity under neutral wording for both GPT deployments at fixed correct-draw budgets, with intervals excluding zero; Grok’s intervals include zero (App. K.2). These comparisons condition on jointly eligible prompts and do not match overall accuracy. The different responses to wording concern different models and evaluation populations. Cross-level narrowing describes concentration under the specified prompts; it does not isolate task difficulty from prompting or deployment differences.

O COMPARISON ACROSS HOSTED DEPLOYMENTS

Figure 39 compares accuracy and success-conditional solution-mode diversity across hosted deployments. Its accuracy row uses a common formatting normalizer and the revised Opus 5 Python instructions at each level; hollow marks show that deployment’s original Python condition. Other accuracy cells use the original instructions. The diversity row uses strict grades and original instructions for every deployment, with the prompt-level estimand and eligibility criteria of App. N.1. Accuracy includes every response, including refusals and verification failures. Deployment settings differ, so the figure describes the stated configurations and does not rank models under matched prompts and compute.

The tables and Graph figure use the original instructions for every deployment. The Python comparison reports both instruction conditions under strict and formatting-normalized grad-

ing. Level 3 extends the test tasks; performance across levels need not follow a common difficulty ordering for all deployments.

Evaluation population. The deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro. The original-instruction cohort contains 107,520 responses: 15,360 per deployment, with the same 128 held-out prompts in each of five domains and three levels, and eight stateless responses per prompt. Requests use no model-side tools or conversation history, and the evaluation includes no task training or replay intervention.

Provider configurations. All deployments request medium reasoning effort and an 8,192-token native output limit. Claude Opus 5 and Claude Opus 4.8 use Anthropic Messages with adaptive thinking; the other deployments use hosted API routes.

The requested reasoning setting and output limit do not match compute across providers: effort labels and token accounting have provider-specific meanings. Temperature and nucleus sampling use provider defaults, whose exact values are unknown when the provider does not return them.

Strict grading checks the original executable answer; formatting-normalized grading applies the same typography corrections across deployments before the executable checks. The accuracy row of the main display uses normalized grades and the revised Opus 5 Python instruction at all three levels; its other accuracy cells use the original instructions. The diversity row uses strict grades and original instructions for every deployment. The tables below report original-instruction results under both grading rules. Prompt preferences and the Level-1 PantryPlan interface difference limit comparisons across levels. Countdown has no certified finite-total-support uniform reference. This inference-only comparison does not identify a training cause, a model-size effect, or a difficulty effect. Collision intervals are pointwise estimates for individual cells, not intervals for paired deployment differences.

Response and cost coverage: DeepSeek V4 Pro. The 15,360 terminal responses include verification failures and truncations. Outside this cohort, 13 HTTP-200 responses have neither a terminal finish reason nor a visible answer, and 192 request attempts have unknown outcomes and usage. These incomplete attempts are not scored as model responses. Token totals cover known usage only and can therefore understate the usage and cost of all attempted requests.

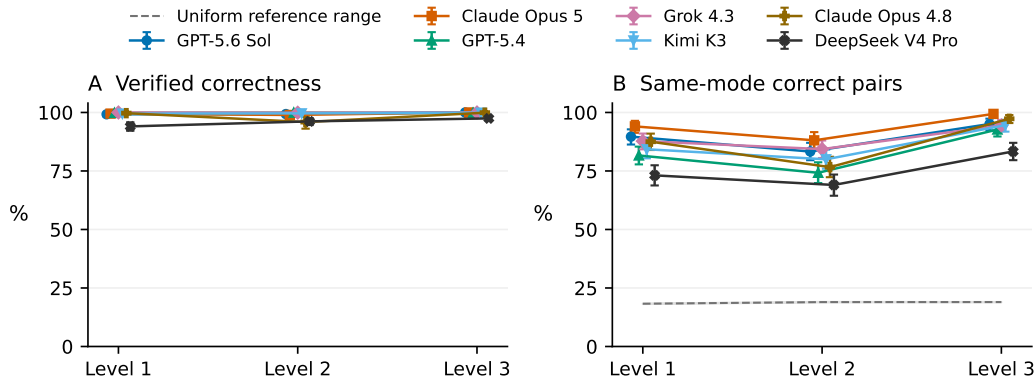


Figure 37: **Hosted Graph answers are usually correct but concentrated relative to uniform sampling.** Panel A shows strict per-response accuracy; panel B shows the fraction of correct pairs sharing a solution mode, pooling pairs across prompts. Colors and markers identify seven deployments, each with the same 128 prompts per level and eight stateless responses per prompt. Error bars are pointwise 95% intervals from resampling whole prompts. The gray band spans the deployments’ uniform-correct-mode references, weighted by their observed correct-pair counts; the dashed line is their mean. Level labels do not imply equal or monotonically increasing difficulty across deployments.

O.1 PYTHON PROMPT SENSITIVITY

Opus 5 receives the same 384 Python tasks under the original instructions and a direct arithmetic-expression instruction without a system message. Each condition contains eight responses from separate requests per task across three levels. Adaptive thinking, medium effort, an 8,192-token output limit, allowed operators, public inputs, and graders are the same in both conditions. The revised instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed input, without a preferred divisor. Task wording and system-message presence both change; provider state and collection time can also differ. The comparison describes these two configurations and does not isolate which change accounts for their difference. All responses contribute to accuracy and mode counts, including provider refusals.

Table 45: Opus 5 rarely refuses Python tasks with direct instructions and no system message. Each row contains 128 tasks with eight responses each (1,024 responses). R counts provider-declared refusals; A is per-response accuracy (%); D is mean `distinct@8`; C is correct-pair collision (%), pooling pairs within each level. Both conditions use the same executable grader and formatting normalizer. Original Level-2/3 collision is undefined because no task has two correct responses.

Condition	L	R	Strict			Normalized		
			A	D	C	A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4
	2	1021	0.3	0.02	–	0.3	0.02	–
	3	1024	0.0	0.00	–	0.0	0.00	–
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9
	2	0	71.8	1.31	84.7	100.0	1.38	85.7
	3	3	76.2	1.38	82.7	99.7	1.41	84.1

The revised condition has no API failures or token-limit truncations. Strict grading accepts 2,282 responses; formatting normalization accepts 787 additional responses by converting escaped percent signs (`\%`) to Python modulo (`%`). Every non-refused answer therefore passes the executable checks after normalization. Normalized collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The three refusals occur on two tasks whose other identical requests produce accepted answers. These results use all eight responses per task, without selecting or replacing outputs according to their validity.

Accuracy differences use a paired bootstrap of whole eight-response task groups across conditions (20,000 replicates). Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2, 100.0] percentage points (pointwise 95% intervals). Tasks are paired; output draws are not paired across conditions. Collision weights tasks by their numbers of correct pairs, which differ between conditions. Its raw difference therefore compares different correct-pair populations and does not estimate a prompt effect at matched accuracy or on a common set of pairs.

First-valid retry selection. A retry condition replaces each of the revised-wording cohort’s three refused responses with the first verified-valid answer to an additional request. All three replacements pass formatting-normalized verification on the first retry and reproduce a solution mode already observed for that task. Selecting 3,072 valid responses therefore requires 3,075 requests. The selected set’s 100% accuracy is a consequence of selection, not unconditional model accuracy. Its `distinct@8` is unchanged at 1.492, 1.375, 1.414 at Levels 1–3. Level-3 correct-pair collision changes from 84.0909% to 84.1518%; the other levels are unchanged. This diagnostic uses variable request effort. The main figure uses the fixed eight-response condition, including all three refusals, without these replacements.

O.2 REQUESTED-TEMPERATURE SENSITIVITY

Grok 4.3 and Kimi K3 each receive requested temperatures $T = 1.0$ and $T = 1.5$ on the same 120 tasks: eight prompts in each of five domains and three levels. Eight responses from separate requests per prompt give 960 responses per deployment–temperature condition and 3,840 in total. Within each deployment, the two conditions share all non-temperature request settings and run concurrently. Accuracy and mode counts include all responses, including verification failures and truncated outputs.

The providers accept both requested temperature settings, but their responses do not report effective sampling temperatures. DeepSeek is outside this comparison because its thinking-mode interface

ignores the temperature setting. The results concern these requested settings on this 120-task subset and do not establish a general temperature response for all hosted deployments.

Each level summary averages five domains equally; All averages the 15 domain-level cells equally. Collision pools correct pairs within each cell, so tasks with more correct responses receive more weight. Eligible tasks and pair weights can differ between temperatures; the contrast does not hold accuracy or the correct-pair population fixed. An undefined cell makes the corresponding average undefined. Pointwise 95% intervals use 20,000 paired whole-prompt bootstrap resamples, stratified by domain and level. Each resampled prompt includes all eight responses. Prompts are paired across conditions; output draws are not. Intervals have no multiplicity adjustment.

Observed sensitivity. For Grok, the observed diversity change is +0.050 [-0.058, +0.158] modes, and the collision change is -0.78 [-3.86, +2.12] percentage points. Both intervals include zero, so neither shows a clear diversity gain. The per-response accuracy change is -2.29 [-3.85, -0.94] percentage points. This does not establish invariance to temperature in general.

Kimi returns 696/960 token-limited outputs at $T = 1.5$, compared with 0 at $T = 1.0$. Strict accuracy falls from 96.56% to 21.25%, while raw diversity falls from 1.933 to 0.983 modes. The large increase in truncation accompanies losses in accuracy and distinct modes. These results do not isolate correct-output concentration at matched accuracy. The five-domain collision mean is undefined at $T = 1.5$ because some cells have no eligible correct pairs. Neither model has provider-declared refusals in this experiment.

† marks a defined contrast with an omitted interval because at least one bootstrap replicate has an undefined cell. For Kimi collision, only 11,815 and 11,616 of 20,000 replicates define the contrast at Levels 1 and 2. Percentiles over those subsets would condition on bootstrap estimability. The five-domain average requires all five cell estimates; an undefined cell does not contribute zero or disappear from its denominator.

Countdown has no finite-total-support uniform reference; uniform-reference comparisons therefore use the other four domains. Cross-level differences also reflect different task populations and do not establish a common ordering of model difficulty. These inference-only results do not identify a training or parameter-scale cause of concentration.

O.3 GPT-5.6 SOL: TEMPERATURE, PASS@8 AND VERIFIED MODES

Figure 38 plots empirical `pass@8` against `distinct@8` for $T \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$, with reasoning `none`. Each condition uses the same 480 held-out prompts (32 per domain-level cell), with eight stateless responses per prompt: 3,840 responses per temperature and 19,200 in total. A prompt contributes one to `pass@8` if at least one of its eight answers is verified correct, and contributes its number of distinct correct canonical keys to `distinct@8`. Both metrics average over all prompts, including those with no correct answer. The empirical `pass@8` uses the observed eight-response groups; $1 - (1 - p)^8$ applied to pooled accuracy would ignore differences among prompts. Raw `distinct@8` depends on both success and variation among correct solution modes, so its changes need not track `pass@8`.

Prompts are selected by an outcome-independent hash ranking within each domain-level cell. Ranks 1–8 form a 120-prompt subset (4,800 responses); ranks 9–32 form a 360-prompt subset (14,400 responses). These subsets have different collection times. The 360-prompt requests interleave all five temperatures within each prompt/draw slot. For the 120-prompt subset, zero-temperature responses and nonzero-temperature responses come from separate collection periods. Prompt composition and service changes can therefore contribute to differences between subsets and conditions.

Requests use the same prompt wording, verifiers and sampling controls except temperature, with an 8,192-token output cap. Returned metadata identifies `gpt-5.6-sol-2026-07-09`, reasoning `none`, `top_p = 0.98` and the requested temperature in every condition. The exposed settings do not establish unchanged internal service behavior or independent provider randomness. A zero-temperature request does not establish deterministic outputs. The deployment rejects non-default temperatures with medium reasoning and rejects temperature 2.5; this sweep therefore does not test temperature robustness under medium reasoning.

Table 46: **Higher requested temperature yields no clear Grok diversity gain and much lower Kimi accuracy.** Strict executable grades compare $T = 1.5$ with $T = 1.0$. Each condition contains eight responses on each of 120 tasks. Level summaries average five domains equally; All averages 15 domain-level cells. A is per-response accuracy (%); D is mean `distinct@8`; C is correct-pair collision (%), pooling pairs within each cell. Differences subtract $T = 1.0$ from $T = 1.5$: mode counts for D and percentage points for A/C. Brackets give pointwise 95% intervals from paired whole-prompt bootstrap resampling. † marks an omitted interval when some resamples have undefined cells; – denotes an undefined estimate or omitted interval.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.50	-70.00 [-75.62, -63.75]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.12	-13.11 [-]†
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-]†
	3	A	99.06	20.94	-78.12 [-83.75, -72.81]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.33	–	–
	All	A	96.56	21.25	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.25	–	–

Table 47: **Formatting normalization leaves the temperature sensitivity largely unchanged.** Only rows differing from strict grading appear; all other rows are identical. Each condition contains eight responses on each of 120 tasks. Level summaries average five domains equally; All averages 15 domain-level cells. A is per-response accuracy (%); D is mean `distinct@8`; C is correct-pair collision (%), pooling pairs within each cell. Differences subtract $T = 1.0$ from $T = 1.5$: mode counts for D and percentage points for A/C. Brackets give pointwise 95% intervals from paired whole-prompt bootstrap resampling. † marks an omitted interval when some resamples have undefined cells; – denotes an undefined estimate or omitted interval.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Kimi K3	1	A	92.50	22.81	-69.69 [-75.31, -63.44]
	1	C	66.23	53.61	-12.62 [-]†
	3	A	99.38	20.94	-78.44 [-84.06, -73.12]
	3	C	78.18	–	–
	All	A	96.67	21.35	-75.31 [-78.44, -72.19]
	All	C	72.20	–	–

Strict grading applies the executable validators directly. Normalized grading uses the typography rules in App. N before the same validators, preserving strict successes and their mode keys.

Each level averages five domains equally; All averages all 15 cells. Pointwise 95 percent intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, keeping all eight responses together and matching prompts across temperatures. Individual model responses are not paired across conditions. There is no multiplicity adjustment. A zero-width empirical interval reflects constant observed groups, not certain success on unseen prompts.

Table 48: **Nonzero temperatures improve eight-draw coverage in this sweep.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). G is the grading: S strict, N formatting-normalized. Each temperature uses the same 480 prompts (32 per domain–level cell), eight responses each, and reasoning none. Level rows average five domains equally; All averages all 15 cells. Brackets give pointwise 95% whole-prompt bootstrap intervals.

T	Level	G	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	S	55.0 [48.8, 61.3]	0.569 [0.500, 0.637]	53.9 [47.7, 60.1]
	2		45.0 [39.4, 50.6]	0.456 [0.400, 0.512]	39.5 [34.4, 44.7]
	3		49.4 [42.5, 56.2]	0.519 [0.444, 0.594]	44.5 [38.3, 50.9]
	All		49.8 [46.2, 53.3]	0.515 [0.475, 0.554]	46.0 [42.6, 49.4]
0.5	1	S	70.6 [64.4, 76.9]	0.956 [0.850, 1.069]	57.5 [51.9, 63.0]
	2		50.0 [44.4, 55.6]	0.525 [0.463, 0.594]	38.6 [33.9, 43.4]
	3		54.4 [48.1, 60.6]	0.581 [0.506, 0.662]	43.0 [37.1, 48.9]
	All		58.3 [54.8, 61.9]	0.688 [0.640, 0.738]	46.4 [43.3, 49.5]
1.0	1	S	75.6 [69.4, 81.2]	1.181 [1.050, 1.312]	57.3 [52.0, 62.7]
	2		51.9 [45.6, 58.1]	0.581 [0.506, 0.662]	37.6 [33.2, 42.0]
	3		60.6 [54.4, 66.9]	0.713 [0.625, 0.806]	42.7 [37.3, 48.1]
	All		62.7 [59.2, 66.2]	0.825 [0.767, 0.885]	45.9 [42.9, 48.8]
1.5	1	S	80.6 [75.0, 86.2]	1.413 [1.269, 1.556]	56.6 [51.7, 61.4]
	2		58.1 [51.9, 64.4]	0.650 [0.569, 0.738]	38.4 [34.3, 42.7]
	3		65.6 [59.4, 71.2]	0.838 [0.731, 0.956]	43.0 [38.0, 48.2]
	All		68.1 [64.8, 71.5]	0.967 [0.900, 1.035]	46.0 [43.3, 48.8]
2.0	1	S	83.8 [78.1, 88.8]	1.519 [1.375, 1.663]	59.7 [54.8, 64.5]
	2		54.4 [48.1, 60.6]	0.600 [0.525, 0.675]	35.3 [31.4, 39.3]
	3		61.9 [56.2, 67.5]	0.819 [0.719, 0.925]	40.5 [35.8, 45.5]
	All		66.7 [63.3, 70.0]	0.979 [0.915, 1.044]	45.2 [42.6, 47.8]
0.0	1	N	70.6 [65.0, 76.2]	0.725 [0.662, 0.788]	70.5 [64.5, 76.2]
	2		60.6 [55.0, 66.2]	0.613 [0.556, 0.669]	54.5 [49.0, 59.9]
	3		65.6 [59.4, 71.9]	0.681 [0.606, 0.756]	61.4 [55.2, 67.6]
	All		65.6 [62.3, 69.0]	0.673 [0.635, 0.710]	62.1 [58.8, 65.5]
0.5	1	N	76.9 [71.2, 82.5]	1.038 [0.931, 1.150]	71.7 [66.4, 77.0]
	2		68.1 [62.5, 73.8]	0.706 [0.644, 0.769]	53.7 [48.8, 58.6]
	3		71.9 [65.6, 77.5]	0.775 [0.700, 0.850]	58.1 [52.3, 63.9]
	All		72.3 [69.2, 75.4]	0.840 [0.792, 0.890]	61.2 [58.2, 64.3]
1.0	1	N	79.4 [73.8, 84.4]	1.238 [1.106, 1.369]	70.4 [65.2, 75.5]
	2		69.4 [63.7, 75.0]	0.756 [0.681, 0.831]	51.6 [47.1, 56.2]
	3		76.9 [71.2, 82.5]	0.894 [0.806, 0.988]	55.2 [49.8, 60.7]
	All		75.2 [72.1, 78.3]	0.963 [0.904, 1.021]	59.1 [56.2, 62.0]
1.5	1	N	82.5 [77.5, 87.5]	1.494 [1.344, 1.644]	68.5 [63.7, 73.3]
	2		75.0 [69.4, 80.6]	0.844 [0.762, 0.931]	49.8 [45.5, 54.3]
	3		81.9 [76.2, 87.5]	1.038 [0.931, 1.156]	54.7 [49.6, 59.8]
	All		79.8 [76.7, 82.9]	1.125 [1.056, 1.196]	57.7 [54.9, 60.5]
2.0	1	N	85.0 [80.0, 90.0]	1.606 [1.456, 1.762]	69.6 [64.8, 74.3]
	2		72.5 [66.9, 78.1]	0.800 [0.731, 0.869]	45.7 [41.7, 49.8]
	3		80.6 [75.0, 86.2]	1.031 [0.931, 1.137]	52.5 [47.7, 57.5]
	All		79.4 [76.2, 82.5]	1.146 [1.079, 1.212]	55.9 [53.3, 58.6]

Coverage and sampled solution modes. At $T = 0.0$ and $T = 2.0$, normalized pass@8 is 65.62% and 79.38%, respectively, while distinct@8 is 0.673 and 1.146. Across these 5 measured temperatures, the highest observed aggregate pass@8 occurs at $T = 1.5$ and the highest distinct@8 occurs at $T = 2.0$. These five temperatures do not establish a population optimum or a continuous Pareto boundary, and level-specific curves differ.

Per-response accuracy changes from 62.11% to 55.94% between the endpoints. That diagnostic is distinct from `pass@8`: success within eight draws and the reliability of an individual draw need not change together as temperature varies.

The paired $T = 2.0$ minus $T = 0.0$ contrast is 13.8 [10.8, 16.7] `pass@8` percentage points and 0.473 [0.417, 0.529] distinct correct modes, with pointwise 95% intervals.

Response outcomes. At every temperature ($T = 0.0$, $T = 0.5$, $T = 1.0$, $T = 1.5$ and $T = 2.0$), 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. Unsuccessful answers remain in the eight-response groups. API errors are retried without adding responses to those groups; the 19,200-response budget excludes failed API attempts.

Sensitivity across prompt subsets. We compare the same temperature contrast within each hash-rank subset. The subsets differ in prompts and collection periods, so differences between them do not isolate a causal effect of time.

Across all 480 prompts, the paired $T = 2$ minus $T = 1.5$ contrast is -0.4 [-2.5, 1.7] `pass@8` percentage points and 0.021 [-0.029, 0.071] distinct modes (95% intervals). Ranks 1–8 (120 prompts): the paired $T = 2$ minus $T = 1.5$ contrast is -1.7 [-5.0, 0.8] `pass@8` percentage points and -0.008 [-0.083, 0.067] distinct modes (95% intervals). Ranks 9–32 (360 prompts): the paired $T = 2$ minus $T = 1.5$ contrast is 0.0 [-2.8, 2.8] `pass@8` percentage points and 0.031 [-0.033, 0.094] distinct modes (95% intervals).

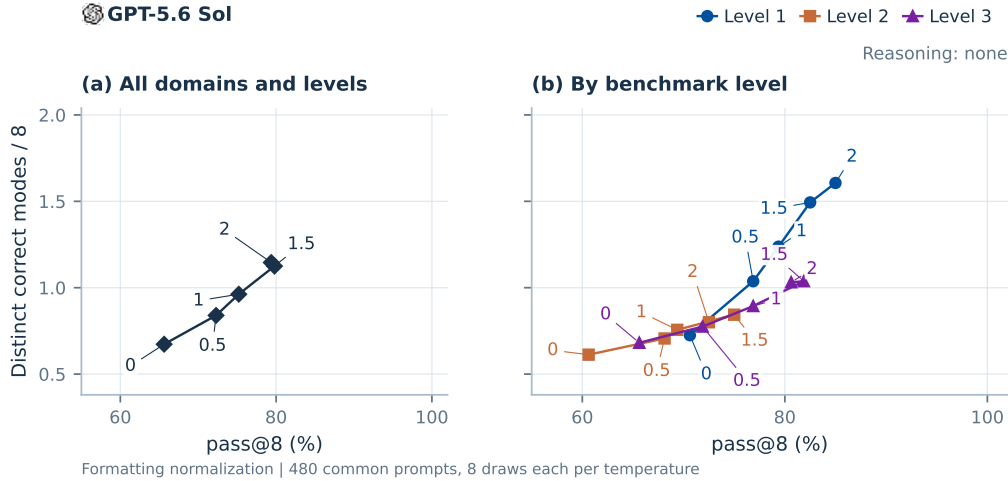


Figure 38: **Nonzero temperatures yield greater observed eight-draw coverage.** GPT-5.6 Sol with reasoning none, typography normalization, and the same 480 prompts at each of five temperatures (32 per domain–level cell, eight responses each). Left: an equal mean over all 15 domain–level cells; right: equal means over five domains at each level, identified by color and marker. The horizontal axis is empirical `pass@8`; the vertical axis averages distinct correct solution modes per eight responses, including unsolved prompts as zero. Labels give requested temperatures; segments connect measured point estimates in temperature order. Table 48 gives pointwise 95% whole-prompt bootstrap intervals. Distinct-mode counts depend on both success and variation among correct answers, and need not move with `pass@8`.

O.4 NATIVE PROVIDER OUTCOMES AND UNDEFINED CONCENTRATION

Provider metadata distinguish refusal and filtering from executable correctness. A refusal does not establish that the underlying model cannot solve the task. Classification uses native stop reasons, refusal fields and filter metadata; it does not detect refusals expressed only in ordinary answer text. An empty visible answer is a failed draw, including when the response contains reasoning output. Empty-answer and refusal counts therefore need not agree.

Correct-pair collision requires at least two correct draws on one prompt. It is undefined otherwise, as is a five-domain mean containing any undefined domain. Dashes denote these undefined values. Accuracy and distinct-mode counts include all responses, with zero correct modes on unsuccessful prompts.

Native completion states over the 15,360 original-instruction responses per deployment are:

- GPT-5.6 Sol — completed: 15,360
- Claude Opus 5 — end_turn: 12,838, refusal: 2,522
- GPT-5.4 — completed: 15,355, incomplete: 5
- Grok 4.3 — stop: 15,360
- Kimi K3 — length: 15, stop: 15,345
- Claude Opus 4.8 — end_turn: 15,355, max_tokens: 5
- DeepSeek V4 Pro — length: 632, stop: 14,728

Claude Opus 5 returns provider-declared refusals for 409, 1,021, 1,024 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. Across those three levels, the provider labels the refusals cyber: 2,454. These labels describe service behavior on factor-selection tasks; they do not measure concentration among correct Python outputs.

Table 49: **Provider-declared refusals occur only for Opus 5 in this cohort.** Each original-instruction cell contains 128 prompts with eight responses each (1,024 responses). R counts native refusal signals, F native content filters, and E responses without visible answer text, including reasoning-only outputs. Counters can overlap and are separate from executable grades. Only nonzero cells appear; the other 83 of 105 cells have $R = F = E = 0$, including all cells for GPT-5.6 Sol, Grok 4.3, Kimi K3. Zero R does not exclude refusals expressed only in answer text.

Deployment	L	Domain	R	F	E
Claude Opus 5	1	Python	409	0	409
	2	Python	1021	0	1021
	2	MathIR	35	0	35
	3	Python	1024	0	1024
	3	MathIR	33	0	32
GPT-5.4	1	PantryPlan	0	0	1
	2	Countdown	0	0	1
	3	Countdown	0	0	1
	3	PantryPlan	0	0	2
Claude Opus 4.8	3	Countdown	0	0	3
	3	PantryPlan	0	0	1
DeepSeek V4 Pro	1	Graph	0	0	59
	1	MathIR	0	0	4
	1	PantryPlan	0	0	13
	2	Graph	0	0	38
	2	Countdown	0	0	1
	2	MathIR	0	0	59
	2	PantryPlan	0	0	80
	3	Graph	0	0	24
	3	Countdown	0	0	7
	3	MathIR	0	0	100
	3	PantryPlan	0	0	246

Table 51: **Verified hosted outputs concentrate on a few observed solution modes.** Each original-instruction cell contains 128 prompts with eight responses each. M/P are mean@8/pass@8 (%); D averages distinct correct solution modes per prompt, including zeros. C is the fraction of correct pairs sharing a mode (%), pooling pairs across prompts; it differs from one minus prompt-averaged $\mathbb{P}_{\text{CMD}}$. Intervals are pointwise 95% percentiles from 2,000 whole-prompt bootstrap resamples within each cell. U is the uniform-correct-mode reference (%), using certified support counts and the same pair weights. A dash in C denotes no eligible pairs; U is also undefined when total support is unknown (Countdown). Grades use the original executable answers.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
GPT-5.6 Sol	1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
	1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	—
	1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
	1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
	1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
	2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
	2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	—
	2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
	2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
	2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9

Strict executable grades, continued.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
Claude Opus 5	3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
	3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
	3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
	3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
	3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8
	1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
	1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
	1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
	1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
	1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
	2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
	2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
	2	Python	0.3	2.3	0.02	–	–
	2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
	2	PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4
	3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
GPT-5.4	3	Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–
	3	Python	0.0	0.0	0.00	–	–
	3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
	3	PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5
	1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
	1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–
	1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4
	1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
	1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
	2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
	2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–
	2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0
	2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
	2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
	3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
	3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–
Grok 4.3	3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1
	3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
	3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6
	1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
	1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
	1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
	1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
	1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
	2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
	2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
	2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
	2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
	2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
	3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
	3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
	3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
Kimi K3	3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
	3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5
	1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
	1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–
	1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4
	1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
	1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
	2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
	2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–
	2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1
	2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
	2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6
	3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
	3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–
	3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2
	3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
Claude Opus 4.8	3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5
	1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
	1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
	1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
	1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
	1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
	2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
	2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
	2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
	2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
	2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
	3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
	3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–

Strict executable grades, continued.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
DeepSeek V4 Pro	3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
	3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
	3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5
	1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
	1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
	1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
	1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
	1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
	2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
	2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
	2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
	2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
	2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
	3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
	3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
	3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
	3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
	3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 52: **Formatting normalization increases accuracy without consistently reducing collision.** Each original-instruction cell contains 128 prompts with eight responses each. M/P are mean@8/pass@8 (%); D averages distinct correct solution modes per prompt, including zeros. C is the fraction of correct pairs sharing a mode (%), pooling pairs across prompts; it differs from one minus prompt-averaged PCMD. Intervals are pointwise 95% percentiles from 2,000 whole-prompt bootstrap resamples within each cell. U is the uniform-correct-mode reference (%), using certified support counts and the same pair weights. A dash in C denotes no eligible pairs; U is also undefined when total support is unknown (Countdown). Typography corrections precede the same executable verifier; mathematical values and program logic are not repaired. Only cells with at least one changed displayed entry appear; omitted cells match at the printed precision. Every displayed entry is unchanged for Grok 4.3.

Deployment	Level	Domain	M	P	D	C (95% interval)	U
GPT-5.6 Sol	1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
	1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
	2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
	2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
	2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
	3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
Claude Opus 5	3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
	3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7
	1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
	2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
	3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
	3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5
GPT-5.4	1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
	1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
	2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
	2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
	3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
	3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
Kimi K3	1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
	1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
	2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
	2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
	2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
	3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
Claude Opus 4.8	3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
	1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
	3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5
DeepSeek V4 Pro	1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
	2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
	3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2

Table 50: **Hosted deployments average fewer than three distinct correct modes per prompt.** Each row averages five domains equally under strict grading and original instructions, with 128 prompts per domain and eight responses per prompt. M/P are mean@8/pass@8 (%); D averages distinct correct solution modes, including zeros. C is correct-pair collision (%), pooling pairs within each domain before averaging domains. Pointwise 95% intervals use 2,000 whole-prompt bootstrap resamples within domains. A dash denotes an undefined domain collision and hence an undefined five-domain mean. Provider settings differ, so these are descriptive deployment comparisons.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	—
Claude Opus 5	3	77.8	79.8	1.07	—
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Model	Level 1				Level 2				Level 3			
	pass@8 (%)	# modes	PCMD		pass@8 (%)	# modes	PCMD		pass@8 (%)	# modes	PCMD	
GPT-5.6 Sol	98.0	1.80	.256		98.4	1.50	.229 [†]		99.2	1.45	.183 [†]	
Claude Opus 5	98.3 [‡]	1.43 [‡]	.140		99.7 [‡]	1.44 [‡]	.164 [†]		99.8 [‡]	1.36 [‡]	.121 [†]	
GPT-5.4	98.1	2.11	.338		98.4	1.72	.247		99.5	1.68	.224	
Grok 4.3	99.5	1.97	.275		100.0	1.57	.179		100.0	1.55	.171	
Kimi K3	98.4	2.31	.374		100.0	2.05	.307		100.0	1.90	.253	
Claude Opus 4.8	98.8	1.64	.217		100.0	1.67	.229		99.8	1.51	.168	
DeepSeek V4 Pro	98.4	2.01	.289		99.1	1.84	.319		99.7	1.62	.232	

Table 53: **High prompt success coexists with few verified solution modes.** Each deployment requests medium reasoning effort, with 128 prompts per domain and level and eight responses per prompt. pass@8 and # modes use formatting-normalized grading and average the five domains equally: pass@8 is empirical prompt success; # modes is mean distinct@8, including zero for prompts without a verified response. [‡] marks the three Claude Opus 5 cells using revised Python wording without a system message (App. O.1). PCMD instead uses strict grading and the original wording throughout, averaging per-prompt diversity over prompts with at least two verified draws and then equally over reportable domains. These are the per-level columns of Table 44, not its common-cell macro. [†] marks four cells omitting Python because fewer than thirty prompts qualify: GPT-5.6 Sol has strict-formatting failures; Opus 5 has provider refusals. All entries are point estimates. Table 54 gives normalized success and mode counts on the four domains with common wording.

Comparison with common prompts. The accuracy panels of Fig. 39 and the pass@8/distinct@8 columns of Table 53 use normalized grading, with revised Python wording for Claude Opus 5. The lower figure panels and the table’s PCMD columns instead use strict grading and original wording. Table 54 compares normalized success and mode counts over Graph, Countdown, MathIR and Pantry, the four domains with identical prompts

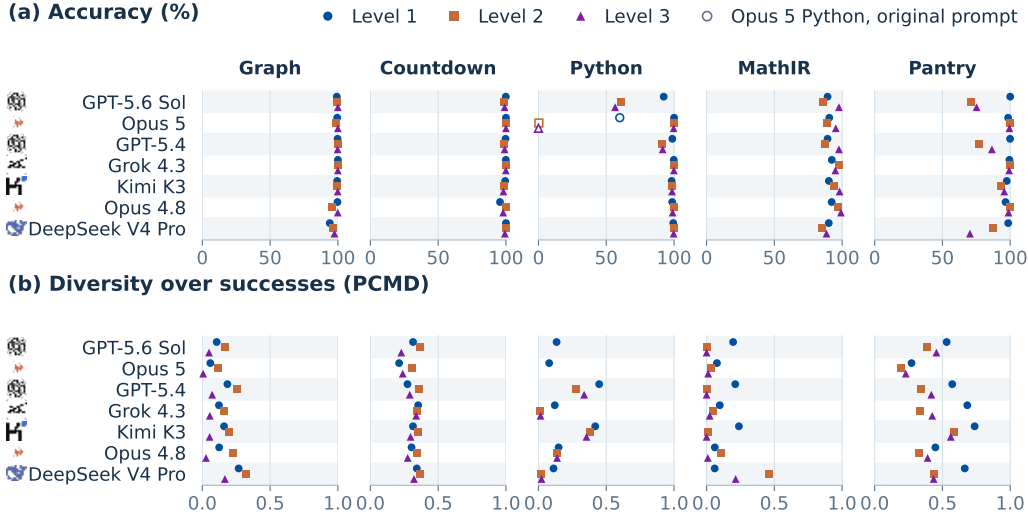


Figure 39: **Accurate hosted responses can still concentrate on few solution modes.** Columns show domains; colors and marker shapes distinguish Levels 1–3. Each deployment–domain–level cell uses 128 prompts and eight responses per prompt. Top: formatting-normalized response accuracy. Solid Opus 5 Python marks use revised wording without a system message; hollow marks show original wording. Bottom: promptwise PCMD under strict grading and original wording for every deployment, averaging over prompts with at least two verified draws. Cells with fewer than thirty eligible prompts are omitted. Thus the two rows use different grading and, for Opus 5 Python, different prompt conditions. All marks are point estimates. Table 53 gives per-level means; Table 54 gives normalized success and mode counts on the four domains with common wording.

across deployments. Claude Opus 5 averages 1.42/1.45/1.35 verified modes at Levels 1–3, compared with 1.43/1.44/1.36 over five domains using revised Python wording and 1.36/1.17/1.08 using original Python wording. It has the lowest mean `distinct@8` at each level in all three comparisons. The revised condition increases its verified Python mode count, which also depends on response success; these means do not compare diversity at matched accuracy. The ordering of the other deployments depends on whether Python is included.

A separate Python comparison in App. K.4 uses original and neutral wording on three deployments, with 16 prompts and 64 responses per prompt at Levels 2 and 3. Pair-pooled diversity $1 - C$ rises by +.024 to +.131 across its six deployment–level cells; the largest value under either wording is .369 (effective-mode transform 1.58). These values weight correct pairs equally, unlike promptwise PCMD. They describe that neutral-wording comparison and do not bound the effect of other wordings or the revised no-system condition on untested deployments.

Model	Level 1		Level 2		Level 3	
	pass@8 (%)	# modes	pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	97.5	1.90	98.2	1.59	99.4	1.54
Claude Opus 5	97.9	1.42	99.6	1.45	99.8	1.35
GPT-5.4	97.7	2.00	98.0	1.68	99.4	1.62
Grok 4.3	99.4	2.12	100.0	1.70	100.0	1.68
Kimi K3	98.0	2.26	100.0	1.95	100.0	1.79
Claude Opus 4.8	98.4	1.70	100.0	1.75	99.8	1.56
DeepSeek V4 Pro	98.0	2.16	98.8	2.04	99.6	1.75

Table 54: **Few verified modes remain when all deployments share the same prompts.** Equal-domain means over Graph, Countdown, MathIR and Pantry, with 128 prompts per domain and level and eight responses per prompt, under normalized grading. Python is excluded for every deployment. `pass@8` is empirical prompt success; # modes is mean `distinct@8` over all prompts, including those with no verified response. Entries are point estimates; response accuracy and provider compute are not matched.

O.5 THE REVISED PYTHON WORDING ON A SECOND DEPLOYMENT

The original and revised Python conditions of App. O.1 are evaluated on GPT-5.6 Sol through `api.openai.com`. Each condition uses the same 384 problems, with 128 prompts per level and eight responses per prompt, medium reasoning effort and an 8,192-token output cap. The original condition uses the benchmark request; the revised condition asks for a direct expression without a system message. Public task inputs, allowed operators and the verifier are unchanged, but wording and system-message presence change together. All responses report `gpt-5.6-sol`; build equality with the Azure deployment used in the hosted averages is not established. These are separate populations.

Verified success and mode counts decrease under the revised condition. There are no provider-declared refusals among the 6,144 responses in this comparison. The Azure GPT-5.6 Sol cohort also contains no provider-declared refusals; its missing strict Python diversity entries reflect too few verified draws. Under normalized grading, the revised condition lowers accuracy and mean `distinct@8` at each level (Table 55). Revised-minus-original differences use all 128 prompts per level, pairing problems across conditions and resampling whole eight-response groups: 20,000 replicates give pointwise 95% percentile intervals. Accuracy differences are fractions of responses; mode-count differences are modes per prompt: accuracy $-.689 [-.745, -.631]$, $-.420 [-.467, -.374]$, $-.370 [-.418, -.320]$ at Levels 1–3; `distinct@8` $-.945 [-1.078, -.820]$, $-.703 [-.812, -.594]$, $-.656 [-.766, -.547]$ respectively. The revised normalized cells have 28, 26, 25 prompts with at least two verified responses, all below the 30-prompt reporting threshold. Their success-conditional diversity is therefore omitted; these cells do not support a diversity contrast at the stated reporting threshold.

Formatting failures accompany the lower scores. The literal `\text{}` marker appears in 862 original-condition and 2,089 revised-condition responses (28% and 68%). The formatting normalizer does not remove this wrapper; none of these responses verifies. Among responses without the marker, normalized success is 94.9% and 59.3%, respectively. Absence of this marker does not guarantee parseability. These condition-dependent subsets are not matched semantic comparisons and provide no bound on a content or capability effect.

Prompt-interface sensitivity differs across deployments. The revised condition increases normalized success and verified mode counts for Opus 5 (App. O.1) but decreases both for this GPT-5.6 Sol deployment. This comparison does not isolate task wording from the system message, establish that either task is generally easier, or identify the effect on the remaining hosted deployments. The four-domain comparison in Table 54 holds the task prompts fixed across all seven deployments by excluding Python.

Table 55: The revised Python condition lowers normalized success and verified mode counts on GPT-5.6 Sol. Both conditions use the same 128 problems per level and eight responses per prompt through `api.openai.com`, with medium reasoning effort and an 8,192-token output cap. R counts provider-declared refusals; A is response accuracy (%); D is mean `distinct@8` over all prompts. $1 - C$ is pair-pooled diversity: the fraction of correct pairs with different solution modes, weighting each correct pair equally across prompts. Dashes mark cells with fewer than 30 prompts containing at least two verified draws. Strict grading uses the original output; normalized grading applies the same typography rules to both conditions. Entries are point estimates; paired uncertainty for accuracy and mode-count differences is given in the text.

Wording	L	R	Strict			Normalized		
			A	D	$1 - C$	A	D	$1 - C$
Original	1	0	37.0	1.02	.106	89.8	1.36	.114
	2	0	10.4	0.14	—	58.7	1.15	.053
	3	0	13.0	0.16	—	56.3	1.13	.048
Revised, no system	1	0	18.7	0.27	—	20.9	0.41	—
	2	0	13.4	0.26	—	16.7	0.45	—
	3	0	15.5	0.25	—	19.3	0.48	—

Model	Level 1			Level 2			Level 3		
	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD
🌀 GPT-5.6 Sol	82.5	1.31	.226	66.9	0.76	.063	76.2	0.88	.095
🌀 GPT-5.4	82.5	1.67	.347	71.2	0.92	.134	70.0	0.94	.160
✂️ Grok 4.3	82.5	1.64	.493	86.9	1.41	.296	90.0	1.45	.314
🐼 Kimi K3	92.5	1.91	.595	92.5	1.37	.335	95.6	1.59	.361
🌟 Claude Opus 4.8	91.2	1.46	.216	100.0	1.62	.226	100.0	1.51	.185

Table 56: Verified solution diversity varies across deployments with explicit reasoning disabled. Each level contains 32 prompts per domain and eight responses per prompt, with formatting-normalized grading. `pass@8` is the fraction with any verified response; `# modes` is mean `distinct@8`, including zero-success prompts. These columns average five domains equally. PCMD instead weights equally the prompts with at least two verified responses in both this condition and the medium condition of Table 57. This selected population can differ between deployments and levels; it does not match overall accuracy. Entries are point estimates.

O.6 MATCHED REASONING-CONTROL COMPARISON

Five deployments each answer the same 480 prompts under medium reasoning and disabled explicit reasoning. The sample uses rows 0–31 in each of five domains and three levels, selected without reference to model outputs. Each condition has eight responses per prompt, or 3,840 responses per deployment. The system and user prompts are identical across conditions, including the original Python wording. These 32-prompt cells are a subset of the 128-prompt medium-reasoning evaluation; the Opus 5 Python wording change in Fig. 39 is outside this comparison.

Both conditions have an 8,192-token output cap. GPT-5.6 Sol and GPT-5.4 request `reasoning.effort=None`; Grok 4.3 and Kimi K3 request `reasoning_effort=None`. Claude Opus 4.8 requests `thinking.type=disabled` with `output_config.effort=medium`. These settings disable explicit reasoning through the provider interface; they do not establish the absence of hidden computation or equal compute across conditions or providers. Both conditions use the same formatting normalization and canonical-key grader. Their responses were collected at different times. Temperature and `top_p` use provider defaults, which can change over time, and matching sample indices do not imply shared random seeds.

Accuracy and conditional diversity use different populations. `pass@8` and `distinct@8` use all 160 prompts at each level, with zero modes for unsolved prompts. Their overall means give every domain and level equal weight. PCMD gives each jointly eligible prompt equal weight: a prompt must contain at least two verified responses in both conditions. The level estimates pool such prompts across domains, and the *All* estimate pools them across domains and levels. Thus their weights depend on eligibility and are not equal-domain averages or means of the three level estimates. A reported PCMD requires at least thirty jointly eligible prompts in the displayed level or overall population. Joint eligibility makes the two conditions comparable on that selected population; it does not hold policy accuracy fixed or describe prompts that only one condition solves twice.

Model	Level	Medium reasoning			Reasoning disabled		
		pass@8 (%)	# modes	PCMD	pass@8 (%)	# modes	PCMD
GPT-5.6 Sol	1	96.9	1.81	.311	82.5	1.31	.226
	2	98.8	1.54	.117	66.9	0.76	.063
	3	98.8	1.51	.195	76.2	0.88	.095
	All	98.1	1.62	.226	75.2	0.99	.145
GPT-5.4	1	96.9	2.06	.338	82.5	1.67	.347
	2	99.4	1.68	.162	71.2	0.92	.134
	3	99.4	1.82	.247	70.0	0.94	.160
	All	98.5	1.85	.255	74.6	1.18	.224
Grok 4.3	1	100.0	1.93	.211	82.5	1.64	.493
	2	100.0	1.62	.136	86.9	1.41	.296
	3	100.0	1.54	.121	90.0	1.45	.314
	All	100.0	1.70	.154	86.5	1.50	.364
Kimi K3	1	96.9	2.27	.343	92.5	1.91	.595
	2	100.0	2.02	.277	92.5	1.37	.335
	3	100.0	1.89	.264	95.6	1.59	.361
	All	99.0	2.06	.293	93.5	1.62	.426
Claude Opus 4.8	1	98.1	1.61	.202	91.2	1.46	.216
	2	100.0	1.67	.218	100.0	1.62	.226
	3	100.0	1.57	.185	100.0	1.51	.185
	All	99.4	1.62	.202	97.1	1.53	.209

Table 57: **Lower success with disabled reasoning accompanies different changes in conditional diversity.** Formatting-normalized grading, eight responses per prompt. At each level, pass@8 and # modes use all 160 prompts across five domains; All uses all 480 prompts. # modes is mean distinct@8. PCMD weights jointly eligible prompts equally, using 65–159 prompts per level and 245–458 overall. Each displayed population exceeds the thirty-prompt reporting threshold. Eligibility can vary by domain, level and deployment, so PCMD uses different weights from the other columns. Entries are descriptive point estimates without uncertainty intervals.

Response coverage and control compliance. DeepSeek V4 Pro has 3,839 disabled-condition responses; one Level-1 PantryPlan prompt has seven instead of eight. The missing response has an unknown outcome and is not scored as a failure. Under strict grading, the 437 jointly eligible prompts have PCMD .282 with medium reasoning and .369 with disabled reasoning, a difference of +.087. The seven-response prompt has only one verified answer and is excluded. This estimate uses different grading from the normalized comparisons.

For Claude Opus 5, a response includes a thinking block and forty thinking tokens despite an explicit request to disable thinking. This violates the requested control, so its responses do not provide a validated reasoning-disabled comparison.

Observed differences and limits. All five tabulated deployments have lower overall pass@8 and distinct@8 under disabled reasoning. Changes in conditional diversity differ: overall PCMD falls for both GPT deployments, rises for Kimi K3 and Grok 4.3, and changes from .202 to .209 for Opus 4.8. For Grok, PCMD rises from .154 to .364 while mean distinct modes falls. The common reduction in raw mode counts therefore does not imply a common increase in concentration among successful responses.

The level estimates also differ by deployment. GPT-5.6 Sol’s PCMD falls at every level; Grok 4.3 and Kimi K3 rise at every level. GPT-5.4 rises by .009 at Level 1 and falls at Levels 2 and 3, so its change reverses sign. Opus 4.8’s level changes range from approximately .0002 to .0142; these small positive estimates do not establish invariance. There are no uncertainty intervals for these comparisons. Eligibility changes the population being summarized, and collection time and provider defaults can confound the requested-control contrast. These observations describe output distributions under the tested configurations, not a causal effect of training or an isolated effect of internal deliberation.

Part VI. Idealized analysis

P MODE COLLAPSE AND VERIFIED REPLAY

For a fixed prompt, the binary-reward score estimators defined below have mean updates along the correctness gradient, while their sampling covariances can differ. With independently optimized categorical logits, these updates amplify differences among equally rewarded modes, and sampling can break an initially exact symmetry. Fixed-bank replay supplies probability floors, restoration thresholds, and recovery bounds under the stated update assumptions. Neural retention requires additional conditions on complete-response likelihoods and local updates. These results do not certify the implemented AdamW/PPO trajectories. Table 2 defines the notation.

P.1 CATEGORICAL ABSTRACTION AND MEAN UPDATES

The categorical model assigns one independent logit to each outcome category. Its Euclidean mean flow and the subsequent finite-step, sampled, and neural updates have different assumptions; a neural solution mode need not have an independently optimized logit.

Assumption P.1 (Categorical mean-flow model). Fix one isolated prompt and a finite category set $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$ with $m \geq 2$ correct solution modes and $n \geq 1$ incorrect categories, and set $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$, so that P measures correctness while q describes how that correct probability is allocated among the individual correct modes.

- (A1) *Independent logits*. The policy carries one independently optimized logit for every category in $\mathcal{A}$, including one separate logit for each canonical correct mode.
- (A2) *Common length normalization*. Every response uses the same fixed length-normalization factor, absorbed into the positive coefficient of the group estimator.
- (A3) *Euclidean mean flow*. Updates are the infinitesimal expected updates in these logits, taken without optimizer preconditioning and without momentum carried between steps.
- (A4) *No competing regularizer*. Explicit reference KL is zero and no entropy term is present.
- (A5) *Nondegenerate start*. Logits are finite initially, $0 < P(0) < 1$, and the group size is $G \geq 2$.

Assumption P.1 includes all five conditions. AdamW’s preconditioning and momentum violate (A3); finite, repeated PPO updates are outside its infinitesimal on-policy limit. Response-specific length normalization violates (A2), while reference KL and entropy regularization violate (A4).

A language model induces probabilities over canonical outcomes, but those probabilities alone do not determine its update geometry. Aggregating several response strings into a mode does not generally commute with gradient flow in response logits. The independent-category model therefore requires an additional assumption beyond a neural policy and its validator equivalence classes.

Draw a group $Z_1, \dots, Z_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[Z_i \in \mathcal{C}]$, and let $R = \sum_i R_i$. At the on-policy point where the PPO ratio equals one, consider the common-length estimator

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(R) \left(R_i - \frac{R}{G} \right) \nabla_z \log p_{Z_i}. \quad (14)$$

For the Dr.GRPO abstraction (Liu et al., 2025), $w(R) = 1$ after absorbing the fixed positive length-normalization factor. Reward-standardized GRPO (Shao et al., 2024) with fixed common response-length normalization takes $w(R)$ to be the reciprocal group reward standard deviation on mixed groups (with a fixed numerical stabilizer if used). Standard variable-length GRPO also weights each response by its own inverse length; that response-specific factor cannot generally be absorbed into $w(R)$ and is outside this equivalence. In either included case, $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. The entire update is zero on all-equal groups, without evaluating an undefined reciprocal standard deviation; likewise, $w(R)R(G-R)$ below is defined as zero when $R = 0$ or $R = G$. All advantages and normalization weights are detached in the score estimator.

Lemma P.2 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected update in Equation (14) is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(R)R(G-R)]}{G^2P(1-P)} > 0. \quad (15)$$

For the Dr.GRPO abstraction, $c_G(P) = (G-1)/G$. Group centering and reward standardization rescale the expected gradient without changing its direction under these assumptions.

Proof. Condition on the complete binary reward vector, whose sum is R . The score identities give

$$\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 0] = \nabla_z \log(1 - P).$$

There are R correct rows with centered reward $(G-R)/G$ and $G-R$ incorrect rows with centered reward $-R/G$. Therefore the conditional expectation of the unaveraged sum in (14) is

$$w(R) \frac{R(G-R)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(R) \frac{R(G-R)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $R \sim \text{Binomial}(G, P)$ yield (15). For $w \equiv 1$, $\mathbb{E}[R(G-R)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1) P^j (1-P)^{G-2-j}.$$

The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. Thus c_G extends smoothly and positively to $P = 0, 1$.

The practical MaxRL estimator has the same mean-gradient direction (Tajwar et al., 2026, Section 4.3 and App. D, Theorem 5). Its all-failure convention determines the finite-group coefficient.

Lemma P.3 (Binary MaxRL has the same correctness-gradient direction). *Under Assumption P.1, let MaxRL assign $A_i^{\text{MaxRL}} = GR_i/R - 1$ when $R > 0$ and zero when $R = 0$, as in Section 3. For $\hat{g}_G^{\text{MaxRL}} = G^{-1} \sum_i A_i^{\text{MaxRL}} \nabla_z \log p_{Z_i}$ at the on-policy point, the expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1-P)^{G-1}}{P} > 0, \quad (16)$$

with continuous endpoint values $c_G^{\text{MaxRL}}(0) = G-1$ and $c_G^{\text{MaxRL}}(1) = 1$.

Proof. With $N = G$ and pass rate P , the estimator in Tajwar et al. (2026, Theorem 5) equals $\hat{g}_G^{\text{MaxRL}}$. Its expected coefficient is $\sum_{j=0}^{G-2} (1-P)^j$, whose geometric sum gives Equation (16) and its endpoint values. The identity requires on-policy independent draws and the score identity. $\square$

Integrating this coefficient gives the potential

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1-P)^k}{k}. \quad (17)$$

In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, best@ k equals pass@ k , so this is a harmonic mixture of sampling objectives for $k = 1, \dots, G-1$. The upper limit is $G-1$ because the implemented centered estimator sets the entire update to zero on all-failure groups. Subtracting an unconditional score baseline even on those groups instead gives the order- G estimator. These are exact finite-group expectations at the on-policy point; subsequent clipping and adaptive optimization can alter the expected update.

P.2 WINNER-TAKE-ALL COLLAPSE IN THE CATEGORICAL MEAN FLOW

Consider $\dot{z} = c_G(P)\nabla_z P$, with any of the smooth positive coefficients established above: Dr.GRPO, common-length reward-standardized GRPO, or the specified centered binary MaxRL estimator. This infinitesimal expected-update model has no sampling noise and uses independently optimized Euclidean logits. The conditional dynamics follow the frequency-dependent replicator system with fitness $f_c(q) = q_c$ (Hofbauer & Sigmund, 1998; Harper, 2009). Related analyses of collapse among equally rewarded outcomes include Sinha et al. (2026); Lochab et al. (2026).

Theorem P.4 (Winner-take-all collapse in the categorical mean flow). *Under Assumption P.1, if the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \rightarrow 1, \quad q_{c^*}(t) \rightarrow 1, \quad q_c(t) \rightarrow 0 \quad (c \neq c^*).$$

Thus these categorical mean flows converge to a single correct solution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward.

Remark P.5. Theorem P.4 is asymptotic and requires (A1)–(A5). Finite-time support loss and convergence of the stochastic neural optimizers used in the experiments are outside its scope.

Proof. The coefficient c_G is smooth and bounded on $[0, 1]$, and the softmax reward gradient is bounded. Thus the logit flow exists for all finite times and cannot develop infinite logits at finite time. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P)p_c(1 - P), \quad \dot{z}_a = -c_G(P)p_aP.$$

Also $\dot{P} = c_G(P)\|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (18)$$

The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ; normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly uniform correct distribution is therefore a special initial condition. The theorem concerns limiting concentration: finite softmax logits already give positive probabilities at every finite time, so finite-time positivity alone is not a retention result.

P.3 CONDITIONAL TRAJECTORIES OF OUTCOME-BINARY MEAN FLOWS

Under Assumption P.1, every outcome-binary estimator with a positive correctness coefficient induces the same conditional trajectory in q . Its advantage changes only the rate at which that trajectory is traversed. Conditioning on correctness averages over all correct modes, so a reward-dependent advantage cannot distinguish their identities.

Proof of Theorem 3.1. The score identities used in Lemma P.2 give $\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 1] = \nabla_z \log P$ and $\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 0] = \nabla_z \log(1 - P)$. Independence of the draws means that conditioning also on the other rows' rewards does not change either conditional mean. Since $a(R_i, R)$ depends only on the binary rewards, conditioning on their complete vector and summing the R successful and $G - R$ failed rows gives

$$\mathbb{E}[\hat{g}_a] = \frac{1}{G} \left(\mathbb{E}[R a(1, R)] \nabla_z \log P + \mathbb{E}[(G - R) a(0, R)] \nabla_z \log(1 - P) \right).$$

Substituting $\nabla_z \log P = \nabla_z P / P$ and $\nabla_z \log(1 - P) = -\nabla_z P / (1 - P)$ yields Equation (3). Hence $\frac{d}{dt} \log(q_c/q_d) = \dot{z}_c - \dot{z}_d = c_a(P) P(1 - P)(q_c - q_d)$. For $c_a > 0$, the positive time change $d\tau = c_a(P) P(1 - P) dt$ gives the replicator dynamics in Equation (18). $\square$

For a fixed reward-dependent advantage, the binomial identities $R \binom{G}{R} = G \binom{G-1}{R-1}$ and $(G - R) \binom{G}{R} = G \binom{G-1}{R}$ give

$$c_a(P) = \sum_{j=0}^{G-1} \binom{G-1}{j} P^j (1 - P)^{G-1-j} [a(1, j+1) - a(0, j)], \quad (19)$$

a polynomial in P of degree at most $G - 1$. Finite advantage values therefore give a smooth, bounded coefficient on $[0, 1]$.

The coefficients in this Bernstein expansion are

$$d_j := a(1, j+1) - a(0, j), \quad j = 0, \dots, G-1. \quad (20)$$

With the other $G - 1$ rows fixed at j successes, d_j is the change in a row's advantage when its reward changes from failure to success. The Bernstein basis is nonnegative on $[0, 1]$ and sums to one, so $c_a(P)$ lies between $\min_j d_j$ and $\max_j d_j$. Each basis function is strictly positive on $(0, 1)$; hence nonnegative gaps, with at least one positive, imply $c_a(P) > 0$ throughout the interior. This condition is sufficient: mixed-sign gaps alone do not determine the sign of c_a .

For Dr.GRPO, every $d_j = (G - 1)/G$, giving the constant coefficient in Lemma P.2. For the centered MaxRL estimator, $d_0 = G - 1$ and $d_j = G/(j + 1)$ for $1 \leq j \leq G - 1$. In particular, $a(1, G) = 0$ and $a(0, G - 1) = -1$ give $d_{G-1} = 1$. Substitution into Equation (19) gives $[1 - (1 - P)^{G-1}]/P$, as in Lemma P.3.

Corollary P.6 (Collapse for correctness-improving outcome-binary mean flows). *Under Assumption P.1, let $a(R_i, R)$ be a fixed, bounded advantage depending only on the binary rewards. Suppose the gaps in Equation (20) satisfy $d_j \geq 0$ for every j , with at least one strictly positive gap. If $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

The surviving correct mode is the one with the largest initial probability; the choice of a changes the rate along the conditional trajectory.

Proof. By Theorem 3.1, the flow is $\dot{z} = c_a(P) \nabla_z P$. Equation (19) makes c_a smooth and bounded on $[0, 1]$. If $d_{j_0} > 0$, the nonnegative-gap hypothesis gives

$$c_a(P) \geq \binom{G-1}{j_0} P^{j_0} (1 - P)^{G-1-j_0} d_{j_0} > 0 \quad (0 < P < 1).$$

These are the coefficient properties used in the proof of Theorem P.4: the logit flow exists for all finite times, $P \rightarrow 1$, and the effective time $\tau \rightarrow \infty$. The conditional replicator equation (18) then eliminates all coordinates strictly below the initial maximum. It contains neither the advantage coefficients nor the rollout group size, so the surviving mode depends only on $q(0)$. $\square$

The nonnegative-gap condition is sufficient rather than necessary. An estimator with mixed-sign gaps also has the same limiting behavior if $c_a(P) > 0$ throughout $(0, 1)$. An interior zero of c_a is a stationary correctness level for the mean flow, while a negative coefficient gives $\dot{P} = c_a(P) \|\nabla_z P\|_2^2 < 0$ at an interior policy. The collapse conclusion requires positivity and an initial imbalance under Assumption P.1; it does not describe shared-prompt or sampled neural updates.

Remark P.7 (Group size in the deterministic conditional flow). For each $G \geq 2$, group size enters the mean flow through $c_a(P)$. When this coefficient is positive, the time change in Theorem 3.1 gives the same conditional replicator equation. Increasing G can change the speed of the mean flow but cannot change the surviving mode under the conditions of Corollary P.6. Group size also changes the probability of a mixed-reward group in Lemma P.18, which is small near both $P = 0$ and $P = 1$, and the MaxRL coefficient in Corollary P.8. These properties concern total correctness P ; they do not provide a probability floor for an individual correct mode. Gradient covariance and finite-sample symmetry breaking can depend on G , so sampled updates need not share this equivalence.

Corollary P.8 (MaxRL amplification of the correctness gradient). *For the coefficients of Lemmas P.2 and P.3,*

$$\frac{c_G^{\text{MaxRL}}(P)}{c_G^{\text{Dr.GRPO}}} = \frac{G[1 - (1 - P)^{G-1}]}{(G - 1)P}.$$

This ratio is nonincreasing in P , with limits G as $P \rightarrow 0$ and $G/(G - 1)$ as $P \rightarrow 1$. It is strictly decreasing for $G > 2$ and constant at 2 for $G = 2$. At the same policy, MaxRL therefore multiplies the Dr.GRPO correctness-gradient coefficient by a factor approaching G when correctness is small. The probability of a nonzero fresh task component remains bounded by the mixed-group probability in Lemma P.18 for both estimators.

Proof. Divide $c_G^{\text{MaxRL}}(P) = [1 - (1 - P)^{G-1}]/P$ by $(G - 1)/G$. The finite-series identity $c_G^{\text{MaxRL}}(P) = \sum_{j=0}^{G-2} (1 - P)^j$ gives the endpoint limits and monotonicity. For $G = 2$ the sum is the constant 1; for $G > 2$ it contains the strictly decreasing term $1 - P$. $\square$

P.4 NEURAL SCORE IDENTITIES AND THE LIMITS OF CONDITIONAL INERTNESS

Expected binary reward equals the total correct mass; conditional diversity depends on its allocation among solution modes. Sinha et al. (2026, Theorem 3.1, v1) derive the expected-return log-ratio dynamics for softmax outcome logits, and Lochab et al. (2026, Section 4.1, v1) identify binary reward’s indifference to allocation within the correct set. For differentiable neural policies, outcome-binary group estimators have per-prompt means proportional to the correctness gradient but can differ in covariance. Finite steps, shared prompts, and response-dependent weighting limit the resulting trajectory equivalence.

Theorem P.9 (Per-prompt neural mean and covariance). *Fix a prompt and a differentiable policy π_θ on a finite or countable response space with common positive support, where θ lies in an open subset of $\mathbb{R}^d$. Let $\mathcal{C}$ be a fixed verifier-positive event, $P = \Pr_\theta(Y \in \mathcal{C}) \in (0, 1)$, and $s(Y) = \nabla_\theta \log \pi_\theta(Y)$. Assume differentiation may be interchanged with the probability sums over the full support and over $\mathcal{C}$, and that $\mathbb{E}\|s(Y)\|_2^2 < \infty$. For G independent responses, write $R_i = \mathbf{1}[Y_i \in \mathcal{C}]$, $R = \sum_i R_i$, and let $a : \{0, 1\} \times \{0, \dots, G\} \rightarrow \mathbb{R}$ be any finite, detached advantage. Define*

$$\hat{g}_a = \frac{1}{G} \sum_i a(R_i, R) s(Y_i), \quad v = \nabla_\theta P, \quad \Sigma_r = \text{Cov}(s(Y) \mid \mathbf{1}[Y \in \mathcal{C}] = r).$$

With

$$\beta_r = \frac{1}{G} \left(\frac{r a(1, r)}{P} - \frac{(G - r) a(0, r)}{1 - P} \right), \quad c_a(P) = \mathbb{E}[\beta_R],$$

where $R \sim \text{Binomial}(G, P)$, the exact identities are

$$\mathbb{E}[\hat{g}_a] = c_a(P) v, \tag{21}$$

$$\text{Cov}(\hat{g}_a) = \text{Var}(\beta_R) v v^\top + \frac{1}{G^2} \mathbb{E}[R a(1, R)^2 \Sigma_1 + (G - R) a(0, R)^2 \Sigma_0]. \tag{22}$$

In particular, c_a is the coefficient in Equation (3), without an independent-logit assumption.

Proof. The conditional-score identity (Tajwar et al., 2026, Theorem 1) (also He et al., 2026, Lemma B.1) applied to the correct and incorrect events gives

$$\mathbb{E}[s(Y) \mid Y \in \mathcal{C}] = v/P, \quad \mathbb{E}[s(Y) \mid Y \notin \mathcal{C}] = -v/(1 - P).$$

Conditioned on the complete reward vector, the response scores remain independent. The conditional mean of $\hat{g}_a$ is $\beta_R v$ and its conditional covariance is $G^{-2}[Ra(1, R)^2 \Sigma_1 + (G - R)a(0, R)^2 \Sigma_0]$. Taking expectations proves Equation (21); the law of total covariance proves Equation (22). $\square$

For one prompt, a common locally Lipschitz preconditioner $M(\theta)$ and positive coefficients $c_a(P)$ give deterministic mean fields $\dot{\theta} = c_a(P)M(\theta)\nabla_\theta P$ that trace the same orbit under a positive change of time, wherever the flows exist uniquely. This is a statement about the mean direction. Equation (22) shows that outcome-binary estimators can have different sampling covariance, including after their mean magnitudes are matched.

Corollary P.10 (A finite-step comparison in neural parameters). *Under Theorem P.9, suppose $c_a(P) > 0$ and take $\theta^+ = \theta + \eta \hat{g}_a / c_a(P)$ with $\eta > 0$. Let f be a twice differentiable scalar observable with $\|\nabla^2 f\|_{\text{op}} \leq L_f$ along every update segment, which is assumed to remain in its domain. Then, conditional on the current policy,*

$$|\mathbb{E}[f(\theta^+) - f(\theta)] - \eta \nabla f(\theta)^\top v| \leq \frac{L_f \eta^2}{2} \left(\|v\|_2^2 + \frac{\text{tr Cov}(\hat{g}_a)}{c_a(P)^2} \right). \quad (23)$$

Proof. Taylor’s theorem bounds the remainder after the linear term by $L_f \eta^2 \|\hat{g}_a\|_2^2 / (2c_a(P)^2)$. Take expectations and use Equation (21) together with $\mathbb{E}\|\hat{g}_a\|_2^2 = c_a(P)^2 \|v\|_2^2 + \text{tr Cov}(\hat{g}_a)$. $\square$

For two estimators satisfying the corollary, division by their respective coefficients $c_a(P)$ gives the same mean step ηv ; the training updates do not use this normalization. Their expected changes in f then differ by at most the sum of the two second-order bounds. Conditional solution mode probabilities or PCMD may be used as f where the stated smoothness bounds hold. Mean-direction equivalence is therefore a first-order statement for stochastic finite updates; it does not determine their trajectories or eventual surviving solution modes. The bound assumes fresh on-policy score updates and does not cover Adam’s moment estimates or subsequent PPO epochs.

Remark P.11 (Shared prompts need not follow the same trajectory). For a fixed prompt-sampling distribution μ_x and shared parameters, the aggregate mean, assuming integrable updates, is

$$\mathbb{E}[\hat{g}_a] = \sum_x \mu_x c_a(P_x) \nabla_\theta P_x. \quad (24)$$

Different objectives generally assign different relative coefficients to the prompt gradients, so the per-prompt identity does not imply a common aggregate orbit. An explicit example also shows a difference in conditional diversity. Let $\theta = (u, v)$, $\sigma(t) = (1 + e^{-t})^{-1}$, and sample two prompts A, B equally. Set

$$P_A = \sigma(u + v), \quad P_B = \sigma(u - v), \quad q_A = (\sigma(v), 1 - \sigma(v)).$$

For prompt A , the probabilities of two correct responses and one incorrect response are $(P_A \sigma(v), P_A [1 - \sigma(v)], 1 - P_A)$; for B , use $(P_B/2, P_B/2, 1 - P_B)$. At $(u, v) = (0, \log 3)$, $P_A = 3/4$, $P_B = 1/4$, and $\nabla P_A = (3/16, 3/16)$, $\nabla P_B = (3/16, -3/16)$. For $G = 3$, Lemmas P.2 and P.3 give $c_{\text{Dr}} = 2/3$ and $c_{\text{Max}}(P) = 2 - P$, hence

$$\dot{\theta}_{\text{Dr}} = (1/8, 0), \quad \dot{\theta}_{\text{Max}} = (9/32, -3/64).$$

Prompt A has conditional disagreement $D_A = 1 - \sum_c q_{A,c}^2 = 2\sigma(v)[1 - \sigma(v)]$, so direct differentiation gives

$$\dot{D}_{A,\text{Dr}} = 0, \quad \dot{D}_{A,\text{Max}} = 9/1024 > 0.$$

Both prompts’ correctness increases under both mean fields, since $\dot{P}_A = (3/16)(\dot{u} + \dot{v}) > 0$ and $\dot{P}_B = (3/16)(\dot{u} - \dot{v}) > 0$ at this point. Changing a binary objective can therefore change conditional diversity through shared prompt gradients, despite the exact per-prompt identity.

Proposition P.12 (The residual from response-dependent normalization). *Under Theorem P.9, multiply each response’s score by a detached function $\omega(Y)$ with finite conditional second moments; inverse response length is the case $\omega(Y) = 1/L(Y)$ with $L(Y) > 0$. Define*

$$\hat{g}_{a,\omega} = \frac{1}{G} \sum_i a(R_i, R) \omega(Y_i) s(Y_i), \quad A_1 = \frac{\mathbb{E}[Ra(1, R)]}{G}, \quad A_0 = \frac{\mathbb{E}[(G - R)a(0, R)]}{G},$$

and, conditioning on $\mathbf{1}[Y \in \mathcal{C}] = r$, let $\bar{\omega}_r = \mathbb{E}[\omega(Y) \mid r]$, $K_r = \text{Cov}(\omega(Y), s(Y) \mid r)$, and $\sigma_{\omega,r}^2 = \text{Var}(\omega(Y) \mid r)$. Then

$$\mathbb{E}[\hat{g}_{a,\omega}] = \tilde{c}_a \nabla_\theta P + A_1 K_1 + A_0 K_0, \quad \tilde{c}_a = \frac{A_1 \bar{\omega}_1}{P} - \frac{A_0 \bar{\omega}_0}{1 - P}, \quad (25)$$

$$\|A_1 K_1 + A_0 K_0\|_2 \leq \sum_{r=0}^1 |A_r| \sigma_{\omega,r} \sqrt{\text{tr } \Sigma_r}. \quad (26)$$

Proof. Conditioning on the reward vector gives $\mathbb{E}[\hat{g}_{a,\omega}] = A_1 \mathbb{E}[\omega s \mid 1] + A_0 \mathbb{E}[\omega s \mid 0]$. For each outcome, $\mathbb{E}[\omega s \mid r] = \bar{\omega}_r \mathbb{E}[s \mid r] + K_r$; substituting the conditional score means proves Equation (25). Cauchy–Schwarz gives $\|K_r\|_2 \leq \sigma_{\omega,r} \sqrt{\text{tr } \Sigma_r}$; summing these bounds with weights $|A_r|$ proves Equation (26). $\square$

The residual is a within-outcome covariance between response weighting and the policy score. For inverse-length weighting, a common response length makes it zero; equal mean lengths for correct and incorrect responses need not. Response-dependent weights need not be functions of binary outcomes alone. IPS uses outcome-frequency weights (Sinha et al., 2026, Section 4.4, v1), and UCPO adds a conditional-uniformity term (Lochab et al., 2026, Section 6, v1); these interventions are outside the outcome-binary class analyzed above.

The verified-key diversity metric is a specialization of SetPO’s distributional functional. Its influence gives an exact metric gradient for shared neural parameters and fixed verified execution keys.

Corollary P.13 (Verified-key specialization of SetPO). *Fix a prompt, a verifier-positive response event $\mathcal{C}$, and a deterministic key map $V : \mathcal{C} \rightarrow \{1, \dots, m\}$, all independent of θ . Let π_θ be differentiable with common positive support, $P_\theta = \Pr_\theta(\mathcal{C}) > 0$, and $Q_\theta = \pi_\theta(\cdot \mid \mathcal{C})$. Assume the score $s_\theta(Y) = \nabla_\theta \log \pi_\theta(Y)$ is integrable under Q_θ and differentiation commutes with the response sums over each key. Write $q_c = Q_\theta(V(Y) = c)$ and $S_2 = \sum_c q_c^2$. For $k(y, y') = \mathbf{1}[V(y) = V(y')]$ and $g(u) = 1 - u$, the functional $\mathcal{F}(Q) = \mathbb{E}_{Y \sim Q}[g(\mathbb{E}_{Y' \sim Q} k(Y, Y'))]$ is exactly $\text{PCMD}(q)$. The influence formula of Li et al. (2026a, Theorem 4.2, v1) specializes, for $y \in \mathcal{C}$, to*

$$\left. \frac{d}{d\varepsilon} \mathcal{F}((1 - \varepsilon)Q_\theta + \varepsilon \delta_y) \right|_{\varepsilon=0} = 2(S_2 - q_{V(y)}), \quad (27)$$

$$\nabla_\theta \text{PCMD}(q) = 2\mathbb{E}_{Y \sim Q_\theta}[(S_2 - q_{V(Y)})s_\theta(Y)]. \quad (28)$$

Proof. The equality kernel and g satisfy SetPO’s Assumption 4.1, and its local mass is $q_{V(y)}$. Substitution into its influence formula proves the first identity. The influence has mean zero under Q_θ , so the $-\nabla_\theta \log P_\theta$ term in the conditional score cancels, giving the second identity. $\square$

These identities permit shared neural parameters. Along an update, the sign of the diversity change depends on its alignment with this metric gradient; the influence formula alone supplies no monotonicity guarantee for a given training direction.

P.5 DIVERSITY AT FINITE STEPS AND SAMPLING BUDGETS

Softmax outcome logits amplify probability differences among equally rewarded solutions (Sinha et al., 2026, Theorem 3.1 and App. A.1), and sampling imbalances can reinforce this effect (Lochab et al., 2026, Section 4.2). Under the categorical model, PCMD decreases from every nonuniform correct-mode distribution along both the mean flow and finite expected-gradient steps. Sampled updates can also break an initially uniform distribution’s symmetry. The bounds below relate concentration to the probability of observing a competing solution mode within a finite inference budget.

Proposition P.14 (Exact diversity drift and matched correctness). *Under Assumption P.1, consider $\dot{z} = c(P)\nabla_z P$ with $c(P) > 0$. Write $D(q) = 1 - \sum_c q_c^2$, $S_2 = \sum_c q_c^2$, and $V(q) = \sum_c q_c^3 - S_2^2 = \text{Var}_{C \sim q}(q_C)$. For incorrect conditional probabilities $r_a = p_a/(1 - P)$, set $T_2 = \sum_a r_a^2$. In effective time $d\tau = c(P)P(1 - P)dt$,*

$$\frac{dD}{d\tau} = -2V(q), \quad \frac{dD}{d \log[P/(1 - P)]} = -\frac{2V(q)}{S_2 + T_2}. \quad (29)$$

Thus diversity strictly decreases at every nonuniform interior q . Positive-coefficient mean flows from the same initial logits follow the same D -versus- P curve, independently of their chosen outcome-binary advantage or group size.

Proof. For the replicator flow (18), the potential $U(q) = S_2/2$ has fitness $\partial U/\partial q_c = q_c$. The fitness-variance identity of Harper (2009, Theorems 1–2) gives $dU/d\tau = V(q)$, hence $dD/d\tau = -2V(q)$. The variance vanishes exactly when the positive coordinates of q are equal. Also,

$$\dot{P} = c(P)\|\nabla_z P\|_2^2 = c(P)P^2(1 - P)^2(S_2 + T_2),$$

so $d \log[P/(1 - P)]/d\tau = S_2 + T_2 > 0$, proving the second identity. Finally, the full logit equation in effective time is $dz_c/d\tau = q_c$ and $dz_a/d\tau = -r_a$; it contains no c . The same initial logits therefore give the same full trajectory after the time change, and hence the same curve at matched correctness for every positive-coefficient outcome-binary estimator considered here. $\square$

Theorem P.15 (Collapse under finite expected-gradient steps). *Retain (A1), (A2), (A4), and (A5) of Assumption P.1, and replace (A3) by*

$$z_{k+1} = z_k + \eta_k c(P_k) \nabla_z P_k, \quad 0 < \eta_k < \infty, \quad \sum_{k=0}^{\infty} \eta_k = \infty, \quad (30)$$

where c is continuous and positive on $(0, 1)$. Each step strictly decreases $D(q_k)$ unless q_k is uniform. If q_0 has a unique largest coordinate c^ , then $P_k \rightarrow 1$ and $q_{c^*,k} \rightarrow 1$. No upper bound on the finite step sizes is required.*

Proof. Let $h_k = \eta_k c(P_k) P_k (1 - P_k) > 0$. The exact updates are $z_c^+ = z_c + h_k q_c$ and $z_a^+ = z_a - h_k r_a$, so

$$q_c^+ = \frac{q_c e^{h_k q_c}}{\sum_d q_d e^{h_k q_d}}. \quad (31)$$

For fixed q , put $u_c(s) = q_c e^{s q_c} / \sum_d q_d e^{s q_d}$. Differentiation gives

$$\frac{d}{ds} \sum_c u_c(s)^2 = \sum_{c,d} u_c(s) u_d(s) (u_c(s) - u_d(s)) (q_c - q_d) \geq 0.$$

The ordering of $u(s)$ is that of q , and the inequality is strict whenever q is nonuniform. Integrating from 0 to h_k proves the diversity claim. Equation (31) also preserves the leader and makes its conditional probability nondecreasing.

Every correct logit increases and every incorrect logit decreases, hence P_k increases. Jensen's inequality gives

$$\log \frac{P_{k+1}}{1 - P_{k+1}} - \log \frac{P_k}{1 - P_k} = \log \sum_c q_c e^{h_k q_c} - \log \sum_a r_a e^{-h_k r_a} \geq h_k \sum_c q_c^2 \geq h_k/m.$$

If P_k had an interior limit, continuity and positivity of c would imply $\sum_k h_k = \infty$, contradicting bounded correctness odds. Thus $P_k \rightarrow 1$. Moreover, $\sum_k h_k = \infty$ is necessary: otherwise every logit would have finite total variation and converge to a finite value, contradicting $P_k \rightarrow 1$. Set $\alpha = q_{c^*,0}$ and $\xi_{c,k} = q_{c,k}/q_{c^*,k}$. For $c \neq c^*$,

$$\xi_{c,k+1} = \xi_{c,k} e^{-h_k (q_{c^*,k} - q_{c,k})}, \quad q_{c^*,k} - q_{c,k} \geq \alpha(1 - \xi_{c,0}) > 0.$$

The divergent sum of h_k forces every such ratio to zero, proving $q_{c^*,k} \rightarrow 1$. $\square$

The update uses the exact expected gradient. Finite steps preserve the concentration conclusion but can change the trajectory across estimators. The identical diversity-versus-correctness curve in Proposition P.14 requires infinitesimal mean flow; it does not extend to these finite updates.

Finite multinomial counts can break a uniform policy’s symmetry (Lochab et al., 2026, Proposition A.1). For the centered group estimator, the leading expected one-step loss in PCMD is quadratic in the learning rate, with a coefficient determined by group size and correctness.

Proposition P.16 (Sampled updates break conditional symmetry). *Retain (A1), (A2), (A4), and (A5) of Assumption P.1, and replace (A3) by one sampled update $z^+ = z + \eta \hat{g}_G$ from Equation (14), with $\eta > 0$. Suppose $0 < P < 1$ and $q_c = 1/m$ for every correct mode. Then*

$$\mathbb{E}[D(q^+)] < 1 - 1/m, \quad \mathbb{E}[q_c^+] = 1/m. \quad (32)$$

As $\eta \rightarrow 0$, for the fixed initial policy and estimator,

$$\mathbb{E}[D(q^+)] = 1 - \frac{1}{m} - \eta^2 \frac{m-1}{m^3} \mathbb{E} \left[\frac{w(R)^2 R(G-R)^2}{G^4} \right] + O(\eta^3), \quad (33)$$

where the summand is zero on $R = 0, G$. For Dr.GRPO, $w = 1$ and

$$\mathbb{E} \left[\frac{R(G-R)^2}{G^4} \right] = \frac{G-1}{G^3} P(1-P) [1 + (G-2)(1-P)]. \quad (34)$$

Proof. Let N_c count occurrences of correct mode c . The centered advantages sum to zero, so the common softmax-score term cancels and $\hat{g}_c = \kappa_R N_c$, where $\kappa_R = w(R)(G-R)/G^2$ on mixed groups and $\kappa_R = 0$ otherwise. Conditional on R , the correct counts are $N \sim \text{Multinomial}(R; 1/m, \dots, 1/m)$, and

$$q_c^+ = \frac{e^{\eta \kappa_R N_c}}{\sum_d e^{\eta \kappa_R N_d}}.$$

Diversity is at most $1 - 1/m$, with equality only at uniform q^+ . The event $R = 1$ has positive probability; on this event one correct logit increases and the others do not, so q^+ is nonuniform. This proves the strict inequality for every $\eta > 0$. Permutation symmetry of the correct labels gives $\mathbb{E}[q_c^+] = 1/m$.

For a fixed vector v , Taylor expansion about uniform logits gives

$$\sum_c \text{softmax}(\eta v)_c^2 = \frac{1}{m} + \frac{\eta^2}{m^2} \sum_c (v_c - \bar{v})^2 + O(\eta^3), \quad \bar{v} = \frac{1}{m} \sum_c v_c.$$

Use $v_c = \kappa_R N_c$ and $\mathbb{E}[\sum_c (N_c - R/m)^2 \mid R] = R(1 - 1/m)$. There are finitely many possible groups and all mixed-group weights are finite, so the Taylor remainder is uniform over those groups. Averaging proves Equation (33). Finally,

$$\mathbb{E}[R(G-R)^2] = G(G-1)(G-2)P(1-P)^2 + G(G-1)P(1-P),$$

by the binomial factorial moments, yielding Equation (34). $\square$

At fixed $P \in (0, 1)$, the Dr.GRPO coefficient in Equation (34) is $P(1-P)^2/G + O(G^{-2})$ as $G \rightarrow \infty$. The coefficient of the quadratic learning-rate term therefore decays asymptotically as $1/G$; it need not decrease at every increment in finite group size. This sampling effect is absent from the uniform mean flow. The result concerns one step from a uniform correct-mode distribution and gives neither monotone expected diversity from an arbitrary initial policy nor stochastic-optimizer convergence.

Corollary P.17 (Finite-budget disappearance of competing modes). *In the mean flow of Theorem P.4, let c^* be the unique initial correct leader and define*

$$\alpha = q_{c^*}(0), \quad \Gamma_0 = \sum_{c \neq c^*} \frac{q_c(0)}{\alpha - q_c(0)}.$$

At effective time τ ,

$$1 - q_{c^*}(\tau) \leq \Gamma_0 e^{-\alpha \tau}, \quad D(q(\tau)) \leq 2\Gamma_0 e^{-\alpha \tau}. \quad (35)$$

For $K \geq 1$ independent full-policy draws at that checkpoint,

$$\Pr(\text{any correct mode other than } c^* \text{ is observed}) \leq K\Gamma_0 e^{-\alpha \tau}. \quad (36)$$

In particular, if $\tau \geq \max\{0, \alpha^{-1} \log(K\Gamma_0/\delta)\}$ for $0 < \delta < 1$, no competing correct mode is observed with probability at least $1 - \delta$, although every mode retains positive probability.

Proof. For $c \neq c^*$, write $\xi_c = q_c/q_{c^*} < 1$. The leader is nondecreasing, and Equation (18) gives

$$\frac{d}{d\tau} \log \frac{\xi_c}{1 - \xi_c} = -q_{c^*} \leq -\alpha.$$

Hence $\xi_c(\tau) \leq [q_c(0)/(\alpha - q_c(0))]e^{-\alpha\tau}$. Now $1 - q_{c^*} = \sum_{c \neq c^*} \xi_c / (1 + \sum_{c \neq c^*} \xi_c)$ and $D \leq 1 - q_{c^*}^2 \leq 2(1 - q_{c^*})$, proving the first bounds. A single full-policy draw hits a competing correct mode with probability $P(1 - q_{c^*}) \leq 1 - q_{c^*}$; a union bound over K draws completes the proof. $\square$

The bound gives a finite-budget interpretation of the rare-output pruning mechanism discussed by Lochab et al. (2026, Section 4.2.5). It can also be indexed by correctness: Proposition P.14 and $S_2 + T_2 \leq 2$ imply $\tau \geq \frac{1}{2} \log\{[P/(1 - P)]/[P(0)/(1 - P(0))]\}$. These are sufficient visibility bounds within the categorical model; they do not require literal support loss or identify an empirical collapse time for neural training.

P.6 A REPLAY SIGNAL WHEN FRESH GROUPS SUPPLY NONE

Lemma P.18 (Fresh-group starvation). *Under Assumption P.1 with correctness probability P , draw G fresh categories independently. The probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

Every all-equal group has zero fresh task component for Equation (14) and for the specified centered MaxRL estimator, so a nonzero fresh task component requires a mixed group, an event whose probability is at most $h_G(P)$ at every correctness level.

Proof. The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$. A mixed group requires both a correct and an incorrect sample, so the union bound supplies each of the two stated bounds, GP and $G(1 - P)$, and the minimum of the pair is therefore also a bound. $\square$

Lemma P.19 (Replay ascent persists as fresh signal vanishes). *Under Assumption P.1, let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed verified bank with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ with $\rho_w > 0$, and extend w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is*

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p),$$

which vanishes only at $p = \bar{w}$. If $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$, even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes and fixed $\rho_w > 0$, as $p \rightarrow \mathbf{e}_b$ at one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$.

Proof. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$, proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k - 1)/k^2 = 1 - 1/k,$$

which gives the strictly positive limiting norm for $k \geq 2$. $\square$

Remark P.20. Together the two lemmas say only that a banked target can supply a verified signal when the fresh task gradient is unavailable. Lemma P.18 bounds an opportunity, not a gradient magnitude, and says nothing about other loss terms, clipping, or momentum carried across updates. The vector in Lemma P.19 averages the fixed bank, so a stochastic exemplar draw realizes it only in expectation, and a nonzero logit coordinate can still be annihilated by a degenerate neural Jacobian. Replay also need not increase the next mixed-group probability, since $h_G(P)$ is largest at $P = 1/2$.

P.7 FIXED-BANK REPLAY AND QUALITATIVE RETENTION

Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (37)$$

For the uniform bank target w , this is the established likelihood-replay objective $R_B = H(w) + \text{KL}(w||p)$ (Li et al., 2025, App. D.1, version 4). The target assigns mass $1/k$ to each banked mode and zero elsewhere. Decreasing this loss can raise total bank mass as well as redistribute mass within the bank; the bank is not the conditioning event for p .

In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting; it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P)\nabla_z P - \rho\nabla_z R_B(z). \quad (38)$$

Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas P.2 and P.3, c_G has a finite positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G-1)/G$ and MaxRL has $c_G = [1 - (1-P)^{G-1}]/P$. Nothing below uses more than that. By Theorem 3.1 and Equation (19), every bounded reward-measurable objective contributes $c_a(P)\nabla_z P$ with c_a a polynomial, so it admits the same Ψ_a and the retention argument runs unchanged: what follows is a statement about verified replay added to any outcome-binary fresh objective, of which Re:Dr and Re:Max are the two this paper trains. Define

$$F_B(z) = \rho R_B(z) - \Psi(P(z)). \quad (39)$$

Then (38) is exactly gradient flow $\dot{z} = -\nabla_z F_B$.

Theorem P.21 (Verified replay prevents banked-mode extinction). *Suppose logits are finite at time T and thereafter the nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective replay dose ρ , the fresh objective being any bounded reward-measurable one in the sense of Theorem 3.1. Set $\Psi_{\max} = \max_{s \in [0,1]} \Psi(s)$, which is finite, and under (38) define*

$$C_T = R_B(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho}.$$

Then for every $b \in \mathcal{B}$ and every $t \geq T$,

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (40)$$

Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including asymptotically, even when the bank does not cover every correct mode. This is stronger than finite-time positivity, which already follows from finite softmax logits without replay.

Proof. The smooth vector field has bounded velocity because c_G , $\nabla_z P$, and $\nabla_z R_B$ are bounded, so finite initial logits give a flow for every finite $t \geq T$. Along this flow,

$$\frac{d}{dt} F_B(z(t)) = -\|\nabla_z F_B(z(t))\|_2^2 \leq 0.$$

Since $\Psi(P(t)) \leq \Psi_{\max}$,

$$R_B(z(t)) \leq R_B(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho} = C_T.$$

Every $-\log p_b$ is nonnegative and their sum is kR_B . Therefore $-\log p_b(t) \leq kC_T$ for every banked mode, which is exactly Equation (40) for every time after T . $\square$

A sharper certificate from normalization. Write $\text{kl}(a, r) = a \log(a/r) + (1-a) \log[(1-a)/(1-r)]$ with continuous endpoint conventions. For $0 < a < 1$ and $D \geq 0$, let $\ell_-(a, D)$ be the unique $r \in (0, a]$ satisfying $\text{kl}(a, r) = D$; set $\ell_-(1, D) = e^{-D}$. In particular, $\ell_-(a, 0) = a$.

Corollary P.22 (Replay certificate by KL data processing). *Let $w_b > 0$, $\sum_{b \in \mathcal{B}} w_b = 1$, and suppose a probability law p satisfies $R_w(p) = -\sum_{b \in \mathcal{B}} w_b \log p_b \leq C < \infty$. With $D = C - H(w)$ and $H(w) = -\sum_b w_b \log w_b$, necessarily $D \geq 0$, and*

$$p_b \geq \ell_-(w_b, D) > 0, \quad p(\mathcal{B}) \geq e^{-D}, \quad p_b \geq \max\{0, w_b - \sqrt{D/2}\}. \quad (41)$$

The binary-KL coordinate bound is sharp on the closed probability simplex when the alphabet has at least two categories. In Theorem P.21, take $w_b = 1/k$ and $C = C_T$ to obtain these bounds simultaneously for every $t \geq T$.

Proof. Extend w by zero outside $\mathcal{B}$. Then $\text{KL}(\bar{w}||p) = R_w(p) - H(w) \leq D$. Data processing under the partitions $\{b\}, \{b\}^c$ and $\mathcal{B}, \mathcal{B}^c$ gives $\text{kl}(w_b, p_b) \leq D$ and $-\log p(\mathcal{B}) \leq D$ (van Erven & Harremoës, 2014, Theorem 9). Binary inversion gives the first bound; Pinsker gives the third (van Erven & Harremoës, 2014, Theorem 31). For $w_b < 1$, equality in the first is attained by $p_b = \ell_-(w_b, D)$ and $p_j = (1 - p_b)w_j / (1 - w_b)$ on the remaining bank, with zero mass elsewhere. For $w_b = 1$, put the remaining probability on another category. For $D > 0$, full-support laws approach the same bound by assigning vanishing mass off the bank and slightly increasing the protected coordinate while keeping the divergence within the allowed budget. $\square$

For illustration, a uniform bank of 16 with $C = \log 16 + 0.01$ gives $p_b \geq 0.03397$, whereas the termwise bound e^{-16C} is 4.62×10^{-20} . These are illustrative inputs, not measured training losses. A large excess loss D can still make the sharper certificate weak.

For the nonnegative coefficients of the named fresh estimators, $\Psi_{\max} = \Psi(1)$, recovering the simpler constant. Signed coefficients also satisfy the retention bound, but need not yield $P \rightarrow 1$. The elementary floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not the replay coefficient alone.

A common nonnegative, locally integrable rate $\eta(t)$ multiplying both terms of (38) preserves the energy bound, since $\dot{F}_{\mathcal{B}} = -\eta(t)\|\nabla F_{\mathcal{B}}\|_2^2 \leq 0$. The convergence statements below additionally require $\int_T^\infty \eta(t) dt = \infty$ and then follow by reparameterizing optimization time. Infinitely many replay visits alone establish neither this condition nor a fixed replay-to-fresh weighting. Different time-varying relative weights are outside the fixed-potential argument.

Full coverage and the categorical target.

Corollary P.23 (Conditional full-coverage categorical limit). *Assume the conditions of Theorem P.21 and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Assume additionally $c_G(P) \geq 0$ on $[0, 1]$, as for the three named estimators. Let $u_c = 1/m$ on $\mathcal{C}$. Then the categorical mean flow converges to*

$$P(t) \rightarrow 1, \quad q(t) \rightarrow u.$$

Consequently

$$\min_{c \in \mathcal{C}} q_c(t) \rightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \rightarrow \log m,$$

and conditional correct-mode collapse is impossible.

Proof. For full coverage,

$$R_{\mathcal{C}} = \log m - \log P + \text{KL}(u||q). \quad (42)$$

The potential is bounded below by $-\Psi_{\max}$, so its descent gives $\int_T^\infty \|\nabla F_{\mathcal{C}}\|_2^2 dt < \infty$. The finite-group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded. Thus the Hessian of $F_{\mathcal{C}}$ and the velocity are bounded; the gradient along the trajectory is uniformly continuous. Square integrability then implies $\nabla F_{\mathcal{C}} \rightarrow 0$: recurring gradients of fixed positive size would otherwise contribute energy over intervals of a uniform minimum length.

Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness of this limit establish convergence of the probability vector. $\square$

Corollary P.24 (Uniform allocation maximizes finite-budget coverage). *Fix correctness $P \in (0, 1]$ and $m \geq 2$ possible correct keys. Let $D_K(P, q)$ count distinct verified keys in K independent draws with key probabilities $Pq_1, \dots, Pq_m$ and failure probability $1 - P$. For $u = (1/m, \dots, 1/m)$, every $K \geq 1$ and $r = 1, \dots, m$ satisfy*

$$\Pr[D_K(P, u) \geq r] \geq \Pr[D_K(P, q) \geq r]. \quad (43)$$

Thus uniform conditional allocation maximizes $\text{distinct}@K$ at fixed correctness, and more strongly maximizes every coverage tail probability. The conclusion applies Anceaume et al. (2015, Theorem 3, v1) to a frozen policy with a fixed correctness level and a finite catalogue of verified execution keys under independent sampling.

Proof. Map each correct key to a coupon of mass Pq_c and every failure to the null coupon of mass $1 - P$. The cited theorem makes the time to collect r distinct non-null coupons stochastically smallest when each has mass P/m . The event that this time is at most K is exactly $D_K \geq r$. Zero coordinates and $P = 1$ follow by continuity of finite-horizon probabilities. Summing the tail inequalities over $r = 1, \dots, m$ gives the same ordering of expected distinct counts for every finite sampling budget at the same correctness level and over the same catalogue of possible keys. $\square$

This sampling optimum does not assert that a training algorithm reaches the uniform allocation.

Positive weights and the implemented length normalization. Uniformity of the target is separate from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w = -\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi_{\max} - \Psi(P(T))}{\rho_w}.$$

Corollary P.22 sharpens this floor with $C = C_{T,w}$ and the same normalized target w . For the following convergence statements assume $c_G \geq 0$. If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w \| q)$, and the independent-logit limit is $P \rightarrow 1, q \rightarrow w$. The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate equations forces $P = 1$, then each equation forces $p_c = w_c$.

A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$, so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-bank or stochastic neural training.

This distinction matters for the implemented mean-token exemplar loss. Even if each key were represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

Its effective categorical replay coefficient would be $\rho_w = \rho A$. The full-coverage limit would therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual language model, $\pi_\theta(e_b \mid x)$ is only one contribution to the total key probability. Equal averaging of per-token exemplar scores does not imply uniform learned key probabilities; the categorical uniformity corollary is not that claim.

Full support is an additional premise, not a consequence of replay. In Python factors, evaluation prompts admit 16–1,512 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects only admitted banked modes; it supplies no probability floor for an unbanked key.

P.8 WHEN REPLAY REVERSES CONCENTRATION, AND HOW FAST

The restoring logit field $\rho(\bar{w} - p)$ has a shared foundation with inverse probability scaling: the ideal IPS field of Sinha et al. (2026, Theorem 4.1), normalized to a fixed target w , is $\bar{w} - p$. The

uniform-correct target and its forward conditional-KL formulation are also established by Lochab et al. (2026). Here the target is restricted to a discovered bank and evaluated from stored exemplars. The question below is how its restoring field competes with the fresh binary objective over a finite interval. Neither a uniform target nor its cross-entropy gradient is a new contribution of this analysis.

Fix a bank $\mathcal{B} \subseteq \mathcal{C}$ of $k \geq 2$ modes, with uniform categorical weights, and consider Equation (38). Write $c(P)$ for its fresh coefficient and define

$$M_{\mathcal{B}} = \sum_{b \in \mathcal{B}} p_b, \quad r_b = \frac{p_b}{M_{\mathcal{B}}}, \quad C_{\mathcal{B}} = \sum_{b \in \mathcal{B}} r_b^2, \quad (44)$$

$$a_{\rho}(t) = \rho - c(P(t))(1 - P(t)).$$

Thus $1 - C_{\mathcal{B}}$ is PCMD conditional on the bank. It equals the paper’s success-conditional PCMD when $\mathcal{B} = \mathcal{C}$; otherwise it measures only allocation inside the bank.

Theorem P.25 (Exact replay threshold for bank diversity). *Under the categorical mean-flow assumptions and a fixed uniform bank,*

$$\frac{d}{dt} \log \frac{p_i}{p_j} = -a_{\rho} M_{\mathcal{B}} (r_i - r_j), \quad i, j \in \mathcal{B}, \quad (45)$$

$$\dot{r}_b = -a_{\rho} M_{\mathcal{B}} r_b (r_b - C_{\mathcal{B}}), \quad b \in \mathcal{B}, \quad (46)$$

$$\frac{d}{dt} (1 - C_{\mathcal{B}}) = 2a_{\rho} M_{\mathcal{B}} \left(\sum_{b \in \mathcal{B}} r_b^3 - C_{\mathcal{B}}^2 \right). \quad (47)$$

For a nonuniform r , bank-conditional diversity strictly increases when $\rho > c(P)(1 - P)$, strictly decreases when $\rho < c(P)(1 - P)$, and has zero instantaneous derivative at equality. These conclusions require no coverage of correct modes outside the bank.

Proof. For $b \in \mathcal{B}$, the score identities give $\dot{z}_b = \rho/k - a_{\rho} p_b$. Subtracting two such equations gives Equation (45). The distribution r is the softmax of the logits restricted to $\mathcal{B}$, so $\dot{r}_b = r_b(\dot{z}_b - \sum_j r_j \dot{z}_j)$, proving Equation (46). Differentiating $C_{\mathcal{B}}$ gives Equation (47). Its parenthesis is $\text{Var}_{b \sim r}(r_b)$, which is strictly positive for a nonuniform distribution with positive coordinates. $\square$

A positive dose therefore need not improve diversity immediately. The thresholds for Dr.GRPO and the specified MaxRL estimator are, respectively,

$$\rho > \frac{G-1}{G}(1-P), \quad \rho > \frac{(1-P)[1 - (1-P)^{G-1}]}{P}.$$

The asymptotic result for every $\rho > 0$ and this finite-time condition are compatible: the fresh concentration pressure vanishes as $P \rightarrow 1$.

Theorem P.26 (Finite recovery from accumulated replay exposure). *In the setting of Theorem P.25, suppose $a_{\rho}(u) \geq 0$ for $u \in [T, t]$, and define*

$$S(T, t) = \int_T^t a_{\rho}(u) M_{\mathcal{B}}(u) du, \quad D(t) = \log \frac{\max_{b \in \mathcal{B}} r_b(t)}{\min_{b \in \mathcal{B}} r_b(t)}.$$

Then

$$e^{D(t)} - 1 \leq (e^{D(T)} - 1)e^{-S(T, t)/k}, \quad \min_b r_b(t) \geq \frac{1}{1 + (k-1)e^{D(t)}}. \quad (48)$$

For $0 < \delta < 1/k$, put $D_{\delta} = \log[(1/\delta - 1)/(k - 1)]$. If $D(T) > D_{\delta}$, the condition

$$S(T, t) \geq k \log \frac{e^{D(T)} - 1}{e^{D_{\delta}} - 1} \quad (49)$$

is sufficient for every $r_b(t) \geq \delta$. If $D(T) \leq D_{\delta}$, this conclusion already holds and is preserved on the interval. In particular, when $a_{\rho} \geq a_0 > 0$ and $M_{\mathcal{B}} \geq M_0 > 0$ on the interval, its required duration is at most the right-hand side of Equation (49) divided by $a_0 M_0$. At that endpoint every unconditional bank probability is at least $M_0 \delta$.

Proof. Bank-coordinate ordering is preserved by Equation (45): a difference of two bank logits satisfies a scalar equation proportional to that difference, and ties persist. Writing $r_{\max}$ and $r_{\min}$ for the extremal coordinates,

$$\dot{D} = -a_\rho M_{\mathcal{B}}(r_{\max} - r_{\min}), \quad r_{\max} - r_{\min} = r_{\max}(1 - e^{-D}) \geq \frac{1 - e^{-D}}{k}.$$

Thus $\frac{d}{dt}(e^D - 1) \leq -(a_\rho M_{\mathcal{B}}/k)(e^D - 1)$; integration proves the first bound. Since $1 \leq r_{\min} + (k - 1)r_{\max}$, the second follows. Substituting D_δ and solving the first bound for the exposure proves the remaining claims. When $D(T) = 0$, uniformity is preserved directly. $\square$

The bound specifies the interval on which restoration occurs. It does not assume that bank mass is monotone or that correctness stays above its value at T . Both enter through the displayed exposure; a bound on unconditional visibility additionally needs the stated bank-mass lower bound.

Proposition P.27 (Discrete exact-gradient contraction). *Replace the continuous update by simultaneous exact mean-gradient steps, while retaining independent logits and a fixed uniform bank:*

$$z_{n+1} = z_n + \eta_n \{c(P_n) \nabla_z P_n + \rho(\bar{w} - p_n)\}, \quad \eta_n \geq 0.$$

Define D_n as above and $\gamma_n = \eta_n M_{\mathcal{B},n}[\rho - c(P_n)(1 - P_n)]$. If $0 \leq \gamma_n \leq 2$, the order of all bank probabilities is preserved at this step and

$$e^{D_{n+1}} - 1 \leq \left(1 - \frac{1 - e^{-\gamma_n}}{k}\right) (e^{D_n} - 1). \quad (50)$$

Consequently, if $0 \leq \gamma_n \leq 1$ for $n = 0, \dots, N - 1$, then

$$e^{D_N} - 1 \leq (e^{D_0} - 1) \exp\left(-\frac{1 - e^{-1}}{k} \sum_{n=0}^{N-1} \gamma_n\right).$$

Proof. For two bank logits with difference $d = z_i - z_j \geq 0$,

$$d_{\text{next}} = d - \gamma_n(r_i - r_j), \quad r_i - r_j = (r_i + r_j) \tanh(d/2) \leq d/2.$$

Hence $d_{\text{next}} \geq 0$ when $\gamma_n \leq 2$. The same extremal indices therefore give $D_{n+1} = D_n - \gamma_n(r_{\max} - r_{\min})$. For $x \in [0, 1]$, convexity of $e^{-\gamma_n x}$ yields $1 - e^{-\gamma_n x} \geq (1 - e^{-\gamma_n})x$. Set $x = r_{\max} - r_{\min}$ and use $e^{D_n} x \geq (e^{D_n} - 1)/k$ from the preceding proof. This gives Equation (50). Multiplication over steps, $1 - y \leq e^{-y}$, and $1 - e^{-\gamma} \geq (1 - e^{-1})\gamma$ for $0 \leq \gamma \leq 1$ prove the last bound. $\square$

These discrete statements concern exact mean gradients, without clipping, sampling noise, optimizer state or alternating replay visits. They establish a finite-step result for the specified update, rather than a guarantee for AdamW. Uniform categorical weights are also essential to the stated target. For fixed nonuniform weights w_b , the exact conditional equation is instead

$$\dot{r}_b = r_b \left\{ \rho \left(w_b - \sum_j r_j w_j \right) - a_\rho M_{\mathcal{B}}(r_b - C_{\mathcal{B}}) \right\}. \quad (51)$$

The inverse-length weights induced by the implemented mean-token loss belong to this weighted case. The uniform contraction theorem therefore describes an equal-length categorical specialization, and does not assert uniform key probabilities for neural exemplar replay.

P.9 COMPLETE RESPONSES AND FINITE EVALUATION VISIBILITY

The learner scores a particular response string, while evaluation groups strings by execution key. Event containment provides a conditional bridge: one complete verified response is one way of producing its key. This bridge also makes the loss's length factor explicit. Here $p_\theta(b \mid x) = \Pr_{Y \sim \pi_\theta(\cdot \mid x)}[V(Y, s_x) = b]$ is the unconditional probability of producing key b , rather than the probability conditional on correctness.

Lemma P.28 (Retention from a joint exemplar potential). *Fix finitely many verified complete-response exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j | x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$, every protected exemplar satisfies

$$p_\theta(b_j | x_j) \geq \pi_\theta(e_j | x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

Proof. The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative, so $-\log \pi_\theta(e_j | x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key b_j , which proves the stated lower bound on every protected key’s probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \max_{s \in [0,1]} \Psi_x(s)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them. Its bound applies only to the listed exemplar–prompt pairs. Shared parameters do not extend it to held-out prompts; transfer to those prompts remains an empirical question.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or a score under another decoding law does not supply this bound without a separate event or likelihood comparison. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

A certificate for an execution key. For one fixed prompt x , take distinct complete verified responses e_j with lengths L_j , keys b_j , and coefficients $\alpha_j > 0$. Define

$$A_x = \sum_j \frac{\alpha_j}{L_j}, \quad w_j = \frac{\alpha_j / L_j}{A_x}, \quad W_b = \sum_{j: b_j=b} w_j.$$

If their mean-token replay loss is at most C_x , namely $\sum_j (\alpha_j / L_j) [-\log \pi_\theta(e_j | x)] \leq C_x$, then

$$\text{kl}(W_b, p_\theta(b | x)) \leq D_x := C_x / A_x - H(w), \quad p_\theta(b | x) \geq \ell_-(W_b, D_x) \quad (W_b > 0). \quad (52)$$

Indeed, the response-level divergence from the normalized exemplar target is at most D_x ; deterministic execution followed by the indicator of key b is data processing (van Erven & Harremoës, 2014, Theorem 9), followed by Corollary P.22. This uses the same complete response and decoding law as Lemma P.28. Normalize separately for each prompt; a bound on total replay loss also bounds each prompt’s nonnegative summand. A checkpoint certificate supplies no trajectory guarantee without a uniform loss bound.

Allowing stochastic energy excursions. Exact descent is sufficient but not necessary. The following is a direct application of Ville’s inequality (Howard et al., 2020, Lemma 1).

Corollary P.29 (Retention from a conditional energy budget). *Use the fixed complete exemplars, weights and potential F of Lemma P.28. Let $D_t = F(\theta_t) + J_{\max} \geq 0$ be adapted and integrable, with deterministic finite D_0 . For $N \in \mathbb{N} \cup \{\infty\}$, suppose deterministic $d_t \geq 0$ satisfy $\mathbb{E}[D_{t+1} | \mathcal{F}_t] \leq D_t + d_t$ for $t < N$, with finite total budget $B_N = \sum_{t < N} d_t$. These hypotheses control expected upward energy changes without requiring every realized step to decrease the joint loss. For every $0 < \delta < 1$, with probability at least $1 - \delta$,*

$$p_{\theta_t}(b_j | x_j) \geq \pi_{\theta_t}(e_j | x_j) \geq \exp\left[-\frac{L_j(D_0 + B_N)}{\alpha_j \delta}\right] \quad \text{for all } t \leq N \text{ and all } j. \quad (53)$$

The sharper per-prompt certificate in Equation (52) also applies on this event. For $N = \infty$, every protected probability has an almost surely positive, potentially random, infimum over the iterations.

Proof. $X_t = D_t + \sum_{s=t}^{N-1} d_s$ is a nonnegative supermartingale. Ville’s inequality bounds its supremum by $(D_0 + B_N)/\delta$ with probability at least $1 - \delta$. Every replay summand is nonnegative and bounded by D_t , so exponentiation and event containment give the claim without a union factor over exemplars. If $D_0 + B_N = 0$, then $X_t = 0$ almost surely. For $N = \infty$, Ville’s inequality with thresholds tending to infinity gives $\sup_t X_t < \infty$ almost surely. $\square$

For example, the one-step estimate underlying Wang et al. (2017, Lemma 2.1, Eq. (2.13)) supplies this drift for the update $\theta_{t+1} = \theta_t - \eta_t H_t g_t$ with an L -smooth F , $\mathbb{E}[g_t | \mathcal{F}_t] = \nabla F(\theta_t)$, $\mathbb{E}[\|g_t - \nabla F(\theta_t)\|^2 | \mathcal{F}_t] \leq \sigma^2/m_t$, and preconditioners measurable before the current gradient noise with $\underline{\kappa}I \preceq H_t \preceq \bar{\kappa}I$. Deterministic steps $\eta_t \leq \underline{\kappa}/(L\bar{\kappa}^2)$ and batch sizes m_t permit $d_t = L\eta_t^2 \bar{\kappa}^2 \sigma^2/(2m_t)$. An infinite horizon additionally needs the displayed finite budget; constant steps and persistent noise generally supply only a finite-horizon bound. The current-gradient AdamW moments, clipping, changing banks and cyclic replay schedule are not certified by these premises: their bias and energy effects require separate control. No such budget is established for the experimental runs.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K\delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero (Clopper & Pearson, 1934). For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

P.10 WHEN A REPLAY GRADIENT IMPROVES A NEURAL EXEMPLAR

Lemma P.28 permits shared parameters by assuming descent of a joint potential. A local statement can instead expose the condition under which a realized replay update helps a particular exemplar. This condition involves interference among scores and the optimizer’s response, quantities that the categorical abstraction suppresses.

Proposition P.30 (Local replay response under shared parameters). *For a selected bank of k complete exemplars, define*

$$\ell_b(\theta) = \frac{\log \pi_\theta(e_b | x_b)}{L_b}, \quad h_b = \nabla_\theta \ell_b(\theta), \quad \bar{h} = \frac{1}{k} \sum_{j=1}^k h_j.$$

Suppose a step has the form $\Delta = \eta H(g + \lambda \bar{h}) + u$, where $\eta, \lambda \geq 0$, H is a fixed symmetric positive semidefinite matrix for this step, g is the fresh ascent direction, and u is the remaining update. Assume ℓ_b is twice differentiable and its Hessian has operator norm at most B_b along $\{\theta + s\Delta : 0 \leq s \leq 1\}$. Set

$$K_{bj} = h_b^\top H h_j, \quad \kappa_b = \frac{1}{k} \sum_j K_{bj}, \quad m_b = \eta h_b^\top H g + \eta \lambda \kappa_b + h_b^\top u - \frac{B_b}{2} \|\Delta\|_2^2.$$

Then

$$\ell_b(\theta + \Delta) - \ell_b(\theta) \geq m_b, \quad \pi_{\theta+\Delta}(e_b | x_b) \geq \pi_\theta(e_b | x_b) e^{L_b m_b}. \quad (54)$$

In particular, $m_b > 0$ guarantees that this update increases the complete exemplar’s probability, whether or not it appeared among the fresh samples.

Proof. Taylor’s theorem with the Hessian bound gives $\ell_b(\theta + \Delta) - \ell_b(\theta) \geq h_b^\top \Delta - (B_b/2)\|\Delta\|_2^2$. Substitution of the stated step and $h_b^\top H h = k^{-1} \sum_j K_{bj}$ proves the first inequality. Multiplication by L_b and exponentiation prove the second. $\square$

The required replay response is a row average κ_b of the score Gram matrix, not only its nonnegative diagonal entry K_{bb} . A sufficient interference condition is $\kappa_b > 0$, with replay strength satisfying

$$\eta \lambda \kappa_b > -\eta h_b^\top H g - h_b^\top u + \frac{B_b}{2} \|\Delta\|_2^2.$$

This inequality includes the step’s curvature cost, which itself depends on the chosen dose; arbitrarily increasing λ is not a sufficient rule.

The interference condition cannot be dropped. For example, consider three categorical logits sharing one scalar parameter, with slopes $(1, -2, 0)$. At a point with probabilities $(s, s, 1 - 2s)$, where $0 < s < 1/4$, let the first two categories be correct and banked. Their unit-length log-score gradients are $h_1 = 1 + s$ and $h_2 = -2 + s$. Uniform replay alone follows $\bar{h} = s - 1/2$, so

$$\left. \frac{d}{dt} \log p_1 \right|_{\theta=\bar{h}} = (1 + s)(s - 1/2) < 0.$$

Thus shared parameters can make replay initially reduce the probability of a rare stored mode while improving the average replay score.

For an AdamW step, replay can alter both moment estimates. The proposition therefore does not identify its H with an unchanged AdamW preconditioner. It applies to a specified decomposition of the realized step, with any remaining contribution included in u ; alternatively its Taylor bound can be evaluated directly from $h_b^\top \Delta$. Neither the interference margin nor a curvature bound is established for the experimental runs here. The statement concerns complete stored responses under the sampling law specified in App. P.9. Raising one such probability strengthens that exemplar’s contribution to its key, but need not increase total key probability or transfer to a held-out prompt.

P.11 REFERENCE KL RETAINS THE REFERENCE, NOT THE DISCOVERY

Assumption P.1(A4) sets the reference-KL coefficient to zero, which is what our runs train at (Table 6, $\beta_{\text{KL}} = 0$). The collapse theorem therefore says nothing about the regularizer most RLVR recipes carry, and a reader is entitled to ask whether that regularizer already does the work the bank is introduced for. This subsection separates the categorical stationary target, exact-gradient convergence, and recovery rates. For Dr.GRPO and MaxRL, the full-support stationary conditional is the reference’s own. Corollary P.33 applies entropy-regularized softmax theory to prove convergence for exact Dr.GRPO gradient steps. The recovery comparison below concerns a specified two-mode face: reference KL’s restoring logit force vanishes as a mode vanishes, whereas replay supplies a nonvanishing banked-coordinate force. App. P.12 compares these mechanisms with measured runs.

Replace (A4) by

(A4’) *Reference KL.* A fixed reference μ with full support on the category set $\mathcal{A}$ enters with a positive coefficient β , and no entropy term is present alongside it.

The low-variance estimator standard in GRPO implementations is unbiased for $\text{KL}(p \parallel \mu)$ under samples drawn from p (Shao et al., 2024), so the reverse direction is the one to analyze; the forward direction is treated at the end of this subsection and behaves differently in exactly the way that matters. Write $D(z) = \text{KL}(p(z) \parallel \mu)$. The prompt-isolated categorical mean flow is

$$\dot{z} = c_G(P) \nabla_z P - \beta \nabla_z D(z), \quad \nabla_{z_b} D = p_b (\log(p_b/\mu_b) - D), \quad (55)$$

which is exact gradient flow of the potential $F_\beta(z) = -\Psi(P(z)) + \beta D(z)$, built exactly as Equation (39) was built for verified replay.

Lemma P.31 (Reverse-KL sublevels and replay barriers). *For a full-support reference, $D = \text{KL}(p \parallel \mu) \leq \log(1/\min_a \mu_a)$ on the simplex, while $R_B \rightarrow \infty$ if a banked coordinate vanishes. More precisely,*

$$\inf_{p:p_b=0} D(p) = \log \frac{1}{1 - \mu_b}.$$

Thus every finite replay-loss sublevel bounds each protected probability away from zero. A reverse-KL sublevel $\{D \leq B\}$ does so for coordinate b if $B < -\log(1 - \mu_b)$, but includes a point with $p_b = 0$ if $B \geq -\log(1 - \mu_b)$.

Proof. The global bound follows from $\sum_a p_a \log p_a \leq 0$ and $-\sum_a p_a \log \mu_a \leq \log(1/\min_a \mu_a)$. On the face $p_b = 0$, write $\tilde{\mu}_a = \mu_a/(1 - \mu_b)$ for $a \neq b$. Then $D(p) = \text{KL}(p \parallel \tilde{\mu}) - \log(1 - \mu_b)$, whose minimum is attained at $\tilde{\mu}$. Continuity and compactness exclude that face from any strictly lower sublevel and give a positive coordinate minimum. For replay, $-\log p_b \leq kR_B$ gives the bound directly. $\square$

Descent of F_β gives $D(p(t)) \leq [F_\beta(z(T)) + \Psi_{\max}]/\beta$. This yields a floor when its right side is below the displayed face threshold, but need not do so for an arbitrary initial energy. These are statements about loss sublevels. A particular optimizer can still admit a trajectory-dependent probability floor, as Corollary P.33 establishes for exact Dr.GRPO gradient steps with a fixed reference.

Proposition P.32 (The retained conditional is the reference's). *Under Assumption P.1 with (A4'), using the Dr.GRPO or centered MaxRL coefficient, the flow (55) has exactly one stationary policy of full support,*

$$p_a^* \propto \mu_a e^{\lambda R_a}, \quad \lambda = c_G(P^*)/\beta, \quad R_a = \mathbf{1}[a \in \mathcal{C}],$$

with $\lambda = (G - 1)/(G\beta)$ in closed form for the Dr.GRPO coefficient, and a unique λ for the MaxRL coefficient. Consequently

$$q^* = \mu|_{\mathcal{C}}, \quad \mathcal{H}(q^*) = \mathcal{H}(\mu|_{\mathcal{C}}), \quad \text{logit } P^* = \text{logit } \mu(\mathcal{C}) + \lambda.$$

This stationary conditional does not depend on β : the coefficient sets the correctness level there and leaves the allocation among correct modes at the reference's.

Proof. Stationarity of (55) in a coordinate with $p_b > 0$ reads $c_G(P)(R_b - P) = \beta(\log(p_b/\mu_b) - D)$, so $\log(p_b/\mu_b) = D + (c_G(P)/\beta)(R_b - P)$ and the b -independent terms are absorbed by normalization, giving the stated form with $\lambda = c_G(P)/\beta$. Conversely each $\lambda \geq 0$ determines $P(\lambda) = \mu(\mathcal{C})e^\lambda/(\mu(\mathcal{C})e^\lambda + 1 - \mu(\mathcal{C}))$, strictly increasing from $\mu(\mathcal{C})$ to 1. The consistency requirement $\lambda = c_G(P(\lambda))/\beta$ has a solution by the intermediate value theorem, since the left side increases from 0 without bound while the right side is positive and bounded; it is unique whenever c_G is non-increasing, which covers the constant Dr.GRPO coefficient and $c_G^{\text{MaxRL}}(P) = [1 - (1 - P)^{G-1}]/P$, decreasing from $G - 1$ to 1. Two correct modes give $\log(p_c/\mu_c) = \log(p_d/\mu_d)$, hence $q^* \propto \mu$ on $\mathcal{C}$, which is β -free. The logit identity is $P^*/(1 - P^*) = [\mu(\mathcal{C})/(1 - \mu(\mathcal{C}))]e^\lambda$. $\square$

For Dr.GRPO, the constant fresh coefficient permits a stronger conclusion by direct application of established optimization theory. The following specializes Mei et al. (2020, Theorem 5 and Lemma 13); its Gibbs target is also the standard regularized maximizer of Geist et al. (2019, Proposition 1).

Corollary P.33 (Exact Dr.GRPO with reference KL converges). *Fix one prompt, a finite category set $\mathcal{A} = \mathcal{C} \cup \mathcal{N}$ with both parts nonempty, and independently optimized logits $p_n = \text{softmax}(z_n)$. Let z_1 be finite, let μ be a reference distribution with $\mu_a > 0$ for every category, and take $\beta > 0$ and $\alpha = (G - 1)/G$ for $G \geq 2$. Define*

$$\mathcal{J}_\beta(p) = \alpha P(p) - \beta \text{KL}(p \parallel \mu).$$

For constant $0 < \eta \leq 1/\beta$, consider the exact gradient updates

$$z_{n+1} = z_n + \eta \nabla_z \mathcal{J}_\beta(p_n). \quad (56)$$

Then $p_n \rightarrow p^*$, where

$$p_a^* = \frac{\mu_a \exp(\alpha R_a/\beta)}{\sum_b \mu_b \exp(\alpha R_b/\beta)}, \quad R_a = \mathbf{1}[a \in \mathcal{C}], \quad q^* = \mu|_{\mathcal{C}}.$$

There are constants $\underline{p} > 0$ and $C < \infty$, depending only on $|\mathcal{A}|, z_1, \mu, \alpha, \beta$, such that for every $n \geq 1$,

$$\min_a p_{n,a} \geq \underline{p}, \quad \mathcal{J}_\beta(p^*) - \mathcal{J}_\beta(p_n) = \beta \text{KL}(p_n \parallel p^*) \leq C e^{-2\beta \eta \underline{p}(n-1)}. \quad (57)$$

Proof. Set $\tilde{r}_a = \alpha R_a + \beta \log \mu_a$. Then $\mathcal{J}_\beta(p) = \langle p, \tilde{r} \rangle + \beta H(p)$. For $B = \max\{1, \max_a \tilde{r}_a - \min_a \tilde{r}_a\}$, define $r'_a = (\tilde{r}_a - \min_b \tilde{r}_b)/B \in [0, 1]$, $\tau = \beta/B$, and $\eta' = \eta B$. Equation (56) is exactly the entropy-regularized bandit update of Mei et al. (2020) with reward r' , temperature τ , and step $\eta' \leq 1/\tau$. Their Lemma 13 supplies $\underline{p}$, and their Theorem 5 supplies the displayed rate after multiplying the objective by B . The optimum softmax(r'/τ) is p^* ; substitution gives the KL gap identity. The gap tends to zero, so Pinsker's inequality also gives convergence of the full categorical probability law and its conditional allocation among the correct categories. $\square$

This positive trajectory floor depends on initialization and the reference; it need not provide useful finite-sample visibility. It is compatible with the vanishing coordinate force below and with increasingly slow recovery as the initial probability decreases. The result concerns exact discrete gradients with a fixed reference and the stated step size, not sampled KL gradients, PPO epochs, or AdamW. For centered MaxRL with $G \geq 3$, $c_G(P)$ is nonconstant, so its potential cannot be absorbed into the fixed reward vector used in this reduction; Proposition P.32 continues to specify its stationary target without asserting convergence.

The reference-conditional target differs from full-coverage uniform replay: Corollary P.23 gives $q \rightarrow u$, whereas Corollary P.33 gives $q_n \rightarrow \mu|_C$ for exact Dr.GRPO steps. A broader reference changes that allocation; replay constructs its target from verified discoveries. For MaxRL, Proposition P.32 identifies the stationary conditional without a convergence claim. Neither result bounds transient diversity by the reference's diversity or certifies a neural optimizer trajectory.

Remark P.34. The correctness prediction also depends on reference correctness $s_\mu = \mu(C)$: $1 - P^* = (1 - s_\mu)/(1 - s_\mu + s_\mu e^\lambda)$. For Dr.GRPO with $G = 16$ and $\beta \leq .1$, $\lambda \geq 9.375$; this amplifies the reference correctness odds by at least $e^{9.375}$ but does not give a reference-independent bound on the correctness deficit.

Lemma P.35 (Reference KL has no probability-independent boundary force). *Under (A4') the logit velocity contributed by the reference-KL term is*

$$\dot{z}_b^{\text{KL}} = \beta p_b (\log(\mu_b/p_b) + D) \xrightarrow{p_b \rightarrow 0} 0, \quad |\dot{z}_b^{\text{KL}}| \leq \beta \left(\frac{\mu_b}{e} + \log \frac{1}{\min_a \mu_a} \right),$$

whereas the verified-replay term contributes $\rho(w_b - p_b) \rightarrow \rho w_b > 0$ at the same limit. This is a property of the exactly evaluated penalty. Its score contains an unbounded log ratio, so it is not an instance of the uniformly bounded-score premise in Lemma P.38; both comparisons nevertheless have a vanishing coordinate force.

Proof. The displayed velocity is the softmax score identity used above. For the bound, $\sup_{0 < s \leq 1} s \log(\mu_b/s) = \mu_b/e$ at $s = \mu_b/e$, the same expression is bounded below by $\log \mu_b$ at $s = 1$, and $0 \leq D \leq \log(1/\min_a \mu_a)$ by Lemma P.31. The replay expression is Lemma P.19. $\square$

Corollary P.36 (Recovery on a two-correct-mode face). *Consider the continuous extension of the categorical probability flow to the face supported on two correct modes b, c , with $p_b = s$, $p_c = 1 - s$. This boundary reduction replaces the full-support start in (A5); here $P = 1$ and the fresh correctness gradient is zero. For reference KL take fixed $\beta > 0$ and $\mu_b, \mu_c > 0$. For replay take the uniform bank $\{b, c\}$ and fixed $\rho > 0$. Fix $0 < \delta < \min\{1/2, \mu_b/(\mu_b + \mu_c)\}$. The times to reach $p_b = \delta$ from $s \downarrow 0$ satisfy*

$$T_{\text{KL}}(s) = \frac{1 + o(1)}{2\beta s \log(1/s)}, \quad T_{\text{replay}}(s) = \frac{\log(\delta/s)}{\rho} + O(1).$$

In particular their ratio diverges. The KL order is $\Theta(1/[s \log(1/s)])$, and the replay order is $\Theta(\log(1/s))$.

Proof. Write $u = p_b$ along this face. The softmax identity $\dot{u} = u(1 - u)(\dot{z}_b - \dot{z}_c)$ gives exactly

$$\dot{u}_{\text{KL}} = 2\beta u^2(1 - u)^2 \log \frac{(1 - u)\mu_b}{u\mu_c}, \quad \dot{u}_{\text{replay}} = \rho u(1 - u)(1 - 2u).$$

Both velocities are positive up to δ by its stated restriction. Separating variables expresses each hitting time as the integral from s to δ of the reciprocal velocity. For KL the integrand is $(1 + o(1))/(2\beta u^2 \log(1/u))$ as $u \downarrow 0$, and its integral is asymptotic to $1/(2\beta s \log(1/s))$ (equivalently, apply l'Hopital's rule to the two divergent functions). For replay the integrand is $1/(\rho u) + O(1)$, giving the second expression for any fixed admissible target δ . $\square$

Table 58: **Time for a depleted correct mode to return to $p_b = 0.05$, by the depth it starts from.** Numerical integration of the categorical mean flow with $m = 4$ correct modes, $G = 16$, and a uniform reference, in the flow’s own time units. Reference KL multiplies its recovery time by 7.3–8.2 per decade of depth; verified replay adds approximately a constant per decade. Corollary P.36 proves the contrasting asymptotic orders on a separate two-mode face.

Regularizer	$s = 10^{-3}$	10^{-4}	10^{-5}	10^{-6}
Reference KL, $\beta = 0.04$	2.12e+03	1.55e+04	1.20e+05	9.83e+05
Reference KL, $\beta = 0.1$	8.49e+02	6.19e+03	4.81e+04	3.93e+05
Verified replay, $\rho = 0.01$	4.07e+02	6.37e+02	8.68e+02	1.10e+03
Verified replay, $\rho = 0.1$	4.07e+01	6.37e+01	8.68e+01	1.10e+02

Table 58 is a numerical illustration in a larger categorical system, with 4 correct and three incorrect categories. Its reference is uniform, its other depleted logits start at -40 , and its target is $p_b = 0.05$. The KL times grow by a factor 7.3–8.2 per decade of initial depth; replay adds approximately $\log(10)/\rho$ per decade. These finite-depth integrations illustrate the recovery contrast but do not prove the two-mode asymptotic constants for the larger system.

The direction of the divergence decides whether it is a barrier. Forward KL to the same reference is a different object: $\text{KL}(\mu \| p) = -\sum_a \mu_a \log p_a - \mathcal{H}(\mu)$ is exactly the weighted replay loss R_w of App. P.7 with $w = \mu$, so it is unbounded at the boundary and does supply the floor $p_a \geq \exp(-C_{T,\mu}/\mu_a)$. Three differences remain, and they are the design of the bank. Its weights are the reference’s own probabilities, so a mode the base model rarely emits receives a correspondingly weak floor, where uniform replay weights every admitted key alike. Its support is all of $\mathcal{A}$, so it protects incorrect categories on the same footing as correct ones, where the bank is verifier-filtered. And it cannot be estimated from on-policy samples without importance weights or draws from μ , where the bank is a stored sample of the policy’s own verified history. Verified replay is the barrier construction with the target replaced by a policy-generated, validator-accepted, uniformly weighted one.

Correctness at the analyzed targets. For Dr.GRPO and MaxRL with a full-support reference, the interior KL stationary point has $P^* < 1$, because the reference assigns positive mass to incorrect categories. Verified-only cross-entropy instead has the proved limit $P \rightarrow 1$ when $c_G \geq 0$. These statements concern the identified stationary point and convergence theorem, respectively. They do not exclude boundary stationary points of the continuously extended reverse-KL probability flow or establish a universal correctness cost for neural KL training.

Stationary targets and finite training budgets. Full-coverage uniform replay has the same categorical limit for every $\rho > 0$, but this does not make dose selection irrelevant. The finite-time analysis above compares replay with the instantaneous fresh concentration pressure $c_G(P)(1 - P)$. Small doses can allow diversity to decline before the asymptotic target is approached.

For reference KL, increasing β changes stationary correctness through $\logit P^* = \logit \mu(\mathcal{C}) + c_G(P^*)/\beta$. Its boundary recovery time also depends on β ; on the two-mode face the leading scale is $1/[2\beta s \log(1/s)]$. Both coefficients therefore matter at a finite horizon. We have not swept the replay dose empirically, so the finite-time comparison is a prediction of the specified dynamics rather than a measured dose response.

Scope of the comparison. The stationary and rate statements concern independently optimized categorical logits. Schedules, changing references, momentum, and shared neural parameters require separate analysis. The implemented sampled KL term also differs from an exactly differentiated distributional KL: an absent response supplies no direct sampled score, although normalization and shared parameters can still change its probability. Reference KL has guarantees for regularized control and proximity to a reference (Geist et al., 2019; Vieillard et al., 2020). The results here distinguish its stationary target and boundary force from verified replay; they do not exclude useful finite-time diversity preservation by KL.

Corollary P.37 (Two limits and a KL stationary value in the diversity metric). *The diversity axis this paper reports is $PCMD = 1 - C(q) = 1 - \sum_{c \in \mathcal{C}} q_c^2$. In the categorical mean flow, a bounded outcome-binary objective satisfying the positivity and unique-maximum hypotheses of Corollary P.6 sends $PCMD \rightarrow 0$, and adding full-coverage uniform verified replay sends $PCMD \rightarrow 1 - 1/m$ for*

every dose $\rho > 0$ (Corollary P.23). With the Dr.GRPO or centered MaxRL coefficient, adding a reference KL instead gives a single interior stationary point, at which

$$PCMD = 1 - \sum_{c \in \mathcal{C}} \left(\frac{\mu_c}{\mu(\mathcal{C})} \right)^2 \quad \text{for every } \beta > 0,$$

the frozen reference’s own success-conditional diversity (Proposition P.32).

Proof. Substitute $\mathbf{e}_{c^*}$, the uniform u on $\mathcal{C}$, and $\mu|_{\mathcal{C}}$ into $1 - \sum_c q_c^2$, using continuity of C on the simplex. $\square$

The first two statements are convergence results under their stated hypotheses. The third also holds as a limit for the exact Dr.GRPO steps of Corollary P.33; for centered MaxRL with $G \geq 3$, it identifies the stationary value without a convergence claim. Lemma P.31 separately describes which KL sublevels exclude a boundary face. Neither the stationary value nor the two-mode recovery rate is a bound on transient neural diversity. The distinction between the reference allocation and the discovered-bank target remains useful without that stronger claim.

Two boundaries, and which instrument reaches each. The results in this appendix separate along the two boundaries of the simplex that training actually visits, and they are reached by different means.

At $P \rightarrow 0$ no verified success is available anywhere, and Lemma P.18 bounds the probability of a mixed group — the only kind that carries fresh signal — by GP . This is a *discovery* boundary, and a fresh objective is the instrument for it: Corollary P.8 gives MaxRL a factor approaching G there, and Equation (17) identifies what it is maximizing as a harmonic mixture of $\text{pass}@k$ for $k = 1, \dots, G-1$.

At $p_b \rightarrow 0$ for a discovered correct mode, the bounded sampled-coordinate force vanishes, while verified replay contributes $\rho(w_b - p_b)$. The reference-KL force also vanishes, at a different rate. These are coordinate-level comparisons in the stated geometry: they do not exclude retention through parameter sharing or other update rules.

The fresh and replay terms compose into $F = \rho R_B - \Psi_{\text{MaxRL}}(P)$. Its descent gives the fixed-bank retention guarantee, and the finite-time analysis quantifies when replay overcomes concentration. Their gradients can oppose one another, so composition does not imply an absence of interference. Changing fresh advantages adds arithmetic to the existing update; replay adds stored targets and gradient computation. Equation (58) and App. Q quantify that cost separately from the mathematical comparison.

P.12 TWO CEILINGS, AND WHICH ONE A DOMAIN SITS UNDER

The results above are statements about a flow. This subsection reads them against measured cells, and the reading changes the conclusion in one place and sharpens it in two others. Table 59 reports terminal PCMD for Dr.GRPO with the same update and fresh-rollout budgets and a reference KL at 5 coefficients, 0.001, 0.01, 0.04, 0.1, 0.2, across 5 domains at Qwen2.5-0.5B and Level 1, beside the matched control and verified-replay arms. Every cell is the control configuration with β alone moved.

What the flow got right. The observed concentration is consistent with the categorical mechanism in Corollary P.6, although neural training does not satisfy that corollary’s update assumptions. The control with the same update and fresh-rollout budgets reaches PCMD at or below .004 in every domain in the table: not reduced diversity but effectively

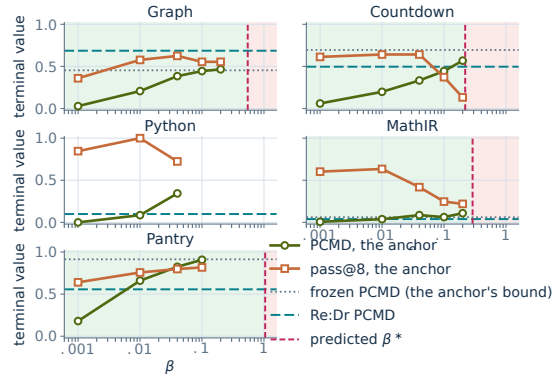


Figure 40: **What a coefficient buys, and where it starts spending.** Green is where Proposition P.32 still holds stationary correctness above one half, red the range past β^* ; the damage follows that boundary in all four domains carrying one. Python has neither, solving too little when frozen to define them.

Table 59: **Terminal PCMD beside categorical reference values.** Columns 11–13 give the frozen reference’s conditional diversity and the replay occupancy summaries $1 - 1/\mathbb{E}|\mathcal{B}|$. The former is the interior stationary value in Proposition P.32, not a transient upper bound. The latter is a Jensen upper bound on the mean uniform-bank limit $\mathbb{E}[1 - 1/|\mathcal{B}|]$ for nonempty fixed categorical banks; it does not certify held-out neural diversity. Re:Dr and Re:Max have separate summaries because their discovered banks differ. Every column but the β ones is the 32-disjoint-stream resample; the β columns use the same estimator on their own disjoint draws. Dashes are coefficients a domain does not yet carry, and a superscript on a β cell is a seed count below the five every other column holds, so a value resting on one run is marked where it is read rather than in a note. The last column is $\beta^* = c_G/(-\log \mu(\mathcal{C}))$, the coefficient at which Proposition P.32 puts stationary correctness at one half, computed from each domain’s own frozen correctness.

Domain	no retention		+ reference KL, by β					+ memory		reference values			
	Dr:GRPO	MaxRL	.001	.01	.04	.1	.2	Re:Dr	Re:Max	KL	Re:Dr	Re:Max	β^*
Graph	0.003	0.148	0.029	0.207	0.385	0.444	0.465	0.686	0.625	0.45	0.75	0.70	0.55
Countdown	0.004	0.022	0.060	0.198	0.333	0.444	0.568	0.497	0.531	—	0.56	0.52	0.22
Python	0.000	0.000	0.000	0.086	0.344	0.225 ³	—	0.099	0.239	—	0.15	0.42	—
MathIR	0.003	0.008	0.007	0.038	0.086	0.088	0.103	0.039	0.018	0.06	0.15	0.12	0.29
Pantry	0.000	0.000	0.180	0.660	0.824	0.901	0.925	0.557	0.556	0.91	0.85	0.85	1.05

none, which is what a winner-take-all limit predicts and what no reweighting of binary outcomes was able to avoid.

Measured checkpoints differ from the stationary predictions. Proposition P.32 makes the stationary conditional β -free. The measured coefficients order PCMD monotonically instead, and the ordering is stable across seeds. The same proposition puts stationary correctness at essentially one for every coefficient at or below .1, even at the cohort’s narrowest reference ($\mu(\mathcal{C}) = 0.0138$, measured on the frozen model in Countdown); the $\beta = .1$ cells instead sit near $\mu(\mathcal{C})$ itself rather than near that limit.

These discrepancies show that the stationary categorical predictions do not quantitatively describe the measured checkpoints. Finite training, shared parameters, and the sampled adaptive optimizer are all possible sources of the difference; this comparison does not identify their separate effects. The boundary-force identities and the two-mode $\Theta(1/[s \log(1/s)])$ versus $\Theta(\log(1/s))$ recovery comparison motivate mechanism checks, rather than certify the measured neural rates.

Reference diversity and bank occupancy. The table compares quantities with different meanings. For a nonempty fixed uniform categorical bank of size k , the limiting diversity is $1 - 1/k$. Across such banks, its mean is $\mathbb{E}[1 - 1/k] \leq 1 - 1/\mathbb{E}[k]$ by Jensen’s inequality. The occupancy columns report the latter summary. They are not exact mean limits when bank sizes differ, and training-bank occupancy does not furnish a bound on held-out neural PCMD. The KL column reports the reference conditional, which is the analyzed interior stationary value rather than a bound on transient diversity.

The measurements nevertheless motivate an occupancy explanation: where replay discovers few keys, it has few distinct targets to rehearse. Where the frozen model is broader, KL can preserve useful diversity without requiring further admission. These are interpretations of the observed comparison; the categorical results do not identify which mechanism causes a particular neural outcome.

The fresh objective moves the bound, not just the rate. Re:Dr and Re:Max carry separate occupancy columns because a bank is filled by discovery, and Corollary P.8 gives MaxRL a coefficient approaching the group size exactly where correctness is scarce. If that advantage is real outside the flow it should show up as a fuller bank in the domain with the least correctness to begin with, and not elsewhere.

That is what the occupancy columns do. In Python, the domain whose control reaches the lowest correctness of the five, the Re:Max bank holds 1.73 keys per prompt against Re:Dr’s 1.18, lifting the occupancy summary from .15 to .42 and terminal PCMD from .099 to .239. In the four domains that do not start starved, the two occupancies differ by at most .6 of a key and the terminal values track them: Re:Max is ahead in Countdown, level in Pantry, and behind in Graph and MathIR, where its bank is the slightly emptier one. The prediction was specific — a discovery advantage confined to small P — and the one domain it picks out is the one where it appears.

These measurements are consistent with separating fresh learning and memory. The per-prompt mean-direction identity does not itself predict the shared neural trajectory: MaxRL alone reaches .148 PCMD in Graph and at or below .022 elsewhere, no better than Dr.GRPO alone in four domains. The larger Python bank motivates the discovery interpretation, but does not isolate it from prompt weighting, stochasticity, or optimizer effects.

Consequences for the comparison. The claim that a reference KL is the wrong instrument for diversity does not survive this table and is withdrawn. What survives is narrower and better specified. Reference KL retains what its reference had; verified replay rehearses what its bank holds; neither control retains anything. Which of the first two reaches further is a property of the domain — the frozen model’s diversity against the bank’s realized occupancy — and not a property of the method. Sorting the domains by what the best coefficient buys gives three cases rather than a ranking. In Python and Pantry an anchor carries more diversity than either memory arm at no cost in `pass@8`. In Countdown and MathIR it passes them on diversity only at a coefficient that gives correctness up, which is a different claim and is read as one below. In Graph no coefficient reaches either memory arm at all.

The bank-capacity premise of Corollary P.23 helps interpret these differences. Lower replay diversity in domains whose banks remain small is consistent with limited discovery, but occupancy alone does not separate discovery from retention or bound held-out neural PCMD. The coefficient sweep measures a correctness–diversity tradeoff at finite checkpoints; it does not establish that the policies reached their stationary targets. The replay dose also controls restoration time in Theorem P.26. A matched replay-dose sweep would be needed to compare those sensitivities empirically.

P.13 WHY A MEMORY RATHER THAN AN ANCHOR, ONCE THE REFERENCE IS COLLAPSED

App. P.12 withdrew the claim that a reference KL is the wrong instrument for diversity, because in 2 of 5 domains it reaches further than either memory arm and gives up no correctness doing it. This subsection states what remains of the case for a memory, and it is not a restatement of the withdrawn claim. The argument is about which of the two quantities a practitioner is relying on, and about what that quantity is worth in the setting this paper measures.

The case for the anchor, at its strongest. For every $\beta > 0$, the analyzed reference-KL flow has a full-support stationary point. It needs no replay bank, admission rule, or bank capacity, although it retains a reference policy, and adds a familiar loss term. Its best cell is Pantry at $\beta = 0.2$, where it reaches $\text{PCMD} = 0.925$ against the better memory arm’s 0.557 with `pass@8` of 0.812 against 0.797: more diversity and no less correctness. That cell rests on 5 of the five seeds its column will carry, which is why the table marks it, but nothing in this appendix disputes the measurement, and a reader choosing a method for a domain that resembles Pantry should choose the anchor.

What the anchor is actually buying. Proposition P.32 says the quantity being bought is the reference’s own success-conditional diversity, and App. P.12 finds the measured arms at 91 to 100 percent of it. This empirical agreement is consistent with reference retention, while the stationary formula alone does not describe the finite training trajectory or establish a bound on its diversity.

This is why Pantry is the case it is. The frozen Qwen2.5-0.5B has $\text{PCMD} = 0.91$ there, and .85 is the most a capacity-16 bank assembles from 18.19 admissible keys per prompt. The anchor wins because the reference is broader than anything the bank can hold — not because copying is a better mechanism than accumulating, but because in that one cell there is more worth copying than the bank can accumulate.

How often is there that much worth copying? This paper measures the answer for deployed models. Section 2.3 reports 7 hosted deployments across the benchmark; their macro PCMD spans 0.14–0.31, with Kimi K3 highest at 0.31. Of the 101 reportable model–domain–level cells in that cohort, 7 reach $\text{PCMD} = .50$ and the highest of any of them is 0.74. Pantry’s frozen 0.91 is not a typical reference; it is higher than every cell of every frontier model we measured, including their own Pantry cells.

Under Proposition P.32, a deployed model used as the reference would determine the categorical stationary conditional. Its measured diversity therefore describes a candidate target; it is not a ceiling on a retrained neural policy. The Re:Dr result of 0.686 in Graph concerns a different evaluation population from the hosted macro average. The structural distinction is

the source of the target: verified replay uses discovered bank membership, whereas the analyzed reference-KL target uses the fixed reference distribution.

Three further asymmetries, in decreasing strength. The measured knee is the sharpest. In Countdown and MathIR every coefficient that carries the anchor past the memory arms on diversity also costs correctness, and the steepest of them is MathIR at $\beta = 0.2$, where PCMD reaches 0.103 against 0.039 while `pass@8` falls to 0.214 from 0.816. The diversity is real and so is the price; in those domains no coefficient buys the first without the second.

The coefficient is also domain-dependent in a way the dose is not, and the table’s last column says why: β^* , the coefficient at which Proposition P.32 halves stationary correctness, is set by each domain’s own frozen correctness and runs from 0.22 to 1.05 across the domains that carry one. A practitioner must therefore locate a window whose position depends on the reference’s correctness, the group size and the estimator, per domain, before training. Replay has the same categorical limit for every positive dose, but its finite-time restoration still depends on the dose relative to fresh concentration pressure.

Last and most structural: an anchor requires a reference that is worth anchoring to, and successive rounds of RL do not supply one. The bank is assembled from the policy’s own verified successes and is available on the second round for the same reason it was on the first.

What would change this reading. The argument rests on a measured claim about references, not a theorem about mechanisms, so it is defeasible in an obvious way: a setting whose base policies carry diversity like Pantry’s, rather than like the hosted cohort’s, is a setting where anchoring is the better primitive and this appendix says so. The evidence here is also thin exactly where it is most favourable to the anchor. 1 of the 24 measured coefficient cells stand on fewer than five seeds and the thinnest on 3, the table marks each one, and the cells still filling are the high- β ones the anchor’s case rests on. MathIR’s support is keyed to derivations rather than to answers besides, so a mode certified there need not be one any policy produces. The anchor’s free wins are also the domains where the bank is furthest from holding what the prompt admits, which is the reading App. P.12 already gives them: they are discovery failures on the memory side rather than retention successes on the anchor’s. One of them does not even fit the inheritance account, since a reference that verifies nothing has no success-conditional diversity to pass on, so what the coefficient supplies there cannot be inferred from the stationary conditional formula. Zero observed successes also does not establish zero population correctness. We report them as wins regardless.

P.14 SCOPE RELATIVE TO ENTROPY, KL, ADMISSION, AND NEURAL OPTIMIZATION

Dependence on optimization geometry. For a smooth parameterization $z = z(\theta)$ with Jacobian $J_\theta = \partial z / \partial \theta$, Euclidean parameter flow instead induces $\dot{z} = c_G(P) J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization metric. Equal binary reward alone does not force this winner selection. The natural policy gradient update of Agarwal et al. (2021, Lemma 15), specialized to this bandit, is $p_a^+ \propto p_a \exp(\eta c_G(P) R_a)$ and preserves every ratio of correct probabilities at each finite step. Its continuous version has $\dot{z}_a = c_G(P)(R_a - P)$, with $R_a = \mathbf{1}[a \in \mathcal{C}]$. The distinction between Euclidean and natural updates also underlies the tabular entropy identities of Cui et al. (2025, Theorems 1–2); entropy and verified-mode diversity remain different observables.

The mean-gradient identity has broader scope than the collapse conclusion. For one prompt and a common autonomous preconditioner $M(\theta)$, positive coefficients give flows $c_a(P) M(\theta) \nabla_\theta P$ that follow the same orbit after a change of time. Outcome-dependent covariance, response-length weights, shared prompts, and optimizer memory can break the corresponding training-trajectory equivalence, as the preceding neural analysis shows. Parameter count alone does not identify this geometry. The empirical scale comparisons change pretrained models and schedules; they do not establish the cause of cross-scale differences or the training history of a hosted model.

A bounded advantage based on fresh samples has a different boundary behavior. The next bound includes group-dependent detached scores and distinguishes sampled updates from gradients of an exactly evaluated entropy functional.

Lemma P.38 (Bounded sampled scores have no probability-independent boundary force). *Under Assumption P.1, condition on the training history, draw $Z_1, \dots, Z_G$ independently from p , and let*

a_i be any detached, possibly group-dependent advantages with $|a_i| \leq B$. To interpret the bound uniformly as $p_b \rightarrow 0$, assume a finite B independent of that limit. At the on-policy point define

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (e_{Z_i} - p).$$

For every category b ,

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.

Proof. Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{Z_i = b\} - p_b| = 2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

The bound compares the isolated logit-gradient components. It does not prove inevitable collapse, rule out retention by a bounded estimator, or guarantee that replay dominates the full update. An unsampled mode can still change through softmax normalization or shared parameters. Exact entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero; the lemma is not an impossibility result for entropy regularization. Nor does the absence of a mode from a finite sample establish zero probability.

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q\|u), \quad R_u(p) = \log m - \log P + \text{KL}(u\|q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$ have the unique global optimum $P = 1, q = u$ in probability space on the simplex with $P > 0$, for positive coefficients. This is a boundary limit for finite-logit softmax policies. The optimum follows from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets (Pereyra et al., 2017; Agarwal et al., 2021): replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions (Mei et al., 2020); our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes. In particular, the fixed-weight argument does not cover a target whose weights continue to change with fresh observation counts.

For a fixed joint potential, a discrete bound $F(\theta_{n+1}) \leq F(\theta_n) + \epsilon_n$ with $\epsilon_n \geq 0$ and $\sum_{n \geq n_0} \epsilon_n = E < \infty$ would extend Lemma P.28, using $D = F(\theta_{n_0}) + J_{\max} + E$ for steps $n \geq n_0$. This follows by telescoping and bounds retention, not convergence. Corollary P.29 instead permits conditional expected drift with a controlled accumulated error budget, and yields a simultaneous probability statement over iterations and protected exemplars. An optimizer-specific argument is still needed to establish either form of energy control. A positive replay coefficient or small nominal learning rate does not establish it. Adam moments, clipping, momentum, prompt

scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not established the required joint-energy or update-error assumptions. The mathematical contribution is the stated categorical mechanism and conditional exemplar argument; empirical support and longitudinal exemplar scores remain separate measurements.

Q WHAT VERIFIED REPLAY COSTS

Q.1 WHAT THE CONTROL MATCHES, AND WHAT IT DOES NOT

The core factorial matches initialization, data, optimizer settings, 3,072 optimizer updates, and $G = 16$ fresh rollouts per visited prompt. Both arms maintain banks and use the same configured replay traversal; the control replaces the replay score derivative with exact zeros. Replay gradients join the fresh-task gradients before the existing optimizer step. Activating replay therefore does not add an optimizer step or a fresh rollout, and the control already incurs replay computation.

This design isolates the effect of applying the replay learning signal under the common schedule. It does not match total training compute: policies can produce different rollout lengths and populate different banks, changing whether replay executes and how many tokens it processes. Equal code paths and update counts do not imply equal FLOPs or wall-clock time. More fundamentally, the control’s replay work contributes no learning signal and is not reassigned to useful baseline updates. We do not report a control that spends an equivalent budget on extra task-gradient steps, extra fresh rollouts, or a longer training horizon. The results therefore do not establish that replay is the best use of an equal total-compute budget.

The uniform-versus-frequency weighting ablation addresses a different question: how to allocate replay weight among stored modes. It retains the same bank construction, scheduling, and update rule, but its policies can also develop different workloads. It supports a contribution from balancing modes; it is not a comparison of performance at equal total compute.

Q.2 TOKEN-BASED COST ACCOUNTING AND ITS LIMITS

The retained accounting uses diagnostics from 75 replay runs: five seeds in each of five domains at Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B. Table 60 summarizes those logs without new training. It compares components of replay and a hypothetical reference-KL implementation; it is not a measured replay/control runtime comparison.

Resident state. Under full fine-tuning, a frozen reference copy requires $\Theta(|\theta|)$ parameter bytes. A replay bank stores prompt and exemplar token identifiers, whose cost depends on the number of prompts, retained keys, and token lengths rather than directly on parameter count. The estimated mean bank footprint is at most 444 kB across these families, while the reference copy grows from 0.99 to 6.17 GB. The corresponding reference-to-bank ratios are 2,554–15,697. These are component storage estimates, not peak accelerator-memory measurements: bank bytes assume four bytes per token and exclude bookkeeping, optimizer state, and activations. A reference recovered by disabling an adapter need not incur a separate resident parameter copy.

A likelihood-pass subtotal, not total training FLOPs. Let T_{replay} include the prompt and response tokens processed by a selected bank, and T_{fresh} the corresponding fresh-group token estimate. Approximating a forward pass by $2|\theta|T$ and its backward by $4|\theta|T$, the table prices one differentiable replay forward/backward at $6|\theta|T_{\text{replay}}$ and one reference forward at $2|\theta|T_{\text{fresh}}$. Thus its reported ratio is

$$\frac{\text{replay likelihood-pass FLOPs}}{\text{reference-forward FLOPs}} \simeq \frac{3T_{\text{replay}}}{T_{\text{fresh}}} \approx \frac{3\mathbb{E}|\mathcal{B}|}{G}, \quad (58)$$

where the final approximation additionally assumes comparable fresh and stored sequence lengths. Stored exemplars and current rollouts need not have the same lengths, so occupancy alone does not determine the ratio. The token-based cell means range from 0.19 to 1.64; the subtotal is below one in 12 of 15 cells. The three exceptions are PantryPlan. This denominator is a reference forward, not the baseline’s complete training cost.

The maintained implementation also makes a detached scoring forward before the differentiable replay pass. Under the same approximation, these passes together would cost about $8|\theta|T_{\text{replay}}$, before

Table 60: **Component cost estimates from retained replay diagnostics.** The first row per family gives mean selected-bank occupancy, reference parameter bytes, estimated bank token bytes, and their storage ratio. The second row is the likelihood-pass/reference-forward FLOP ratio of Equation (58); it excludes the detached scoring pass and is not a replay/control overhead ratio. Diagnostics are averaged within each run, then across five seeds per cell (75 runs overall).

Family	Graph	Countd.	Python	MathIR	Pantry	Ref (GB)	Bank (kB)	Ratio
Qwen2.5-0.5B	3.98	2.28	1.18	1.17	6.74	0.99	387	2,554
ratio	0.75	0.43	0.22	0.22	1.26			
Falcon3-1B	3.83	1.88	1.24	1.20	8.76	3.34	444	7,520
ratio	0.72	0.35	0.23	0.23	1.64			
Qwen2.5-3B	3.69	1.50	1.27	1.03	6.46	6.17	393	15,697
ratio	0.69	0.28	0.24	0.19	1.21			

activation recomputation and other system costs. The table’s $6|\theta|T$ subtotal omits that scoring pass and cannot certify actual historical FLOPs. Both the active and exact-zero control paths retain scoring and backward computation; this subtotal must not be interpreted as their measured cost difference.

Measurement scope. The builder averages retained rows with replay diagnostics and an available fresh-length measurement, then averages runs within each model–domain cell. It does not deduplicate repeated step records or truncate them to the 3,072-update horizon. Fresh-group token counts are reconstructed from logged response lengths and a prompt-length estimate derived from replay rows. These are descriptive workload summaries, not a cumulative compute audit. They exclude rollout generation, padding, attention-specific costs, activation recomputation, verification, bank maintenance, optimizer work, communication, and evaluation. Hardware pools also differ across cohorts. Neither these estimates nor the reference-KL comparison establishes an end-to-end speedup or resolves the missing budget control.

Q.3 THE MISSING COMPARISON AT EQUAL TOTAL COMPUTE

A separate comparison should give the baseline a fixed cumulative training budget and let it spend replay’s allocated work on useful learning. Relevant controls are additional task-gradient passes over fresh groups already collected, larger fresh-rollout groups, and additional fresh-sampling updates beyond the eight-pass horizon. These alternatives change different resources; none is represented by setting the replay derivative to zero. Their correctness and PCMD should be evaluated at the same total-compute budget, with the same initialization, task splits, seeds, evaluation requests, and hardware configuration.

Such a comparison needs either measured cumulative FLOPs or timing on controlled hardware, including rollout generation and every learner pass. It should report both the budget achieved and any overshoot from a final indivisible update, with evaluation costs treated consistently. Until that experiment is available, our empirical conclusion is that applying verified replay gradients improves the reported aggregate outcomes under the fixed update and fresh-sampling schedule. Whether a baseline can match those outcomes by spending the same total compute on more useful training remains open.

Part VII. Extended comparison

R DIVERSITY-PRESERVING RLVR: EXTENDED COMPARISON

That RLVR concentrates its outputs is not our discovery, and this appendix states what three contemporary lines establish and exactly which of our claims each one already covers. Cui et al. (2025) identify entropy collapse in reasoning RL, attribute the fall to a covariance between token probability and advantage, and fit a ceiling on downstream reward as a function of remaining entropy; Clip-Cov and KL-Cov restrict the high-covariance tokens and postpone the fall. Lochab et al. (2026) show that RLVR’s objective is indifferent to how correct probability is divided among correct answers, characterize the uniform-correct policy as the optimum of a robustness and entropy-regularized criterion, and add a conditional uniformity penalty to GRPO. Li et al. (2026b) attribute the same concentration to reverse-KL mode seeking and replace it with an approximate forward-KL objective that aligns the group to a target proportional to sampled rewards. We take the phenomenon as established by this literature and do not claim it. Our three separable claims are a success-conditional measurement, an intervention that is not a fresh-update objective, and a stated reason the two classes differ in reach.

The axis each work reads. Cui et al. (2025) track policy entropy over all sampled tokens. That quantity mixes correct and incorrect mass, so it falls both when a policy stops offering alternative correct answers and when it stops offering wrong ones, and a training run that improves only by suppressing errors registers as a loss of diversity. Lochab et al. (2026) report $\text{pass}@K$ and an equation-level count within the correct set; Li et al. (2026b) read diversity through downstream score on problems with many feasible solutions. $\text{pass}@K$ and downstream score rise with the success rate on their own: (1) makes the dependence explicit, and across the 375 untrained measurements of App. B.5 $\text{distinct}@8$ correlates .936 with $\text{pass}@8$. PCMD removes that dependence by construction — it is a function of q alone — so correctness and diversity are two axes a method can move in opposite directions, and the concentration Sec. 5.1 reports is not a restatement of a change in the success rate. MODEBENCH also fixes sameness with the executable check that assigns reward, so the equivalence used to define a mode is the one used to grant credit, with no judge or embedding threshold between them.

Where the intervention can act. All three interventions modify the fresh on-policy update. Clip-Cov and KL-Cov reweight tokens of the current rollouts; UCPO reallocates a prompt group’s positive advantage mass across that group’s verifier-positive rows; DMPO forms its target over the trajectories the group sampled. Each therefore requires a mode to be present in the current group before it can act on that mode, and that requirement is the pressure App. P formalizes. Lemma P.38 applies to this whole class whenever the per-row scores a_i are detached and uniformly bounded by some B : the expected logit velocity on category b is at most $2Bp_b(1 - p_b)$, which vanishes with p_b , and on a group containing no draw of b the component is exactly $-p_b G^{-1} \sum_i a_i$. UCPO’s estimator satisfies the premise as implemented here. Write G^+ for the number of verifier-positive rows in the group, A^+ for the per-row positive advantage a uniform split would assign, $\hat{q}_i$ for row i ’s estimated within-group mode frequency, and $\omega \in [0, 1]$ for the weight UCPO puts on inverse-frequency rather than uniform reallocation. Such a row receives $A_i = G^+ A^+ ((1 - \omega)/G^+ + \omega v_i / \sum_j v_j)$ with $v_i = 1/\hat{q}_i$ self-normalized within the group, so $|A_i| \leq G^+ |A^+|$ and the total positive mass is conserved — it is a redistribution among sampled correct rows, not an addition to an absent one. Verified replay is outside the premise because its target is not a function of the current group: for a banked key the logit velocity is $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ as $p_b \rightarrow 0$. This is a statement about which mechanism retains a signal at the boundary, and App. P.14 records what it is not: it does not prove that collapse is inevitable, does not rule out retention by a bounded estimator, and is not an impossibility result for entropy regularization, whose surprisal is unbounded near zero and therefore fails the bounded-score premise.

The target is shared; the weighting is not. UCPO’s uniform-correct target is the target our replay loss descends, and App. P.14 shows that exact conditional entropy shares that optimum too, the three differing in the direction of the KL and hence in what happens to the loss when a mode’s probability vanishes. So the disagreement with Lochab et al. (2026) is not about where to aim. DMPO is the one of the three that aims elsewhere: a target proportional to rewards over

sampld trajectories is, under a binary verifier, uniform over the correct sampled trajectories, which places mass on each mode in proportion to how many times the group produced it. That is frequency weighting, and it is the control we run directly. Holding the bank and the reuse fixed and weighting modes by rediscovery frequency instead of equally costs $+.318$ extra verified modes per prompt ($[+.272, +.358]$) with no aggregate correctness difference (App. F.6). Reuse is not the active ingredient; declining to let popularity set the rehearsal rate is.

What we measured rather than cited. Two of the three lines above are implemented directly and run as matched comparators rather than only discussed; the third is covered by the frequency-weighting control already described, and sparse RLEP-Dr is added beside them for the memory side. UCPO is implemented from its Equations 16, 47 and 51, and runs against seed-matched Dr.GRPO at Qwen2.5-0.5B and Falcon3-1B (App. F.4); its gains are local, raising Countdown correctness and PCMD there while its MathIR effects are inconclusive. An entropy-shaped objective on the success-conditional axis is implemented as Fixed Semantic MaxEnt (App. E), which supplies a bounded surprisal bonus to underrepresented verified outcomes that appear in a fresh rollout; its effects are likewise heterogeneous, and App. E states why it carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee. Sparse RLEP-Dr covers the memory side (Zhang et al., 2025): the same reuse of verified successes, without uniform canonical-key weighting.

Scope of the comparison. These comparators are read at the coverage they have. UCPO and sparse RLEP-Dr run at two of our three scales on MODEBENCH, whose executable domains are not the mathematical reasoning suites of Lochab et al. (2026) or the NP-Bench problems of Li et al. (2026b), and the token-level interventions of Cui et al. (2025) act at a different granularity from the outcome-level comparators we run. Nothing here ranks these methods on their own benchmarks. The claim is narrower and is the one the lemma and the weighting ablation support jointly: an objective confined to the current group and a memory over verified keys reach different modes, and uniform rehearsal over those keys is what separates our memory from replay that revisits successes as often as it found them.

Part VIII. Records

S SOURCE INTEGRITY AND REPRODUCIBILITY

S.1 ADMISSIBLE SOURCES AND MISSING CHECKPOINTS

Falcon3-1B Countdown Re:Dr seed 59 contains two complete four-draw terminal evaluation sequences with identical request identities but conflicting metrics. That source is excluded at every checkpoint and from every efficacy endpoint, paired contrast, and aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs: 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial intersections are descriptive and receive no confidence interval.

A second kind of conflict is more common and has a diagnosable cause. The job watchdog can requeue a run; the restart finds no resumable checkpoint, begins from initialization, and evaluates the initial model a second time, appending a second step-0 block to the same record. Separate attempts can land on different GPU models, and GPU model changes floating-point reduction order and therefore the sampled tokens, so the two blocks disagree and the checkpoint is invalidated under the rule below. This is why several MaxRL initial checkpoints are unavailable, leaving the longitudinal contrast with three defined seeds in Falcon3-1B Graph and one in Falcon3-1B PantryPlan. The same hardware sensitivity means that initial evaluations of the identical frozen base model agree within a GPU pool and can differ across pools, which is one reason initial evaluations are measured within each method rather than shared across arms; a before–after contrast compares a method with its own initial checkpoint and is unaffected. Of the step-0 blocks in a run’s records, exactly one is the block its admitted trajectory continues from, and App. S.2 admits that block for the 27 cells where it is unique.

Trajectory records use admitted sources and approved continuations or replacements, with superseded attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy and mode-count views use the same populations and gap rules. Supporting methods retain their explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the curves. A curve point is drawn from the cohort seeds that report at that checkpoint and is ringed where that is fewer than all of them, so a checkpoint whose evidence exists for part of the cohort is read as the partial measurement it is rather than as an absence; a checkpoint no seed reports remains a gap. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule, and source hashes.

S.2 CONFLICTED INITIAL CHECKPOINTS, ADMITTED BY PROVENANCE

A conflicted step-0 endpoint is a requeue duplicate, not a missing evaluation, and the two attempts disagree only because they executed on different GPU models. For each affected cell the analysis admits the step-0 block that the cell’s admitted trajectory continues from — chosen by file order, step number and the presence of training rows, never by a metric value. The rule resolves 27 of the 36 conflicted cells and leaves nine, where more than one such block exists, unavailable.

Two checks bound what that admission moves. The twelve Dr.GRPO and GRPO before–after comparisons that carry Section 5.1 are unaffected and reproduce to 10^{-9} , and no terminal replay-effect block changes at all, since those compare two trained checkpoints and use no initial endpoint. What it buys is precision rather than a new direction: every affected block moves from one to four defined seeds up to all five and receives a nominal interval, while its sign is unchanged.

It does not license a broader collapse claim. Falcon3-1B Graph under MaxRL moves the other way, with an interval excluding zero, as does its replay variant; MaxRL concentrates on Countdown, MathIR and PantryPlan at that scale but not on Graph. We therefore state the training-induced concentration result for the RLVR-only objectives it was measured on, and report the MaxRL longitudinal cohort as the mixed result it is.

S.3 CONFLICTED SCHEDULED CHECKPOINTS, ADMITTED BY PROVENANCE

The requeue that duplicates a step-0 block duplicates blocks at scheduled training steps in the same way, and the frozen snapshot invalidates those checkpoints under the same rule, which the training-curve figures draw as gaps. The step-0 rule above extends to any scheduled step: within a source file whose attempt trained past that step, a block of four draws counts as continued when the next evaluation row carries a later step, and the checkpoint is admitted only where exactly one continued block exists across the cell’s files. At step 0 this is that rule verbatim, so the 27 initial admissions are carried by the same mechanism. It reads file order, step numbers, draw indices and the presence of later training rows, never a metric value.

Of the 70 conflicted scheduled checkpoints away from step 0, the rule resolves 27. The remaining 43 carry more than one continued block, because the run was requeued and more than one attempt trained past that step, so nothing in the source structure separates them. They stay unavailable, and the figures draw them as gaps rather than interpolating across them. Only the terminal export of each run was retained, so the mid-training weights those checkpoints evaluated no longer exist and no re-evaluation can settle them.

S.4 DATA, MODELS, AND SCIENTIFIC RECORDS

The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row identities, and construction and admission records. Its loading guide and evaluation guide identify the reported splits and preserve auxiliary data separately. The model archive links each available model–domain–method–seed export to an immutable upload revision containing original weights, configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license. The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-weighting ablation, and supporting controls to their model families; the scientific-record index and restore guide identify preserved analysis inputs and SHA-256 verification procedures.

Each empirical figure has a machine-readable record identified in the figure manifest, containing its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper versions to their builders, records, and reproduction commands. Detailed source and recovery inventories remain in those records and the repository’s archived audit documents. Model availability may extend beyond the frozen analysis population; it does not change a figure’s source-admission or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories require the recorded evaluation checkpoints.

S.5 EXACT CONTINUATION AND VALIDATION

Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream state in App. D.4, in addition to model weights. State mismatch, non-finite values, missing completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures. Placement-only recovery may change hardware or wall time but must preserve the scientific update. Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints or silently incorporated from a newer archive.